Introduction to IoT Systems

Building connected devices from the breadboard up

Sergio Gago-Masagué

Contents

Preface . . . 17
Who this book is for . . . 17
How the book is organized . . . 17
Two tracks . . . 17
Conventions . . . 17
License and contributions . . . 17
Sources and originality . . . 17
Acknowledgments . . . 18

Using This Book with the IoT@UCI Kit . . . 19
What is in the kit . . . 19
How each chapter is organized . . . 19
Safety . . . 20

I Part I — Foundations

1 What Is the Internet of Things? . . . 23
1.1 Why it matters . . . 23
1.2 Concepts . . . 23
1.2.1 A working definition . . . 23
1.2.2 Related terms . . . 23
1.2.3 From the Internet of People to the Internet of Things . . . 24
1.2.4 Three kinds of connection . . . 24
1.2.5 A reference architecture . . . 24
1.2.6 Application domains . . . 25
1.2.7 The questions every IoT system must answer . . . 25
1.3 In depth (optional) . . . 26
1.3.1 Back-of-the-envelope engineering . . . 26
1.4 Design trade-offs . . . 27
1.5 Common pitfalls . . . 27
1.6 Check your understanding . . . 27
1.7 Further reading . . . 28

2 Development Boards and the IoT Kit . . . 29
2.1 Why it matters . . . 29
2.2 Concepts . . . 29
2.2.1 Voltage, current and resistance . . . 29
2.2.2 Lighting an LED safely . . . 29
2.2.3 Breadboards . . . 30
2.2.4 From fixed circuits to microcontrollers . . . 30

2.2.5 What a development board adds . . . 30
2.2.6 Arduino and the ESP family . . . 30
2.2.7 The ESP32 DevKit V1 . . . 31
2.2.8 The IoT@UCI kit . . . 32
2.3 In depth (optional) . . . **33**
2.3.1 Currents, resistors and heat on the kit . . . 33
2.4 Design trade-offs . . . **33**
2.5 Common pitfalls . . . **34**
2.6 Check your understanding . . . **34**
2.7 Further reading . . . **34**

3 Programming the Device . . . **35**
3.1 Why it matters . . . **35**
3.2 Concepts . . . **35**
3.2.1 The toolchain . . . 35
3.2.2 The Arduino programming model . . . 35
3.2.3 Digital input and output . . . 36
3.2.4 Pull-up and pull-down resistors . . . 36
3.2.5 Debouncing . . . 36
3.2.6 Polling and interrupts . . . 37
3.2.7 Analog input . . . 38
3.2.8 Analog output: DAC and PWM . . . 38
3.2.9 Time without blocking . . . 38
3.3 In depth (optional) . . . **39**
3.3.1 Bounce, time and wrap-around . . . 39
3.4 Design trade-offs . . . **40**
3.5 Common pitfalls . . . **40**
3.6 Check your understanding . . . **41**
3.7 Further reading . . . **41**

4 Embedded Systems and Operating Systems . . . **43**
4.1 Why it matters . . . **43**
4.2 Concepts . . . **43**
4.2.1 What makes a system “embedded” . . . 43
4.2.2 Microprocessors, microcontrollers and SoCs . . . 44
4.2.3 Real-time systems . . . 44
4.2.4 Architecture 1: the super-loop . . . 44
4.2.5 Architecture 2: foreground/background . . . 44
4.2.6 Architecture 3: a real-time kernel . . . 44
4.2.7 FreeRTOS on the ESP32 . . . 45
4.2.8 Do you need an RTOS? . . . 46
4.3 In depth (optional) . . . **46**
4.3.1 Will every task meet its deadline? . . . 46
4.4 Design trade-offs . . . **47**
4.5 Common pitfalls . . . **47**
4.6 Check your understanding . . . **47**
4.7 Further reading . . . **48**

II Part II — The Things

5 Serial Communication: UART, SPI and I²C . . . 51
5.1 Why it matters . . . 51
5.2 Concepts . . . 52
5.2.1 Protocols, parallel and serial . . . 52
5.2.2 UART: asynchronous serial . . . 52
5.2.3 SPI: the fast synchronous bus . . . 54
5.2.4 I²C: two wires, many devices . . . 55
5.3 In depth (optional) . . . 58
5.3.1 Bit budgets and bus electrics . . . 58
5.4 Design trade-offs . . . 59
5.5 Common pitfalls . . . 59
5.6 Check your understanding . . . 60
5.7 Further reading . . . 60

6 Sensors . . . 61
6.1 Why it matters . . . 61
6.2 Concepts . . . 61
6.2.1 What a sensor does . . . 61
6.2.2 Smart sensors . . . 62
6.2.3 Classifying sensors . . . 62
6.2.4 Common sensor types . . . 62
6.2.5 Reading a sensor's specifications . . . 63
6.2.6 The kit's sensors . . . 63
6.2.7 Making readings trustworthy . . . 65
6.3 In depth (optional) . . . 66
6.3.1 From voltage to physics . . . 66
6.4 Design trade-offs . . . 67
6.5 Common pitfalls . . . 67
6.6 Check your understanding . . . 68
6.7 Further reading . . . 68

7 Actuators and the Observe–Decide–Act Loop . . . 69
7.1 Why it matters . . . 69
7.2 Concepts . . . 69
7.2.1 What an actuator is . . . 69
7.2.2 The Observe–Decide–Act loop . . . 69
7.2.3 Classifying actuators . . . 70
7.2.4 Families of actuators . . . 70
7.2.5 Non-mechanical actuators . . . 71
7.2.6 Driving loads safely . . . 72
7.2.7 Controlling a servo . . . 72
7.2.8 Deciding: simple control strategies . . . 72
7.2.9 Fail-safe design . . . 73
7.3 In depth (optional) . . . 73
7.3.1 PWM, inductors and capacitors . . . 73
7.4 Design trade-offs . . . 74

7.5 Common pitfalls . . . 74
7.6 Check your understanding . . . 75
7.7 Further reading . . . 75

III Part III — Connecting Things

8 Communication Models: OSI and TCP/IP . . . 79
8.1 Why it matters . . . 79
8.2 Concepts . . . 79
8.2.1 Protocols and layers . . . 79
8.2.2 The OSI reference model . . . 79
8.2.3 Encapsulation . . . 80
8.2.4 TCP/IP: the internet that won . . . 81
8.2.5 Four kinds of address . . . 81
8.2.6 How a device joins a network . . . 82
8.2.7 NAT . . . 82
8.2.8 TCP and UDP . . . 82
8.2.9 TCP/IP on the ESP32 . . . 83
8.3 In depth (optional) . . . 83
8.3.1 Overhead and delay, counted . . . 84
8.4 Design trade-offs . . . 84
8.5 Common pitfalls . . . 84
8.6 Check your understanding . . . 85
8.7 Further reading . . . 85

9 IoT Network Requirements and the Protocol Stack . . . 87
9.1 Why it matters . . . 87
9.2 Concepts . . . 87
9.2.1 Requirement 1: support for constrained devices . . . 87
9.2.2 Requirement 2: massive scalability . . . 88
9.2.3 Requirement 3: determinism . . . 88
9.2.4 Requirement 4: security and privacy . . . 88
9.2.5 Requirement 5: application interoperability . . . 89
9.2.6 The IoT protocol stack . . . 89
9.3 In depth (optional) . . . 90
9.3.1 The limits every radio lives within . . . 90
9.4 Design trade-offs . . . 90
9.5 Common pitfalls . . . 91
9.6 Check your understanding . . . 91
9.7 Further reading . . . 91

10 The Link Layer: Short and Long Range . . . 93
10.1 Why it matters . . . 93
10.2 Concepts . . . 93
10.2.1 What IoT asks of the link layer . . . 93
10.2.2 Energy and duty cycling . . . 93
10.2.3 IEEE 802.15.4: low-rate personal area networks . . . 94
10.2.4 Bluetooth . . . 95

10.2.5 Wi-Fi . . . 96
10.2.6 Long range: LPWANs . . . 96
10.2.7 Wired: time-sensitive networking . . . 98
10.2.8 Choosing a link technology . . . 98
10.3 In depth (optional) . . . **99**
10.3.1 Link budgets and time on air . . . 99
10.4 Design trade-offs . . . **100**
10.5 Common pitfalls . . . **100**
10.6 Check your understanding . . . **101**
10.7 Further reading . . . **101**

11 The Internet Layer: 6LoWPAN and RPL . . . **103**
11.1 Why it matters . . . **103**
11.2 Concepts . . . **103**
11.2.1 Why IP for IoT? . . . 103
11.2.2 The internet layer, briefly . . . 103
11.2.3 IPv6 in a nutshell . . . 103
11.2.4 Low-power and lossy networks . . . 104
11.2.5 6LoWPAN: IPv6 over small frames . . . 104
11.2.6 RPL: routing in lossy networks . . . 104
11.2.7 Thread: putting it together . . . 105
11.2.8 Mobility and security at the internet layer . . . 106
11.3 In depth (optional) . . . **106**
11.3.1 Fragments, links and routes . . . 106
11.4 Design trade-offs . . . **107**
11.5 Common pitfalls . . . **107**
11.6 Check your understanding . . . **108**
11.7 Further reading . . . **108**

12 Application Protocols: HTTP, CoAP, MQTT and More . . . **109**
12.1 Why it matters . . . **109**
12.2 Concepts . . . **109**
12.2.1 Two interaction patterns . . . 109
12.2.2 HTTP and REST . . . 109
12.2.3 CoAP: REST for constrained devices . . . 110
12.2.4 MQTT: publish/subscribe telemetry . . . 111
12.2.5 Other application protocols . . . 114
12.2.6 Payload formats . . . 114
12.2.7 Choosing a protocol . . . 115
12.3 In depth (optional) . . . **115**
12.3.1 Reliability and overhead, quantified . . . 115
12.4 Design trade-offs . . . **116**
12.5 Common pitfalls . . . **116**
12.6 Check your understanding . . . **117**
12.7 Further reading . . . **117**

13 Interoperability and Standards . . . **119**
13.1 Why it matters . . . **119**
13.2 Concepts . . . **119**

13.2.1 What interoperability means . . . 119
13.2.2 From silos to horizontal platforms . . . 120
13.2.3 The application services layer . . . 120
13.2.4 Standards for the application services layer . . . 121
13.2.5 Matter: interoperability in the smart home . . . 122
13.3 In depth (optional) . . . **122**
13.3.1 Why common models scale, and what encodings cost . . . 123
13.4 Design trade-offs . . . **123**
13.5 Common pitfalls . . . **123**
13.6 Check your understanding . . . **124**
13.7 Further reading . . . **124**

IV Part IV — Trustworthy IoT

14 Security and Privacy . . . **127**
14.1 Why it matters . . . **127**
14.2 Concepts . . . **128**
14.2.1 Why IoT is hard to secure . . . 128
14.2.2 Security requirements . . . 128
14.2.3 Threat modeling with STRIDE . . . 129
14.2.4 A three-domain view of IoT security . . . 129
14.2.5 Cryptographic building blocks . . . 131
14.2.6 Privacy by design . . . 132
14.3 In depth (optional) . . . **133**
14.3.1 How strong is strong enough? . . . 133
14.4 Design trade-offs . . . **134**
14.5 Common pitfalls . . . **134**
14.6 Check your understanding . . . **134**
14.7 Further reading . . . **135**

15 Device Identity and Secure Elements . . . **137**
15.1 Why it matters . . . **137**
15.2 Concepts . . . **137**
15.2.1 Identifiers and credentials . . . 137
15.2.2 Shared secrets and public keys . . . 137
15.2.3 Certificates and PKI . . . 138
15.2.4 Protecting the private key . . . 138
15.2.5 Provisioning: giving a device its identity . . . 140
15.2.6 Identity beyond authentication . . . 140
15.3 In depth (optional) . . . **140**
15.3.1 Inside an ECDSA signature . . . 140
15.4 Design trade-offs . . . **141**
15.5 Common pitfalls . . . **141**
15.6 Check your understanding . . . **142**
15.7 Further reading . . . **142**

16 Over-the-Air Updates and the Device Lifecycle . . . **143**
16.1 Why it matters . . . **143**

16.2 Concepts . . . 143
16.2.1 The device lifecycle . . . 143
16.2.2 Over-the-air updates . . . 144
16.2.3 A/B partitions on the ESP32 . . . 145
16.2.4 Fetching updates . . . 145
16.2.5 Secure boot and the chain of trust . . . 146
16.2.6 eFuses . . . 146
16.2.7 Flash encryption . . . 147
16.2.8 Operating a fleet . . . 147
16.3 In depth (optional) . . . 147
16.3.1 The numbers behind a safe rollout . . . 148
16.4 Design trade-offs . . . 148
16.5 Common pitfalls . . . 148
16.6 Check your understanding . . . 149
16.7 Further reading . . . 149

17 Compliance and Regulation . . . 151
17.1 Why it matters . . . 151
17.2 Concepts . . . 151
17.2.1 Why connected products are regulated . . . 151
17.2.2 Radio and electromagnetic compatibility . . . 152
17.2.3 Safety and environmental rules . . . 152
17.2.4 Cybersecurity regulation . . . 152
17.2.5 Privacy and data protection . . . 153
17.2.6 Sector-specific rules . . . 153
17.2.7 From regulation to engineering tasks . . . 153
17.3 In depth (optional) . . . 154
17.3.1 Decibels, power limits and duty cycles . . . 154
17.4 Design trade-offs . . . 155
17.5 Common pitfalls . . . 155
17.6 Check your understanding . . . 156
17.7 Further reading . . . 156

V Part V — Intelligence at the Edge

18 AI on IoT and Edge ML . . . 159
18.1 Why it matters . . . 159
18.2 Concepts . . . 159
18.2.1 The limits of cloud-only processing . . . 159
18.2.2 The device–edge–cloud continuum . . . 159
18.2.3 Edge AI and TinyML . . . 160
18.2.4 A machine-learning refresher . . . 160
18.2.5 Features from sensor data . . . 161
18.2.6 Why model size matters . . . 161
18.2.7 The TinyML toolchain . . . 161
18.2.8 A tiny example: classifying motion with a decision tree . . . 162
18.2.9 Evaluating an edge model . . . 162
18.2.10 Larger models at the edge . . . 162
18.3 In depth (optional) . . . 163

18.3.1 Fitting a model into a microcontroller . . . 163
18.4 Design trade-offs . . . 164
18.5 Common pitfalls . . . 164
18.6 Check your understanding . . . 164
18.7 Further reading . . . 164

19 Where IoT Is Heading . . . 165
19.1 Why it matters . . . 165
19.2 Concepts . . . 165
19.2.1 Looking back: one connected picture . . . 165
19.2.2 Scalable deployment: CI/CD for firmware . . . 165
19.2.3 Directions to watch . . . 166
19.2.4 Evaluating emerging technologies: the case of blockchain . . . 166
19.3 In depth (optional) . . . 167
19.3.1 Can a device run on harvested energy? . . . 167
19.4 Design trade-offs . . . 168
19.5 Common pitfalls . . . 168
19.6 Check your understanding . . . 169
19.7 Further reading . . . 169

VI Part VI — Building an IoT System

20 From Idea to Vertical Slice . . . 173
20.1 Why it matters . . . 173
20.2 Concepts . . . 173
20.2.1 From idea to problem statement . . . 173
20.2.2 Requirements . . . 173
20.2.3 Architecture . . . 174
20.2.4 Risks . . . 174
20.2.5 The proposal . . . 174
20.2.6 The vertical slice . . . 174
20.2.7 Milestones . . . 175
20.2.8 Working as a team . . . 175
20.3 In depth (optional) . . . 176
20.3.1 Sizing and reliability of a whole system . . . 176
20.4 Design trade-offs . . . 177
20.5 Common pitfalls . . . 177
20.6 Check your understanding . . . 177
20.7 Further reading . . . 177

21 Testing in the Field . . . 179
21.1 Why it matters . . . 179
21.2 Concepts . . . 179
21.2.1 Levels of testing . . . 179
21.2.2 Designing for testability . . . 180
21.2.3 Fault injection . . . 180
21.2.4 Long-duration reliability testing . . . 181
21.2.5 Field testing . . . 181

21.2.6 Preparing the final demonstration . . . 181
21.3 In depth (optional) . . . 182
21.3.1 How much testing is enough? . . . 182
21.4 Design trade-offs . . . 183
21.5 Common pitfalls . . . 183
21.6 Check your understanding . . . 183
21.7 Further reading . . . 183

22 Reviewing Someone Else's System . . . 185
22.1 Why it matters . . . 185
22.2 Concepts . . . 185
22.2.1 Ground rules . . . 185
22.2.2 Preparing for a review . . . 185
22.2.3 The device review . . . 186
22.2.4 The security code review *(Advanced track)* . . . 186
22.2.5 Writing findings . . . 187
22.2.6 Responding to a review . . . 187
22.3 In depth (optional) . . . 188
22.3.1 How many defects are left? . . . 188
22.4 Design trade-offs . . . 188
22.5 Common pitfalls . . . 188
22.6 Check your understanding . . . 189
22.7 Further reading . . . 189

VII Part VII — Lab Companion

Lab 0: Bench Safety and Soldering . . . 193
Goals . . . 193
Safety first . . . 193
Background . . . 193
Tools and materials . . . 193
Procedure: soldering header pins . . . 194
Recognizing good and bad joints . . . 194
What to show . . . 194
Questions . . . 194

Lab 1: Build the Kit in Stages . . . 197
Goals . . . 197
The layout . . . 197
The stages . . . 198
Stage 1: basic I/O and the display (this lab) . . . 198
Stage 2: the I²C bus (before the serial-protocols activity) . . . 198
Stage 3: the servo rail (before the actuators activity) . . . 199
Stage 4: the LoRa radio (before the LoRa activity) . . . 199
Procedure . . . 199
What to show and submit . . . 199

Troubleshooting . . . 200
Questions . . . 200

Lab 2: Traffic Controller . . . 201

Goals . . . 201
The problem . . . 201
Design . . . 201
A starting structure . . . 202
Testing . . . 203
What to show and submit . . . 203
Analysis questions . . . 204

Lab 3: Serial Communication . . . 205

Goals . . . 205
Part 1: I²C, reading the BMP280 from its datasheet . . . 205
Part 2: SPI, showing data on the OLED . . . 206
Part 3: UART, talking to another device . . . 206
Stretch: hardware vs. software SPI . . . 206
What to show and submit . . . 206
Analysis questions . . . 206

Lab 4: BLE Step Counter . . . 207

Goals . . . 207
Part 1: detecting steps . . . 207
Part 2: publishing over BLE . . . 208
Part 3: evaluating accuracy . . . 208
What to show and submit . . . 208
Analysis questions . . . 208

Lab 5: Cloud Communication . . . 209

Goals . . . 209
What you need . . . 209
Part A: the cloud side . . . 209
A1. Create a broker . . . 209
A2. Node-RED and the dashboard . . . 210
Part B: credentials in flash, not in code . . . 210
Part C: TLS, Last Will, back-off and telemetry . . . 210
Part D: dashboard and alert . . . 211
Part E: project analysis . . . 211
What to show and submit . . . 211
Troubleshooting . . . 211
Stretch goal . . . 212
Analysis questions . . . 212

Lab 6: Harden Your Device . . . 213
Goals . . . 213
Part 1: threat model . . . 213
Part 2: verify the broker . . . 213
Part 3: get secrets out of the source code . . . 213
Part 4: authenticate commands . . . 214
Part 5: reduce the attack surface . . . 214
Part 6: eFuses, read-only . . . 214
What to show and submit . . . 214
Analysis questions . . . 214

VIII Part VIII — Lecture Activities

About the Activities . . . 217
How each activity works . . . 217
Helpers used by the starter projects . . . 217

Activity 1: Blink and Basic Circuits . . . 219
Goal . . . 219
Set up the simulator . . . 219
Build the circuit . . . 219
Starter code . . . 219
Hints . . . 220
Expected result . . . 220
Extension (optional) . . . 220
Troubleshooting . . . 220
Questions . . . 220

Activity 2: Buttons, Debouncing and Interrupts . . . 221
Goal . . . 221
Build the circuit . . . 221
Starter code . . . 221
Hints . . . 222
Expected result . . . 222
Extension (optional) . . . 222
Troubleshooting . . . 222
Questions . . . 222

Activity 3: Scan the I²C Bus, Report on SPI . . . 223
Goal . . . 223
What the starter does . . . 223
Expected result . . . 223
Extension (optional) . . . 224

Troubleshooting . . . 224
Questions . . . 224

Activity 4: The Observe–Decide–Act Loop . . . 225
Goal . . . 225
Before you start . . . 225
TODOs . . . 225
Expected result . . . 226
Extension (optional) . . . 226
Troubleshooting . . . 226
Questions . . . 226

Activity 5: A Classroom Poll over HTTP . . . 227
Goal . . . 227
Before you start . . . 227
Steps and TODOs . . . 227
Expected result . . . 228
Status codes you may see . . . 228
Extension (optional) . . . 228
Questions . . . 228

Activity 6: A LoRa Emergency Beacon . . . 229
Goal . . . 229
TODOs . . . 229
Expected result . . . 229
Class signal test . . . 230
Extension (optional) . . . 230
Troubleshooting . . . 230
Questions . . . 230

Activity 7: MQTT Command Center and Picking the QoS . . . 231
Goal . . . 231
Part A: the command center . . . 231
Part B: pick the QoS . . . 231
Extension (optional) . . . 232
Troubleshooting . . . 232
Questions . . . 232

Activity 8: Signed Messages and STRIDE . . . 233
Goal . . . 233
TODOs . . . 233
Running it . . . 233
Extension (optional) . . . 234
Questions . . . 234

Appendices . . . 235

A Kit Reference . . . 237
A.1 What the finished kit looks like . . . 238
A.2 Bill of materials . . . 239
A.3 The printed holder . . . 241
A.4 Wiring by stage . . . 242
A.5 Pin map . . . 245
A.6 `iot_pins.h` . . . 245
A.7 Power . . . 246
A.8 Kit care . . . 246

B Toolchain Setup . . . 247
B.1 What you need . . . 247
B.2 Installing PlatformIO . . . 247
B.3 USB drivers . . . 247
B.4 Opening a project . . . 247
B.5 Build, upload and monitor . . . 248
B.6 Secrets . . . 248
B.7 Wokwi . . . 248
B.8 Networks . . . 248

C Troubleshooting . . . 249
C.1 Power and upload . . . 249
C.2 Serial Monitor . . . 249
C.3 GPIO, buttons and analog input . . . 249
C.4 I²C . . . 250
C.5 SPI and the OLED . . . 250
C.6 LoRa . . . 250
C.7 Wi-Fi and MQTT . . . 251
C.8 When you are stuck . . . 251

D Glossary . . . 253

E Setting Up a Teaching Lab . . . 257
E.1 The room . . . 257
E.2 Shared equipment (per 10 soldering stations) . . . 258
E.3 Before the first lab . . . 258
E.4 Running costs . . . 259

References . . . 261

Bibliography . . . 263

Preface

This book teaches the Internet of Things the way IoT systems are actually built: one wire, one bus, one protocol layer at a time, on hardware you assemble yourself.

It was written for a ten-week, hands-on course in IoT software and systems. Every concept is paired with something you can run on a small ESP32 kit, so that the ideas in each chapter turn into working circuits, packets and devices before the week is over.

Who this book is for

The book assumes you can program in C or C++ and have met basic computer organization and networking ideas. It does not assume any electronics background. Chapters that need circuit knowledge build it as they go, and the Lab Companion walks through soldering and wiring from first principles.

How the book is organized

The book has two layers.

- **The core text** (Parts I–VI) covers ideas that change slowly: how devices talk to sensors, how IoT networks are layered, how to make devices trustworthy, and how to build and test a complete system.
- **The Lab Companion** (Part VII) covers the specific kit, pin assignments, starter code and lab procedures. It is updated every term as hardware and libraries change.

Each core chapter follows the same pattern: learning objectives, a motivating example, the concepts, a short exercise on your kit, design trade-offs, common pitfalls, review questions, and further reading.

Two tracks

Every chapter has a core path that all readers should follow. Some chapters also contain **Advanced track** sections, marked with a distinct callout. These go deeper into standards, design reviews and engineering practice. They are optional on a first reading.

Conventions

- Code that runs on the kit is written for the Arduino framework on the ESP32, built with PlatformIO.
- Course helper functions use the `iot` prefix (for example `iotStartSerial()`), and pin assignments live in `iot_pins.h` under the `IoTPins::` namespace.
- I²C addresses are written as 7-bit hexadecimal values, such as `0x68`.

License and contributions

The text and figures are licensed under CC BY 4.0. Code is licensed under the MIT License. Corrections and suggestions are welcome as issues or pull requests in the book's repository.

Sources and originality

The text, figures and code in this book were written for it. Where the book follows an existing framework, taxonomy or teaching sequence, it says so in the text and cites the source. In particular:

- The overall sequence of Parts I–IV, and several frameworks used in them (the IoT reference framework, the classification of sensors and actuators, the Observe–Decide–Act loop, IoT networking requirements, the layered IoT protocol stack and the three-domain view of IoT security), follow Rayes and Salam's *Internet of Things from Hype to Reality* [1] and its security chapter by Dabbagh and Rayes [2]. These are used as frameworks with attribution; no text or figures are reproduced from them.

- Facts, figures and standards are cited to their primary sources: datasheets, RFCs, specifications and official regulatory texts.
- Code examples are original, or minimal uses of public library APIs written in the style of their documentation.
- Product names and trademarks belong to their owners and are used only to identify products.

If you believe any passage is insufficiently attributed, please open an issue in the book's repository; it will be corrected promptly.

Acknowledgments

Several of the labs grew out of earlier lab handouts written with **Biswadip Maity**. The Fall 2026 offering of the course, on which the Lab Companion and the lecture activities are based, was supported by teaching assistants **Hamidreza Alikhani Koshkak**, **Nitish Nagesh**, **Jay Chen** and **Leming**, and earlier activity materials were developed with **Manuel Segura**. The printed kit holder is the Breadboard Snap-On Component Tray designed by **Alvin Ma**, used with his permission. Thanks also to the students of the course, whose questions and feedback shaped every chapter.

Using This Book with the IoT@UCI Kit

Most chapters include a **Try it on your kit** box. These exercises assume the IoT@UCI team kit: an ESP32 DevKit V1 on two 830-point breadboards, with a set of sensors, actuators, a display, a secure element and a LoRa radio.

What is in the kit

Part	Role	Bus	Used from
ESP32 DevKit V1	Microcontroller with Wi-Fi and Bluetooth	—	Chapter 2.
LEDs, push buttons, potentiometer, buzzer	Basic I/O	GPIO / ADC / PWM	Chapter 3.
128×64 OLED display (SSD1306)	Text and graphics output	SPI	Chapter 5.
BMP280	Temperature and pressure sensor	I²C	Chapter 5.
MPU-6050	Accelerometer and gyroscope	I²C	Chapter 5.
ATECC608A	Secure element (device identity, signatures)	I²C	Chapter 5., Chapter 15.
Servo motor	Actuator on its own 5 V rail	PWM	Chapter 7.
RFM95W (SX1276)	LoRa radio	SPI	Chapter 10.

The kit is built in stages, so that each part is wired just before the chapter that first needs it. The full build is described in Lab 1, and the complete pin map is in Appendix A..

How each chapter is organized

Every chapter follows the same pattern, so you can choose how deep to go:

Element	What it is	Required?
Concepts	The core ideas, with diagrams	Yes
Try it on your kit	Short exercises on the kit	Recommended
In depth (optional)	The math, physics and technical detail behind the chapter: formulas, worked examples and an exercise	No: clearly marked and skippable
Advanced track	Extra tasks for graduate students and those who want more	No
Check your understanding	Review questions	Recommended

Part VII (the Lab Companion) and Part VIII (the Lecture Activities) turn the chapters into step-by-step handouts.

Safety

Always unplug the USB cable before changing any wiring. Never connect the servo's 5 V rail to the 3.3 V rail. Read Lab 0 before you solder.

I

Part I — Foundations

1 What Is the Internet of Things? 23
1.1 Why it matters 23
1.2 Concepts 23
1.3 In depth (optional) 26
1.4 Design trade-offs 27
1.5 Common pitfalls 27
1.6 Check your understanding 27
1.7 Further reading 28

2 Development Boards and the IoT Kit . 29
2.1 Why it matters 29
2.2 Concepts 29
2.3 In depth (optional) 33
2.4 Design trade-offs 33
2.5 Common pitfalls 34
2.6 Check your understanding 34
2.7 Further reading 34

3 Programming the Device 35
3.1 Why it matters 35
3.2 Concepts 35
3.3 In depth (optional) 39
3.4 Design trade-offs 40
3.5 Common pitfalls 40
3.6 Check your understanding 41
3.7 Further reading 41

4 Embedded Systems and Operating Systems 43
4.1 Why it matters 43
4.2 Concepts 43
4.3 In depth (optional) 46
4.4 Design trade-offs 47
4.5 Common pitfalls 47
4.6 Check your understanding 47
4.7 Further reading 48

1. What Is the Internet of Things?

Learning objectives

After this chapter you should be able to:

1. Define the Internet of Things and name the four capabilities every IoT system combines.
2. Explain how IoT differs from the "Internet of People" that came before it.
3. Describe a four-level reference architecture and place real components in it.
4. Recognize the major application domains and the engineering questions they raise.
5. List the cross-cutting concerns (security, interoperability, reliability, efficiency) that the rest of this book addresses.

1.1 Why it matters

In 2024 there were about 18.5 billion connected IoT devices in the world, more than two for every person. Analysts expected roughly 21 billion in 2025 and close to 40 billion by 2030 [3]. Most of them are not phones or laptops. They are thermostats, parking sensors, infusion pumps, irrigation valves, shipping-container trackers and factory motors.

Each of these devices is a small computer that senses something in the physical world, decides what it means, and either acts on it or reports it to someone who will. Building such devices well requires knowledge that usually lives in separate courses: electronics, embedded programming, networking, distributed systems and security. This book brings them together around one question: **how do you build a small device that reliably senses, decides, communicates and acts?**

1.2 Concepts

1.2.1 A working definition

There is no single official definition of the Internet of Things, but most definitions agree on four ingredients. An IoT system is a network of physical things that have:

- **Sensors**, to collect information about the world;
- **Identifiers**, so that every piece of data can be traced to the device that produced it;
- **Software**, to process and interpret the data;
- **Connectivity**, so that data and commands can travel between devices, people and services.

Putting these together, in a formulation close to that of Rayes and Salam [1]:

> The **Internet of Things** is the network of physical objects that are embedded with sensors, software and network connectivity, have a unique identity, and can collect and exchange data and act on the physical world.

Notice the last clause. Many IoT devices do not just observe; they also **actuate**: they open valves, lock doors, dim lights and move motors. A device that can change the physical world raises the stakes of every design decision, from how fast it responds to who is allowed to command it.

1.2.2 Related terms

You will meet several overlapping terms in the literature:

Term	Emphasis
M2M (machine-to-machine)	Devices talking directly to each other, often over cellular links, without a person in the loop. The older industrial term.
Wireless sensor networks	Large numbers of small, battery-powered sensing nodes; mostly research-driven.
Industrial IoT (IIoT)	Factories, energy and process control, where reliability and safety dominate.
Web of Things	Using web standards (HTTP, URLs, JSON) to expose devices as resources.
Internet of Everything	A marketing term that adds people, processes and data to the picture.
Cyber-physical systems	The academic term for tightly coupled computation and physical processes.

These are not competing technologies. They are different lenses on the same idea.

1.2.3 From the Internet of People to the Internet of Things

The internet has gone through several phases, each of which digitized something new [1], [4]:

1. **Connectivity** digitized *access to information*: email, the web, search.
2. **The networked economy** digitized *business*: e-commerce, online banking, supply chains.
3. **Immersive experiences** digitized *interaction*: social media, video, mobile apps.
4. **The Internet of Things** digitizes *the physical world itself.*

The first three phases mostly connected people to applications. Almost every packet started or ended at a person looking at a screen. In the IoT, most traffic is between machines, and a person may never see the raw data at all. This changes the design priorities. A web page can take a second to load; a braking system cannot. A phone is recharged daily; a soil sensor may need to run for five years on one battery.

Connecting objects is not a goal in itself. The value comes from turning the data into **better decisions** (for example, scheduling maintenance before a pump fails) and delivering the right information to the right person or machine **at the right time**.

1.2.4 Three kinds of connection

It helps to classify IoT interactions by who is talking:

- **People to things.** A person monitors or controls a device: checking a home camera, adjusting a thermostat, reading an ECG from a wearable.
- **Things to people.** A device notifies a person: a leak alert, a low-battery warning, a door left open.
- **Things to things.** Devices coordinate without a person: a motion sensor turns on a light, a traffic signal adapts to a queue of cars, a fleet of warehouse robots avoids collisions.

The third category grows fastest and is the hardest to get right, because no person is in the loop to notice when something goes wrong.

1.2.5 A reference architecture

Real IoT systems are built from many products made by different vendors. A **reference architecture** gives you a common vocabulary for where each part belongs. A widely used model, adapted here from Rayes and Salam [1], has four levels (Figure 1.1):

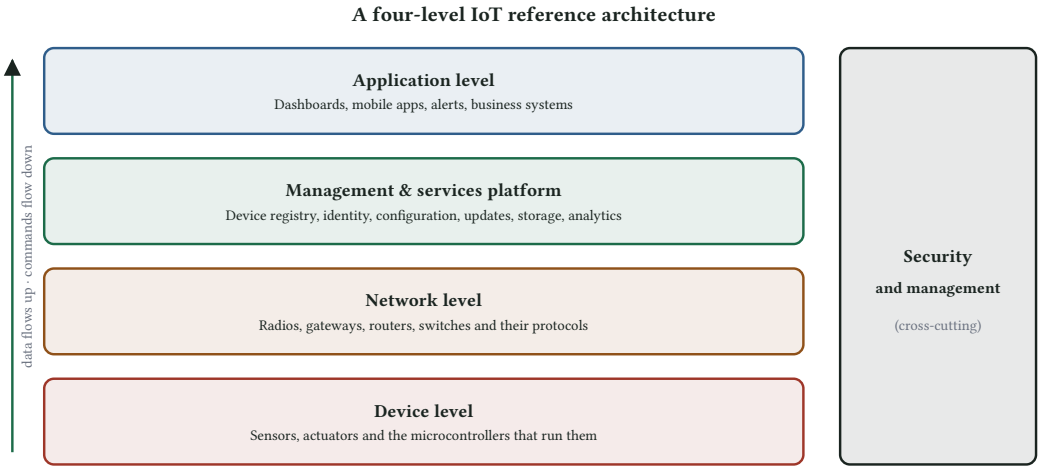


Figure 1.1: A four-level IoT reference architecture. Data generally flows upward; commands flow downward.

1. **Device level.** The *things*: sensors, actuators and the microcontrollers that run them. Your kit lives here.
2. **Network level.** Everything that moves data: radios, gateways, routers and switches, and the protocols they use. A gateway often translates between a local radio (Bluetooth, LoRa, Zigbee) and the internet.
3. **Management and services platform level.** Software that manages fleets of devices and their data: device registration and identity, configuration, firmware updates, data storage, rules engines and analytics. Cloud IoT platforms and MQTT brokers live here.
4. **Application level.** The applications people and businesses actually use: dashboards, mobile apps, alerting, and integration with business systems.

Two concerns cut across every level: **security** (who may read, send or change what) and **management** (how you find, configure, update and retire devices once they are deployed). Much of this book is organized around this picture. Parts I and II focus on the device level, Part III on the network level, Part IV on the cross-cutting concerns, and Parts V and VI on how the levels come together into a working system.

1.2.6 Application domains

IoT appears in almost every industry. A few domains recur throughout the book:

- **Smart homes and buildings**: lighting, heating and cooling, locks, appliances and energy metering. Interoperability between brands is the central challenge (Chapter 13.).
- **Smart cities**: parking, street lighting, waste collection, air-quality monitoring and traffic management. Devices are spread over kilometers and must run for years, which favors long-range, low-power radios (Chapter 10.).
- **Healthcare and wellness**: wearables, remote patient monitoring and connected medical devices. Privacy, safety and regulation dominate.
- **Transportation and logistics**: vehicle telematics, fleet tracking and cold-chain monitoring of goods in transit.
- **Industry and agriculture**: predictive maintenance, process monitoring, irrigation control and livestock tracking.

Each domain weighs the same trade-offs differently. A smart-home light switch must respond in well under a second but can draw mains power. A soil-moisture sensor can report once an hour but must run on a battery for years.

1.2.7 The questions every IoT system must answer

The basic promise of IoT is to **monitor and control things from anywhere**. Delivering that promise raises a set of questions that this book takes up one at a time:

Question	Where it is covered
How does a device sense and act on the world?	Parts I and II
How does it reach the network, and the network reach it?	Part III
Who is allowed to monitor and control it, and how is that enforced?	Part IV
How do devices from different makers work together?	Chapter 13.
How do we make it reliable when it is unattended for years?	Chapter 16., Chapter 21.
How do we make it efficient in energy, bandwidth and cost?	Throughout
Where should the "thinking" happen: device, edge or cloud?	Chapter 18.

Try it on your kit: place the kit in the architecture

Look at the kit tour in *Using This Book with the IoT@UCI Kit.*

1. For each part in the kit, decide which level of Figure 1.1 it belongs to.
2. The kit has no gateway. Which kit component will eventually play that role for the LoRa radio, and what will it connect to?
3. Name one application-level product you use daily that sits on top of a device-level thing. What are its sensors, identifiers, software and connectivity?

1.3 In depth (optional)

Note

Optional, in-depth explanation. This section goes beyond the rest of the chapter into the math and physics behind it. Nothing later in the book depends on it; skip it on a first reading.

1.3.1 Back-of-the-envelope engineering

Most early IoT design questions are answered with two quick estimates: **how long will the battery last**, and **how much data will the fleet produce**. Both are simple averages, and both are worth doing before writing any code.

Average current. A device that is active for t_a seconds in every period T has a duty cycle $D = t_a/T$. Its average current is

$$I_{\text{avg}} = D\, I_{\text{active}} + (1 - D)\, I_{\text{sleep}}. \tag{1.1}$$

For a sensor that wakes for 0.5 s every 10 minutes, draws 80 mA while awake (Wi-Fi on) and 10 µA asleep: $D = 0.5/600 \approx 8.3 \times 10^{-4}$, so $I_{\text{avg}} \approx 67\ \mu\text{A} + 10\ \mu\text{A} \approx 77\ \mu\text{A}$. Notice that the 10 µA sleep current is already 13% of the total: at low duty cycles, **sleep current dominates**.

Lifetime. A battery of capacity Q (in mAh) lasts roughly Q/I_{avg} hours. A 225 mAh coin cell would give $225/0.077 \approx 2900$ h, about four months, on paper.

The physics the average hides. A battery is not an ideal voltage source; it has an internal resistance R_{int}. The terminal voltage under load is $V = V_{\text{oc}} - I R_{\text{int}}$. A coin cell's internal resistance is typically 10–30 Ω and rises as it discharges, so an 80 mA pulse drops $0.08 \times 15 = 1.2$ V: the 3 V cell sags to 1.8 V and the microcontroller resets. The average said four months; the peak says the design does not work at all.

This is why Wi-Fi devices use AA or lithium-ion cells, and why coin-cell devices use BLE or LoRa, whose transmit current is far lower, often with a capacitor to supply the pulse.

Fleet data volume. If N devices each send b bytes every T seconds, the fleet produces Nb/T bytes per second. One million smart meters sending 100 bytes every 15 minutes produce $10^6 \times 100/900 \approx 111$ kB/s on average, or about 9.6 GB per day before protocol overhead (Chapter 8. shows the overhead can be many times the payload). The rate is modest; the challenge is the *number of connections* and the *bursts* when every meter reports at the same moment.

Exercise. Repeat the lifetime estimate for a device that reports every minute instead of every 10 minutes. Which term now dominates the average?

Sources for this section: [5].

1.4 Design trade-offs

Every IoT design balances a small set of competing goals. It is worth naming them now, because they will reappear in every chapter:

Goal	Typically in tension with
Long battery life	Frequent sensing, fast response, high data rates
Long radio range	Data rate and energy per bit
Low unit cost	Security hardware, extra sensors, certification
Rich local processing	Power, cost and complexity of the device
Strong security	Usability, cost and development time
Openness and interoperability	Speed to market and vendor control

There is no universally right answer. A good IoT engineer can state which goals matter most for a given product and justify the trade-offs that follow.

1.5 Common pitfalls

- **Connecting things because you can.** A device should be connected because the data or the remote control creates value, not because a radio is cheap.
- **Designing for the demo, not the deployment.** A prototype on a desk has unlimited power, perfect Wi-Fi and a developer watching it. A deployed device has none of these.
- **Treating security as a feature to add later.** Retrofitting identity, encryption and updates into deployed devices is expensive and sometimes impossible.
- **Ignoring the people involved.** Someone installs, maintains and eventually discards every device. Their time and skills are part of the system.

> **i** Advanced track
>
> **Reference architectures in standards.** The four-level model above is a simplified version of several formal architectures, including the ITU-T Y.2060 IoT reference model, the IEEE 2413 architectural framework and ISO/IEC 30141. They differ in naming and in how they divide responsibilities, but they all separate devices, networks, services and applications, and they all treat security and management as cross-cutting.
>
> **Exercise.** Choose a commercial IoT product (for example a video doorbell, a connected car or a smart meter). Draw it on the four-level architecture, naming the actual products, protocols and companies at each level. Identify which party controls each level, and what happens to the product if that party goes out of business.

1.6 Check your understanding

1. Name the four ingredients that most definitions of the IoT share, and give an example of each from a smart thermostat.

2. Why is “connecting objects together” not, by itself, the goal of an IoT system?
3. Explain one way in which IoT traffic differs from traffic in the “Internet of People”.
4. At which level of the reference architecture would you place: a LoRa gateway; a firmware-update service; a temperature sensor; a mobile app showing a room’s temperature?
5. Give one domain where battery life is the dominant constraint, and one where response time is.
6. Why are security and management drawn as cross-cutting concerns rather than as a fifth level?

1.7 Further reading

- IoT Analytics, *State of IoT 2025* [3], for current device counts and connectivity trends.
- A. Rayes and S. Salam, *Internet of Things from Hype to Reality* [1], whose framing of IoT definitions and reference architecture this chapter follows.
- ITU-T Recommendation Y.2060, *Overview of the Internet of Things* [6], the source of many reference-architecture terms.

2. Development Boards and the IoT Kit

Learning objectives

After this chapter you should be able to:

1. Use voltage, current and Ohm's law to choose a current-limiting resistor for an LED.
2. Build simple circuits on a solderless breadboard and explain how its strips are connected.
3. Explain what a microcontroller is and why it replaced fixed-function circuits in most products.
4. Identify the parts of a development board and what each one does.
5. Describe the ESP32 and the DevKit V1 board used in the kit, including the pins you must avoid.

2.1 Why it matters

Every IoT device starts as a circuit on a bench. Before you can write code that senses or acts, you need to know how electricity reaches a component, how much of it the component can take, and how a microcontroller board fits into the circuit. Most "my code doesn't work" problems in a first IoT course turn out to be circuit problems: an LED wired backwards, a missing ground, a component on the wrong breadboard row.

This chapter builds the smallest useful circuit, makes it blink, and then explains the development board that will run all the code in the rest of the book.

2.2 Concepts

2.2.1 Voltage, current and resistance

Three quantities describe what happens in a simple circuit:

- **Voltage** (V, measured in volts) is the electrical "pressure" between two points. It is what pushes charge around a circuit. A USB port supplies 5 V; the ESP32's pins work at 3.3 V.
- **Current** (I, measured in amperes or milliamperes, mA) is the rate at which charge flows. Components are damaged by too much current, not by voltage alone.
- **Resistance** (R, measured in ohms, Ω) opposes current.

They are related by **Ohm's law**:

$$V = I \times R \tag{2.1}$$

Current only flows around a **closed loop**: from the supply, through the components, and back to **ground** (GND), the reference point that all voltages are measured from. If any part of the loop is open, nothing happens.

2.2.2 Lighting an LED safely

A light-emitting diode (LED) conducts in one direction only. Its longer leg, the **anode**, must face the positive supply; the shorter leg, the **cathode**, faces ground. Once it conducts, an LED holds a roughly fixed voltage across itself, called its **forward voltage** (V_f): about 2.0 V for a red LED, and around 3 V for blue or white ones.

An LED has almost no resistance of its own once it conducts. Connect it straight across a supply and the current rises until something burns out. A **current-limiting resistor** in series sets the current:

$$R = \frac{V_{supply} - V_f}{I_{LED}} \tag{2.2}$$

For a red LED driven from a 3.3 V ESP32 pin, aiming for 10 mA:

$$R = \frac{3.3\ \text{V} - 2.0\ \text{V}}{0.010\ \text{A}} = 130\ \Omega \tag{2.3}$$

Resistors come in standard values, and it is always safe to round **up**. The kit uses 220 Ω, which gives about 6 mA: plenty for an indicator LED, and gentle on the microcontroller pin.

2.2.3 Breadboards

A **solderless breadboard** lets you build and change circuits without soldering. Inside, metal clips connect groups of holes:

- In the main area, each **row of five holes** is connected. The rows on either side of the central gap are *not* connected to each other. Chips and modules straddle this gap so that each pin gets its own row.
- The long **power rails** along the edges run the length of the board. By convention, the red rail carries the supply voltage and the blue rail carries ground. On some boards the rails are split in the middle, so check before relying on them.

The kit uses two 830-point breadboards side by side, with the ESP32 bridging the seam. The rails carry 3.3 V and GND, except for one short rail section reserved for the servo's 5 V supply (Lab 1).

2.2.4 From fixed circuits to microcontrollers

Suppose you want the LED to blink once per second. One classic solution is a **555 timer**: a chip whose output switches on and off at a rate set by a resistor and a capacitor. It works, but changing the behavior means changing components. Making it blink twice and pause, or blink faster when a button is pressed, means designing a new circuit.

A **microcontroller** replaces the fixed circuit with software. It is a small computer on a single chip, with a processor, memory and input/output pins. The same chip can blink an LED, read a sensor or run a network stack, depending on the program you load:

```
void loop() {
  digitalWrite(LED_PIN, HIGH);
  delay(1000);
  digitalWrite(LED_PIN, LOW);
  delay(1000);
}
```

Microcontrollers are cheap (often well under a dollar for simple ones), so it is usually cheaper to use one than to design a custom circuit. This is a large part of why connected devices became affordable.

2.2.5 What a development board adds

A bare microcontroller chip is hard to use on a bench: its pins are tiny, it needs a stable power supply, and loading a program onto it requires special hardware. A **development board** wraps the chip with everything needed to experiment:

- **A power circuit**: a voltage regulator that turns 5 V from USB into the 3.3 V the chip needs.
- **A programming interface**: usually a USB-to-UART bridge chip (Chapter 5.), so a laptop can upload programs and exchange text with the board.
- **Basic I/O**: at least one LED and a reset button, often a second "boot" button.
- **Header pins**: the chip's I/O pins brought out at breadboard-friendly spacing.

2.2.6 Arduino and the ESP family

The **Arduino** project made microcontroller development accessible to people with no electronics background. It combined an inexpensive open-hardware board, the Arduino Uno based on an 8-bit AVR microcontroller, with a simple programming environment and a large library ecosystem. Its programming model (a `setup()` function and a `loop()` function, Chapter 3.) is still the most common entry point to embedded programming.

The original Uno has no network connection. Adding Wi-Fi meant stacking an expansion board, called a **shield**, which added cost and size. The **ESP8266** and later the **ESP32**, both from Espressif Systems,

changed this by putting a capable processor and a Wi-Fi radio on the same inexpensive chip. A chip that combines a microcontroller with radios and other advanced peripherals is often called a **system on a chip** (SoC).

Table 2.1: The ESP32 compared with the classic Arduino Uno [7].

	Arduino Uno (ATmega328P)	ESP32 (ESP32-WROOM-32 module)
CPU	8-bit AVR, 16 MHz	32-bit dual-core Xtensa LX6, up to 240 MHz
RAM	2 KB	520 KB
Program flash	32 KB	4 MB (external, on the module)
Logic level	5 V	3.3 V
Radios	None	Wi-Fi 802.11 b/g/n, Bluetooth Classic and BLE
ADC	10-bit, 6 inputs	12-bit, up to 18 inputs
Serial buses	1 UART, 1 SPI, 1 I²C	3 UART, 2 SPI, 2 I²C (freely remappable)

The ESP32 can still be programmed with the Arduino framework, so the simple programming model carries over unchanged.

2.2.7 The ESP32 DevKit V1

The kit uses the **ESP32 DevKit V1**, a widely cloned board built around the ESP32-WROOM-32 module. Its parts are:

- The **ESP32-WROOM-32 module** (the metal can), which contains the ESP32 chip, its flash memory, a crystal and a printed antenna. Keep the antenna end clear of metal and wires.
- A **USB-to-UART bridge** (typically a CP2102 or CH340 chip) for uploading and for the Serial Monitor. If your laptop does not recognize the board, the bridge's driver is the usual culprit.
- A **3.3 V regulator** fed from USB or from the VIN pin.
- Two buttons: **EN** resets the chip; **BOOT** (GPIO 0) puts it into upload mode if automatic upload fails.
- A power LED and, on most versions, a blue LED on GPIO 2.

Not every pin is equally safe to use. These rules prevent most beginner problems:

Table 2.2: ESP32 DevKit V1 pin restrictions [7].

Pins	Restriction
GPIO 6–11	Connected to the module's flash memory. **Never use.**
GPIO 34, 35, 36, 39	**Input only**, with no internal pull-up or pull-down resistors. Good for analog input (the kit's potentiometer uses GPIO 34).
GPIO 0, 2, 5, 12, 15	**Strapping pins**, read at reset to choose the boot mode. Usable, but an external component holding them high or low can stop the board from booting or uploading.
GPIO 1, 3	UART0 TX and RX, used for uploading and the Serial Monitor.
ADC2 pins	Cannot be used for analog input while Wi-Fi is active. Use ADC1 pins (GPIO 32–39) for analog sensors.

Each pin can safely source or sink only a few tens of milliamperes. Drive LEDs through resistors, and never power motors directly from a GPIO pin.

2.2.8 The IoT@UCI kit

The kit gives each team one ESP32 and a set of parts that together exercise every topic in this book. It is built in stages, so that each part is wired just before the chapter that first needs it:

Stage	Parts	First used in
1	ESP32, power rails, three LEDs, two buttons, potentiometer, buzzer, OLED display	Chapter 3., Chapter 5.
2	I²C sensors: BMP280, MPU-6050; ATECC608A secure element	Chapter 5., Chapter 6.
3	Servo motor on its own 5 V rail, with a 470 µF capacitor	Chapter 7.
4	RFM95W LoRa radio	Chapter 10.

The complete pin map is in Appendix A., and the build procedure is in Lab 1.

Try it on your kit: blink and basic circuits

This exercise corresponds to the first lecture activity and can be done in the Wokwi simulator before you have hardware.

1. Calculate the resistor needed to run a red LED at 8 mA from 3.3 V. Which standard value would you choose?
2. Build an LED and resistor circuit on GPIO 2 and run the blink program above.
3. Reverse the LED. What happens, and why does nothing break?
4. Move the LED to GPIO 34 and try again. Explain the result using Table 2.2.
5. Change the program so the LED blinks "SOS" in Morse code. What did changing the behavior cost you, compared with a 555 timer circuit?

A guided, step-by-step version for a lecture session is Activity 1.

2.3 In depth (optional)

Note

Optional, in-depth explanation. This section goes beyond the rest of the chapter into the math and physics behind it. Nothing later in the book depends on it; skip it on a first reading.

2.3.1 Currents, resistors and heat on the kit

Sizing an LED resistor. A lit LED holds a roughly constant forward voltage V_f across itself. The resistor takes the rest of the supply, so by Ohm's law

$$I = \frac{V_{\text{GPIO}} - V_f}{R}. \tag{2.4}$$

With a 3.3 V pin, a red LED ($V_f \approx 2.0$ V) and 220 Ω, $I = 1.3/220 \approx 5.9$ mA: bright enough to see, well within a pin's limits. A green or blue LED (V_f from about 2.2 to 3.0 V) draws less, and appears dimmer, with the same resistor. The resistor dissipates $P = I^2 R \approx 7.7$ mW, far below its ¼ W rating.

Without the resistor, the only thing limiting current is the pin driver's own output resistance (tens of ohms) and the LED's small dynamic resistance: the current can exceed the pin's rating, and the pin or the LED can be damaged. The simulator in Activity 1 does not model this.

Pin limits add up. The ESP32 lets you set each pin's drive strength; at the default setting a pin sources roughly 20 mA, and the absolute maximum is around 40 mA. What matters as much is the *total* drawn through the chip's power pins. Use pins to signal, not to power things: anything over a few milliamperes deserves a transistor.

Pull-up resistors. With `INPUT_PULLUP`, an internal resistor of roughly 45 kΩ connects the pin to 3.3 V. When the button closes, the current is $3.3/45\,000 \approx 73$ μA: negligible for an LED-sized budget, but not for a coin-cell device that holds a button down for hours.

The regulator gets warm. The DevKit's linear regulator drops USB's 5 V to 3.3 V and turns the difference into heat:

$$P_{\text{reg}} = (V_{\text{in}} - V_{\text{out}})\, I_{\text{load}}. \tag{2.5}$$

During Wi-Fi bursts of 250 mA, $P = 1.7 \times 0.25 \approx 0.43$ W. A small regulator package rises in temperature by its thermal resistance θ_{JA} times that power; at a typical 90 °C/W, about 38 °C above ambient. It works, but leaves little margin, which is one reason not to hang extra loads on 3V3.

Exercise. You want an LED current of 10 mA from a 3.3 V pin with a green LED of $V_f = 2.2$ V. Which standard resistor value would you choose, and what current do you actually get?

Sources for this section: [7].

2.4 Design trade-offs

Choice	Consider
Development board vs. custom board	Development boards are ideal for prototypes and small runs. Products use a custom circuit board with only the parts they need, which lowers cost, size and power.
Module vs. bare chip	A pre-certified radio module (like the WROOM-32) avoids designing and certifying an antenna circuit yourself.
Arduino framework vs. vendor SDK (ESP-IDF)	Arduino is faster to learn and has many libraries. The vendor SDK gives full control of power modes, the RTOS and security features.

Choice	Consider
5 V vs. 3.3 V parts	Mixing them needs level shifting. Prefer 3.3 V parts with a 3.3 V microcontroller.

2.5 Common pitfalls

- **LED backwards.** It simply stays dark. Flip it.
- **No resistor.** The LED may glow briefly and fail, and the GPIO pin can be damaged.
- **Component in the wrong row.** Both legs of a resistor in the same five-hole row connect it to nothing.
- **Split power rails.** Half the board has no power.
- **Charge-only USB cables.** The board powers up but the computer never sees it. Use a data cable.
- **Strapping pin held at the wrong level.** The board refuses to upload or boot. Disconnect the component on that pin and try again.

> **i** Advanced track
>
> **Reading a datasheet.** Find the ESP32-WROOM-32 datasheet and locate: the absolute maximum voltage on any I/O pin; the recommended operating voltage range; the current drawn during Wi-Fi transmission; and the current in deep sleep. Compare the transmit and deep-sleep figures. By what factor do they differ, and what does that imply for a battery-powered design?
>
> **Beyond the ESP32.** Newer Espressif chips use RISC-V cores (ESP32-C3, C6, H2) and add radios such as 802.15.4 for Thread and Zigbee. Other families popular in IoT include Nordic's nRF52 and nRF54 (Bluetooth Low Energy), Silicon Labs' EFR32 (multi-protocol) and the Raspberry Pi RP2040/RP2350 (no radio, but very low cost). Compare one of them with the ESP32 on power, radios and security features.

2.6 Check your understanding

1. A green LED has a forward voltage of 2.2 V. What resistor would you use to run it at 5 mA from a 3.3 V pin?
2. Why does a breadboard have a gap down the middle?
3. What advantage does a microcontroller have over a 555 timer for blinking an LED? What does it cost?
4. Name the four main parts a development board adds to a microcontroller chip.
5. You connect a potentiometer to GPIO 4 and readings are fine until you enable Wi-Fi. What happened?
6. Why should you never use GPIO 6–11 on the DevKit V1?

2.7 Further reading

- Espressif, *ESP32 Series Datasheet* [7], for pin functions, electrical limits and power figures.
- M. Margolis, *Arduino Cookbook* [8], for many small, tested circuit and code recipes.

3. Programming the Device

Learning objectives

After this chapter you should be able to:

1. Set up a PlatformIO project for the ESP32 using the Arduino framework.
2. Read and write digital pins, and use pull-up and pull-down resistors correctly.
3. Debounce a push button and explain why it is necessary.
4. Choose between polling and interrupts, and write a safe interrupt service routine.
5. Read analog voltages with the ADC, and produce analog-like outputs with the DAC and PWM.
6. Write non-blocking code with `millis()` so that several things can happen "at once".

3.1 Why it matters

A push button looks like the simplest input imaginable: pressed or not pressed. Yet a naïve button handler on a microcontroller will often count one press as three, miss presses entirely, or freeze the rest of the program while it waits. Getting the basics of input, output and timing right is what separates a demo that works once from a device that works every time.

This chapter covers the programming model you will use for the rest of the book, and the handful of techniques that make simple I/O reliable.

3.2 Concepts

3.2.1 The toolchain

Programming a microcontroller involves a **cross-compiler** (it runs on your laptop but produces code for the ESP32), libraries for the chip's hardware, and an uploader that writes the compiled program into the board's flash memory.

The two most common environments are the **Arduino IDE** and **PlatformIO**. This book uses PlatformIO, an extension for VS Code, because it:

- supports several *frameworks* for the same chip (Arduino and Espressif's own ESP-IDF);
- manages library dependencies per project, with version numbers, in a single `platformio.ini` file;
- integrates building, uploading, the Serial Monitor and debugging into one editor.

A minimal kit project looks like this:

```
; platformio.ini
[env:esp32dev]
platform = espressif32@^6.9.0      ; pins the Arduino-ESP32 core (2.x series)
board = esp32dev
framework = arduino
monitor_speed = 115200
lib_deps =
    adafruit/Adafruit SSD1306@^2.5.13
```

Pinning the platform version matters: different versions of the Arduino-ESP32 core have different APIs for some peripherals, and an unpinned project can stop compiling when a new version is released. Setup instructions are in Appendix B..

3.2.2 The Arduino programming model

An Arduino program (a **sketch**) defines two functions:

- `setup()` runs once after power-up or reset. Configure pins, start serial ports and initialize devices here.
- `loop()` runs repeatedly, forever, until the board loses power.

```
#include <Arduino.h>
#include "iot_pins.h"                // IoTPins::LED_1 (red), LED_2 (yellow), LED_3
(green), BUTTON_1, ...

void setup() {
  pinMode(IoTPins::LED_1, OUTPUT);
  pinMode(IoTPins::BUTTON_1, INPUT_PULLUP);
}

void loop() {
  bool pressed = (digitalRead(IoTPins::BUTTON_1) == LOW);   // active low, see below
  digitalWrite(IoTPins::LED_1, pressed ? HIGH : LOW);
}
```

Behind the scenes the Arduino framework provides `main()`, calls `setup()` once, and then calls `loop()` in an endless loop. On the ESP32, that loop actually runs as a task under a real-time operating system (Chapter 4.), but for now you can think of it as a simple infinite loop.

Arduino code is C++. Most sketches use a C-like subset, but classes, references and the standard library are all available. Because `int` is 16 bits on some boards and 32 bits on others, prefer the fixed-width types `uint8_t`, `int16_t`, `uint32_t` and so on whenever size matters.

3.2.3 Digital input and output

A **digital** signal has two levels. On the ESP32, a pin reads `HIGH` (1) near 3.3 V and `LOW` (0) near 0 V.

Function	Purpose
`pinMode(pin, mode)`	Configure a pin as `OUTPUT`, `INPUT`, `INPUT_PULLUP` or `INPUT_PULLDOWN`
`digitalWrite(pin, value)`	Drive an output pin `HIGH` or `LOW`
`digitalRead(pin)`	Read an input pin as `HIGH` or `LOW`

3.2.4 Pull-up and pull-down resistors

A button connected between a pin and ground only defines the pin's level while it is pressed. When it is released, the pin is connected to nothing. It is **floating**, and it will pick up electrical noise and read random values.

A **pull-up resistor** connects the pin weakly to 3.3 V, so it reads `HIGH` until the button pulls it firmly to ground. A **pull-down resistor** does the opposite. The ESP32 has internal pull-up and pull-down resistors (roughly 45 kΩ) on most pins, enabled with `INPUT_PULLUP` or `INPUT_PULLDOWN`.

The kit's buttons are wired from GPIO 32 and 33 to ground and use `INPUT_PULLUP`. They are therefore **active low**: `digitalRead()` returns `LOW` when the button is pressed. Forgetting this inverts every button test.

Remember that GPIO 34–39 have no internal pull resistors. A button on one of those pins needs an external resistor.

3.2.5 Debouncing

Mechanical switch contacts bounce when they close or open. For a few milliseconds the pin rapidly flips between `HIGH` and `LOW` before settling. A microcontroller samples fast enough to see every bounce, so a single press can be counted many times.

Debouncing filters out the bounces. The simplest software approach accepts a new state only once it has been stable for a short time, such as 20–50 ms:

```
constexpr uint32_t DEBOUNCE_MS = 30;
```

```
bool readButtonDebounced(uint8_t pin) {
  static bool stableState = HIGH;     // last accepted state (released)
  static bool lastReading = HIGH;
  static uint32_t lastChange = 0;

  bool reading = digitalRead(pin);
  if (reading != lastReading) {       // something changed: restart the timer
    lastChange = millis();
    lastReading = reading;
  }
  if (millis() - lastChange >= DEBOUNCE_MS) {
    stableState = reading;            // stable long enough: accept it
  }
  return stableState;
}
```

Detecting double clicks, long presses and so on builds on the same idea. Well-tested libraries such as Button2 handle debouncing and click patterns for you, and are usually the right choice in a project.

3.2.6 Polling and interrupts

The loop above **polls** the button: it checks the pin on every pass, even though the answer is almost always "no change". Polling is simple and adequate when the loop is fast and the event is slow.

An **interrupt** lets the hardware notify the processor when a pin changes. The processor pauses whatever it was doing, runs a short function called an **interrupt service routine** (ISR), and then resumes. Every ESP32 GPIO pin can trigger an interrupt.

```
volatile uint32_t pressCount = 0;     // shared with the ISR: must be volatile

void IRAM_ATTR onButton() {           // keep the ISR in fast internal RAM
  pressCount++;                       // do the minimum, then return
}

void setup() {
  Serial.begin(115200);
  pinMode(IoTPins::BUTTON_1, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(IoTPins::BUTTON_1), onButton, FALLING);
}

void loop() {
  static uint32_t lastReported = 0;
  uint32_t count = pressCount;        // copy once, then work with the copy
  if (count != lastReported) {
    Serial.printf("Presses: %lu\n", count);
    lastReported = count;
  }
}
```

The trigger **mode** says which change causes the interrupt: `RISING`, `FALLING`, `CHANGE`, `LOW` or `HIGH`. Because the button is active low, a press is a `FALLING` edge.

ISRs come with strict rules:

- **Keep them short.** Set a flag, increment a counter or record a timestamp, and return. Do the real work in `loop()`.
- **Do not block or print.** No `delay()`, no `Serial.print()`, no I²C or SPI transactions.
- **Mark shared variables `volatile`**, so the compiler always reads them from memory instead of a stale copy in a register.
- **Place the ISR in IRAM** with `IRAM_ATTR` on the ESP32, so it can run even while the flash memory is busy.

Interrupts do not remove bouncing. The ISR above will count bounces as presses. A common fix is to record the time in the ISR and ignore edges that arrive within the debounce window of the previous one.

3.2.7 Analog input

The physical world is mostly **analog**: temperature, light level and the position of a knob vary continuously. A computer can only store numbers, so an **analog-to-digital converter** (ADC) measures a voltage and **quantizes** it into one of a fixed number of steps.

The number of steps is set by the converter's **resolution**. An n-bit ADC divides its input range into 2^n levels. The ESP32's ADCs are 12-bit by default, so `analogRead()` returns a value from 0 to 4095 over roughly 0 to 3.3 V, and each step is about 0.8 mV.

```
int raw = analogRead(IoTPins::POT_ADC);                  // 0..4095
float volts = raw * 3.3f / 4095.0f;                       // approximate
int percent = map(raw, 0, 4095, 0, 100);                  // rescale
```

The ESP32 uses **successive-approximation** ADCs, which find the value bit by bit, like a binary search. Keep three practical limits in mind:

- Only the **ADC1** pins (GPIO 32–39) work while Wi-Fi is on.
- The ESP32's ADC is **not perfectly linear**, especially near 0 V and near the top of the range. For accurate measurements, use the calibrated `analogReadMilliVolts()` function, or an external ADC.
- Readings are **noisy**. Averaging several samples is cheap and effective.

3.2.8 Analog output: DAC and PWM

There are two ways to produce an output between "fully on" and "fully off".

A **digital-to-analog converter** (DAC) produces a real intermediate voltage. The original ESP32 has two 8-bit DAC channels, on GPIO 25 and 26: `dacWrite(25, 128)` outputs about half of 3.3 V.

Pulse-width modulation (PWM) is far more common. The pin switches rapidly between `HIGH` and `LOW`, and the fraction of time it spends `HIGH`, the **duty cycle**, sets the *average* power. An LED switched fast enough appears dimmer; a motor runs slower. A servo reads the width of each pulse as a position command (Chapter 7.).

The ESP32 generates PWM with its **LEDC** peripheral, which has 16 channels with configurable frequency and resolution:

```
// Arduino-ESP32 core 2.x API (the kit's pinned version)
constexpr int LED_CHANNEL = 1, PWM_FREQ = 5000, PWM_BITS = 8;

void setup() {
  ledcSetup(LED_CHANNEL, PWM_FREQ, PWM_BITS);
  ledcAttachPin(IoTPins::LED_3, LED_CHANNEL);
}

void loop() {
  for (int duty = 0; duty <= 255; duty += 5) {   // fade in
    ledcWrite(LED_CHANNEL, duty);
    delay(20);
  }
}
```

Version 3.x of the Arduino-ESP32 core replaced `ledcSetup()` and `ledcAttachPin()` with a single `ledcAttach(pin, freq, bits)` and lets `ledcWrite()` take the pin directly. This is one reason the kit pins its platform version. Also note that `tone()` and the servo library both use LEDC channels; the kit's starter code assigns them separate channels so they do not interfere.

3.2.9 Time without blocking

`delay(ms)` stops the program completely. While it waits, the loop cannot read buttons, refresh the display or answer network messages. A better pattern is to check how much time has passed and act only when it is due:

```
void loop() {
  static uint32_t lastRed = 0, lastYellow = 0;
  uint32_t now = millis();
```

```
  if (now - lastRed >= 1000) {           // every 1 s
    lastRed = now;
    digitalWrite(IoTPins::LED_1, !digitalRead(IoTPins::LED_1));
  }
  if (now - lastYellow >= 1500) {        // every 1.5 s
    lastYellow = now;
    digitalWrite(IoTPins::LED_2, !digitalRead(IoTPins::LED_2));
  }
  // ... the loop stays free to read inputs here
}
```

`millis()` returns the milliseconds since start-up as a 32-bit unsigned number, which wraps around to zero after about 49.7 days. Writing the comparison as `now - last >= interval`, with unsigned arithmetic, gives the correct answer even across the wraparound. Writing it as `now >= last + interval` does not.

Try it on your kit: buttons and interrupts

This exercise corresponds to the second lecture activity and runs in Wokwi or on Stage 1 of the kit.

1. Count button presses by polling without any debouncing, and print the count. Press the button 20 times. What count do you get?
2. Add the debounce function and repeat. What changed?
3. Rewrite the counter with an interrupt. Does it count bounces? Fix it by ignoring edges that arrive within 30 ms of the previous one.
4. Make the red LED blink every 500 ms and the green LED every 700 ms using `millis()`, while the button still toggles the yellow LED instantly.
5. Read the potentiometer and use it to set the green LED's brightness with PWM.

A guided, step-by-step version for a lecture session is Activity 2.

3.3 In depth (optional)

Note

Optional, in-depth explanation. This section goes beyond the rest of the chapter into the math and physics behind it. Nothing later in the book depends on it; skip it on a first reading.

3.3.1 Bounce, time and wrap-around

Why buttons bounce. A pushbutton is two springy metal contacts. When they close, they collide, separate and collide again several times before settling, typically for 1–10 ms. A pin sampled every microsecond sees each collision as a separate edge.

Hardware debouncing with an RC filter. A resistor R and capacitor C smooth the signal. When the button opens, the pull-up charges the capacitor:

$$V(t) = V_{DD}\left(1 - e^{-t/RC}\right). \tag{3.1}$$

The pin reads HIGH once V crosses its input threshold V_{IH}, at time $t = -RC\ln(1 - V_{IH}/V_{DD})$. With $R = 10\ \text{k}\Omega$, $C = 1\ \mu\text{F}$ ($RC = 10$ ms) and $V_{IH} = 0.75\,V_{DD}$, that is $10\ln 4 \approx 14$ ms: long enough to ride out the bounce. A slow edge can make an ordinary input chatter as it crosses the threshold, which is why hardware debouncers often add a Schmitt-trigger input with two thresholds.

Software debouncing replaces the capacitor with a time window: accept an edge only if more than t_d has passed since the last accepted one. The window must be longer than the worst bounce and shorter than the fastest intentional double press (about 100 ms for a person).

Timer arithmetic. `millis()` counts milliseconds in an unsigned 32-bit integer, so it wraps to zero after 2^{32} ms ≈ 49.7 days. The idiom

```
if (now - last >= interval) { ... }
```

still works across the wrap, because unsigned subtraction is arithmetic modulo 2^{32}: if `last` is just before the wrap and `now` just after, `now - last` is still the small, correct difference. The tempting alternative `if (now >= last + interval)` fails for the interval around the wrap. A device meant to run for months must get this right.

Hardware timers. The ESP32's general-purpose timers count a clock (80 MHz by default) through a prescaler P. One tick lasts $P/80$ MHz; with $P = 80$, one tick is 1 µs, and an alarm after N ticks fires every N µs. This is how `micros()` and periodic timer interrupts are built.

Exercise. A wearable samples a button every 5 ms and declares a press when it has seen k consecutive LOW samples. What is the smallest k that rejects bounces of up to 8 ms? What is the longest delay the user can notice?

Sources for this section: [9].

3.4 Design trade-offs

Choice	Prefer it when
Polling	The loop is fast, the event is slow, and simplicity matters
Interrupts	Events are short or rare, or the processor should sleep while waiting
Internal pull-ups	Short wires and simple buttons
External pull-ups (e.g. 10 kΩ)	Long wires, noisy environments, or input-only pins
DAC	You need a true analog voltage (audio, a reference level)
PWM	Dimming LEDs, controlling motor speed or driving servos
`delay()`	Throwaway tests only
`millis()` scheduling	Anything that must stay responsive

3.5 Common pitfalls

- **Floating inputs.** An input with no pull resistor reads random values.
- **Forgetting active-low logic.** With `INPUT_PULLUP`, pressed is `LOW`.
- **Long or blocking ISRs.** Printing, delaying or using I²C inside an ISR crashes the ESP32 or causes watchdog resets.
- **Missing `volatile`.** The main loop never notices that the ISR changed a variable.
- **Integer overflow and unsigned loops.** `for (uint8_t i = 255; i >= 0; i--)` never ends, because an unsigned value is always ≥ 0.
- **ADC2 with Wi-Fi.** Analog readings fail silently once Wi-Fi starts.
- **Unpinned platform versions.** Code that compiled last week fails after a core update.

> **i** Advanced track
>
> **Hardware debouncing.** A resistor and capacitor (an RC filter) on the button line, followed by a Schmitt-trigger input, removes bounce in hardware. Calculate an RC time constant that filters bounces of up to 5 ms, and discuss when hardware debouncing is worth the extra parts.
>
> **Atomic access.** Copying a 32-bit counter on the ESP32 is atomic, but a 64-bit value or a structure is not: an interrupt can arrive halfway through the copy. On the ESP32, protect such data with `portENTER_CRITICAL_ISR()` / `portEXIT_CRITICAL_ISR()` and a `portMUX_TYPE` spinlock, or pass events through a FreeRTOS queue (Chapter 4.).
>
> **Hardware timers.** For precise periodic work, the ESP32 has hardware timers whose interrupts fire at exact intervals regardless of what the main loop is doing. The `esp_timer` API offers microsecond-resolution software timers built on them.

3.6 Check your understanding

1. What does a floating input read, and how do pull-up resistors fix it?
2. A button on GPIO 32 uses `INPUT_PULLUP`. What does `digitalRead()` return while it is pressed?
3. Why must a variable shared with an ISR be declared `volatile`?
4. A 12-bit ADC with a 3.3 V range reads 2048. Approximately what voltage is on the pin?
5. Explain how PWM makes an LED look half as bright without producing 1.65 V.
6. Why does `now - last >= interval` keep working after `millis()` wraps around, while `now >= last + interval` does not?
7. Find the bug: `for (uint8_t i = 255; i >= 0; i--) { dacWrite(25, i); }`.

3.7 Further reading

- Espressif, *Arduino-ESP32 documentation* [10], for GPIO, ADC, LEDC and interrupt APIs, including the 2.x-to-3.x migration guide.
- PlatformIO documentation [11], for project configuration and library management.

4. Embedded Systems and Operating Systems

Learning objectives

After this chapter you should be able to:

1. Describe the characteristics that distinguish embedded systems from general-purpose computers.
2. Distinguish microprocessors, microcontrollers and systems on a chip.
3. Explain hard and soft real-time requirements with examples.
4. Compare three software architectures: the super-loop, foreground/background, and a real-time kernel.
5. Create, schedule and synchronize FreeRTOS tasks on the ESP32.
6. Decide when an IoT device needs an RTOS and when it does not.

4.1 Why it matters

Your laptop runs hundreds of programs at once, can be restarted when something hangs, and has gigabytes of memory to spare. An IoT device may run one program for ten years in a place no one can reach, with a few hundred kilobytes of memory and a battery that must last the whole time. It may also have to react to an event within milliseconds, every time.

Writing software for these conditions is different from writing desktop or web software. This chapter introduces the vocabulary of embedded systems and the two main ways to structure their software: a single loop, or a set of tasks managed by a real-time operating system.

4.2 Concepts

4.2.1 What makes a system “embedded”

There is no precise definition of an embedded system, but embedded systems share a recognizable set of characteristics:

- They do **one job**, or a small set of jobs, defined when they are built. The user does not install new programs.
- They are **resource-constrained**: limited memory, processing power and energy, and a low cost per unit.
- They **interact with the physical world** through sensors and actuators, and often have **real-time** constraints.
- They run **unattended**, often for years, and may be physically inaccessible after deployment.

Programmers moving from PCs to embedded systems should adjust a few habits:

1. **Assume the device is unreachable.** Over-the-air updates help (Chapter 16.), but a device that crashes in a way that stops updates is lost.
2. **Expect surprising inputs.** A sensor can fail and report nonsense, or a network can deliver a burst of messages at once. Plan for it.
3. **Log well.** Faults may be impossible to reproduce on the bench, so error messages must carry enough context to diagnose them from a report.
4. **Plan testing from the start.** Testing on real hardware is slow; design the code so that most of it can be tested without the hardware.
5. **Read datasheets.** Embedded programmers write code that talks directly to hardware, and the datasheet is the specification.

4.2.2 Microprocessors, microcontrollers and SoCs

These terms are often confused because the categories overlap:

- A **microprocessor** (MPU) is a CPU on a chip. It needs external memory and peripherals to be useful. Desktop and laptop processors are microprocessors.
- A **microcontroller** (MCU) puts a CPU, memory (flash for the program, RAM for data) and peripherals on one chip: GPIO, ADC, timers, UART, SPI and I²C controllers, and an interrupt controller.
- A **system on a chip** (SoC) integrates a microcontroller or microprocessor with larger subsystems, such as radios, graphics or cryptographic accelerators. The ESP32 is an SoC: a dual-core microcontroller plus Wi-Fi and Bluetooth radios.

Arm's processor families show the same split. **Cortex-M** cores are designed for microcontrollers. **Cortex-A** cores are application processors with a memory-management unit that can run Linux (the Raspberry Pi uses them). **Cortex-R** cores target hard real-time, safety-critical systems such as disk controllers and automotive braking.

4.2.3 Real-time systems

A **real-time system** must respond to external events within a specified time. Being correct means producing the right answer *on time*; a right answer that arrives late is wrong.

Real-time systems are classified by what happens when a deadline is missed:

- In a **hard real-time** system, a missed deadline is a failure. An airbag that deploys 200 ms late, or a motor controller that misses its control period, has failed.
- In a **soft real-time** system, a missed deadline degrades quality but the system still works. A video stream drops a frame; a thermostat updates its display a little late.

"Real-time" does not mean "fast". The deadline might be microseconds or hours. What matters is that the response time is **bounded and predictable** (deterministic).

4.2.4 Architecture 1: the super-loop

The simplest structure is the one you have used so far: a single infinite loop that does each job in turn.

```
void loop() {
  readSensors();
  updateControl();
  refreshDisplay();
  sendTelemetry();
}
```

Super-loops are small, easy to understand and need no operating system. Their weakness is that every job waits for every other job. If `sendTelemetry()` blocks for two seconds waiting for Wi-Fi, the control loop stops for two seconds too. Running jobs at different rates also gets awkward quickly, as you saw with the `millis()` scheduler in Chapter 3..

4.2.5 Architecture 2: foreground/background

A common improvement adds interrupts. The super-loop runs in the **background**, while interrupt service routines handle urgent events in the **foreground**. Most low-cost microcontroller products are built this way.

The weakness moves rather than disappears. Urgent work must either be done inside the ISR, making the ISR long and delaying other interrupts, or deferred to the background loop, where it waits until the loop gets around to it.

4.2.6 Architecture 3: a real-time kernel

A **real-time kernel** divides the program into independent **tasks** (also called threads), each with its own stack and a **priority**. A **scheduler** decides which task runs at any moment. In a **preemptive** kernel, a higher-priority task that becomes ready immediately interrupts a lower-priority one.

The kernel also provides tools for tasks to cooperate safely:

- **Delays** that put a task to sleep without wasting processor time;
- **Queues** for passing messages between tasks, and from ISRs to tasks;
- **Semaphores and mutexes** to signal events and protect shared resources, such as an I²C bus;
- **Software timers** that run a function at a set interval.

A **real-time operating system** (RTOS) is a real-time kernel plus higher-level services: file systems, network stacks, device drivers and so on.

The kernel costs some memory and processor time. In return, you get more predictable timing, a program split into smaller and more manageable pieces, and priorities that can be changed in software.

4.2.7 FreeRTOS on the ESP32

Many small RTOSes exist: FreeRTOS, Zephyr, RIOT, Apache Mynewt, Azure RTOS ThreadX (now Eclipse ThreadX), and commercial systems such as VxWorks. **FreeRTOS** is the most widely used. It is MIT-licensed, has been ported to dozens of processor families, and has been maintained by Amazon since 2017.

The ESP32's software is built on FreeRTOS. Even a plain Arduino sketch runs on it: the framework starts a task called `loopTask` that calls `loop()` repeatedly, while Wi-Fi and Bluetooth run in their own tasks, mostly on the other core. You can create your own tasks alongside it:

```
#include <Arduino.h>
#include "iot_pins.h"

void blinkTask(void* param) {
  uint8_t pin = (uint8_t)(uintptr_t)param;
  uint32_t periodMs = (pin == IoTPins::LED_1) ? 1000 : 1500;
  pinMode(pin, OUTPUT);
  for (;;) {                                    // a task never returns
    digitalWrite(pin, !digitalRead(pin));
    vTaskDelay(pdMS_TO_TICKS(periodMs));        // sleep; other tasks run
  }
}

void setup() {
  xTaskCreatePinnedToCore(blinkTask, "red",    2048, (void*)(uintptr_t)IoTPins::LED_1,
1, nullptr, 1);
  xTaskCreatePinnedToCore(blinkTask, "yellow", 2048, (void*)(uintptr_t)IoTPins::LED_2,
1, nullptr, 1);
}

void loop() {
  vTaskDelay(pdMS_TO_TICKS(1000));              // loop() is just another task
}
```

The arguments to `xTaskCreatePinnedToCore()` are the task function, a name, the stack size in bytes, a parameter, the priority, an optional handle, and the core to run on.

Queues are the safe way to pass data between tasks, or from an ISR to a task:

```
QueueHandle_t buttonEvents = xQueueCreate(8, sizeof(uint32_t));

void IRAM_ATTR onButton() {
  uint32_t t = millis();
  BaseType_t woken = pdFALSE;
  xQueueSendFromISR(buttonEvents, &t, &woken);   // the ISR only posts an event
  portYIELD_FROM_ISR(woken);
}

void handlerTask(void*) {
  uint32_t t;
  for (;;) {
    if (xQueueReceive(buttonEvents, &t, portMAX_DELAY)) {  // sleeps until an event
arrives
      Serial.printf("Button at %lu ms\n", t);             // safe to print here
```

```
    }
  }
}
```

This pattern keeps ISRs tiny, as Chapter 3. recommended, while guaranteeing that every event is processed promptly by a task whose priority you control.

4.2.8 Do you need an RTOS?

Not always. Many IoT devices are naturally **event-driven**: "sleep until something happens, handle it, go back to sleep". A foreground/background design with deep sleep may be simpler and use less energy. Some designs use only part of a kernel, such as a scheduler and queues.

An RTOS earns its place when a device has several activities with different timing needs, such as a control loop, a user interface and a network connection, and when one slow activity must not stall the others.

Try it on your kit: one loop versus three tasks

1. Using a super-loop with `delay()`, try to blink LED 1 every 1 s, LED 2 every 2 s and LED 3 every 1.5 s. Why is this awkward?
2. Implement the same behavior with three FreeRTOS tasks. How many lines does each version take?
3. Add a fourth task that simulates a slow network call with `vTaskDelay(pdMS_TO_TICKS(3000))`. Does the blinking change? What would happen in the super-loop version?
4. Replace the button-counter ISR from Chapter 3. with the queue pattern above.

4.3 In depth (optional)

Note

Optional, in-depth explanation. This section goes beyond the rest of the chapter into the math and physics behind it. Nothing later in the book depends on it; skip it on a first reading.

4.3.1 Will every task meet its deadline?

A real-time operating system schedules tasks by priority. For periodic tasks, there is a classic way to check, before running anything, that every task will finish before its next period starts.

Describe each task i by its worst-case execution time C_i and its period T_i (its deadline is the end of the period). Its **utilization** is $U_i = C_i/T_i$, the fraction of the CPU it needs.

Necessary condition. The total utilization must not exceed 1: $U = \sum_i C_i/T_i \le 1$.

A sufficient test for fixed priorities. With **rate-monotonic** priorities (shorter period, higher priority), Liu and Layland showed that n independent periodic tasks always meet their deadlines if

$$U \le n\left(2^{1/n} - 1\right). \tag{4.1}$$

The bound is 1.0 for one task, 0.828 for two, 0.780 for three, and approaches $\ln 2 \approx 0.693$ as n grows.

Example. A kit runs three tasks: read the IMU every 10 ms (1 ms of work), update the OLED every 100 ms (25 ms), and publish to MQTT every 1000 ms (60 ms). Then $U = 0.10 + 0.25 + 0.06 = 0.41 \le 0.780$: schedulable.

Response-time analysis gives an exact answer when the bound fails. The worst-case response time R_i of task i is the smallest solution of

$$R_i = C_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i}{T_j} \right\rceil C_j, \tag{4.2}$$

where $hp(i)$ are the tasks with higher priority. Solve it by iteration from $R_i = C_i$. For the OLED task: $R = 25 + \lceil 25/10 \rceil \cdot 1 = 28$, then $25 + \lceil 28/10 \rceil \cdot 1 = 28$: converged, well under 100 ms.

What breaks the model. Real tasks share resources. If a low-priority task holds a mutex that a high-priority task needs, while a medium-priority task runs, the high-priority task waits indefinitely: **priority inversion**. FreeRTOS mutexes use priority inheritance to bound it; binary semaphores do not. Interrupts, Wi-Fi stack activity on the other core, and flash writes also steal time that a simple C_i does not show, so leave margin.

Exercise. Add a fourth task that reads the BMP280 every 50 ms for 5 ms. Is the set still guaranteed schedulable by the utilization bound? If not, apply response-time analysis to the lowest-priority task.

Sources for this section: [12], [13].

4.4 Design trade-offs

Architecture	Strengths	Weaknesses	Typical use
Super-loop	Tiny, simple, easy to reason about	Every job waits for every other	Single-purpose, low-cost devices
Foreground/background	Responsive to events, no kernel needed	Hard to balance ISR and loop work	Most simple products
RTOS	Priorities, isolation, predictable timing	Memory per task, concurrency bugs	Devices with networking and control together
Embedded Linux	Full OS, rich software ecosystem	Needs Cortex-A-class hardware and much more power	Gateways, cameras, smart displays

4.5 Common pitfalls

- **Stack overflow.** A task with too small a stack corrupts memory or crashes. Start with generous stacks and measure with `uxTaskGetStackHighWaterMark()`.
- **Starving tasks.** A high-priority task that never blocks prevents all lower-priority tasks from running, and triggers the watchdog timer.
- **Using `delay()` instead of yielding.** On the ESP32 `delay()` calls `vTaskDelay()`, but a busy-wait loop does not, and it starves other tasks.
- **Unprotected shared resources.** Two tasks using the same I²C bus or `Serial` port at the same time interleave their messages. Protect them with a mutex.
- **Priority inversion.** A low-priority task holding a mutex can block a high-priority task. Use mutexes (which support priority inheritance) rather than plain semaphores for shared resources.

> **i** Advanced track

Scheduling theory. In rate-monotonic scheduling, tasks with shorter periods get higher priorities. Liu and Layland showed that n periodic tasks are guaranteed to meet their deadlines under this policy if total processor utilization is at most $n(2^{1/n} - 1)$, which approaches about 69% for large n. Compute whether three tasks with execution times 1, 2 and 3 ms and periods 10, 20 and 40 ms are schedulable.

Watchdog timers. A watchdog is a hardware timer that resets the processor unless software "feeds" it regularly. The ESP32's task watchdog fires when a task hogs a core. Design a watchdog strategy for a field device: what should be fed, by whom, and how do you avoid feeding it from a loop that is itself stuck?

Zephyr. The Zephyr RTOS, hosted by the Linux Foundation, has become a common choice for new IoT products because it includes networking, Bluetooth, security and update support in one project. Compare its configuration model (Kconfig and devicetree) with the Arduino approach.

4.6 Check your understanding

1. List four characteristics of embedded systems.
2. Is the ESP32 a microprocessor, a microcontroller or an SoC? Justify your answer.

3. Give one example each of a hard and a soft real-time requirement in an IoT product.
4. Why does “real-time” not necessarily mean “fast”?
5. What problem does a preemptive scheduler solve that a super-loop cannot?
6. Why should an ISR post to a queue rather than print a message directly?
7. Describe a device that does not need an RTOS, and explain why.

4.7 Further reading

- R. Barry, *Mastering the FreeRTOS Real Time Kernel* [13], a free, practical guide to tasks, queues and synchronization.
- Espressif, *ESP-IDF FreeRTOS documentation* [14], for the ESP32′s dual-core extensions.
- C. L. Liu and J. W. Layland, “Scheduling algorithms for multiprogramming in a hard-real-time environment” [12].

II

Part II — The Things

5 Serial Communication: UART, SPI and I²C 51
5.1 Why it matters 51
5.2 Concepts 52
5.3 In depth (optional) 58
5.4 Design trade-offs 59
5.5 Common pitfalls 59
5.6 Check your understanding 60
5.7 Further reading 60

6 Sensors 61
6.1 Why it matters 61
6.2 Concepts 61
6.3 In depth (optional) 66
6.4 Design trade-offs 67
6.5 Common pitfalls 67
6.6 Check your understanding 68
6.7 Further reading 68

7 Actuators and the Observe–Decide–Act Loop 69
7.1 Why it matters 69
7.2 Concepts 69
7.3 In depth (optional) 73
7.4 Design trade-offs 74
7.5 Common pitfalls 74
7.6 Check your understanding 75
7.7 Further reading 75

5. Serial Communication: UART, SPI and I²C

Learning objectives

After this chapter you should be able to:

1. Explain why embedded devices move data serially, and distinguish synchronous from asynchronous links.
2. Describe how UART, SPI and I²C frame, clock and address their data.
3. Choose the right bus for a given peripheral, and justify the choice in terms of wires, speed, distance and number of devices.
4. Connect, scan and read devices on the kit's I²C and SPI buses from ESP32 code.
5. Diagnose the most common wiring and configuration faults on each bus.

5.1 Why it matters

Open almost any IoT device and you will find a microcontroller surrounded by smaller chips: a temperature sensor, a motion sensor, a display, a radio, a secure element. The microcontroller is the only part that runs your code. Everything the device *senses*, *shows* or *transmits* reaches it over a handful of wires.

Your kit is a good example. Its ESP32 talks to a computer, two sensors, a security chip, a display and a LoRa radio over about a dozen signal wires, most of them shared. It manages this because it uses three different serial protocols, each designed for a different job (Figure 5.1).

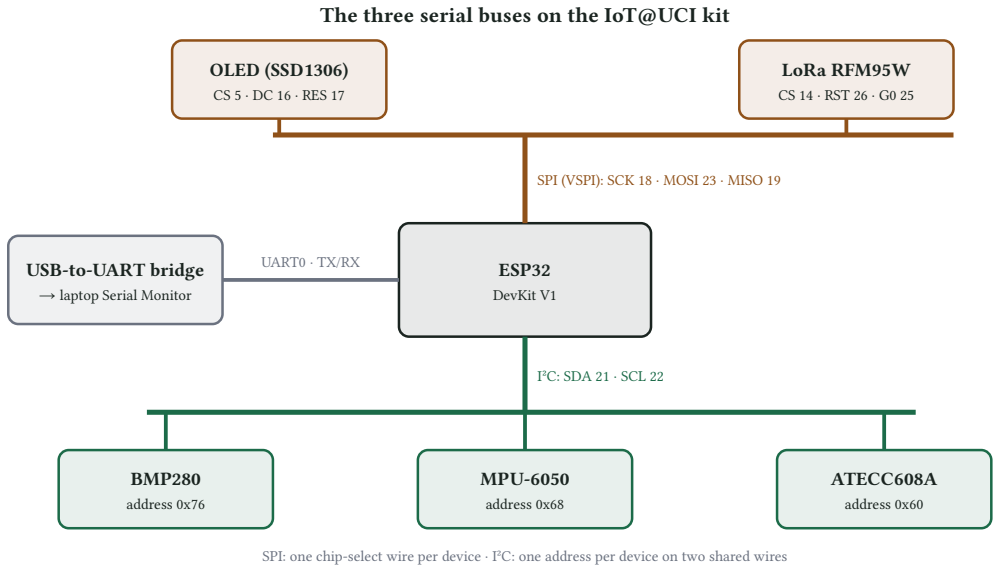


Figure 5.1: The kit uses all three buses covered in this chapter. UART connects the ESP32 to your laptop. SPI connects fast devices that each get their own chip-select wire. I²C connects slower devices that share two wires and are told apart by address.

Choosing the wrong bus, or wiring the right one carelessly, is one of the most common reasons an IoT prototype does not work. This chapter gives you the vocabulary and the mental model to get it right.

5.2 Concepts

5.2.1 Protocols, parallel and serial

A **communication protocol** is an agreement between two or more parties about how information is exchanged: the *syntax* (how bits are grouped), the *semantics* (what they mean), and the *synchronization* (when each bit is valid). Protocols also say what happens when something goes wrong.

Suppose you want to move one byte from one ESP32 to another. The simplest approach is **parallel** communication: use eight wires, one per bit, and set them all at once. Parallel buses are fast and are still used inside computers, for example between a processor and its memory.

Parallel links do not scale well for small devices, though. Each extra bit costs a pin, a trace and a wire, and microcontrollers have few pins to spare. Over longer wires, the bits also stop arriving at exactly the same moment. **Serial** communication solves both problems by sending the bits one after another over a single data line, and adding just enough control information for the receiver to reassemble them.

Two properties describe any serial link:

- **Synchronization.** In a **synchronous** link, one side drives a separate *clock* wire that tells the receiver exactly when to read each bit. In an **asynchronous** link there is no clock wire, so both sides must agree on the bit rate in advance and resynchronize on every frame.
- **Direction.** A **simplex** link carries data one way only. A **half-duplex** link carries data both ways, but only one way at a time. A **full-duplex** link carries data both ways simultaneously.

> **i** A note on terminology
>
> Older documents describe bus roles as *master* and *slave*, and name SPI pins MOSI and MISO. Current specifications use **controller** and **target** for I²C [15], and many vendors now use **controller** and **peripheral** for SPI, with pins named COPI (controller out, peripheral in) and CIPO. This book uses the modern terms. You will still see the older labels printed on modules and in the Arduino API, and they mean the same thing.

5.2.2 UART: asynchronous serial

A **UART** (Universal Asynchronous Receiver/Transmitter) is a hardware block, not a protocol in itself. It turns bytes into a timed stream of bits on a transmit (TX) wire, and turns the bit stream arriving on a receive (RX) wire back into bytes.

5.2.2.1 The frame

Because there is no clock wire, every byte is wrapped in a **frame** that the receiver can recognize on its own (Figure 5.2):

1. The line **idles high**.
2. A **start bit** pulls the line low for one bit time. This falling edge tells the receiver a byte is coming and where its bits begin.
3. Five to nine **data bits** follow, *least-significant bit first*. Eight is almost universal.
4. An optional **parity bit** allows simple error detection.
5. One or two **stop bits** return the line high, so the next start bit always produces a fresh falling edge.

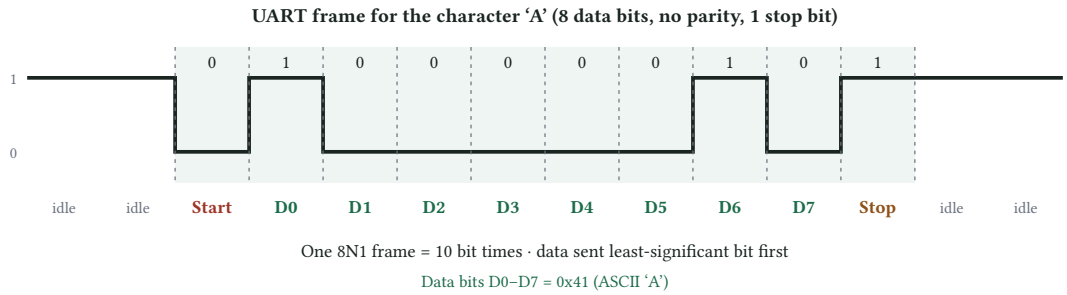


Figure 5.2: One UART frame carrying the ASCII character 'A' (0x41) in the common 8N1 format.

A configuration is usually written in shorthand such as **8N1**: 8 data bits, no parity, 1 stop bit. The receiver samples each bit near the middle of its time slot, which gives it some tolerance for timing error.

5.2.2.2 Baud rate and throughput

The **baud rate** is the number of bit times per second. Both sides must be set to the same value. Common rates are 9600 and 115200 baud.

The framing bits are overhead. In 8N1, each byte costs 10 bit times, so a link at 115200 baud moves at most

$$\frac{115\,200 \ \text{bits/s}}{10 \ \text{bits/byte}} = 11\,520 \ \text{bytes/s} \ . \tag{5.1}$$

If the two sides disagree on the baud rate, the receiver samples at the wrong moments and the error accumulates across the frame. A mismatch of a few percent is enough to corrupt data. A large mismatch produces the familiar screen full of random characters in the Serial Monitor.

5.2.2.3 Wiring

A UART link is **point-to-point** between two devices:

- Connect each device's **TX to the other's RX**, not TX to TX.
- **Connect the grounds.** The receiver measures voltage relative to its own ground, so without a common reference the levels are meaningless.
- Check the **logic levels**. The ESP32 uses 3.3 V signals. Connecting a 5 V device's TX directly to an ESP32 input can damage it.

5.2.2.4 UART on the ESP32

The ESP32 has three UART controllers. On the DevKit V1, **UART0** is wired to a USB-to-UART bridge chip on the board. That is the link you use every time you upload code or open the Serial Monitor, so avoid using its pins for anything else. The other two UARTs can be routed to almost any free GPIO pins through the ESP32's pin multiplexer [9].

In the Arduino framework, each UART is a `HardwareSerial` object:

```
#include <Arduino.h>

constexpr int UART2_RX = 16;   // example pins; check your pin map first
constexpr int UART2_TX = 17;

void setup() {
  Serial.begin(115200);                                  // UART0 → USB → laptop
  Serial2.begin(9600, SERIAL_8N1, UART2_RX, UART2_TX);   // e.g., a GPS module
}

void loop() {
  if (Serial.available() > 0) {        // bytes waiting in the receive buffer?
    int b = Serial.read();             // take one byte
    Serial.printf("Received 0x%02X ('%c')\n", b, b);
  }
}
```

The course helper `iotStartSerial()` wraps `Serial.begin(115200)` and prints a banner, which is why the Serial Monitor in every activity is set to 115200 baud.

5.2.2.5 Strengths and limits

UART is simple, needs only two wires, and every microcontroller has one. That is why it remains the standard way to talk to a development board, and why modules such as GPS receivers and cellular modems still use it.

Its limits follow from its design. It connects only two devices. It spends 20% of the line on framing. Both sides must agree on the speed in advance. Inside devices, most chip-to-chip links have moved to SPI and I²C.

5.2.3 SPI: the fast synchronous bus

The **Serial Peripheral Interface** (SPI) is a synchronous bus introduced by Motorola in the 1980s [16]. It has no formal standard, but it is supported by almost every microcontroller and by thousands of peripherals.

5.2.3.1 Signals and topology

One **controller** (the ESP32) drives the bus. Three wires are shared by every device:

- **SCK**, the clock, always driven by the controller.
- **COPI** (MOSI), data from controller to peripheral.
- **CIPO** (MISO), data from peripheral to controller.

Each peripheral also gets its own **chip select** (CS) wire. CS is *active low*: the controller pulls one device's CS low to talk to it, and keeps every other CS high. A deselected peripheral must ignore the clock and release CIPO, so devices do not fight over the shared line.

On every clock pulse, the controller shifts one bit out on COPI while the selected peripheral shifts one bit back on CIPO. SPI is therefore **full duplex**, and the hardware inside a simple peripheral can be little more than a shift register. Because the controller generates every clock edge, a peripheral can only send data while the controller is clocking. If the controller wants to read a reply, it must keep clocking (usually by sending dummy bytes) for as many bytes as it expects.

5.2.3.2 Clock modes

Devices disagree about which clock edge marks a valid bit. Two settings describe the convention:

- **CPOL** (clock polarity): whether the clock idles low (0) or high (1).
- **CPHA** (clock phase): whether data is sampled on the first (0) or second (1) clock edge of each bit.

Table 5.1: The four SPI modes. Mode 0 is the most common.

Mode	CPOL	CPHA	Clock idles	Data sampled on
0	0	0	Low	Rising edge
1	0	1	Low	Falling edge
2	1	0	High	Falling edge
3	1	1	High	Rising edge

The controller must use the mode the peripheral expects, which is given in the peripheral's datasheet. Most devices also send the **most significant bit first**.

5.2.3.3 Speed

SPI has no addressing overhead and no framing bits, and its push-pull outputs switch quickly. Clock rates of 10 MHz or more are routine, which makes SPI the usual choice for displays, radios, SD cards and flash memory. The practical limit is set by the slowest device on the bus and by the length of the wires.

5.2.3.4 SPI on the ESP32

The ESP32 has four SPI controllers. Two of them (SPI0 and SPI1) are reserved for the external flash chip that holds your program. The other two, **HSPI** and **VSPI**, are free for peripherals [9]. The kit uses VSPI on its default pins: SCK on GPIO 18, COPI on GPIO 23 and CIPO on GPIO 19. Using a controller's default pins gives the highest clock rates; routing SPI signals to other pins through the GPIO matrix works, but lowers the maximum speed.

The kit puts two devices on this bus (Figure 5.1):

- The **OLED display** (CS on GPIO 5) only receives data, so it never uses CIPO. It has an extra **D/C** pin that tells it whether each byte is a command or pixel data.
- The **LoRa radio** (CS on GPIO 14), added in Chapter 10., both receives commands and returns data, so it uses CIPO.

Because different devices may need different speeds and modes, the Arduino SPI library groups those settings into a *transaction*:

```
#include <SPI.h>

constexpr int LORA_CS = 14;

uint8_t readRadioRegister(uint8_t reg) {
  SPI.beginTransaction(SPISettings(8000000, MSBFIRST, SPI_MODE0));
  digitalWrite(LORA_CS, LOW);          // select the radio
  SPI.transfer(reg & 0x7F);           // send the register address (top bit 0 = read)
  uint8_t value = SPI.transfer(0x00);  // clock out a dummy byte, clock in the reply
  digitalWrite(LORA_CS, HIGH);         // release the bus
  SPI.endTransaction();
  return value;
}

void setup() {
  pinMode(LORA_CS, OUTPUT);
  digitalWrite(LORA_CS, HIGH);         // every CS starts deselected
  SPI.begin(18, 19, 23);               // SCK, CIPO (MISO), COPI (MOSI)
}
```

Notice that reading one byte takes two transfers: the first sends the register address, and the second exists only to generate eight clock pulses so the radio can answer.

5.2.3.5 Strengths and limits

SPI is fast, full duplex and simple to implement in hardware. Its costs are wiring and coordination. Every additional device needs another CS wire. There is no acknowledgment, so the controller cannot tell whether anyone is listening. Peripherals cannot start a conversation on their own, and they cannot talk to each other.

When there are more peripherals than spare pins, designers either add a decoder chip that turns a few pins into many chip selects, or **daisy-chain** devices so that data shifts through one into the next. Daisy-chaining suits output-only devices such as LED drivers.

5.2.4 I²C: two wires, many devices

I²C (Inter-Integrated Circuit, pronounced “I-squared-C”) was created by Philips in 1982 to connect many chips inside a television using as few wires as possible. Today it is the default bus for low-speed sensors and is defined by a published specification [15].

5.2.4.1 Two wires and an address

Every device on an I²C bus shares just two lines:

- **SCL**, the serial clock, driven by the controller.
- **SDA**, serial data, driven by whichever side is sending.

Instead of chip-select wires, each target has a **7-bit address**. The controller begins every transaction by sending an address, and only the target with that address responds. The kit’s three I²C devices sit on the same two wires at addresses 0x76 (BMP280), 0x68 (MPU-6050) and 0x60 (ATECC608A). A few addresses are reserved by the specification, so the usable range is 0x08 to 0x77.

Many breakout boards let you choose between two addresses with a pin or a jumper. The BMP280 answers at 0x76 or 0x77 depending on its SDO pin, and the MPU-6050 at 0x68 or 0x69 depending on its AD0 pin [17], [18]. That is how two identical sensors can share a bus.

5.2.4.2 Open drain and pull-up resistors

I²C outputs are **open drain**: a device can pull a line *low*, but it can never drive it *high*. Instead, a **pull-up resistor** on each line returns it to the supply voltage whenever nobody is pulling down.

This design has two important consequences:

- **No device can short the bus.** If one device holds the line low while another releases it, the line is simply low. This is what makes acknowledgments, arbitration and clock stretching (below) safe.
- **The pull-ups set the speed.** The rising edge is only as fast as the resistor can charge the capacitance of the wires and inputs. Larger resistors save power but slow the edges; smaller ones allow faster edges but force each device to sink more current when it pulls low. Values between about 2.2 kΩ and 10 kΩ are typical.

Most sensor breakout boards include their own pull-up resistors. When several boards share a bus, their pull-ups act in parallel, so the effective resistance drops as you add boards. That is usually harmless with two or three boards, but worth checking as a bus grows.

5.2.4.3 Transactions, ACK and NACK

An I²C transaction is built from 9-bit units: eight data bits, most significant first, followed by one **acknowledge** bit (Figure 5.3).

1. The controller sends a **START** condition: SDA falls while SCL is high. Data never changes while SCL is high, so this pattern is unmistakable.
2. The controller sends the 7-bit address and a **read/write** bit (0 = write, 1 = read).
3. The addressed target pulls SDA low during the ninth clock to **acknowledge** (ACK). If no device has that address, SDA stays high: a **NACK**.
4. Data bytes follow, each acknowledged by the receiver.
5. The controller ends with a **STOP** condition: SDA rises while SCL is high.

Most sensors expose their data as numbered **registers**. To read a register, the controller first *writes* the register number, then uses a **repeated START** (a START without a STOP in between) to switch to *reading* without giving up the bus.

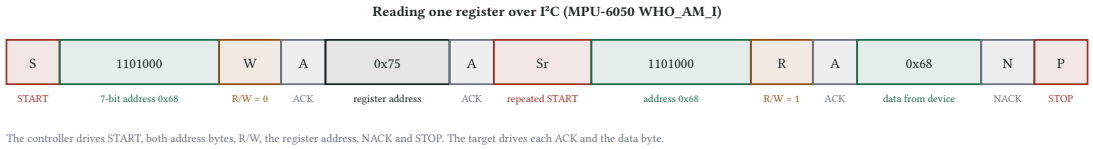


Figure 5.3: Reading the MPU-6050's WHO_AM_I register (0x75). The chip answers 0x68, which identifies it as an MPU-6050.

```
#include <Wire.h>

constexpr uint8_t MPU6050_ADDR = 0x68;
constexpr uint8_t WHO_AM_I     = 0x75;

int readRegister(uint8_t addr, uint8_t reg) {
  Wire.beginTransmission(addr);
  Wire.write(reg);                          // which register we want
  if (Wire.endTransmission(false) != 0) {   // false = repeated START, keep the bus
    return -1;                              // NACK: nobody at this address
  }
  Wire.requestFrom(addr, (uint8_t)1);       // read one byte, then STOP
  return Wire.available() ? Wire.read() : -1;
}

void setup() {
  Serial.begin(115200);
  Wire.begin(21, 22);                       // SDA, SCL
  Serial.printf("WHO_AM_I = 0x%02X\n", readRegister(MPU6050_ADDR, WHO_AM_I));
}

void loop() {}
```

5.2.4.4 Scanning a bus

Because a missing device answers with a NACK, the controller can discover what is connected simply by addressing every possible target and noting which ones acknowledge. This **bus scan** is the first thing to run on any new I²C wiring.

```
#include <Wire.h>

void setup() {
  Serial.begin(115200);
  Wire.begin(21, 22);
  int found = 0;
  for (uint8_t addr = 0x08; addr <= 0x77; addr++) {
    Wire.beginTransmission(addr);
    if (Wire.endTransmission() == 0) {     // 0 = the address was acknowledged
      Serial.printf("Found device at 0x%02X\n", addr);
      found++;
    }
  }
  Serial.printf("%d device(s) found\n", found);
}

void loop() {}
```

5.2.4.5 Speed, clock stretching and multiple controllers

I²C runs at defined speeds: **100 kHz** (Standard-mode), **400 kHz** (Fast-mode) and **1 MHz** (Fast-mode Plus), with faster modes defined for special hardware [15]. Address bytes, acknowledgments and the slow rising edges make it much slower than SPI in practice, which is acceptable for sensors that produce a few readings per second. On the ESP32, `Wire.setClock(400000)` selects Fast-mode.

Open-drain outputs also make two advanced features possible:

- **Clock stretching.** A target that needs more time can hold SCL low after a byte. The controller waits until SCL is released.
- **Multiple controllers.** Two controllers can share a bus. If both start at once, each watches SDA while sending; the first one to send a 1 while the line reads 0 knows it has lost **arbitration** and backs off.

5.2.4.6 Strengths and limits

I²C needs only two wires for up to dozens of devices, confirms that each byte was received, and lets software find devices by scanning. Its costs are speed and fragility: it is half duplex, slower than SPI, sensitive to pull-up values and wire length, and a single misbehaving device can hold the whole bus low.

Try it on your kit: map your buses

This exercise uses the kit wiring from Stage 2 of Lab 1, and corresponds to the I²C and SPI lecture activity.

1. Run the bus scanner above. You should find exactly three devices: 0x60, 0x68 and 0x76. Write down what you see before going on.
2. Unplug the USB cable, remove the MPU-6050's SDA wire, and scan again. Which addresses remain? Why does removing one device not break the others?
3. Restore the wire and run the WHO_AM_I example. Confirm that it prints 0x68.
4. Set the Serial Monitor to 9600 baud while the code still uses 115200. Describe what you see, and explain it using Figure 5.2.
5. Write a line of your choice to the OLED. Which bus carried it, and which chip-select pin went low?

A guided, step-by-step version for a lecture session is Activity 3.

5.3 In depth (optional)

Note

Optional, in-depth explanation. This section goes beyond the rest of the chapter into the math and physics behind it. Nothing later in the book depends on it; skip it on a first reading.

5.3.1 Bit budgets and bus electrics

UART efficiency. An 8N1 frame sends 10 bits (start, 8 data, stop) for every byte, so the useful byte rate is baud/10: 11 520 bytes per second at 115 200 baud.

How much baud-rate error is tolerable? The receiver synchronizes on the start bit's falling edge and samples each bit in its middle. If the two clocks differ by a fraction ε, the sampling point drifts by ε bit-times per bit. By the stop bit, about 9.5 bit-times after the edge, the drift must stay under half a bit:

$$9.5\,\varepsilon < 0.5 \quad \Rightarrow \quad \varepsilon < 5.3\%. \tag{5.2}$$

That is the *combined* error of both ends, before any allowance for slow edges or noise, so designers aim for about 2% on each side. It explains why a 9600-baud monitor on a 115 200-baud stream produces garbage rather than a slowed-down message.

I²C pull-up resistors. I²C lines are open-drain: devices only pull them low, and a pull-up resistor R_p pulls them high through the bus capacitance C_b (wires plus pins). The rise is an RC charge, and the standard measures it from 30% to 70% of V_{DD}:

$$t_r = t_{70\%} - t_{30\%} = R_p C_b \left(\ln \frac{1}{0.3} - \ln \frac{1}{0.7} \right) = R_p C_b \ln \frac{7}{3} \approx 0.847\, R_p C_b. \tag{5.3}$$

Fast mode (400 kHz) allows $t_r \le 300$ ns, so $R_p \le 300\ \text{ns}/(0.847\, C_b)$; for $C_b = 100$ pF, $R_p \le 3.5$ kΩ. The resistor also cannot be too small, or a device cannot pull the line low enough: with a 3 mA sink and a 0.4 V low level, $R_p \ge (3.3 - 0.4)/0.003 \approx 970\ \Omega$. Breakout boards each add their own pull-ups (often 4.7 kΩ or 10 kΩ), and they combine in parallel, so three modules on one bus already give 1.6–3.3 kΩ.

Transfer time on each bus. An I²C byte takes 9 clock cycles (8 data bits plus acknowledge), so a 1024-byte display frame at 400 kHz takes at least

$$\frac{1024 \times 9}{400\,000} \approx 23\ \text{ms}, \tag{5.4}$$

capping the refresh rate near 40 frames per second before any other work. On SPI at 8 MHz, with no acknowledge bit, the same frame takes $1024 \times 8/8\,000\,000 \approx 1$ ms. That factor of twenty is why the kit's display uses SPI.

Exercise. The MPU-6050 delivers 14 bytes per sample (accelerometer, temperature, gyroscope). At 400 kHz, counting the address and register bytes, what is the highest sample rate the bus could sustain? What does that leave for the BMP280?

Sources for this section: [9], [15].

5.4 Design trade-offs

Table 5.2: Comparing the three buses.

	UART	SPI	I²C
Signal wires	2 (TX, RX)	3 shared + 1 CS per device	2 shared
Devices	2 (point-to-point)	1 controller, several peripherals	Many controllers and targets
Clock	None (asynchronous)	Controller's SCK	Controller's SCL
Duplex	Full	Full	Half
How a device is chosen	Not needed	Chip-select wire	7-bit address
Delivery confirmed?	No	No	Yes (ACK/NACK)
Typical speed	9.6–115.2 kbit/s (up to a few Mbit/s)	1–80 MHz clock	100 kHz–1 MHz clock
On the kit	Laptop console	OLED, LoRa radio	BMP280, MPU-6050, ATECC608A

A few rules of thumb follow from the table:

- **Talking to another computer or a module with its own processor** (a laptop, GPS receiver or cellular modem): UART.
- **Moving a lot of data quickly** (displays, radios, memory cards, cameras): SPI.
- **Reading many slow sensors with few pins**: I²C.
- **Running out of pins**: I²C for anything slow enough; a CS decoder or daisy-chain for SPI.
- **Long cables**: none of these three. Use a differential standard such as RS-485 or CAN, or move to a network link (Chapter 10.).

5.5 Common pitfalls

- **TX connected to TX.** UART lines cross over: TX goes to RX.
- **No common ground.** Two boards powered separately must share GND for any of these buses to work.
- **Baud-rate mismatch.** Garbled characters almost always mean the two sides disagree on speed.
- **7-bit versus 8-bit addresses.** Some datasheets give the address already shifted left with the read/write bit included. The ATECC608A, for example, is often listed as 0xC0, which is 0x60 in the 7-bit form the Arduino `Wire` library expects [19].
- **Two devices with the same address.** The scan shows one address, but the readings are nonsense. Move one device to its alternate address.
- **More than one CS low at once.** Two SPI peripherals drive CIPO together and both replies are corrupted. Set every CS pin high in `setup()` before using the bus.
- **The wrong SPI mode.** The device answers, but with shifted or meaningless bits. Check CPOL and CPHA in its datasheet.
- **Trusting the silkscreen.** Many SPI display modules label their data input "SDA" even though it is SPI data, not I²C. The kit's OLED is one of them: its "SDA" pin goes to the SPI COPI line (GPIO 23), not the I²C bus.
- **Using reserved pins.** On the ESP32, GPIO 6 to 11 connect to the flash chip and must never be used.

Advanced track

Seeing the bits. A logic analyzer captures the actual voltages on each wire and decodes them as UART, SPI or I²C frames. Inexpensive USB analyzers, used with open-source software such as PulseView/sigrok, are the fastest way to answer "is the problem in my code or my wiring?" Capture a bus scan and identify every START, address, ACK and STOP.

Logic levels. The ESP32 uses 3.3 V logic. Many older sensors and Arduino shields use 5 V. Connecting them needs a level shifter. For I²C, a bidirectional MOSFET shifter works because the bus is open drain; for SPI, use a dedicated buffer.

Bus recovery. If a controller resets in the middle of a read, a target may still be holding SDA low, waiting for clock pulses that never come. The standard fix is to toggle SCL manually up to nine times until SDA is released, then send a STOP.

Beyond these three. RS-485 carries UART-style data over a differential pair for hundreds of meters. CAN, used in vehicles and factories, adds arbitration and error handling on a differential bus. I3C is a newer successor to I²C with higher speeds, in-band interrupts and dynamic addressing.

Design review question. Your product needs a display updated 30 times per second, four environmental sensors and a radio, and the microcontroller has 10 free GPIO pins. Assign each device to a bus, count the pins, and justify any choice a reviewer might challenge.

5.6 Check your understanding

1. Why might a UART link fail if the two devices use different baud rates, even when the wiring is correct?
2. How many bytes per second can an 8N1 UART carry at 9600 baud?
3. Which of the three buses uses the fewest wires for five devices? How many signal wires would SPI need for the same five devices?
4. Which bus would you choose for high-speed communication with several peripherals, and what does it cost you in wiring?
5. Why must I²C outputs be open drain? Name two features of I²C that would be impossible otherwise.
6. A bus scan finds only one device at 0x68, but you connected an MPU-6050 and a real-time-clock chip that also uses 0x68. What happened, and how do you fix it?
7. In the SPI read listing, why does reading one register take two calls to `SPI.transfer()`?
8. Your OLED and your LoRa radio share an SPI bus. The radio works on its own, but stops working once the display code is added. Give two likely causes.

5.7 Further reading

- NXP Semiconductors, *I²C-bus Specification and User Manual* [15]. The authoritative reference, including timing, pull-up sizing and arbitration.
- Espressif, *ESP32 Technical Reference Manual* [9], chapters on UART, SPI and I²C controllers.
- SparkFun, *Serial Communication* [20]. A readable introduction to UART framing and wiring.
- The datasheets for the kit's devices: BMP280 [17], MPU-6050 [18], ATECC608A [19] and SX1276 [21].

6. Sensors

Learning objectives

After this chapter you should be able to:

1. Describe the signal chain from a physical quantity to a number in software.
2. Explain what makes a sensor "smart", and why most modern sensors are.
3. Classify sensors along several dimensions and read the key specifications on a datasheet.
4. Distinguish accuracy, precision and resolution, and explain calibration and drift.
5. Read the kit's pressure/temperature sensor and inertial measurement unit, and convert raw readings to physical units.
6. Apply simple filtering and sampling rules to make sensor data trustworthy.

6.1 Why it matters

A modern smartphone contains more than a dozen sensors. A fitness band carries an accelerometer, a gyroscope, an altimeter, an optical heart-rate sensor, skin-temperature and ambient-light sensors, and a satellite-positioning receiver. A modern car has well over a hundred: cameras, ultrasonic parking sensors, wheel-speed sensors, seat-occupancy sensors that tune the airbags, battery-temperature sensors, tire-pressure sensors and more.

Sensing is what puts the "things" into the Internet of Things. Every decision a device makes, and every chart on a dashboard, is only as good as the sensor readings behind it. This chapter explains how sensors work, how to read their specifications, and how to turn raw readings into numbers you can trust.

6.2 Concepts

6.2.1 What a sensor does

A **sensor** detects a physical quantity, or a change in one, and produces a corresponding signal that another device can use. Sensors can measure almost anything: temperature, pressure, light, sound, motion, magnetic field, humidity, flow, distance, or chemical and biological properties.

The heart of every sensor is a **transducer**: an element whose electrical properties change with the quantity being measured. A thermistor's resistance changes with temperature; a photodiode's current changes with light; a tiny silicon structure flexes under acceleration and changes its capacitance.

Between the transducer and your code there is a **signal chain** (Figure 6.1):

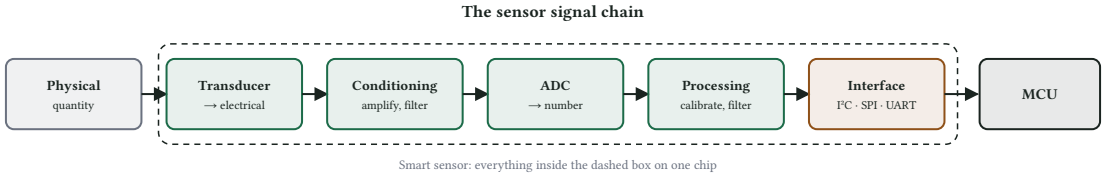


Figure 6.1: The sensor signal chain. In a smart sensor, everything inside the dashed box is on one chip.

1. The **transducer** converts the physical quantity to an electrical quantity.
2. **Conditioning** circuits amplify and filter the signal and make it suitable for conversion.
3. An **ADC** converts it to a number (Chapter 3.).
4. **Processing** applies calibration, compensation and filtering.
5. An **interface** (I²C, SPI or UART, Chapter 5.) delivers the result to the microcontroller.

6.2.2 Smart sensors

In the simplest sensors, such as a potentiometer or a thermistor, the microcontroller's own ADC does the conversion and your code does everything else. A **smart sensor** [1] integrates the whole chain on one small chip, with its own ADC, a small processor and a digital interface. Smart sensors bring several advantages:

- **Self-calibration.** Factory calibration coefficients are stored in the chip, and the sensor or its driver applies them automatically.
- **Communication.** The sensor reports over a digital bus, which is far less sensitive to noise than a small analog voltage on a long wire.
- **Local computation.** Many smart sensors average, filter, detect events (a tap, a fall, a step) or raise an interrupt when a threshold is crossed, so the main processor can sleep.
- **Multi-sensing.** Several sensing elements share one package. The kit's MPU-6050 combines an accelerometer, a gyroscope and a temperature sensor.

Both of the kit's environmental and motion sensors, the BMP280 and the MPU-6050, are smart sensors on the I²C bus.

6.2.3 Classifying sensors

Sensors can be grouped in several useful ways [1]:

Dimension	Options	Example
Power	**Active** (needs an external supply) or **passive** (generates its own signal)	A thermocouple is passive; an ultrasonic range finder is active
Intrusiveness	**Invasive** (part of the system it measures) or **non-invasive**	An implanted glucose sensor versus a camera
Contact	**Contact** or **non-contact**	A thermistor on a pipe versus an infrared thermometer
Reference	**Absolute** or **relative** (measured against a reference)	Absolute pressure versus differential pressure across a filter
Output	**Analog** or **digital**	A potentiometer versus the BMP280

6.2.4 Common sensor types

Sensor	Measures	Common technologies	Typical applications
Inertial (IMU)	Acceleration and rotation rate	MEMS accelerometers and gyroscopes	Activity tracking, drones, vibration monitoring
Tachometer	Rotational speed	Magnetic (Hall effect), optical	Motors, wheels, fans
Proximity	Presence of a nearby object	Capacitive, inductive, infrared, ultrasonic	Phones, parking aids, factory lines
Occupancy	People in a space	Passive infrared (PIR), ultrasonic, mm-wave radar	Lighting and HVAC control
Temperature / humidity	Heat, water vapor	Thermistor, RTD, thermocouple, solid-state	HVAC, cold chain, weather
Pressure	Force per area	Piezoresistive, capacitive MEMS	Weather, altitude, tire pressure

Sensor	Measures	Common technologies	Typical applications
Light	Illuminance	Photoresistor, photodiode	Automatic lighting, display brightness
Soil moisture	Water content	Capacitive, resistive	Irrigation control
Touch / force	Contact and pressure	Capacitive, resistive, piezoresistive	User interfaces, pressure mats
Gas	Concentrations of CO_2, CO, VOCs	Metal-oxide, NDIR, electrochemical	Air-quality monitoring

Most motion and pressure sensors today are **MEMS** (micro-electro-mechanical systems) devices: microscopic mechanical structures etched into silicon alongside their electronics. MEMS made precise motion sensing cheap enough for every phone and wearable.

6.2.5 Reading a sensor's specifications

A datasheet describes a sensor with a handful of terms that are easy to confuse:

- **Range**: the minimum and maximum values the sensor can measure. The BMP280 measures 300 to 1100 hPa.
- **Resolution**: the smallest change the sensor can report. A 16-bit reading across a ±2 g range has a resolution of about 0.06 mg.
- **Accuracy**: how close a reading is to the true value.
- **Precision** (or repeatability): how close repeated readings of the same value are to each other.
- **Sensitivity**: how much the output changes for a given change in the input, such as 16 384 counts per g.
- **Offset (bias)**: the reading when the true input is zero.
- **Drift**: slow change of offset or sensitivity over time or temperature.
- **Noise**: random variation in readings, often given as a noise density or an RMS value.
- **Response time and sampling rate**: how quickly the sensor follows a change, and how often it can produce a reading.

A sensor can be precise but inaccurate (readings tightly clustered around the wrong value) or accurate on average but imprecise (readings scattered around the right value). High resolution does not imply high accuracy: a sensor can report many digits, all of them wrong.

Calibration measures a sensor's offset and sensitivity against a known reference and corrects for them. Many smart sensors ship with factory calibration; others, such as gyroscopes, need a quick calibration at start-up, for example by averaging readings while the device is held still.

6.2.6 The kit's sensors

6.2.6.1 BMP280: pressure and temperature

The Bosch BMP280 measures barometric pressure and temperature [17]. On the kit it sits on the I²C bus at address 0x76. Internally it produces raw readings that must be combined with factory calibration coefficients stored in the chip. The Adafruit library handles this:

```
#include <Wire.h>
#include <Adafruit_BMP280.h>

Adafruit_BMP280 bmp;

void setup() {
  Serial.begin(115200);
  Wire.begin(21, 22);
  if (!bmp.begin(0x76)) {
    Serial.println("BMP280 not found: check wiring and address");
    while (true) delay(1000);
  }
}
```

```
void loop() {
  float tempC = bmp.readTemperature();
  float hPa   = bmp.readPressure() / 100.0f;      // Pa → hPa
  float altM  = bmp.readAltitude(1013.25f);       // sea-level reference pressure
  Serial.printf("%.2f °C  %.2f hPa  %.1f m\n", tempC, hPa, altM);
  delay(1000);
}
```

Pressure falls by roughly 12 Pa for every meter of height near sea level, so a pressure sensor can estimate altitude:

$$h \approx 44\,330\left(1 - \left(\frac{p}{p_0}\right)^{1/5.255}\right)\ \text{m} \tag{6.1}$$

where p_0 is the pressure at sea level. Because weather changes p_0 by tens of hectopascals, absolute altitude estimates are only rough unless you use the local sea-level pressure. *Changes* in altitude, such as climbing a flight of stairs, can be detected much more reliably, because the BMP280's relative accuracy is about ±0.12 hPa, or around ±1 m.

The temperature reading reflects the temperature of the chip, which is warmed slightly by its neighbors on the breadboard. Expect it to read a little above room temperature.

6.2.6.2 MPU-6050: acceleration and rotation

The InvenSense MPU-6050 is an **inertial measurement unit** (IMU) with a three-axis accelerometer and a three-axis gyroscope [18]. On the kit it is at I²C address 0x68.

- The **accelerometer** measures acceleration, including gravity. Lying flat and still, it reads about +1 g on its Z axis and 0 g on X and Y. Its default range is ±2 g, with a sensitivity of 16 384 counts per g.
- The **gyroscope** measures the rate of rotation around each axis. Its default range is ±250 °/s, with a sensitivity of 131 counts per °/s.

Reading the MPU-6050 directly shows how a register-based smart sensor works. It starts in sleep mode, so the first step is to wake it by writing 0 to its power-management register. Then all six motion readings, plus temperature, can be read in one 14-byte burst starting at register 0x3B:

```
#include <Wire.h>

constexpr uint8_t MPU = 0x68;

void writeReg(uint8_t reg, uint8_t value) {
  Wire.beginTransmission(MPU);
  Wire.write(reg);
  Wire.write(value);
  Wire.endTransmission();
}

void setup() {
  Serial.begin(115200);
  Wire.begin(21, 22);
  writeReg(0x6B, 0x00);                     // PWR_MGMT_1: wake up
}

void loop() {
  Wire.beginTransmission(MPU);
  Wire.write(0x3B);                         // ACCEL_XOUT_H
  Wire.endTransmission(false);              // repeated START
  Wire.requestFrom(MPU, (uint8_t)14);

  int16_t raw[7];                           // ax, ay, az, temp, gx, gy, gz
  for (int i = 0; i < 7; i++) {
    raw[i] = (Wire.read() << 8) | Wire.read();   // big-endian, two's complement
```

```
  }
  float ax = raw[0] / 16384.0f, ay = raw[1] / 16384.0f, az = raw[2] / 16384.0f;
  float gz = raw[6] / 131.0f;
  Serial.printf("a = (%.2f, %.2f, %.2f) g   gz = %.1f °/s\n", ax, ay, az, gz);
  delay(100);
}
```

From the accelerometer alone, the device's tilt can be estimated whenever it is not otherwise accelerating, because gravity then points straight down:

$$\text{pitch} = \operatorname{atan2}\left(-a_x, \sqrt{a_y^2 + a_z^2}\right), \qquad \text{roll} = \operatorname{atan2}(a_y, a_z) \tag{6.2}$$

6.2.7 Making readings trustworthy

6.2.7.1 Sampling

How often should you read a sensor? The **Nyquist criterion** says you must sample at more than twice the highest frequency present in the signal to capture it faithfully. Sampling too slowly causes **aliasing**: fast changes masquerade as slow ones. In practice, sample several times faster than the fastest change you care about. Room temperature changes over minutes; footsteps produce accelerations of a few hertz; machine vibration can reach kilohertz.

Sampling faster than necessary wastes energy and bandwidth, which matters for battery-powered devices.

6.2.7.2 Filtering

Real readings are noisy. Two simple filters cover most needs:

- A **moving average** of the last N readings reduces random noise by about $\sqrt{N}$, at the cost of delay.
- An **exponential moving average** needs only one stored value:

$$y_k = \alpha\, x_k + (1 - \alpha)\, y_{k-1}, \qquad 0 < \alpha \le 1 \tag{6.3}$$

A small α smooths heavily but responds slowly; a large α follows changes quickly but passes more noise.

```
float ema(float x) {
  static float y = NAN;
  constexpr float ALPHA = 0.2f;
  y = isnan(y) ? x : ALPHA * x + (1 - ALPHA) * y;
  return y;
}
```

6.2.7.3 Sensor fusion

Different sensors have complementary weaknesses. A gyroscope gives smooth, fast rotation estimates but drifts over time when integrated. An accelerometer gives a drift-free tilt reference but is disturbed by every bump. **Sensor fusion** combines them. The simplest example is a **complementary filter**:

$$\theta_k = \beta\,(\theta_{k-1} + \omega_k\,\Delta t) + (1 - \beta)\,\theta_{\text{accel},k}, \qquad \beta \approx 0.98 \tag{6.4}$$

It trusts the gyroscope over short time scales and the accelerometer over long ones. Phones, drones and wearables use more sophisticated versions, such as Kalman filters, but the idea is the same.

Try it on your kit: from raw counts to physics

This exercise uses Stage 2 of the kit build.

1. Read the BMP280 for one minute and compute the mean and standard deviation of the pressure. Then lift the kit from the floor to a table. Can you see the change?
2. Read the MPU-6050 with the board flat and still. Which axis reads about 1 g? Compute the gyroscope's offsets by averaging 200 samples, and subtract them from later readings.
3. Tilt the board and print pitch and roll from the accelerometer. Now shake it gently. What happens to the tilt estimate?
4. Apply an exponential moving average to the pitch with α = 0.1 and α = 0.5. Describe the trade-off you see.
5. Optional: implement the complementary filter and compare it with the accelerometer-only estimate while shaking the board.

6.3 In depth (optional)

Note

Optional, in-depth explanation. This section goes beyond the rest of the chapter into the math and physics behind it. Nothing later in the book depends on it; skip it on a first reading.

6.3.1 From voltage to physics

Quantization. An N-bit ADC with reference voltage V_{ref} divides the input range into 2^N steps of one **least significant bit**, $\text{LSB} = V_{\text{ref}}/2^N$. For the ESP32's 12-bit ADC over roughly 3.3 V, one LSB is about 0.8 mV. Rounding to the nearest step adds an error spread uniformly over ±½ LSB, whose standard deviation is

$$\sigma_q = \frac{\text{LSB}}{\sqrt{12}}, \tag{6.5}$$

which gives the ideal signal-to-quantization-noise ratio $\text{SQNR} \approx 6.02N + 1.76$ dB (74 dB for 12 bits). Real converters add their own noise and non-linearity; the ESP32's ADC behaves more like a 9- or 10-bit converter, its **effective number of bits** (ENOB).

Averaging. Averaging M independent readings reduces random noise by $\sqrt{M}$: 16 samples give a factor of 4, two extra effective bits. It does not remove systematic errors such as offset or non-linearity; those need calibration.

The potentiometer is a voltage divider. With the wiper at fraction x of the track, $V_{\text{out}} = x\,V_{DD}$ regardless of the total resistance, as long as the ADC draws negligible current. A 10 kΩ potentiometer has a worst-case source resistance of 2.5 kΩ (at mid-travel), low enough for the ADC to sample accurately.

Pressure to altitude. In the standard atmosphere, pressure falls with height. A widely used approximation for the lower atmosphere is

$$h = 44\,330\left[1 - \left(\frac{p}{p_0}\right)^{1/5.255}\right] \text{m}, \tag{6.6}$$

where p_0 is the pressure at sea level (1013.25 hPa by default). Near sea level, pressure falls about 12 Pa per metre, so the BMP280's resolution of a few pascals corresponds to tens of centimetres. Weather changes p_0 by several hectopascals a day, however, so absolute altitude needs a local reference, while *changes* in altitude (a floor climbed) are measured well.

Tilt from an accelerometer. At rest, an accelerometer measures only gravity. With the MPU-6050 at ±2 g, 16 384 counts represent 1 g. Pitch and roll follow from the gravity vector's components:

$$\theta_{\text{pitch}} = \operatorname{atan2}\left(-a_x, \sqrt{a_y^2 + a_z^2}\right), \qquad \phi_{\text{roll}} = \operatorname{atan2}(a_y, a_z). \tag{6.7}$$

Fusing gyro and accelerometer. The accelerometer's angle is correct on average but noisy; the gyroscope's integrated rate is smooth but drifts. A **complementary filter** combines them:

$$\theta_k = \alpha(\theta_{k-1} + \omega_k \, \Delta t) + (1 - \alpha) \, \theta_{\mathrm{acc},k}, \tag{6.8}$$

with α close to 1 (for example 0.98 at 100 Hz). It is a high-pass filter on the gyroscope and a low-pass filter on the accelerometer, with a crossover time constant $\tau = \alpha \, \Delta t / (1 - \alpha) \approx 0.5$ s.

Exercise. You average 64 ADC readings of a still potentiometer whose single readings have a standard deviation of 8 LSB. What standard deviation do you expect for the average, and how many effective bits did you gain?

Sources for this section: [17], [18], [22].

6.4 Design trade-offs

Decision	Considerations
Analog vs. digital (smart) sensor	Analog sensors are cheap and simple but sensitive to noise and wiring; digital sensors are calibrated and robust but cost more and need a bus.
Sampling rate	Faster sampling captures more detail but costs energy, processing and bandwidth.
On-sensor vs. on-MCU processing	Event detection on the sensor lets the MCU sleep; processing on the MCU is more flexible.
Filtering	More smoothing means less noise but more delay.
Accuracy class	Better accuracy costs money and often power. Buy only the accuracy the application needs.

6.5 Common pitfalls

- **Trusting the first reading.** Many sensors need a short warm-up or settling time after power-up or after waking.
- **Forgetting to wake the sensor.** The MPU-6050 starts in sleep mode and returns zeros until it is woken.
- **Wrong byte order or sign.** The MPU-6050 sends 16-bit two's-complement values, high byte first.
- **Confusing resolution with accuracy.** Many digits do not mean a correct value.
- **Self-heating.** A temperature sensor next to a voltage regulator or a radio measures the board, not the room.
- **Integrating gyroscope data without correction.** The angle drifts without bound.
- **Unit mistakes.** Pa versus hPa, g versus m/s², degrees versus radians.

> **i** Advanced track
>
> **Characterizing noise.** Log 10 000 gyroscope samples with the board still and plot their histogram and the Allan deviation. The Allan deviation separates white noise (which averaging removes) from bias instability (which it does not), and is the standard way to compare inertial sensors.
>
> **Sensor-hub architectures.** Phones and wearables offload continuous sensing to a low-power sensor hub or to the sensor itself, so that the main processor can sleep. Modern IMUs such as the Bosch BMI270 or the ST LSM6DSO include step counters, gesture detectors and even small machine-learning cores. Compare the energy cost of counting steps on the MCU with counting them on the sensor.
>
> **Calibration in the field.** Magnetometers need hard-iron and soft-iron calibration after they are mounted, because nearby metal and currents distort the field they measure. Research how a phone guides you through "figure-eight" calibration and what it is estimating.

6.6 Check your understanding

1. List the five stages of a sensor signal chain.
2. What makes a sensor “smart”? Give two advantages.
3. Explain the difference between accuracy, precision and resolution, using a dartboard analogy.
4. The MPU-6050 accelerometer reads 8 192 counts on its Z axis at the ±2 g range. What acceleration is that?
5. Why does a gyroscope-only angle estimate drift, and how does a complementary filter help?
6. You want to detect footsteps, which produce accelerations at up to about 5 Hz. What is the minimum sampling rate, and what would you choose in practice?
7. Your BMP280 reports an altitude of 140 m, but you know you are at 20 m. Give two likely reasons.

6.7 Further reading

- Bosch Sensortec, *BMP280 datasheet* [17].
- InvenSense, *MPU-6000/MPU-6050 Product Specification* and *Register Map* [18].
- A. Rayes and S. Salam [1], chapter on sensors and actuators, which this chapter’s classification follows.
- J. Fraden, *Handbook of Modern Sensors* [22], a comprehensive reference on sensing principles.

7. Actuators and the Observe–Decide–Act Loop

Learning objectives

After this chapter you should be able to:

1. Define an actuator and classify actuators by motion, output, power and energy type.
2. Explain the Observe–Decide–Act (ODA) loop and identify its parts in a real device.
3. Drive loads safely from a microcontroller, using transistors, separate supplies, diodes and capacitors where needed.
4. Control a hobby servo, a buzzer and a display from the ESP32.
5. Implement simple control strategies: on/off with hysteresis, and proportional control.
6. Choose safe default states for actuators when something fails.

7.1 Why it matters

Sensors let a device *observe* the world. Actuators let it *change* the world: open a valve, move a robot arm, sound an alarm, light a warning, or display a message. An IoT system that only senses is a monitoring system. One that also acts becomes a control system, and with that comes responsibility. A misread temperature produces a wrong chart; a misfired actuator can flood a basement or unlock a door.

This chapter covers the main families of actuators, how to drive them without damaging the microcontroller, and the decision loop that connects sensing to action.

7.2 Concepts

7.2.1 What an actuator is

An **actuator** converts a control signal, usually a small electrical signal or a digital command, plus a source of energy into a physical effect: motion, force, light, sound, heat or an image. Actuators are the natural complement to sensors. Many devices contain both: a smart thermostat senses temperature and drives a relay that turns the furnace on.

7.2.2 The Observe–Decide–Act loop

IoT actuation follows a simple cycle, the **Observe–Decide–Act** (ODA) loop described by Rayes and Salam [1] (Figure 7.1):

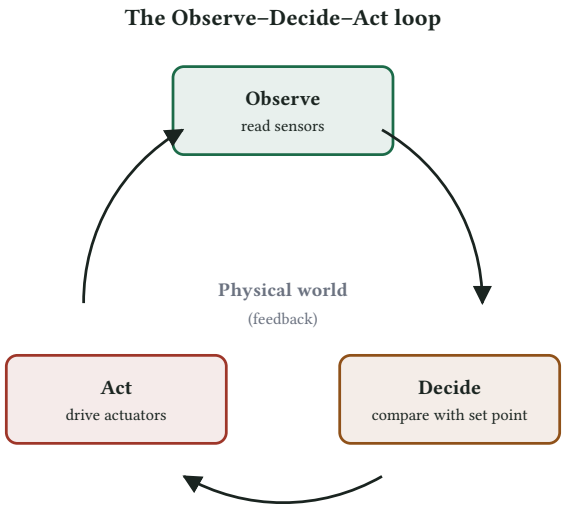


Figure 7.1: The Observe–Decide–Act loop. The physical effect of each action changes what the device observes next.

1. **Observe**: read sensors, or receive data from other devices.
2. **Decide**: compare the observations with a goal (a **set point**) and choose an action. The decision can happen on the device, at a nearby gateway or in the cloud.
3. **Act**: drive the actuators. The action changes the physical world, which changes the next observation.

When the decision uses the result of earlier actions, the loop is a **closed-loop** (feedback) control system. A thermostat is closed-loop: it measures the temperature its furnace has produced. A lawn sprinkler on a timer is **open-loop**: it waters on schedule whether or not the soil is already wet.

Where the decision is made matters. A loop closed on the device can respond in milliseconds and keeps working when the network fails. A loop closed in the cloud can use more data and more computation, but inherits the network's delay and failures. Chapter 18. returns to this choice.

7.2.3 Classifying actuators

Dimension	Options
Type of motion	None (static), linear, rotary; one, two or three axes
Output	**Binary** (on/off, open/closed) or **continuous** (position, speed, brightness)
Power	Micro-power (an LED), low power (a hobby servo), high power (an industrial motor)
Energy source	Electrical, hydraulic, pneumatic, thermal, manual
Application	Consumer, automotive, healthcare, industrial, agricultural

Some binary actuators are **bistable**: they hold their last state without power. A latching relay or a latching solenoid valve stays where it was put, which saves energy in battery-powered devices but means a power failure leaves them in an unknown state.

7.2.4 Families of actuators

- **Electric motors** convert electrical energy into rotation. *DC motors* spin continuously, with speed set by voltage or PWM. *Servo motors* add a gearbox, a position sensor and a controller so they move to a commanded angle. *Stepper motors* move in precise steps and hold their position without feedback.

- **Linear actuators** convert rotation into straight-line motion, typically with a lead screw.
- **Solenoids** are electromagnets that pull a metal plunger. They drive door locks and valves.
- **Relays** are switches operated by an electromagnet (or by a semiconductor, in solid-state relays). They let a low-power circuit switch a high-power one, such as a mains-powered heater.
- **Hydraulic actuators** use pressurized liquid. By Pascal's law, a pressure applied to a confined liquid is transmitted throughout it, so a small piston can drive a large one with great force. They power excavators and aircraft control surfaces.
- **Pneumatic actuators** use compressed air in the same way, trading force for speed and cleanliness. They are common in factory automation.
- **Non-mechanical actuators** produce information rather than motion: lights, displays and speakers.

7.2.5 Non-mechanical actuators

7.2.5.1 Lights

LEDs are the most common actuator of all. They can be binary (a status light) or continuous (dimmed with PWM, Chapter 3.). A traffic light is a familiar example of a binary, stable-state output, and is the subject of Lab 2.

7.2.5.2 Displays

A display converts electrical signals into an image. Common technologies are **LCD** (a backlight shining through liquid crystals), **OLED** (each pixel emits its own light, so black pixels use no power), **LED matrices**, and **e-paper** (which holds an image with no power at all, ideal for labels and signs that change rarely).

The kit's display is a 128 × 64-pixel monochrome OLED driven by an SSD1306 controller over SPI (Chapter 5.). Like most small displays, it keeps a frame buffer in the microcontroller's RAM: your code draws into the buffer, then sends the whole buffer to the display in one transfer.

```
#include <SPI.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include "iot_pins.h"

Adafruit_SSD1306 oled(128, 64, &SPI, IoTPins::OLED_DC, IoTPins::OLED_RES,
IoTPins::OLED_CS);

void setup() {
  oled.begin(SSD1306_SWITCHCAPVCC);
  oled.clearDisplay();
  oled.setTextSize(2);
  oled.setTextColor(SSD1306_WHITE);
  oled.setCursor(0, 0);
  oled.println("Hello,");
  oled.println("IoT!");
  oled.display();                // nothing appears until the buffer is sent
}

void loop() {}
```

7.2.5.3 Sound

A **speaker** moves a diaphragm with a coil in a magnetic field; it needs an amplifier to be driven from a microcontroller. A **piezo buzzer** is simpler: a ceramic disc that flexes when a voltage is applied. A *passive* piezo, like the kit's, makes a tone at whatever frequency you drive it with; an *active* buzzer contains its own oscillator and only beeps at one pitch.

```
tone(IoTPins::BUZZER, 880, 200);   // 880 Hz for 200 ms (uses an LEDC channel)
```

7.2.6 Driving loads safely

A microcontroller pin can supply only a few milliamperes at 3.3 V. Most actuators need more current, a higher voltage, or both. Connecting them directly damages the pin or resets the board. The standard techniques are:

- **Use a transistor or MOSFET as a switch.** The GPIO pin drives the transistor's input; the transistor switches the load's current from a separate supply. Logic-level MOSFETs are the usual choice for DC loads.
- **Use a separate supply for motors**, and **join the grounds**. The servo on the kit runs from its own 5 V rail fed from the board's VIN pin, which carries USB power before the 3.3 V regulator. The servo's signal wire still comes from a GPIO pin, so the two circuits must share ground.
- **Add a flyback diode across inductive loads.** Motors, relays and solenoids are coils. When the current through a coil is switched off, it produces a voltage spike that can destroy the transistor. A diode across the coil gives that energy a safe path.
- **Add a bulk capacitor near motors.** A servo draws large current pulses when it starts moving, which can make the supply voltage dip and reset the ESP32. The kit puts a 470 µF capacitor across the servo rail to supply those pulses locally.
- **Never connect the 5 V servo rail to the 3.3 V rail.** The ESP32's pins are not 5 V tolerant.

7.2.7 Controlling a servo

A hobby servo reads a pulse every 20 ms (50 Hz). The **width** of the pulse sets the angle: typically about 1.0 ms for one end of the travel, 1.5 ms for the center and 2.0 ms for the other end, although many servos accept a wider range of roughly 0.5 to 2.5 ms. The ESP32's LEDC peripheral generates these pulses:

```
// Arduino-ESP32 core 2.x API
constexpr int SERVO_CH = 2;           // channel 0 is used by tone()
constexpr int SERVO_HZ = 50;          // one pulse every 20 ms
constexpr int SERVO_BITS = 16;        // 65 536 steps per 20 ms period

void servoWriteUs(uint32_t us) {
  uint32_t duty = (uint32_t)((uint64_t)us * (1u << SERVO_BITS) / 20000u);
  ledcWrite(SERVO_CH, duty);
}

void servoWriteDeg(float deg) {
  deg = constrain(deg, 0.0f, 180.0f);
  servoWriteUs(500 + (uint32_t)(deg / 180.0f * 2000.0f));   // 0.5–2.5 ms
}

void setup() {
  ledcSetup(SERVO_CH, SERVO_HZ, SERVO_BITS);
  ledcAttachPin(IoTPins::SERVO, SERVO_CH);
  servoWriteDeg(90);                  // start at the center
}

void loop() {}
```

Libraries such as ESP32Servo wrap this, but writing it once shows that a servo command is nothing more than a carefully timed PWM signal.

7.2.8 Deciding: simple control strategies

The "decide" step can be as simple as a threshold or as complex as a machine-learning model. Three strategies cover most IoT devices:

- **On/off control** switches the actuator fully on or off around a set point. Without care it **chatters**: a heater near its set point switches on and off many times a second. **Hysteresis** fixes this by using two thresholds: turn the heater on below 20.5 °C and off above 21.5 °C.

```
void controlHeater(float tempC) {
  static bool heaterOn = false;
  if (!heaterOn && tempC < 20.5f) heaterOn = true;
  if ( heaterOn && tempC > 21.5f) heaterOn = false;
  digitalWrite(HEATER_PIN, heaterOn ? HIGH : LOW);
}
```

- **Proportional control** sets the output in proportion to the **error**, the difference between the set point and the measurement: $u = K_p\,(r - y)$. It responds smoothly, but alone it usually settles slightly off target.
- **PID control** adds an integral term, which removes that steady error, and a derivative term, which damps overshoot. PID controllers run inside most motors, drones and industrial processes.

Every control loop must also run **fast enough**: if the decision step is slower than the physical process it controls, the loop becomes sluggish or unstable.

7.2.9 Fail-safe design

Before writing control code, decide what each actuator should do when something goes wrong: the sensor fails, the network drops, the firmware crashes or the power fails. A **fail-safe** state is one that causes the least harm. A heater should fail *off*; a door lock on a fire exit should fail *unlocked*; a pump protecting a basement from flooding might need to fail *on*. Watchdog timers, sensor plausibility checks and command timeouts are how software enforces those choices.

Try it on your kit: an Observe–Decide–Act loop

This exercise corresponds to the Observe–Decide–Act lecture activity and uses Stage 3 of the kit build (the servo rail).

1. **Observe**: read the potentiometer and smooth it with an exponential moving average.
2. **Decide**: map the reading to a servo angle between 0° and 180°.
3. **Act**: move the servo, and show the angle on the OLED.
4. Add a rule: if the angle exceeds 150°, sound the buzzer and light LED 1. Add hysteresis so the alarm does not chatter near the threshold.
5. Unplug the potentiometer's wiper wire (with the USB cable disconnected first, then reconnect power). What does the servo do? Design a plausibility check and a fail-safe position for this case.

A guided, step-by-step version for a lecture session is Activity 4.

7.3 In depth (optional)

Note

Optional, in-depth explanation. This section goes beyond the rest of the chapter into the math and physics behind it. Nothing later in the book depends on it; skip it on a first reading.

7.3.1 PWM, inductors and capacitors

What PWM delivers. A PWM signal switching between 0 and V_{DD} with duty cycle D has an average value $\bar{V} = D\,V_{DD}$. Loads that respond slowly compared with the period (an LED seen by an eye, a motor with inertia, a heater) respond to the average. A servo is different: it reads the *width* of each pulse as a position command, not the average.

Resolution versus frequency. The ESP32's LEDC peripheral counts a source clock (80 MHz by default). With b bits of resolution, the counter wraps every 2^b clock ticks, so

$$f_{\mathrm{PWM,max}} = \frac{f_{\mathrm{clk}}}{2^b}. \tag{7.1}$$

At 16 bits that is $80 \times 10^6 / 65\,536 \approx 1.2$ kHz, comfortably above a servo's 50 Hz. A 20 kHz motor drive (above the range of hearing) leaves only $\log_2(80 \times 10^6 / 20 \times 10^3) \approx 12$ bits.

Why motors need flyback diodes. A motor or relay coil is an inductor, whose voltage is $V = L\,di/dt$. Switching off 200 mA through 10 mH in 1 µs would demand $10^{-2} \times 0.2 / 10^{-6} = 2000$ V; in practice the voltage spikes until something breaks down, often the transistor. A diode across the coil gives the current a path to decay slowly instead.

Sizing a reservoir capacitor. When the servo starts moving, it draws a current pulse faster than the USB supply and its cable can respond. A capacitor across the rail supplies that pulse. If it must supply current I for time Δt while the rail sags by at most ΔV,

$$C \geq \frac{I\,\Delta t}{\Delta V}. \tag{7.2}$$

A 0.5 A step that the supply takes 0.5 ms to catch up with, allowing 0.5 V of sag, needs $0.5 \times 0.5 \times 10^{-3} / 0.5 = 500$ μF, the order of the kit's 470 µF. A stalled servo draws its high current for much longer; no practical capacitor covers that, which is why the servo has its own 5 V rail.

Hysteresis. An alarm that turns on above 150° and off below 150° chatters when noise straddles the threshold. Use two thresholds, on above θ_{on} and off below $\theta_{\text{off}} < \theta_{\text{on}}$. The gap should exceed the peak-to-peak noise; with ±2° of noise, a 5° gap is enough.

Exercise. Your PWM drives an LED at 1 kHz with 10-bit resolution. What is the smallest change in average current you can make, if full brightness is 6 mA? Why might the LED still look non-linear across the range?

Sources for this section: [9], [23].

7.4 Design trade-offs

Decision	Considerations
Servo vs. stepper vs. DC motor	Servos are simple for positions within 180°; steppers give precise open-loop positioning; DC motors give continuous rotation and speed.
Relay vs. MOSFET	Relays switch AC and isolate circuits but are slow, noisy and wear out; MOSFETs are fast, silent and efficient for DC loads.
Bistable vs. monostable	Bistable actuators save energy but have no natural fail-safe state on power loss.
Local vs. remote decisions	Local loops are fast and survive network failures; remote loops can use more data.
On/off vs. proportional control	On/off is simple and suits slow processes; proportional or PID control is needed for precise, fast processes.

7.5 Common pitfalls

- **Powering motors from a GPIO pin or the 3.3 V rail.** The board browns out and resets whenever the motor moves.
- **No common ground** between the motor supply and the microcontroller. The control signal has no reference.
- **No flyback diode** on relays and solenoids.
- **Forgetting `display()`** on the OLED: drawing commands only change the frame buffer.
- **PWM channel conflicts.** `tone()` and the servo fighting over the same LEDC channel or timer produce erratic movement.
- **Chattering** from on/off control without hysteresis.
- **No fail-safe state**, so a crash leaves an actuator in its last, possibly dangerous, position.

> **i** Advanced track
>
> **Tuning a PID loop.** Build a simple closed loop in which the servo should point at a target angle measured by the MPU-6050 mounted on the servo horn. Implement proportional control, observe the steady-state error, then add integral and derivative terms. Document how you tuned K_p, K_i and K_d, and what limited the loop's speed.
>
> **Actuator safety standards.** Functional-safety standards such as IEC 61508 (general) and ISO 26262 (automotive) require a hazard analysis, defined safe states and evidence that the system reaches them. Choose an IoT actuator (a smart lock, a connected oven, an insulin pump) and write a one-page hazard analysis: what can go wrong, what is the safe state, and how is it enforced?

7.6 Check your understanding

1. Name the three steps of the ODA loop and give an example of each for a smart irrigation controller.
2. What is the difference between open-loop and closed-loop control?
3. Why does the kit's servo have its own 5 V rail and a 470 µF capacitor?
4. What pulse width would you send for a servo's center position, and how often?
5. Why must a relay coil have a flyback diode?
6. A heater controlled at exactly 21 °C switches on and off several times per second. What is happening, and how do you fix it?
7. What should a smart door lock do when its battery is about to run out? Justify your answer.

7.7 Further reading

- A. Rayes and S. Salam [1], on the ODA loop and the classification of actuators.
- K. J. Åström and R. M. Murray, *Feedback Systems* [23], a free textbook on control.
- Adafruit, *Adafruit GFX Graphics Library* guide [24], for drawing text and graphics on small displays.

III Part III — Connecting Things

8 Communication Models: OSI and TCP/IP . . . 79
8.1 Why it matters . . . 79
8.2 Concepts . . . 79
8.3 In depth (optional) . . . 83
8.4 Design trade-offs . . . 84
8.5 Common pitfalls . . . 84
8.6 Check your understanding . . . 85
8.7 Further reading . . . 85

9 IoT Network Requirements and the Protocol Stack . . . 87
9.1 Why it matters . . . 87
9.2 Concepts . . . 87
9.3 In depth (optional) . . . 90
9.4 Design trade-offs . . . 90
9.5 Common pitfalls . . . 91
9.6 Check your understanding . . . 91
9.7 Further reading . . . 91

10 The Link Layer: Short and Long Range . . . 93
10.1 Why it matters . . . 93
10.2 Concepts . . . 93
10.3 In depth (optional) . . . 99
10.4 Design trade-offs . . . 100
10.5 Common pitfalls . . . 100
10.6 Check your understanding . . . 101
10.7 Further reading . . . 101

11 The Internet Layer: 6LoWPAN and RPL . . . 103
11.1 Why it matters . . . 103
11.2 Concepts . . . 103
11.3 In depth (optional) . . . 106
11.4 Design trade-offs . . . 107
11.5 Common pitfalls . . . 107
11.6 Check your understanding . . . 108
11.7 Further reading . . . 108

12 Application Protocols: HTTP, CoAP, MQTT and More . . . 109
12.1 Why it matters . . . 109
12.2 Concepts . . . 109
12.3 In depth (optional) . . . 115
12.4 Design trade-offs . . . 116
12.5 Common pitfalls . . . 116
12.6 Check your understanding . . . 117
12.7 Further reading . . . 117

13 Interoperability and Standards . . . 119
13.1 Why it matters . . . 119
13.2 Concepts . . . 119
13.3 In depth (optional) . . . 122
13.4 Design trade-offs . . . 123
13.5 Common pitfalls . . . 123
13.6 Check your understanding . . . 124
13.7 Further reading . . . 124

8. Communication Models: OSI and TCP/IP

Learning objectives

After this chapter you should be able to:

1. Explain what a network protocol is and why networks are organized in layers.
2. Describe the seven layers of the OSI model and the job of each.
3. Map the OSI model onto the four-layer TCP/IP model actually used on the internet.
4. Trace a message through encapsulation and decapsulation, naming the data unit at each layer.
5. Explain the four kinds of address in TCP/IP (physical, logical, port, name) and how NAT, DHCP and DNS use them.
6. Choose between TCP and UDP for an IoT application, and connect the ESP32 to a Wi-Fi network and a web server.

8.1 Why it matters

The serial buses in Chapter 5. connect chips a few centimeters apart, on one board, under one controller. They cannot scale to billions of devices made by thousands of companies, spread across the planet and connected by every kind of medium from fiber to radio.

The internet solves that problem with **layering**: each layer provides one well-defined service to the layer above, and hides how it is done. A temperature reading from your kit can travel over Wi-Fi, Ethernet, fiber and a cellular link without your code knowing or caring. Understanding the layers is what lets you reason about where a problem is when "the device can't reach the server", and what lets you choose the right protocols for an IoT system in the chapters that follow.

8.2 Concepts

8.2.1 Protocols and layers

A **network protocol** defines the format and order of the messages exchanged between two or more communicating entities, and the actions taken when a message is sent or received, or when some other event happens (such as no reply arriving in time).

Human protocols work the same way. Taking a plane trip involves a sequence of layered steps: you buy a ticket, check your bag, pass security and board the plane; on arrival the steps run in reverse. Each step relies on the one below it and serves the one above. The ticket counter does not care how the plane flies.

Network designers organize protocols the same way, into a **protocol stack**. Each layer:

- offers a **service** to the layer above (for example, "deliver this message to that application");
- uses the service of the layer below;
- talks, conceptually, to the **same layer** on the other device, using that layer's protocol.

A layer can be implemented in hardware, in software, or in both.

8.2.2 The OSI reference model

In the late 1970s and early 1980s, the International Organization for Standardization (ISO) defined the **Open Systems Interconnection** (OSI) model: seven layers meant to describe any communication system, independent of technology (Table 8.1).

Table 8.1: The seven layers of the OSI model.

#	Layer	Responsibility	Data unit	Examples
7	Application	Services to the user's application	Message	HTTP, MQTT, SMTP
6	Presentation	Translation, compression, encryption	Message	Character encoding, TLS (partly)
5	Session	Opening, managing and closing dialogs	Message	Session setup and checkpoints
4	Transport	Process-to-process delivery	Segment (TCP) / datagram (UDP)	TCP, UDP
3	Network	Host-to-host delivery across networks: logical addressing and routing	Packet	IPv4, IPv6
2	Data link	Hop-to-hop delivery: framing, physical addressing, access to a shared medium	Frame	Ethernet, Wi-Fi MAC, BLE link layer
1	Physical	Moving individual bits over a medium	Bit	Voltages, light pulses, radio modulation

Some details of each layer:

- The **physical layer** defines connectors, cables, how bits are represented as signals, the data rate and the line configuration.
- The **data link layer** moves frames from one node to the next. It adds hardware (MAC) addresses, detects errors, and controls access to a shared medium so that devices do not all talk at once.
- The **network layer** delivers packets from the source host to the destination host, possibly across many networks. It provides logical addresses and **routing**.
- The **transport layer** delivers messages from one *process* to another. It adds **port numbers** to identify the application, and may provide segmentation and reassembly, flow control, error control and connection management.
- The **session layer** controls the dialog between two applications, keeping it open long enough to finish and closing it afterwards.
- The **presentation layer** makes data understandable: it translates between data representations, compresses data, and encrypts it.
- The **application layer** provides services that applications use directly, such as transferring web pages or publishing sensor readings.

8.2.3 Encapsulation

As data moves down the stack on the sender, each layer adds its own **header** (and sometimes a trailer) carrying the information its peer needs. This is **encapsulation**. On the receiver, each layer reads and removes its header and passes the rest up: **decapsulation** (Figure 8.1).

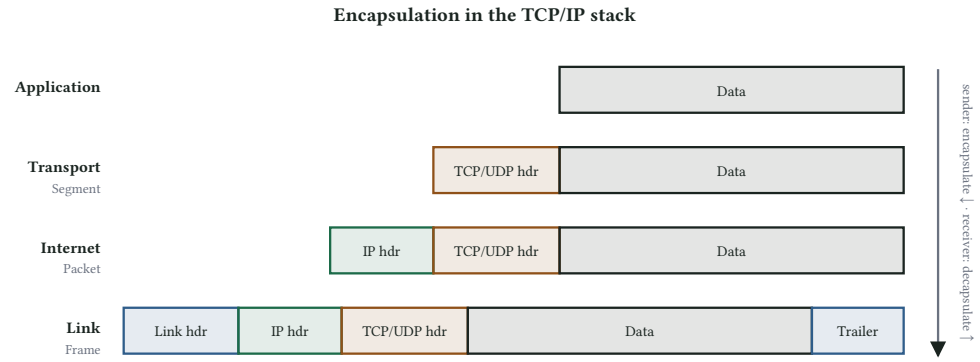


Figure 8.1: Encapsulation: each layer wraps the data from the layer above with its own header.

A useful end-to-end example: when you send an email, the application chooses a protocol (SMTP) and passes the message down; it is segmented by the transport layer, placed in packets by the network layer, framed by the data link layer and sent as bits by the physical layer. At every router along the way, the lower layers are unwrapped and rewrapped for the next link. At the destination, the layers are unwrapped in reverse until the message reaches the recipient's email application.

8.2.4 TCP/IP: the internet that won

The OSI model was developed slowly, through committees that included large telephone companies and computer vendors. Meanwhile, researchers funded by the US Defense Advanced Research Projects Agency (DARPA) built a simpler, working protocol suite, **TCP/IP**, and released it freely. By the 1990s, TCP/IP had become the de facto standard, and "the Internet" meant the network of networks that runs it [25].

OSI survives as a **vocabulary**: engineers still say "a layer-2 switch" or "a layer-7 load balancer". The protocols in use, however, follow the four-layer TCP/IP model (Table 8.2).

Table 8.2: The TCP/IP model and its relationship to OSI.

TCP/IP layer	Corresponding OSI layers	Main protocols	Main device
Application	5, 6, 7	HTTP, MQTT, CoAP, DNS, TLS	End hosts
Transport	4	TCP, UDP	End hosts
Internet	3	IPv4, IPv6, ICMP	Routers
Link (network access)	1, 2	Ethernet, Wi-Fi, BLE, 802.15.4	Switches, access points

TCP/IP has no separate session or presentation layers. If an application needs those services, such as encryption or compression, the application (or a library it uses, like TLS) provides them.

8.2.5 Four kinds of address

TCP/IP uses four kinds of address, each solving a different problem:

1. **Physical (MAC) addresses** identify a network interface on one link, such as the ESP32's Wi-Fi interface (for example `24:6F:28:AB:CD:EF`). They are only meaningful on the local link, and **change at every hop** as a packet is re-framed.
2. **Logical (IP) addresses** identify a host on the internet, such as `192.168.0.17` (IPv4) or `2001:db8::17` (IPv6). They normally stay the same from source to destination.

3. **Port numbers** (16-bit, 0–65535) identify a process or service on a host. Process IDs cannot do this job: they are local to one machine, and two machines may use the same ones. Ports 0–1023 are reserved for **well-known services**: DNS on 53, HTTP on 80, HTTPS on 443. MQTT uses 1883, or 8883 with TLS.
4. **Names**, such as host names and URLs, are what people and configuration files use. The **Domain Name System** (DNS) translates names into IP addresses.

Why not use MAC addresses everywhere? MAC addresses are flat: they say nothing about where a device is. Routing on them would require every router to know the location of every device in the world. IP addresses are **hierarchical**: their prefix identifies a network, so a router only needs to know which way to send each network's traffic.

8.2.6 How a device joins a network

When your ESP32 connects to Wi-Fi, three supporting protocols do work you never see:

- **DHCP** (Dynamic Host Configuration Protocol) gives the device an IP address, a subnet mask, a default gateway and DNS servers when it joins the network.
- **ARP** (Address Resolution Protocol) finds the MAC address that belongs to an IP address on the local network, so frames can be addressed. IPv6 uses Neighbor Discovery for the same job.
- **DNS** turns a name such as `broker.example.org` into an IP address.

8.2.7 NAT

There are only about 4.3 billion IPv4 addresses, far fewer than the number of connected devices. Most homes and campuses therefore use **private addresses** inside their network (such as `192.168.x.x` or `10.x.x.x`) and a single public address on the router.

The router performs **network address translation** (NAT). When a device sends a packet out, the router replaces its private source address and port with the router's public address and a new port, and records the mapping in a translation table. When a reply arrives at that public port, the router looks up the table and forwards it to the right device.

NAT has a major consequence for IoT: **devices behind NAT can easily start connections outward, but nothing outside can start a connection to them.** This is why most IoT devices connect *out* to a cloud service or broker and keep that connection open, rather than running a server that the outside world connects to. IPv6, with its vast address space, removes the need for NAT, although firewalls usually still block unsolicited inbound connections.

8.2.8 TCP and UDP

The transport layer offers two main services:

Table 8.3: Comparing the two main transport protocols.

	TCP	UDP
Connection	Connection-oriented (a handshake first)	Connectionless
Delivery	Reliable, in order, with retransmission	Best effort: datagrams may be lost, duplicated or reordered
Flow and congestion control	Yes	No
Header size	20 bytes or more	8 bytes
Typical IoT uses	HTTP, MQTT, firmware downloads	DNS, CoAP, time sync (NTP), streaming

TCP's reliability costs time and energy: a connection needs a three-way handshake before any data flows, and every segment must be acknowledged. For a sensor that sends one small reading every ten minutes, UDP (often with a lightweight application-level acknowledgment, as in CoAP, Chapter 12.) can be far more efficient. For a firmware update, where every byte must arrive, TCP is the obvious choice.

8.2.9 TCP/IP on the ESP32

The ESP32's Wi-Fi driver and the lwIP TCP/IP stack implement all four layers. From the Arduino framework, joining a network and making an HTTP request takes a few lines:

```
#include <WiFi.h>
#include <HTTPClient.h>
#include "secrets.h"                    // WIFI_SSID, WIFI_PASSWORD, SERVER_BASE_URL

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);                  // station (client) mode
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD); // link layer: associate with the access point
  while (WiFi.status() != WL_CONNECTED) {
    delay(250);
    Serial.print('.');
  }
  // DHCP has now given us an IP address, gateway and DNS server
  Serial.printf("\nIP %s  gateway %s  MAC %s  RSSI %d dBm\n",
                WiFi.localIP().toString().c_str(),
                WiFi.gatewayIP().toString().c_str(),
                WiFi.macAddress().c_str(),
                WiFi.RSSI());

  HTTPClient http;                                  // application layer
  http.begin(String(SERVER_BASE_URL) + "/ping");    // DNS lookup, TCP connect
  int status = http.GET();
  Serial.printf("HTTP status %d: %s\n", status, http.getString().c_str());
  http.end();
}

void loop() {}
```

Keep credentials in a separate `secrets.h` file that is excluded from version control.

> Try it on your kit: a classroom poll over HTTP
>
> This exercise corresponds to the classroom-poll lecture activity. Your kit acts as an HTTP client and submits an answer to a poll server; the instructor's dashboard shows everyone's answers.
>
> 1. Connect to Wi-Fi and print your IP address, gateway, MAC address and signal strength. Which of these would change if you moved to a different building? Which would not?
> 2. Use the buttons to choose an answer and send it to the server with an HTTP POST. Show the server's response on the OLED.
> 3. For one request, identify what each TCP/IP layer contributes: the link (Wi-Fi), internet (IP addresses), transport (ports) and application (HTTP method, path, body).
> 4. Your kit has a private IP address. Explain how the server's reply finds its way back to it.
>
> A guided, step-by-step version for a lecture session is Activity 5.

8.3 In depth (optional)

> Note
>
> **Optional, in-depth explanation.** This section goes beyond the rest of the chapter into the math and physics behind it. Nothing later in the book depends on it; skip it on a first reading.

8.3.1 Overhead and delay, counted

How much of a message is payload? Each layer adds its own header. Consider a 4-byte temperature reading sent in an HTTP POST over Wi-Fi with IPv4:

Layer	Bytes added (typical)
HTTP request line and headers	≈ 150
TCP header	20 (often 32 with options)
IPv4 header	20
802.11 MAC header, LLC/SNAP, FCS	≈ 36

The payload is 4 of roughly 230 bytes, an efficiency $\eta = \text{payload}/\text{total}$ under 2%. Before it, TCP's three-way handshake exchanged three more packets, and the response and connection teardown add more. For a 4-byte reading, protocol choice dominates the energy bill: Chapter 12 compares MQTT and CoAP, which shrink the application header to a few bytes and avoid a new connection per reading.

Where the delay comes from. The delay of one packet across one link is the sum of four parts:

$$d = d_{\text{proc}} + d_{\text{queue}} + \underbrace{\frac{L}{R}}_{d_{\text{trans}}} + \underbrace{\frac{s}{v}}_{d_{\text{prop}}}, \tag{8.1}$$

where L is the packet length in bits, R the link rate, s the distance and v the signal speed (about 2×10^8 m/s in cable). A 230-byte frame at 1 Mb/s takes 1.8 ms to transmit; it travels 300 m of air in 1 µs. On short links, **transmission and queueing dominate**; across continents, propagation does (about 40 ms each way across the United States in fibre).

Round trips set the floor. A new HTTPS request needs one round trip for TCP's handshake, one for TLS 1.3's, and one for the request itself: at least three round-trip times (RTT). With a 50 ms RTT to a cloud region, the minimum is 150 ms before the server even processes the reading. Keeping connections open, or using UDP-based protocols, removes most of it.

The bandwidth-delay product. A sender can only keep a link busy if it has $R \times \text{RTT}$ bits in flight. At 10 Mb/s and 50 ms RTT that is 62.5 kB; a small device with a 4 kB TCP window gets at most $4096 \times 8/0.05 \approx 655$ kb/s, whatever the link speed.

Exercise. In Activity 5's extension you measured the POST round-trip time. Using the formula above, estimate how much of it is Wi-Fi transmission time, and suggest where the rest comes from.

Sources for this section: [26].

8.4 Design trade-offs

Decision	Considerations
TCP vs. UDP	Reliability and simplicity versus latency, overhead and energy
Device connects out vs. accepts connections	Outbound connections work through NAT and firewalls; inbound ones usually do not
IPv4 vs. IPv6	IPv4 is universal but needs NAT; IPv6 gives every device an address and suits low-power networks (Chapter 11.)
Keep connections open vs. reconnect	Open connections give low latency for commands but use memory, and energy for keep-alives

8.5 Common pitfalls

- **Confusing MAC and IP addresses.** MAC addresses are local to a link; IP addresses are end to end.
- **Expecting to reach a device behind NAT from outside.** Design for outbound connections.
- **Hard-coding IP addresses.** Use names and DNS; addresses change.

- **Committing Wi-Fi passwords or API keys** to a public repository.
- **Blocking forever on the network.** Always set timeouts, and handle the case where Wi-Fi drops after it connected.
- **Enterprise Wi-Fi.** Campus networks often use WPA2-Enterprise (802.1X) or captive portals, which small devices cannot easily join. A dedicated IoT network or a travel router is often needed.

> **i** Advanced track
>
> **Watching the layers.** Run Wireshark on a laptop connected to the same access point (or on a mirrored port) while your kit makes an HTTP request. Identify the DHCP exchange, the ARP request, the DNS query, the TCP three-way handshake, the HTTP request and response, and the TCP connection teardown. How many packets and bytes did it take to send a single poll answer?
>
> **Overhead budget.** A sensor sends 20 bytes of data. Estimate the total bytes on the air for that reading when sent (a) as an HTTP POST over a new TCP connection, (b) as an MQTT publish over an existing connection, and (c) as a single UDP datagram. What does that imply for battery life?

8.6 Check your understanding

1. What does a protocol define?
2. Name the data unit at the transport, network and data-link layers.
3. Which OSI layers are combined into the TCP/IP application layer, and who provides their services?
4. Why can't the internet route on MAC addresses?
5. Which address changes at every hop, and which normally stays the same end to end?
6. Why do most IoT devices connect outward to a server instead of running a server themselves?
7. Give one IoT situation where UDP is a better choice than TCP, and one where it is not.
8. A file arrives corrupted. At which layers could error detection have caught it?

8.7 Further reading

- A. Rayes and S. Salam [1], chapter on the internet in IoT, whose sequence this chapter follows.
- J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach* [26], the standard introduction to internet protocols.
- A. L. Russell, "OSI: The Internet that wasn't" [25], on how TCP/IP won.

9. IoT Network Requirements and the Protocol Stack

Learning objectives

After this chapter you should be able to:

1. Explain why the traditional internet protocol stack is a poor fit for many IoT devices.
2. Classify constrained devices and describe the limits they place on protocols.
3. Describe the scalability challenges of addressing, credentials, control and radio spectrum.
4. Explain what deterministic networking means and when an IoT system needs it.
5. Summarize the security, privacy and interoperability requirements of IoT networks.
6. Draw the IoT protocol stack and place the main IoT protocols in it.

9.1 Why it matters

The TCP/IP stack was designed for computers with plenty of memory, wired connections and a person waiting at a screen. It works well for laptops and phones, and for an ESP32 on campus Wi-Fi. But many IoT devices are much smaller: a battery-powered sensor with 10 KB of RAM, a radio that can send only a few hundred bytes per second, and a duty cycle of a fraction of a percent. Others sit in factories or vehicles, where a message that arrives a few milliseconds late is as bad as one that never arrives.

Following the analysis of Rayes and Salam [1], this chapter sets out what IoT asks of a network, and introduces the layered protocol stack that the rest of Part III fills in.

9.2 Concepts

9.2.1 Requirement 1: support for constrained devices

The traditional internet assumes hosts with fast processors, large memories and mains power. Many IoT devices have none of these. RFC 7228 defines three classes of **constrained device** by their memory [27]:

Table 9.1: Classes of constrained devices [27].

Class	RAM (data)	Flash (code)	What it can run
Class 0 (C0)	$\ll$ 10 KB	$\ll$ 100 KB	Cannot run an IP stack securely; needs a gateway or proxy
Class 1 (C1)	$\approx$ 10 KB	$\approx$ 100 KB	A constrained IP stack with CoAP over UDP, but not HTTP and TLS comfortably
Class 2 (C2)	$\approx$ 50 KB	$\approx$ 250 KB	Most standard protocols, with care

The ESP32, with 520 KB of RAM and megabytes of flash, is well beyond Class 2. But the coin-cell sensors, smart locks and wireless switches that make up much of the IoT are Class 0 or 1.

Energy is often a tighter limit than memory. A radio typically uses far more power than the processor, so every byte sent, every acknowledgment and every second spent listening costs battery life. Constrained

devices therefore need protocols with small headers, few round trips, and support for **sleeping**, since a device that is asleep cannot answer a packet.

9.2.2 Requirement 2: massive scalability

Tens of billions of devices stress the network in four ways.

Addressing. IPv4 has about 4.3 billion addresses, fewer than the number of IoT devices already deployed. Devices are hidden behind NAT and gateways (Chapter 8.), which works but makes them hard to reach directly. IPv6, with 128-bit addresses, can give every device its own globally unique address.

Credentials and provisioning. Every device needs network credentials and an identity, and nobody can configure billions of devices by hand. Many devices have no keyboard or screen. IoT systems need **zero-touch or low-touch provisioning**: automated ways to get credentials onto a device and register it with a service. Common approaches for Wi-Fi devices include:

- a **temporary access point**: the device starts its own Wi-Fi network and serves a configuration web page (the WiFiManager library does this on the ESP32);
- **BLE provisioning**: a phone app sends the credentials over Bluetooth Low Energy (ESP-IDF includes this);
- **QR codes** printed on the device, carrying a setup code that a phone app uses to commission it (Matter uses this, Chapter 13.).

The control plane. Network protocols can be divided into the **data plane**, which carries the messages themselves, and the **control plane**, which discovers the topology, reports link health, sets up and tears down sessions, maintains quality of service and reacts to faults. When a network grows from hundreds to millions of devices, the control plane must scale with it, without flooding the network with management traffic.

Wireless spectrum. Most IoT devices are wireless, and the radio spectrum is finite and shared. As devices multiply, and each sends more data, the unlicensed bands, especially 2.4 GHz where Wi-Fi, Bluetooth, Zigbee and Thread all live, become crowded. Spectrum is sometimes called the most valuable real estate nobody can see. Protocols must coexist, retransmit gracefully and stay quiet when they have nothing to say.

9.2.3 Requirement 3: determinism

Many IoT applications only need data to arrive "soon". Some need it to arrive within a **guaranteed** time. A motion-control loop in a factory, a vehicle's braking system or a remote surgical robot fails if a packet is late.

A **deterministic network** is one where the worst-case latency and the **jitter** (variation in latency) of the messages that matter can be calculated in advance from a model of the network. Ordinary internet traffic is not deterministic: packets queue behind other traffic, and Wi-Fi devices wait for a quiet channel.

Achieving determinism requires precise **time synchronization** across devices, **scheduled** transmission slots, and **quality-of-service** reservations along the path. Standards such as IEEE **Time-Sensitive Networking** (TSN) for Ethernet and the IETF's **DetNet** bring these properties to IP-based networks, and the scheduled channel-hopping mode of IEEE 802.15.4 (TSCH) does the same for low-power wireless.

9.2.4 Requirement 4: security and privacy

In 2016, the **Mirai** botnet scanned the internet for IoT devices, mostly cameras and video recorders running a stripped-down Linux, that still used factory-default usernames and passwords. It logged in, installed itself, and used hundreds of thousands of infected devices to launch distributed denial-of-service attacks. One attack on the DNS provider Dyn made many major websites unreachable for hours [28].

Mirai used no clever exploit. It succeeded because the devices were exposed to the internet with default credentials and no way to update them. IoT networks need:

- **lightweight but strong cryptography** that constrained devices can run: AES for encryption, SHA-2 for hashing, elliptic-curve cryptography for keys and signatures, and a good random number generator. The ESP32 has hardware accelerators for AES, SHA, RSA and big-number arithmetic, and a hardware random number generator;
- **authentication and authorization** suited to devices without users;

- **privacy**, because IoT data often reveals personal behavior: when someone is home, how they sleep, where they drive. Identity management should allow devices and data to be decoupled from their owner's identity, while still letting the owner prove ownership.

Part IV covers these topics in depth.

9.2.5 Requirement 5: application interoperability

Two devices can share a network and still fail to understand each other. A light switch from one company and a bulb from another may both speak IP, but unless they agree on what "turn on" looks like, they cannot cooperate. IoT needs agreement not only on how bytes move but on what they **mean**. Chapter 13. is devoted to this problem.

9.2.6 The IoT protocol stack

To meet these requirements, the IoT uses a variant of the TCP/IP stack (Figure 9.1). Two changes stand out.

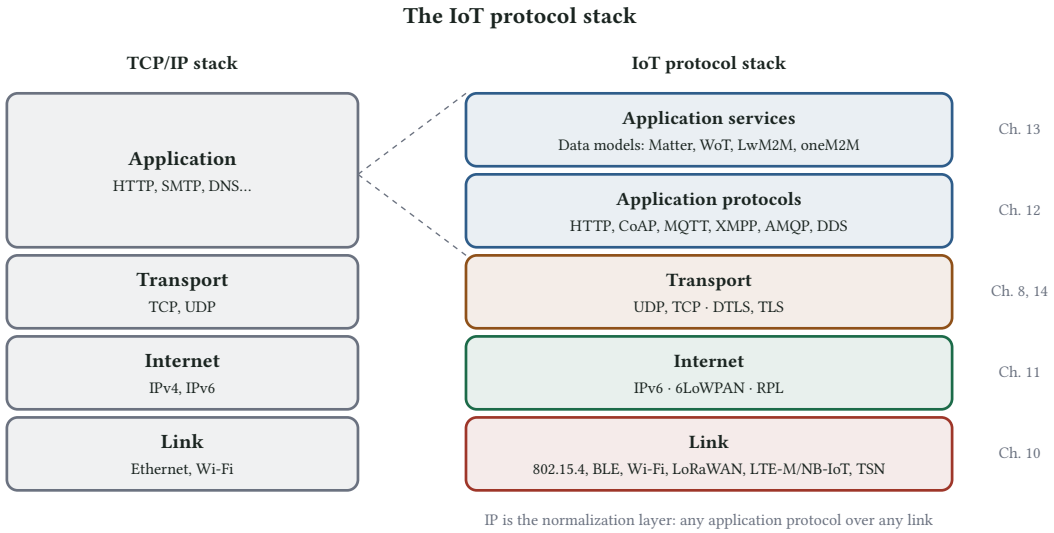


Figure 9.1: The IoT protocol stack compared with the TCP/IP stack. The chapter that covers each layer is shown on the right.

1. **The lower layers gain constrained options.** Alongside Wi-Fi and Ethernet, the link layer includes low-power radios such as Bluetooth Low Energy, IEEE 802.15.4 (used by Zigbee and Thread), LoRa and cellular IoT (LTE-M and NB-IoT). The internet layer adds **adaptation layers** such as 6LoWPAN, which compress IPv6 to fit small radio frames, and routing protocols designed for lossy networks, such as RPL.
2. **The application layer splits in two.**
   - **Application protocols** define how messages are exchanged: HTTP, CoAP, MQTT and others (Chapter 12.).
   - **Application services** define what the messages mean: common data models, device descriptions and interaction patterns, so that devices from different makers interoperate (Chapter 13.).

The IP layer in the middle acts as a **normalization layer**. As long as everything above and below speaks IP, any application protocol can run over any link technology, and each layer can evolve independently.

Not every IoT system uses IP end to end. Classic Bluetooth, LoRaWAN and Zigbee devices use their own stacks and reach the internet through a **gateway** that translates between them and IP. Newer standards, such as Thread, bring IPv6 all the way to the device.

Try it on your kit: provisioning without hard-coded credentials

Until now your Wi-Fi password has been compiled into the firmware in `secrets.h`.

1. List three problems with hard-coding credentials in a product shipped to thousands of customers.
2. Add the WiFiManager library to a project. On first boot, the kit should start an access point; connect to it with your phone and enter your network's credentials. After a reset, it should reconnect automatically.
3. Where are the credentials stored now? Who could read them if they had the device in hand?
4. Add a "factory reset": holding both buttons at power-up erases the stored credentials.

9.3 In depth (optional)

Note

Optional, in-depth explanation. This section goes beyond the rest of the chapter into the math and physics behind it. Nothing later in the book depends on it; skip it on a first reading.

9.3.1 The limits every radio lives within

Shannon's capacity. No modulation scheme, however clever, can carry more than

$$C = B \log_2\left(1 + \frac{S}{N}\right) \text{ bits per second} \tag{9.1}$$

over a channel of bandwidth B with signal-to-noise ratio S/N (as a ratio, not in dB). The formula explains the families in this chapter:

- A 20 MHz Wi-Fi channel at 25 dB SNR ($S/N \approx 316$) is capped at $20 \times 10^6 \log_2 317 \approx 166$ Mb/s. Wide channel, strong signal, high rate.
- A 125 kHz LoRa channel at −10 dB SNR ($S/N = 0.1$) is capped at $125 \times 10^3 \log_2 1.1 \approx 17$ kb/s. LoRa trades almost all of that capacity away to decode signals *below* the noise floor, which is what gives it range.

To get more bits, you can widen the channel (more spectrum, more noise collected, often a regulatory limit) or raise the SNR (more power, better antennas, shorter distance).

Noise floor. Thermal noise in a bandwidth B is $N = kTB$; at room temperature that is $-174\,\text{dBm/Hz} + 10\log_{10} B$. A 125 kHz channel has a floor of $-174 + 51 = -123$ dBm, a 20 MHz channel $-174 + 73 = -101$ dBm, before the receiver adds its own noise figure (typically 6 dB). Narrow channels hear weaker signals.

Energy per bit. For battery devices the useful metric is not the bit rate but the energy spent per delivered bit, $E_b = P_{\text{tx}}/R$, plus the energy of waking up, joining and waiting. A radio at 100 mW and 1 Mb/s spends 100 nJ per bit; one at 50 mW and 5 kb/s spends 10 µJ per bit, 100 times more per bit, yet may still win for a 20-byte daily report because it needs no association, no IP stack and no keep-alive traffic.

Exercise. A 1 MHz channel offers 10 dB of SNR. What is its Shannon capacity? How much SNR would you need to double it, and what does that say about adding power as a way to buy throughput?

Sources for this section: [29].

9.4 Design trade-offs

Requirement	Tension
Small headers and few round trips	Features such as reliability and security add bytes and exchanges
Long sleep periods	Reachability: a sleeping device cannot receive commands

Requirement	Tension
End-to-end IP	Gateways and translation are simpler for very constrained devices
Determinism	Scheduling and synchronization add complexity and cost
Zero-touch provisioning	Convenience versus the risk of someone else claiming the device

9.5 Common pitfalls

- **Assuming every device can run TLS and HTTP.** Many cannot; choose protocols for the smallest device in the system.
- **Ignoring the cost of listening.** A radio that listens continuously can drain a battery faster than one that transmits occasionally.
- **Treating the 2.4 GHz band as empty.** In a dorm or office it is crowded; test in realistic conditions.
- **Shipping default passwords.** Mirai is the textbook consequence.
- **Designing provisioning last.** How a device gets its credentials shapes the whole user experience and much of its security.

> **i** Advanced track
>
> **Energy budget.** A sensor runs on two AA cells (about 2 500 mAh). It sleeps at 10 µA and wakes once every 10 minutes for 2 s at 80 mA to read a sensor and transmit over Wi-Fi. Estimate the battery life. Then repeat the calculation for a LoRa radio that needs only 100 ms at 40 mA per transmission. What dominates the budget in each case?
>
> **Reading an RFC.** Read the introduction and terminology sections of RFC 7228. Then classify three devices you know (for example a smart bulb, a fitness band and a parking sensor) and justify your classification.

9.6 Check your understanding

1. What makes a device “constrained”, according to RFC 7228?
2. Name the four aspects of scalability discussed in this chapter.
3. Describe two ways to give a screenless Wi-Fi device its network credentials.
4. What is the difference between the control plane and the data plane?
5. Define a deterministic network. Give an IoT application that needs one and one that does not.
6. What did the Mirai botnet exploit, and what design practices would have prevented it?
7. Why does the IoT protocol stack split the application layer into two?
8. What role does IP play as a “normalization layer”?

9.7 Further reading

- A. Rayes and S. Salam [1], chapters on IoT networking requirements and the layered IoT protocol stack, on which this chapter’s structure is based.
- C. Bormann, M. Ersue and A. Keranen, RFC 7228, *Terminology for Constrained-Node Networks* [27].
- M. Antonakakis et al., “Understanding the Mirai Botnet” [28].

10. The Link Layer: Short and Long Range

Learning objectives

After this chapter you should be able to:

1. Explain why the link layer dominates an IoT device's energy budget, and estimate battery life from a duty cycle.
2. Use the ESP32's sleep modes to reduce power consumption.
3. Describe IEEE 802.15.4 and the protocols built on it, and compare star, mesh and cluster-tree topologies.
4. Describe Bluetooth Classic and Bluetooth Low Energy, including advertising and the GATT data model.
5. Describe Wi-Fi's role in IoT, including ESP-NOW and Wi-Fi HaLow.
6. Compare long-range options (LoRa and LoRaWAN, cellular IoT) and wired time-sensitive networking.
7. Choose a link technology for a given application from its range, data rate, power and cost.

10.1 Why it matters

The link layer is where an IoT device meets the physical world of radio waves and wires, and it is usually where most of the device's energy goes. Transmitting a packet over Wi-Fi can take more energy than a full second of computation; repairing lost transmissions with retries can account for most of the energy spent on communication.

There is no single best link technology. A wireless earbud, a smart lock, a soil sensor on a farm and a robot on a factory floor need very different combinations of range, data rate, latency, power and cost. This chapter surveys the main options and gives you a framework for choosing among them.

10.2 Concepts

10.2.1 What IoT asks of the link layer

Four groups of characteristics shape the choice of link technology [1]:

- **Device characteristics.** How much energy the device has, and how much processing and memory it can spend on the radio stack. For many devices, communication costs more energy than everything else combined, so low-power radios are essential.
- **Traffic characteristics.** Some devices send short, occasional bursts (a door sensor); others send continuous streams (a camera). Some applications tolerate loss and delay (weather monitoring); others need guaranteed, bounded latency (engine control). The same device can face different requirements in different applications.
- **Access characteristics.** Wired or wireless, mobile or stationary, short range or long range.
- **Scalability characteristics.** Whether to build your own network (Wi-Fi, Zigbee, LoRa) or rent one (cellular), and whether to use a **star** topology, where every device talks to a central hub, or a **mesh**, where devices relay each other's traffic. Meshes extend coverage but add forwarding delay, and they converge more slowly as they grow.

10.2.2 Energy and duty cycling

The ESP32 illustrates how much a radio costs and how sleep modes save it. Approximate figures for the ESP32 chip are [7]:

Table 10.1: ESP32 power modes. Figures are for the chip; a development board's USB bridge, regulator and LEDs add several milliamperes.

Mode	What is running	Approximate current
Active, Wi-Fi transmitting	CPU and radio	160–260 mA
Active, receiving or listening	CPU and radio	80–90 mA
Modem sleep	CPU on, radio off between beacons	≈ 3–20 mA
Light sleep	CPU paused (clock-gated), RAM kept	≈ 0.8 mA
Deep sleep	Only the RTC domain and its memory	≈ 10 µA
Hibernation	Only an RTC timer	≈ 2.5 µA

Light sleep uses **clock gating**: the clock is stopped to most of the chip, so its logic stops switching and stops consuming dynamic power, but its state is preserved. **Deep sleep** powers down the CPU, most of the RAM and the digital peripherals, leaving only the real-time-clock (RTC) controller, its peripherals and a small RTC memory. Waking from deep sleep is like a reset: `setup()` runs again. Variables marked `RTC_DATA_ATTR` survive.

```
RTC_DATA_ATTR uint32_t bootCount = 0;      // kept in RTC memory across deep sleep

void setup() {
  Serial.begin(115200);
  bootCount++;
  Serial.printf("Boot #%lu: measure, transmit, sleep\n", bootCount);
  // ... read the sensor and send the reading here ...
  esp_sleep_enable_timer_wakeup(60ULL * 1000000ULL);   // wake in 60 s
  esp_deep_sleep_start();                              // never returns
}

void loop() {}                                         // never reached
```

Battery life follows from the **average** current:

$$\text{battery life} = \frac{\text{capacity}}{I_{avg}}, \qquad I_{avg} = \sum_{i} I_i \cdot \frac{t_i}{T} \tag{10.1}$$

For a 1 300 mAh battery:

- **Always active** at 250 mA: 1 300 / 250 ≈ **5.2 hours**.
- **Modem sleep**, active 50 ms of every second at 250 mA and in modem sleep (10 mA) the rest: $I_{avg} = 250 \times 0.05 + 10 \times 0.95 = 22$ mA, so about **59 hours**.
- **Hibernation**, active 4 ms of every second at 250 mA and hibernating at 2.5 µA the rest: $I_{avg} \approx 1.0$ mA, so roughly **54 days**.

The lesson is that battery life is set almost entirely by how long the radio is on. Sending less often, sending less data, and reconnecting quickly matter far more than optimizing code.

10.2.3 IEEE 802.15.4: low-rate personal area networks

IEEE 802.15.4 defines the physical and MAC layers for low-rate wireless personal area networks: low cost, low power and low data rate, with battery lives of months to years. Its main features are:

- operation in unlicensed bands: 868 MHz (Europe, one channel), 902–928 MHz (North America) and 2.4 GHz worldwide (16 channels at 250 kbit/s);
- ranges from tens of meters indoors to around a kilometer outdoors with line of sight;
- acknowledged frames for reliability, with frames of at most 127 bytes.

802.15.4 is the foundation for several complete stacks: **Zigbee** (widely used in lighting and home automation), **Thread** (an IPv6-based mesh used by Matter, Chapter 13.), **WirelessHART** and **ISA100.11a** (industrial), and **6LoWPAN** (Chapter 11.).

An 802.15.4 network has three kinds of device:

- a **coordinator**, the most capable node and the root of the network, which chooses the channel, assigns the network identifier and allocates addresses;
- **routers**, which relay traffic and buffer messages for sleeping devices;
- **end devices**, which know only enough to talk to their parent and can sleep most of the time.

These devices can be arranged in three topologies:

Table 10.2: 802.15.4 topologies.

Topology	Description	Strengths	Weaknesses	Typical use
Star	All devices talk to one coordinator	Simple and cheap; end devices can sleep	Single point of failure; range limited to one hop	Smart home, peripherals, health monitors
Mesh	Any device can relay for any other within range	Self-organizing and self-healing; multipath reliability	Forwarding delay; routers must stay awake	Agriculture, environmental monitoring, building automation
Cluster-tree	Hierarchical mesh; routers connect only through parents	Covers large areas with simple routing	Longer latency; less redundancy	Large sensor deployments

The **TSCH** (time-slotted channel hopping) mode, introduced in IEEE 802.15.4e, synchronizes all nodes to a shared schedule of time slots, and changes radio channel on every slot. Synchronization lets nodes sleep until their slot, which saves energy; channel hopping means interference on one frequency only affects some transmissions, which improves reliability. TSCH is the basis of deterministic low-power industrial networks.

10.2.4 Bluetooth

Bluetooth (originally IEEE 802.15.1, now maintained by the Bluetooth SIG) operates in the 2.4 GHz band between 2 402 and 2 480 MHz. It uses **frequency hopping**: devices change channel up to 1 600 times per second, which reduces the impact of interference from Wi-Fi, microwave ovens and other devices sharing the band.

Bluetooth Classic (BR/EDR) organizes devices into **piconets**: one central device (historically called the master) and up to seven active peripherals that share its clock and hopping sequence. Piconets can be linked into larger **scatternets**. Classic Bluetooth suits continuous streams such as audio, but discovery and connection maintenance keep the radio busy, which limits battery life.

Bluetooth Low Energy (BLE), introduced in Bluetooth 4.0, is a separate, much simpler protocol designed for low power. Its main concepts are:

- **Advertising.** A device broadcasts short packets on three dedicated channels to announce itself. Beacons do nothing else; many sensors simply put their readings in their advertisements.
- **Connections.** A central device (such as a phone) can connect to a peripheral. Both then wake only at agreed **connection intervals**, typically every 7.5 ms to 4 s, and sleep in between.
- **GATT.** Connected devices exchange data through the **Generic Attribute Profile**. A peripheral exposes **services**, each containing **characteristics** (values with properties such as read, write and notify). Services and characteristics are identified by UUIDs; standard ones, such as the Heart Rate service, have short 16-bit UUIDs defined by the Bluetooth SIG.

Later versions added features important for IoT: Bluetooth 5 introduced a 2 Mbit/s mode and a long-range **coded PHY**; Bluetooth **Mesh** supports many-to-many networks; and Bluetooth 5.1 and later added direction finding and, more recently, channel sounding for distance measurement.

The original ESP32 supports Bluetooth 4.2, both Classic and BLE. A minimal BLE peripheral that exposes the potentiometer as a readable, notifying characteristic looks like this:

```
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLE2902.h>
#include "iot_pins.h"

#define SERVICE_UUID "6e400001-b5a3-f393-e0a9-e50e24dcca9e"   // example custom UUIDs
#define POT_UUID     "6e400002-b5a3-f393-e0a9-e50e24dcca9e"

BLECharacteristic* potChar;

void setup() {
  BLEDevice::init("IoT-Kit");
  BLEServer* server = BLEDevice::createServer();
  BLEService* service = server->createService(SERVICE_UUID);
  potChar = service->createCharacteristic(
     POT_UUID, BLECharacteristic::PROPERTY_READ | BLECharacteristic::PROPERTY_NOTIFY);
  potChar->addDescriptor(new BLE2902());          // lets clients enable notifications
  service->start();
  BLEDevice::getAdvertising()->addServiceUUID(SERVICE_UUID);
  BLEDevice::startAdvertising();
}

void loop() {
  uint16_t value = analogRead(IoTPins::POT_ADC);
  potChar->setValue(value);                       // 2 bytes, little-endian
  potChar->notify();
  delay(500);
}
```

A generic BLE explorer app on a phone (such as nRF Connect) can now discover the kit, connect and subscribe to the value. Lab 4 builds a BLE step counter on this foundation.

10.2.5 Wi-Fi

Wi-Fi (IEEE 802.11) offers high data rates and uses infrastructure that already exists in most buildings, which makes it the most common link for IoT devices with mains power [3]. Its drawbacks for battery-powered devices are high transmit power, the time needed to associate and obtain an address, and the need to wake regularly to hear the access point's beacons.

Two variants address some of these drawbacks:

- **ESP-NOW** is a connectionless protocol from Espressif that sends short packets (up to 250 bytes) directly between ESP devices using the Wi-Fi radio, with no access point. It supports encrypted and unencrypted peers and delivery callbacks. It is ideal for simple device-to-device control, such as a remote button that switches a light, but it is proprietary and limited to a small number of encrypted peers.
- **Wi-Fi HaLow** (IEEE 802.11ah) moves Wi-Fi to the sub-GHz bands (902–928 MHz in the US) for longer range (around 1 km outdoors) and better penetration through walls. It supports thousands of stations per access point, longer sleep periods and smaller headers, with data rates from about 150 kbit/s to tens of Mbit/s.

Newer mainstream Wi-Fi standards also help IoT: Wi-Fi 6 (802.11ax) adds **target wake time**, which lets a device negotiate when it will next wake, so it can sleep through beacons.

10.2.6 Long range: LPWANs

Low-power wide-area networks (LPWANs) trade data rate for range and battery life. They can reach several kilometers in cities and further in open country, with batteries lasting years, but they carry only a few bytes per message and a limited number of messages per day.

10.2.6.1 LoRa and LoRaWAN

LoRa is a radio modulation (the physical layer) developed by Semtech. It uses **chirp spread spectrum**: each symbol is a frequency sweep, which can be decoded even below the noise floor. Its key parameter is the **spreading factor** (SF7 to SF12). Each step up roughly doubles the time on air and adds range; SF7 at 125 kHz bandwidth carries about 5.5 kbit/s, while SF12 carries under 300 bit/s. In North America LoRa uses the 902–928 MHz band (US915), and in Europe the 863–870 MHz band (EU868), where regulations limit each device to a small duty cycle.

LoRaWAN is the MAC and network protocol built on LoRa, maintained by the LoRa Alliance [30]. Its main features are:

- a **star-of-stars topology**: end devices transmit to any gateways in range; **gateways** are simple bridges that forward everything to a **network server**, which removes duplicates, manages devices and routes data to application servers. The intelligence is in the server, so devices stay simple;
- **device classes** that trade latency for energy: **Class A** devices listen only briefly after each uplink (lowest power); **Class B** devices also open scheduled receive windows; **Class C** devices listen continuously (mains-powered actuators);
- **adaptive data rate**, which picks the fastest spreading factor that reliably reaches a gateway;
- built-in **security**, with separate network and application session keys derived when a device joins (typically over the air, with OTAA).

The kit's RFM95W module (Semtech SX1276) speaks raw LoRa. Without a LoRaWAN network, two kits can talk directly, point to point:

```
#include <SPI.h>
#include <LoRa.h>          // arduino-LoRa library
#include "iot_pins.h"

void setup() {
  Serial.begin(115200);
  LoRa.setPins(IoTPins::LORA_CS, IoTPins::LORA_RST, IoTPins::LORA_G0);
  if (!LoRa.begin(915E6)) {                // US915 band
    Serial.println("LoRa init failed: check wiring");
    while (true) delay(1000);
  }
  LoRa.setSpreadingFactor(7);
  LoRa.setSyncWord(0x34);              // pick a team-specific value to avoid crosstalk
}

void loop() {
  LoRa.beginPacket();
  LoRa.print("beacon team-07 ");
  LoRa.print(millis() / 1000);
  LoRa.endPacket();                        // blocks until sent

  int size = LoRa.parsePacket();           // anything received?
  if (size) {
    String msg;
    while (LoRa.available()) msg += (char)LoRa.read();
    Serial.printf("RX '%s' RSSI %d dBm SNR %.1f dB\n",
                  msg.c_str(), LoRa.packetRssi(), LoRa.packetSnr());
  }
  delay(5000);
}
```

10.2.6.2 Cellular IoT

Cellular networks offer wide coverage without building infrastructure, at the cost of a subscription and higher module prices. Two standards are designed for IoT:

- **LTE-M** (LTE Cat-M1): up to about 1 Mbit/s, supports mobility and voice, suited to trackers and wearables.

- **NB-IoT** (narrowband IoT): tens of kbit/s, excellent building penetration, very low power, suited to static meters and sensors.

Both support **power saving mode** (PSM) and **extended discontinuous reception** (eDRX), which let a device sleep for hours while remaining registered. Many older devices rely on 2G and 3G networks, which carriers are shutting down; new designs should use LTE-M, NB-IoT or 5G variants such as RedCap.

10.2.6.3 Other LPWANs

Sigfox offered an ultra-narrowband network with a very small daily message allowance, and is now operated by UnaBiz. Other options include **Wi-SUN** (used by utilities for smart meters), **Wireless M-Bus** (metering in Europe) and **mioty**.

10.2.7 Wired: time-sensitive networking

Not every IoT link is wireless. Factories and vehicles use Ethernet extended with IEEE **Time-Sensitive Networking** (TSN) standards. TSN combines:

- **precise time synchronization** (IEEE 802.1AS, a profile of the Precision Time Protocol) to better than a microsecond;
- **scheduled and shaped forwarding** (for example IEEE 802.1Qbv and 802.1Qav), so that time-critical traffic gets guaranteed, bounded latency even on links shared with video or file transfers;
- **stream reservation** and **redundant paths** (IEEE 802.1CB) for reliability.

Targets in industrial automation reported in the literature [1] are on the order of microseconds of latency within a machine cell and around 100 µs across a factory.

10.2.8 Choosing a link technology

Figure 10.1 places the main options by range and data rate.

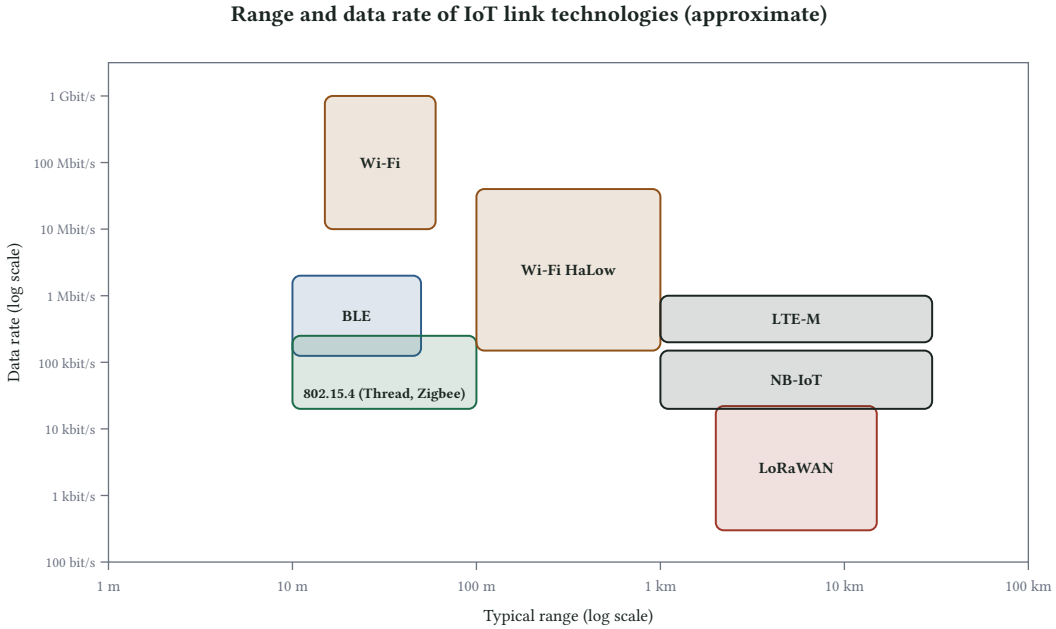


Figure 10.1: Approximate range and data rate of common IoT link technologies. Both axes are logarithmic; real values depend heavily on environment, antennas and configuration.

Table 10.3: Comparing IoT link technologies. Values are typical, not limits.

Technology	Typical range	Data rate	Power	Topology	Infrastructure
BLE	10–50 m	0.125–2 Mbit/s	Very low	Star, mesh	Phone or gateway
Zigbee / Thread (802.15.4)	10–100 m per hop	250 kbit/s	Very low	Mesh	Hub or border router
Wi-Fi	30–50 m indoors	10–1000+ Mbit/s	High	Star	Access point
Wi-Fi HaLow	up to ≈ 1 km	0.15–80 Mbit/s	Low–medium	Star	HaLow access point
LoRaWAN	2–15 km	0.3–22 kbit/s	Very low	Star of stars	Gateways (own or community)
NB-IoT / LTE-M	Cellular coverage	20 kbit/s–1 Mbit/s	Low	Star	Carrier, subscription
Ethernet + TSN	100 m per segment	100 Mbit/s–10 Gbit/s	Mains	Switched	Wired network

Try it on your kit: a LoRa emergency beacon

This exercise corresponds to the LoRa lecture activity and uses Stage 4 of the kit build.

1. Agree on a sync word with a partner team and run the beacon sketch on both kits. Print each received message with its RSSI and SNR.
2. Walk one kit away from the other, down a corridor and through a door. At what RSSI and SNR do packets start to be lost?
3. Change the spreading factor from 7 to 10 on both kits. How do range and time on air change? Measure time on air with `micros()` around `endPacket()`.
4. Turn the beacon into an emergency beacon: pressing a button sends an alert that the other kit shows on its OLED and announces with the buzzer.
5. Estimate battery life for a beacon that wakes from deep sleep once per minute, sends one SF7 packet and goes back to sleep.

A guided, step-by-step version for a lecture session is Activity 6.

10.3 In depth (optional)

Note

Optional, in-depth explanation. This section goes beyond the rest of the chapter into the math and physics behind it. Nothing later in the book depends on it; skip it on a first reading.

10.3.1 Link budgets and time on air

Free-space path loss. In free space, received power falls with the square of distance and with the square of frequency (for fixed-gain antennas). In decibels, with d in metres and f in hertz,

$$\mathrm{FSPL} = 20\log_{10} d + 20\log_{10} f - 147.55\ \mathrm{dB}\ . \tag{10.2}$$

At 915 MHz the loss over 1 km is $60 + 179.2 - 147.6 \approx 91.7$ dB; at 2.4 GHz it is about 8.4 dB more. Real environments lose more, and a **log-distance model** captures it with an exponent n (2 in free space, roughly 2.7–3.5 in towns, 4–6 indoors through walls):

$$PL(d) = PL(d_0) + 10\, n \log_{10} \frac{d}{d_0}. \tag{10.3}$$

The link budget. A link works if

$$P_{\text{tx}} + G_{\text{tx}} + G_{\text{rx}} - PL - L_{\text{misc}} \geq S_{\text{rx}}, \tag{10.4}$$

where S_{rx} is the receiver sensitivity. For an SX1276 at 14 dBm with unity-gain antennas, sensitivity is roughly −123 dBm at SF7 and −137 dBm at SF12 (125 kHz). The difference between transmit power and sensitivity, the **maximum coupling loss**, is 137 dB at SF7 and 151 dB at SF12. Every 6 dB of extra budget doubles the free-space range; with $n = 3$, it multiplies range by $10^{6/30} \approx 1.6$.

LoRa time on air. A chirp spread-spectrum symbol at spreading factor SF and bandwidth BW lasts $T_s = 2^{SF}/BW$. Semtech's formula for the number of payload symbols, for a payload of PL bytes, coding rate $4/(4+CR)$, CRC on (CRC = 1), explicit header ($IH = 0$) and low-data-rate optimization off ($DE = 0$), is

$$n_{\text{payload}} = 8 + \max\left(\left\lceil \frac{8PL - 4SF + 28 + 16\,\text{CRC} - 20\,IH}{4\,(SF - 2DE)} \right\rceil (CR + 4),\, 0\right), \tag{10.5}$$

and the preamble of n_p symbols adds $(n_p + 4.25)\,T_s$.

For Activity 6's 23-byte beacon at SF7, BW 125 kHz, CR = 1 (4/5) and an 8-symbol preamble: $T_s = 1.024$ ms; the fraction is $200/28 = 7.14$, rounded up to 8, times 5 gives 40, so $n_{\text{payload}} = 48$. Time on air is $(12.25 + 48) \times 1.024 \approx 62$ ms. At SF12, $T_s = 32.8$ ms, low-data-rate optimization becomes mandatory ($DE = 1$), and the same packet takes about 1.5 s: the range costs airtime, and airtime costs energy and channel capacity.

Duty-cycle arithmetic. In Europe's 868 MHz bands many sub-bands limit each device to 1% of the time: 36 s per hour, or about 580 of these SF7 packets but only about 24 at SF12. North American rules for 902–928 MHz instead limit the *dwell time* on a channel (400 ms for many configurations), which an SF12 packet of this size already exceeds.

Exercise. Two kits are 300 m apart in a town ($n = 3$, $PL(1\,\text{m}) = 31.7$ dB at 915 MHz). Is SF7 enough? What margin do you have?

Sources for this section: [21], [31], [32].

10.4 Design trade-offs

Decision	Considerations
Range vs. data rate vs. power	You can usually have two of the three
Own network vs. carrier network	Building your own avoids subscription fees but requires deploying and maintaining gateways
Star vs. mesh	Stars are simple and let end devices sleep; meshes extend coverage but need always-on routers
Licensed vs. unlicensed spectrum	Licensed (cellular) bands are managed and predictable; unlicensed bands are free but shared
Standard vs. proprietary	Standards (BLE, Thread, LoRaWAN) bring interoperability; proprietary protocols (ESP-NOW) can be simpler but lock you in

10.5 Common pitfalls

- **Estimating battery life from the chip's datasheet only.** Regulators, USB bridges, LEDs and sensors on the board often draw more than the chip in deep sleep.

- **Ignoring retransmissions.** Lost packets cost energy twice.
- **Using a European LoRa frequency in the US**, or vice versa. Hardware and frequency plans are regional.
- **Exceeding duty-cycle or dwell-time limits** in unlicensed bands, which can make a device illegal to operate.
- **Operating a LoRa module without an antenna.** It can damage the transmitter.
- **Relying on 2G/3G modules** in new designs.
- **Testing radios only on a desk.** Walls, people, metal and other radios change everything.

> **i** Advanced track
>
> **Link budget.** A link works if transmit power plus antenna gains, minus path loss, exceeds the receiver's sensitivity by a margin. With free-space path loss $\mathrm{FSPL(dB)} = 20\log_{10}(d) + 20\log_{10}(f) - 147.55$ (meters and hertz), compute the free-space range of a 915 MHz LoRa link with +14 dBm transmit power, 0 dBi antennas and −130 dBm sensitivity. Then explain why real-world range is far shorter.
>
> **LoRaWAN on campus.** Community networks such as The Things Network provide free LoRaWAN gateways in many cities. Register a device, join it over the air with OTAA, and compare the effort and the result with the point-to-point exercise above. What do you gain from the network server?
>
> **Coexistence.** Wi-Fi channel 1, BLE advertising channels and Thread channel 11 all overlap near 2.4 GHz. Design a test that measures BLE packet loss while a nearby laptop streams video over Wi-Fi, and propose mitigations.

10.6 Check your understanding

1. Why is the link layer often responsible for most of an IoT device's energy use?
2. A device draws 200 mA for 100 ms every 10 s and 20 µA the rest of the time. What is its average current, and how long will a 2 000 mAh battery last?
3. What is preserved in ESP32 deep sleep, and how does the program resume?
4. Name the three device roles in an 802.15.4 network and what each does.
5. What advantages does TSCH bring to 802.15.4?
6. What are BLE services and characteristics? How does a phone receive updates from a characteristic without polling it?
7. Why is LoRaWAN called a "star of stars"? Where is the intelligence?
8. Choose a link technology for: (a) a water meter in a basement; (b) a wireless light switch in an apartment; (c) a livestock tracker on a ranch; (d) a security camera. Justify each.

10.7 Further reading

- A. Rayes and S. Salam [1], on link-layer requirements and technologies.
- Espressif, *ESP-IDF Sleep Modes* documentation [5].
- K. Townsend et al., *Getting Started with Bluetooth Low Energy* [33].
- LoRa Alliance, *LoRaWAN Specification* [30].

11. The Internet Layer: 6LoWPAN and RPL

Learning objectives

After this chapter you should be able to:

1. Explain the benefits of IP-based connectivity over proprietary networks for IoT.
2. Describe IPv6 addressing and autoconfiguration.
3. Define a low-power and lossy network (LLN) and the five challenges it poses for the internet layer.
4. Explain how 6LoWPAN fits IPv6 into small radio frames through compression, fragmentation and forwarding.
5. Describe how RPL builds and maintains routes in an LLN.
6. Explain the role of a border router, and how Thread brings these ideas to products.

11.1 Why it matters

Early sensor networks and home-automation systems used proprietary protocols. Each vendor's devices formed an island, reachable from the internet only through that vendor's gateway, which translated every message. When the vendor changed its product line or went out of business, the island was stranded.

The alternative is to make IoT devices **first-class citizens of the internet**, with their own IP addresses, so that the same open, long-lived standards, tools and skills work everywhere. The obstacle is that IP was designed for links with large frames and reliable delivery, and for hosts that never sleep. This chapter shows how two IETF standards, 6LoWPAN and RPL, adapt IPv6 to the smallest and least reliable networks.

11.2 Concepts

11.2.1 Why IP for IoT?

Moving from proprietary protocols to IP-based connectivity brings:

- **Open, long-lived standards**, maintained by the IETF and implemented by many vendors;
- **A familiar model** with decades of tools, documentation and trained engineers;
- **Transparent internet integration**: no protocol translation at the gateway, so end-to-end data flows and end-to-end security become possible;
- **Maintainability**: the same diagnostic tools (ping, traceroute, packet capture) work everywhere;
- **Global scalability**, given enough addresses, which means IPv6.

11.2.2 The internet layer, briefly

Recall from Chapter 8. that the internet layer delivers packets from a source host to a destination host across many networks. Routers **forward** each packet toward its destination using a routing table, and **routing protocols** build those tables by exchanging information about network links. On the wider internet, routing protocols such as OSPF (within an organization) and BGP (between organizations) do this job.

11.2.3 IPv6 in a nutshell

IPv6 uses 128-bit addresses, written as eight groups of four hexadecimal digits, with leading zeros dropped and one run of zero groups replaced by `::`. For example, `2001:0db8:0000:0000:0000:0000:0000:0017` becomes `2001:db8::17`.

Typically, the first 64 bits are a **network prefix** and the last 64 bits an **interface identifier**. Several features make IPv6 attractive for IoT:

- **Enough addresses** for every device to have a globally unique one, without NAT.
- **Stateless address autoconfiguration** (SLAAC): a device can build its own address from the prefix advertised by a router plus an interface identifier, with no DHCP server.
- **Link-local addresses** (starting `fe80::`) that every interface creates automatically, so devices on the same link can talk with no configuration at all.
- **A simpler, fixed 40-byte header**, with options moved into extension headers.
- **Neighbor Discovery**, which replaces ARP, finds routers and detects duplicate addresses.

11.2.4 Low-power and lossy networks

Many IoT deployments form **low-power and lossy networks** (LLNs): networks of hundreds or thousands of constrained devices (Chapter 9.) connected by links where packet loss is high and variable, such as 802.15.4, BLE or low-power Wi-Fi. LLNs appear in building automation (heating, lighting, access control and fire systems), connected homes, healthcare, environmental monitoring and urban networks such as smart grids.

LLNs pose five challenges for the internet layer [1]:

1. **Very little state.** Nodes have tiny memories, so routing protocols must keep as little per-node state as possible. A protocol that stores a full map of the network on every node will not fit.
2. **Sleeping nodes.** Nodes save energy by sleeping for long periods, so from the network's point of view the topology changes constantly.
3. **Varied traffic patterns.** Traffic can be point-to-point, point-to-multipoint (a controller commanding many lights) or multipoint-to-point (many sensors reporting to one collector), and both unicast and multicast must be supported.
4. **Small frames.** Link technologies such as 802.15.4 carry at most 127 bytes per frame, while IPv6 requires links to carry packets of at least 1 280 bytes.
5. **Unreliable links**, whose loss rates change over time with interference, weather and moving objects.

11.2.5 6LoWPAN: IPv6 over small frames

6LoWPAN (IPv6 over Low-Power Wireless Personal Area Networks) is an **adaptation layer** between the link layer and IPv6 [34], [35]. It was designed for 802.15.4 and later adapted to BLE, DECT ULE and other links.

The size problem is stark. Of the 127 bytes in an 802.15.4 frame, the MAC header can take up to 25 bytes and link-layer security up to 21, leaving about 81. An uncompressed IPv6 header takes 40 of those and a UDP header another 8, leaving only about 33 bytes for application data.

6LoWPAN provides three functions:

1. **Header compression.** Most IPv6 header fields are predictable in an LLN: the version is always 6, hop limits and traffic classes take common values, and the addresses can usually be derived from the link-layer addresses already in the frame. 6LoWPAN's IPHC encoding replaces these fields with a few bits, compressing the 40-byte IPv6 header to as little as **2 bytes**. The UDP header can be compressed similarly, to as little as 4 bytes.
2. **Fragmentation and reassembly.** When a packet does not fit in one frame, 6LoWPAN splits it into fragments with small headers and reassembles it at the other end, so that IPv6's 1 280-byte minimum is met.
3. **Forwarding** across multiple hops. In **mesh-under** forwarding, the adaptation layer relays frames hop by hop, and the whole mesh looks like one IP link. In **route-over** forwarding, each hop is an IP router, and a routing protocol such as RPL decides the path.

A 6LoWPAN network connects to the rest of the internet through a **border router** (also called an edge router), which translates between compressed 6LoWPAN packets on the radio side and ordinary IPv6 packets on the Ethernet or Wi-Fi side. Because addresses and semantics are preserved, this is *adaptation*, not protocol translation: a server on the internet sees ordinary IPv6 packets from each sensor's own address.

11.2.6 RPL: routing in lossy networks

RPL (pronounced "ripple"), the IPv6 Routing Protocol for Low-Power and Lossy Networks, is the IETF's standard routing protocol for LLNs [36].

RPL is a **distance-vector** protocol. Unlike **link-state** protocols such as OSPF, which keep a database of the entire network on every node, a distance-vector node keeps only a little information about its neighbors and their distance to a destination. This fits the "very little state" constraint.

RPL organizes the network as a **destination-oriented directed acyclic graph** (DODAG): a tree-like structure rooted at the border router, in which every node points toward the root through one or more **parents** (Figure 11.1).

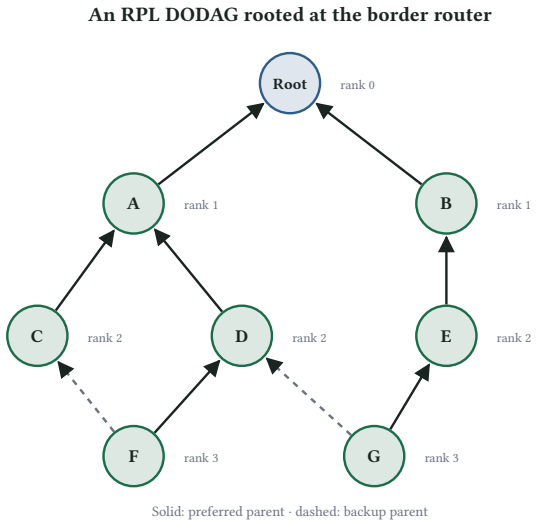


Figure 11.1: An RPL DODAG. Arrows point from each node toward its preferred parent; rank increases with distance from the root.

The DODAG is built with a few control messages:

- The root broadcasts a **DIO** (DODAG Information Object), announcing the graph and its own **rank** (its position in the graph).
- Each node that hears DIOs chooses a **preferred parent** using an **objective function** and computes its own rank, which must be greater than its parent's. It then sends its own DIOs, so the graph spreads outward.
- A new node can ask for DIOs with a **DIS** (DODAG Information Solicitation).
- To make *downward* routes possible, nodes send **DAO** (Destination Advertisement Object) messages toward the root. In **storing mode**, intermediate routers remember downward routes; in **non-storing mode**, only the root does, and it places the full route in the packet header (source routing), which saves memory in the nodes.

The **objective function** defines what "best" means. Common choices minimize hop count (OF0) or the **expected transmission count** (ETX), the average number of transmissions needed to deliver a packet over each link (MRHOF). Other metrics include latency and remaining node energy.

Two properties make RPL suitable for lossy links:

- It is **proactive**: it builds and maintains routes before they are needed, and nodes can keep backup parents, so alternative paths exist before a failure.
- It is **under-reactive**: when a link fails, a node simply switches locally to another parent, instead of triggering a network-wide recomputation.

To keep control traffic low, RPL sends DIOs on a **Trickle timer**: frequently when the network is changing, and exponentially less often when it is stable.

11.2.7 Thread: putting it together

Thread is a complete IPv6 mesh networking stack for homes and buildings, maintained by the Thread Group. It runs on 802.15.4 radios, uses 6LoWPAN for adaptation, adds its own mesh routing and secure commissioning, and connects to Wi-Fi or Ethernet through **border routers**, which are often built into smart speakers, hubs and TVs. Thread is one of the main transports for Matter (Chapter 13.).

The original ESP32 in your kit has no 802.15.4 radio, so it cannot join a 6LoWPAN or Thread network directly. Newer Espressif chips such as the ESP32-C6 and ESP32-H2 do have 802.15.4 radios and support Thread through the OpenThread stack, and dedicated 802.15.4 modules are also available.

11.2.8 Mobility and security at the internet layer

- **Mobility.** When a device moves between networks, its IP address changes and its connections break. Mobile IPv6 lets a device keep a permanent home address while it roams, with a home agent forwarding traffic. In practice, most IoT systems avoid the problem by having devices reconnect and re-register with a cloud service from wherever they are.
- **Security.** IPsec can protect traffic at the internet layer, but it is heavy for constrained nodes. LLNs usually rely on **link-layer encryption** (802.15.4 includes AES-CCM) combined with security at the transport or application layer, such as DTLS or OSCORE (Chapter 12., Chapter 14.).

Try it on your kit: IPv6 on Wi-Fi

Although the kit cannot run 6LoWPAN, its Wi-Fi interface can run IPv6.

1. Enable IPv6 before connecting (`WiFi.enableIpV6()` in the Arduino-ESP32 2.x core; `WiFi.enableIPv6()` in 3.x) and print the link-local address after connecting. How was its interface identifier derived?
2. If your network advertises a global IPv6 prefix, print the global address as well. Is there a NAT between the kit and the internet now?
3. Calculate how many bytes of application payload remain in one 802.15.4 frame (a) without compression and (b) with IPv6 and UDP headers compressed to 2 and 4 bytes respectively, assuming a 25-byte MAC header and 21 bytes of security.
4. Sketch the DODAG that RPL would build for a building with sensors on three floors and a border router in the basement. Which nodes need to stay awake?

11.3 In depth (optional)

Note

Optional, in-depth explanation. This section goes beyond the rest of the chapter into the math and physics behind it. Nothing later in the book depends on it; skip it on a first reading.

11.3.1 Fragments, links and routes

Address space. IPv6 addresses have 128 bits: $2^{128} \approx 3.4 \times 10^{38}$ addresses. A single /64 subnet, the standard size for one link, holds 2^{64} interface identifiers, as many as the entire IPv4 address space squared. Scarcity, and therefore NAT, disappears as a design constraint; reachability and firewalling become policy decisions instead.

Why 6LoWPAN must compress and fragment. IPv6 requires every link to carry packets of at least 1280 bytes. An IEEE 802.15.4 frame carries at most 127 bytes; after the MAC header, security and checksum, roughly 80–100 bytes remain. Two consequences follow:

1. **Compression.** The uncompressed IPv6 header (40 bytes) plus UDP (8) would eat half the frame. Header compression (IPHC) elides fields that can be derived from the link layer, such as addresses built from MAC addresses, and can shrink both to under 10 bytes.
2. **Fragmentation.** A full 1280-byte packet becomes about 14 fragments.

Fragmentation multiplies loss. If each frame is lost independently with probability p, a packet split into k fragments arrives only if *all* fragments do:

$$P_{\text{deliver}} = (1 - p)^k. \tag{11.1}$$

With $p = 5\%$, a single-frame packet arrives 95% of the time, but a 14-fragment packet only $0.95^{14} \approx 49\%$, and a lost fragment forces the whole packet to be resent. Hence the guidance to keep IoT application messages small enough for one frame.

Choosing routes by expected transmissions. RPL routes along the parent that minimizes a cost; a common one is **ETX**, the expected number of transmissions to deliver a frame and receive its acknowledgment across a link:

$$\mathrm{ETX} = \frac{1}{d_f \, d_r}, \tag{11.2}$$

where d_f and d_r are the forward and reverse delivery ratios. A link with 90% delivery each way has ETX $\approx$ 1.23; one with 60% has ETX $\approx$ 2.78. ETX adds along a path, so two good hops (2.46) beat one poor hop (2.78): the shortest route by hop count is not always the cheapest.

Exercise. A sensor is three hops from the border router, each with ETX 1.5, or two hops with ETX 1.2 and 2.0. Which path does RPL prefer under the ETX objective? What is the expected number of radio transmissions per delivered packet on each?

Sources for this section: [35], [36], [37].

11.4 Design trade-offs

Decision	Considerations
End-to-end IPv6 vs. gateway translation	IPv6 enables end-to-end flows and security; gateways allow simpler devices and proprietary radios
Mesh-under vs. route-over	Mesh-under hides the mesh from IP; route-over lets each hop make IP routing decisions
RPL storing vs. non-storing mode	Storing mode spends router memory to save header space; non-storing mode does the opposite
Hop count vs. ETX	Fewer hops is simple; ETX finds more reliable paths that may be longer
Thread vs. Zigbee	Thread is IP-native and part of Matter; Zigbee has a very large installed base

11.5 Common pitfalls

- **Assuming 802.15.4 frames carry IPv6 directly.** They cannot without 6LoWPAN.
- **Ignoring fragmentation costs.** Losing one fragment loses the whole packet; keep application messages small enough to fit in one frame.
- **Forgetting the border router.** An LLN without one is an island.
- **Expecting IPv4 tools to work unchanged.** Many IoT IPv6 addresses are link-local and need a zone (interface) identifier, as in `fe80::1%wlan0`.
- **Assuming IP connectivity means interoperability.** Devices must also agree on application protocols and data models (Chapter 12., Chapter 13.).

> **i** Advanced track
>
> **Simulating an LLN.** Contiki-NG and RIOT include 6LoWPAN and RPL implementations, and the Cooja simulator can run hundreds of virtual nodes. Simulate a 30-node RPL network, kill the root's busiest child, and measure how long the network takes to reconverge and how many control messages it sends.
>
> **Reading IPHC.** Using RFC 6282, decode the two-byte IPHC header `0x7A33`. Which IPv6 fields are elided, which are carried inline after it, and where does the receiver get each elided value from? (Answer check: the traffic class, flow label, hop limit and both addresses are elided; only the next-header field is carried inline.)
>
> **Comparing routing.** Thread does not use RPL; it uses its own distance-vector protocol with a small number of routers (up to 32) and a leader. Compare its design with RPL's in terms of state, scale and behavior when a router fails.

11.6 Check your understanding

1. Give three advantages of IP-based connectivity over proprietary IoT networks.
2. How does a device configure an IPv6 address without a DHCP server?
3. List the five challenges LLNs pose for the internet layer.
4. Why can't an uncompressed IPv6 packet be sent directly over 802.15.4?
5. Name the three main functions of 6LoWPAN.
6. What does a border router do, and why is that not "protocol translation"?
7. What is a DODAG, and what determines a node's rank?
8. Explain what it means for RPL to be proactive and under-reactive.

11.7 Further reading

- Z. Shelby and C. Bormann, *6LoWPAN: The Wireless Embedded Internet* [38].
- RFC 6550, *RPL: IPv6 Routing Protocol for Low-Power and Lossy Networks* [36].
- RFC 6282, *Compression Format for IPv6 Datagrams over IEEE 802.15.4-Based Networks* [35].

12. Application Protocols: HTTP, CoAP, MQTT and More

Learning objectives

After this chapter you should be able to:

1. Distinguish request/response from publish/subscribe interaction, and explain when each fits.
2. Use HTTP and REST from an IoT device, and explain HTTP's costs for constrained devices.
3. Describe CoAP's message model, reliability mechanism, observation and proxying.
4. Describe MQTT's broker architecture, topics, quality-of-service levels, retained messages and wills.
5. Recognize where XMPP, AMQP, DDS and others fit.
6. Choose an application protocol and payload format for a given IoT application.

12.1 Why it matters

Once a device has an IP address and a transport, it still needs a way to exchange meaningful messages with other devices and services. Should a thermostat wait to be asked for the temperature, or announce it whenever it changes? Should a light accept commands directly, or should commands pass through a central server? What happens to a command sent while the light's radio is asleep?

Application protocols answer these questions, and the right choice affects battery life, latency, reliability and how easily the system scales. This chapter covers the three protocols you will meet most often in IoT, HTTP, CoAP and MQTT, and places the others in context.

12.2 Concepts

12.2.1 Two interaction patterns

Request/response. A client sends a request to a server and waits for a response. The web works this way. It is simple and familiar, but the client must know the server's address, the server must be reachable, and getting updates means asking repeatedly (**polling**), which wastes energy and bandwidth when nothing has changed.

Publish/subscribe (pub/sub). Senders (**publishers**) send messages labeled with a **topic** to an intermediary, a **broker**. Receivers (**subscribers**) tell the broker which topics they want, and the broker forwards matching messages. Publishers and subscribers do not need to know about each other, do not need to be online at the same time (with the right settings), and can be added or removed freely. This **decoupling** suits IoT, where many sensors produce data for many consumers.

12.2.2 HTTP and REST

HTTP is the web's application protocol, standardized by the IETF. It is text-based, request/response, and runs over TCP (or, in HTTP/3, over QUIC on UDP). Its design follows the **REST** (Representational State Transfer) architectural style, in which:

- every **resource** has a URI, for example `/devices/kit-07/temperature`;
- a small set of **methods** act on resources: `GET` (read), `POST` (create or submit), `PUT` (replace), `PATCH` (modify) and `DELETE`;
- **status codes** report outcomes: `200 OK`, `201 Created`, `400 Bad Request`, `401 Unauthorized`, `404 Not Found`, `500 Internal Server Error`;
- each request is **self-contained** (stateless), which makes servers easy to scale.

From the ESP32, posting a JSON reading takes a few lines:

```
#include <WiFiClientSecure.h>
#include <HTTPClient.h>

void postReading(float tempC) {
  WiFiClientSecure tls;
  tls.setCACert(ROOT_CA_PEM);                    // verify the server's certificate
  HTTPClient http;
  http.begin(tls, "https://api.example.org/v1/readings");
  http.addHeader("Content-Type", "application/json");
  String body = String("{\"device\":\"kit-07\",\"temp_c\":") + tempC + "}";
  int code = http.POST(body);
  Serial.printf("POST -> %d\n", code);
  http.end();
}
```

HTTP's strengths are universality: every cloud service, programming language and firewall understands it. Its costs for constrained devices are:

- verbose, text-based headers, often hundreds of bytes per request;
- TCP and TLS handshakes before each new connection, which cost several round trips and significant energy;
- no built-in way for a server to push data to a device, other than polling, long polling, WebSockets or server-sent events.

HTTP is a good fit for mains-powered devices, for occasional uploads, and for talking to cloud APIs. It is a poor fit for battery-powered sensors on lossy networks.

12.2.3 CoAP: REST for constrained devices

The IETF's CoRE (Constrained RESTful Environments) working group designed the **Constrained Application Protocol** (CoAP) to bring the REST model to devices and networks that cannot afford HTTP [39]. CoAP keeps HTTP's concepts (URIs, methods, response codes, content types) but changes almost everything underneath:

- It runs over **UDP** (with DTLS for security), avoiding TCP's connection setup.
- Its **binary header** is only 4 bytes, followed by a token and compact options. A typical request fits in a few dozen bytes.
- It supports **multicast**, for example to discover all lights on a network at once.
- It is **asynchronous**: requests and responses are matched by a **token**, not by a connection.

12.2.3.1 Messages and reliability

Because UDP is unreliable, CoAP adds a thin messaging layer with four message types:

- **CON** (confirmable): must be acknowledged; retransmitted with exponential back-off until an ACK arrives.
- **NON** (non-confirmable): fire and forget, for frequent readings where a lost one does not matter.
- **ACK**: acknowledges a CON message. A response can be **piggybacked** on the ACK if it is ready immediately; otherwise the server ACKs first and sends the response later in its own CON message.
- **RST** (reset): "I received your message but cannot process it".

A request is carried in a message, with a method code such as `0.01 GET`, and the response carries a code such as `2.05 Content` or `4.04 Not Found`, mirroring HTTP's 200 and 404.

12.2.3.2 Observe, block transfer and discovery

CoAP adds three extensions that matter for IoT:

- **Observe** [40]: a client registers interest in a resource with one GET, and the server sends a notification whenever the resource changes, a lightweight form of push.
- **Block-wise transfer**: large payloads, such as firmware images, are split into blocks that fit in small packets.

- **Resource discovery**: a GET to `/.well-known/core` returns a list of the device's resources in CoRE Link Format.

12.2.3.3 Caching and proxies

CoAP responses can be cached. The **Max-Age** option says how long a response stays fresh, and the **ETag** option lets a client check whether its cached copy is still valid. **Proxies** use caching to answer on behalf of sleeping devices and to reduce traffic on constrained networks. Because CoAP mirrors HTTP's model, a proxy can also translate between the two, letting a web application `GET` a sensor value over HTTP while the proxy fetches it over CoAP.

CoAP is not a replacement for HTTP, nor a compression of it, nor separate from the web. It is a specialized web protocol for machine-to-machine use on constrained networks. It is used in LwM2M device management, in Thread-based products and in smart-metering and water networks.

12.2.4 MQTT: publish/subscribe telemetry

MQTT is a lightweight publish/subscribe protocol originally developed at IBM for monitoring oil pipelines over satellite links, and now an OASIS standard [41]. (The name once stood for "MQ Telemetry Transport"; despite the "MQ", MQTT has no message queues in the traditional sense.) It runs over TCP, usually with TLS, and has a very small code footprint.

12.2.4.1 Architecture

An MQTT system has two kinds of component (Figure 12.1):

- **Clients** connect to a broker and publish messages to topics, subscribe to topics, or both. Your kit, a phone app and a cloud service can all be clients.
- The **broker** receives all messages and forwards each one to the clients subscribed to its topic. Popular brokers include Eclipse Mosquitto, EMQX and HiveMQ; brokers can also be bridged together.

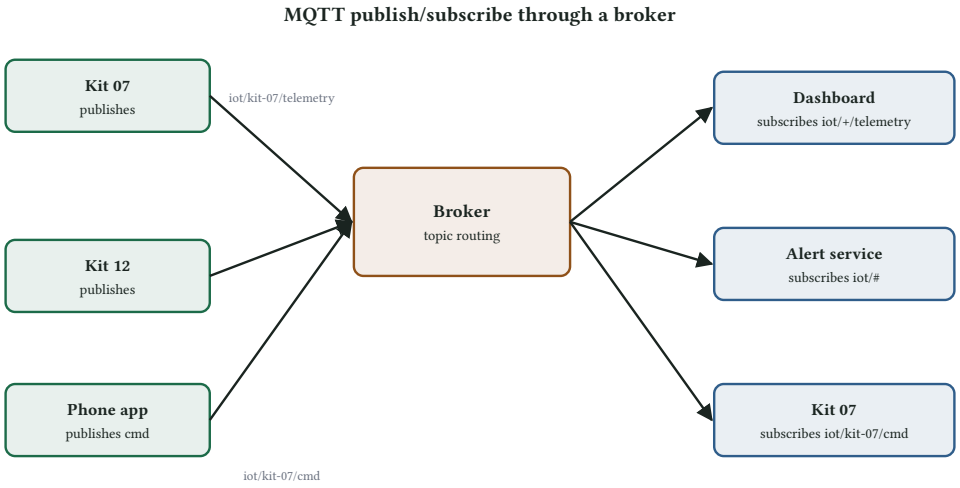


Figure 12.1: MQTT publish/subscribe. Publishers and subscribers only know the broker and the topic names.

Because every client connects *out* to the broker, MQTT works well with NAT and firewalls (Chapter 8.).

12.2.4.2 Topics and wildcards

Topics are hierarchical strings separated by `/`, such as `uci/iot/kit-07/temperature`. Subscribers can use two wildcards:

- `+` matches exactly one level: `uci/iot/+/temperature` matches every kit's temperature.
- `#` matches any number of levels, and must come last: `uci/iot/kit-07/#` matches everything from kit 7.

Messages are opaque bytes: text, JSON, binary or images. MQTT does not care.

12.2.4.3 Quality of service

Each publication and subscription chooses a **quality-of-service** (QoS) level:

Table 12.1: MQTT quality-of-service levels.

QoS	Guarantee	Mechanism	Use
0	At most once	Send and forget; no acknowledgment	Frequent readings where loss is acceptable
1	At least once	PUBLISH → PUBACK; retransmit until acknowledged; duplicates possible	Most commands and events
2	Exactly once	Four-step handshake: PUBLISH → PUBREC → PUBREL → PUBCOMP	Billing, counters, anything where duplicates cause harm

Higher QoS levels cost more messages, more memory and more time. The effective QoS of a delivery is the lower of the publisher's and subscriber's levels.

12.2.4.4 Sessions, retained messages and wills

MQTT includes features for devices that come and go:

- **Persistent sessions.** A client that connects with a clean start (clean session) flag of 0 asks the broker to remember its subscriptions and to queue QoS 1 and 2 messages while it is offline.
- **Retained messages.** A message published with the retain flag is stored by the broker and sent immediately to any new subscriber. This gives late joiners the last known state, such as the current position of a switch.
- **Last will and testament.** When connecting, a client can register a message for the broker to publish on its behalf if it disconnects unexpectedly, for example `status = offline`.
- **Keep-alive.** Clients send small pings when idle, so both sides can detect a dead connection.

MQTT 5, the current version, adds reason codes, message expiry, shared subscriptions for load balancing, topic aliases to shorten repeated topics, and user properties. **MQTT-SN** is a variant for non-TCP networks such as Zigbee and UDP.

12.2.4.5 MQTT on the ESP32

The PubSubClient library is a simple MQTT 3.1.1 client. Connecting securely to a cloud broker (such as an EMQX Serverless deployment) looks like this:

```
#include <WiFiClientSecure.h>
#include <PubSubClient.h>
#include "secrets.h"     // WIFI_*, MQTT_HOST, MQTT_USER, MQTT_PASS, ROOT_CA_PEM
#include "iot_pins.h"

WiFiClientSecure tls;
PubSubClient mqtt(tls);
const char* CMD_TOPIC    = "uci/iot/kit-07/cmd";
const char* STATUS_TOPIC = "uci/iot/kit-07/status";

void onMessage(char* topic, byte* payload, unsigned int len) {
  String msg((char*)payload, len);
  if (msg == "led_on")  digitalWrite(IoTPins::LED_1, HIGH);
  if (msg == "led_off") digitalWrite(IoTPins::LED_1, LOW);
}
```

```
void ensureConnected() {
  while (!mqtt.connected()) {
    // client ID, user, password, will topic, will QoS, will retain, will message
   if (mqtt.connect("kit-07", MQTT_USER, MQTT_PASS, STATUS_TOPIC, 1, true, "offline"))
{
      mqtt.publish(STATUS_TOPIC, "online", true);      // retained
      mqtt.subscribe(CMD_TOPIC, 1);
    } else {
      delay(2000);
    }
  }
}

void setup() {
  pinMode(IoTPins::LED_1, OUTPUT);
  // ... connect Wi-Fi as in Chapter 8 ...
  tls.setCACert(ROOT_CA_PEM);
  mqtt.setServer(MQTT_HOST, 8883);                       // MQTT over TLS
  mqtt.setCallback(onMessage);
  mqtt.setBufferSize(512);                               // default is small
}

void loop() {
  ensureConnected();
  mqtt.loop();                                           // process incoming packets
  static uint32_t last = 0;
  if (millis() - last > 10000) {
    last = millis();
    char buf[64];
    snprintf(buf, sizeof buf, "{\"pot\":%d}", analogRead(IoTPins::POT_ADC));
    mqtt.publish("uci/iot/kit-07/telemetry", buf);       // QoS 0
  }
}
```

PubSubClient publishes only at QoS 0 and subscribes at QoS 0 or 1. Libraries such as the ESP-IDF MQTT client support all three levels and MQTT 5.

12.2.5 Other application protocols

Table 12.2: Other application protocols used in IoT.

Protocol	Standard	Model	Notes
XMPP	IETF	Messaging and presence over XML streams	Originally for chat (Jabber); federated servers; presence ("online/away") is useful for devices. Verbose for constrained devices.
AMQP	OASIS, ISO/IEC 19464	Message queuing over TCP	From the financial sector; rich routing, delivery guarantees and token-based flow control; common between back-end services.
DDS / RTPS	OMG	Data-centric pub/sub, brokerless	Automatic discovery, very rich QoS (deadlines, durability, reliability); used in robotics (ROS 2), defense and vehicles.
SIP	IETF	Session setup	Negotiates voice, video and messaging sessions; used in intercoms and video doorbells.
IEEE 1888	IEEE	Time-series read/write over SOAP and XML	Facility and energy management.
LwM2M	OMA SpecWorks	Device management over CoAP	Standard objects for firmware update, connectivity monitoring and configuration.

12.2.6 Payload formats

The protocol moves bytes; the **payload format** gives them structure:

- **JSON** is human-readable and universal, but verbose.
- **CBOR** (Concise Binary Object Representation) is a binary encoding of the same data model as JSON, often half the size, and is the natural partner of CoAP.
- **Protocol Buffers** and similar schema-based formats are compact and strongly typed, at the cost of a shared schema.
- **Custom binary** structs are the smallest, and the hardest to evolve and debug.

12.2.7 Choosing a protocol

Table 12.3: Comparing the three main IoT application protocols.

	HTTP	CoAP	MQTT
Pattern	Request/response	Request/response + observe	Publish/subscribe via broker
Transport	TCP (+TLS) or QUIC	UDP (+DTLS/OSCORE)	TCP (+TLS)
Header size	Hundreds of bytes	4 bytes + options	2 bytes + topic
Reliability	From TCP	CON messages	QoS 0, 1, 2
Push to device	Polling, WebSockets	Observe	Native
Works through NAT	Device as client: yes	Needs care	Yes (clients connect out)
Best for	Cloud APIs, mains-powered devices	Constrained, low-power, LLNs	Telemetry and commands, many-to-many

Try it on your kit: MQTT command center

This exercise corresponds to the MQTT lecture activity and to Lab 5.

1. Connect the kit to the course broker. Publish the potentiometer reading every 2 s and subscribe to a command topic that controls the LEDs and the buzzer.
2. Use a desktop MQTT client (such as MQTT Explorer) to watch all kits with a wildcard subscription and to send commands to yours.
3. Register a last will. Unplug your kit and observe what other subscribers see.
4. Publish your LED state as a retained message. Restart the desktop client: what does it see immediately?
5. **Pick the QoS**: for each message type (telemetry, an emergency alarm, a "dispense medication" command, a firmware-available notice), choose QoS 0, 1 or 2 and justify your choice. What would duplicates or losses cost in each case?

A guided, step-by-step version for a lecture session is Activity 7.

12.3 In depth (optional)

Note

Optional, in-depth explanation. This section goes beyond the rest of the chapter into the math and physics behind it. Nothing later in the book depends on it; skip it on a first reading.

12.3.1 Reliability and overhead, quantified

Header sizes. For a small reading, the application header is a large share of the message:

Protocol	Fixed header	Plus
MQTT PUBLISH	2 bytes	2-byte topic length + topic; 2-byte packet ID at QoS 1–2
CoAP	4 bytes	Token (0–8) and options, often 10–20 bytes in all

Protocol	Fixed header	Plus
HTTP/1.1	Text request line	Text headers, typically 100–300 bytes

A 4-byte reading published at QoS 0 to topic `iot/t07/temp` (12 characters) is $2 + 2 + 12 + 4 = 20$ bytes of MQTT, against roughly 200 bytes of HTTP. Short topic names matter.

What QoS 1 buys. Suppose each MQTT packet is lost with probability p (after TCP gives up, or across a broker restart). A QoS 1 exchange succeeds when both the PUBLISH and the PUBACK arrive, with probability $s = (1-p)^2$. With up to r attempts, the message is delivered unless every attempt fails:

$$P_{\text{lost}} = (1-s)^r, \qquad E[\text{attempts}] \approx \frac{1}{s}. \tag{12.1}$$

If $p = 10\%$, $s = 0.81$; three attempts leave $0.19^3 \approx 0.7\%$ lost, at an average cost of about 1.23 exchanges. But a PUBACK that is lost *after* the PUBLISH arrived causes a resend the receiver already has: duplicates occur with probability up to $p(1-p)$ per exchange, which is why QoS 1 receivers must be idempotent (or deduplicate by packet ID). QoS 2 removes the duplicates with a four-packet exchange: twice the messages, and state at both ends.

CoAP's retransmission schedule. Confirmable CoAP messages are resent with exponential back-off. RFC 7252's defaults are an initial timeout chosen randomly between `ACK_TIMEOUT` = 2 s and 1.5 × that, doubled after each try, for up to `MAX_RETRANSMIT` = 4 retries. The longest time from first transmission to last retransmission is the `MAX_TRANSMIT_SPAN`:

$$T_{\text{span}} = T_0(2^R - 1)F = 2\,\text{s} \times 15 \times 1.5 = 45\,\text{s}, \tag{12.2}$$

where T_0 is `ACK_TIMEOUT`, R is `MAX_RETRANSMIT` and F is `ACK_RANDOM_FACTOR`.

A sender that hears nothing for 45 s plus the time it waits for the last reply knows the message almost certainly failed. Randomizing the timeout prevents a room of devices from retrying in lockstep.

Exercise. A telemetry stream sends one reading per second at QoS 0 over a link that loses 2% of packets. How many readings are lost per day? Would QoS 1 be worth it if each reading replaces the previous one on the dashboard?

Sources for this section: [39], [41].

12.4 Design trade-offs

Decision	Considerations
Request/response vs. pub/sub	Direct, simple interactions versus decoupled many-to-many distribution
Broker vs. brokerless	A broker simplifies NAT traversal and fan-out but is a central dependency and a target
Higher QoS	Fewer lost or duplicated messages at the cost of traffic, memory and latency
Persistent sessions	Commands survive disconnections, but the broker must store them
JSON vs. binary payloads	Readability and tooling versus size and parsing cost

12.5 Common pitfalls

- **Polling over HTTP from a battery device.** Each poll costs a full TCP and TLS setup.
- **Using one global MQTT topic.** Design a hierarchy that supports fine-grained access control and wildcards.
- **Duplicate client IDs.** Two clients with the same ID keep disconnecting each other.
- **Assuming QoS 1 means "exactly once".** Make command handlers **idempotent**: processing a duplicate should do no harm.

- **Forgetting `mqtt.loop()`.** Incoming messages are never processed and the connection times out.
- **Skipping TLS certificate verification** with `setInsecure()` outside the lab.
- **Messages larger than the client's buffer**, which are silently dropped.

> Advanced track

Measuring overhead. Send the same 20-byte reading 100 times by HTTP POST (new connection each time), HTTP with keep-alive, MQTT QoS 0 and MQTT QoS 1, and capture the traffic with Wireshark. Report bytes on the air and elapsed time per reading, and relate them to battery life.

Securing CoAP. DTLS secures a whole CoAP connection hop by hop; OSCORE (RFC 8613) protects individual CoAP messages end to end, even through proxies. Explain why that difference matters when a proxy caches responses for sleeping devices.

Topic design for access control. Design an MQTT topic hierarchy and broker access-control list for a building with 200 rooms, in which each room's devices may publish only their own telemetry, the building manager's dashboard may read everything, and only the HVAC controller may send commands to thermostats.

12.6 Check your understanding

1. What is decoupled in publish/subscribe that is not in request/response?
2. Why is HTTP polling expensive for a battery-powered device?
3. CoAP runs over UDP. How does it provide reliability when it needs it?
4. What is a piggybacked response in CoAP?
5. How does CoAP's Observe option differ from polling?
6. Which MQTT topic subscriptions would receive a message published to `farm/field3/soil/moisture`: `farm/+/soil/#`, `farm/#`, `farm/+/moisture`, `+/field3/+/moisture`?
7. Describe the message exchange for MQTT QoS 2. Why does it need four messages?
8. What are retained messages and last-will messages for?
9. Recommend a protocol for: (a) a coin-cell sensor on a Thread mesh; (b) a fleet of 10 000 Wi-Fi smart plugs controlled from a phone app; (c) a factory robot cell with hard real-time data distribution.

12.7 Further reading

- RFC 7252, *The Constrained Application Protocol (CoAP)* [39].
- OASIS, *MQTT Version 5.0* [41].
- HiveMQ, *MQTT Essentials* [42], a readable introduction to MQTT features.

13. Interoperability and Standards

Learning objectives

After this chapter you should be able to:

1. Define interoperability and distinguish its technical, syntactic, semantic and organizational levels.
2. Explain why early M2M systems formed vertical silos, and what "inverting the pipes" means.
3. Describe the role of an application services layer, and compare middleware and standards as routes to interoperability.
4. Summarize the main standards efforts: oneM2M, LwM2M, the W3C Web of Things, semantic models and domain standards.
5. Explain how Matter achieves interoperability in the smart home.
6. Describe your own device in a standard, machine-readable way.

13.1 Why it matters

In UCI's SIMHome project, a smart-home testbed built with Southern California Edison, one of the hardest problems was not any single device. It was making sensors, actuators, networks and computers from different companies and research teams work together, when each used its own protocols (sometimes ad hoc), hardware, operating system (sometimes none at all) and data conventions.

Every IoT system that grows beyond a single vendor meets the same problem. Two devices can share a network and an application protocol and still be unable to cooperate, because they disagree on what their messages mean. This chapter explains the levels at which devices must agree, and the standards that help them do so.

13.2 Concepts

13.2.1 What interoperability means

Interoperability is the ability of a system or device to exchange information with another, and to **understand and use** the information exchanged. The second half is the hard part.

The Levels of Conceptual Interoperability Model [43], adopted in IoT by organizations such as the Industrial Internet Consortium, describes interoperability as a stack of levels. Each depends on the ones below:

Table 13.1: Levels of interoperability.

Level	Question	Concerned with	Examples
Technical	Can the bytes get there?	Hardware, operating systems, communication protocols and infrastructure	Both devices speak IPv6 over Thread; both support MQTT
Syntactic	Can the receiver parse them?	Data formats and encodings	JSON, CBOR, XML, ASN.1, a fixed binary layout
Semantic	Does the receiver understand them?	The meaning of the content: units, names, relationships	"temp" is air temperature in °C, measured in the living room
Organizational	Do the organizations cooperate?	Business processes, agreements, policies across organizations, regions and cultures	A utility, a thermostat maker and a homeowner agree on a demand-response program

The protocols in the earlier chapters of Part III solve technical interoperability. Syntactic interoperability needs agreed formats. **Semantic interoperability** is the subject of most of this chapter: two devices may both send `{"temp": 21}`, yet one means degrees Celsius of the water in a boiler and the other means degrees Fahrenheit of a room, or a target rather than a measurement.

In healthcare, for example, standards such as **HL7** (and its modern form, FHIR) define the meaning of clinical information exchanged between hospital systems, while the **ISO/IEEE 11073** family (X73) covers personal health devices across several layers of the stack, so that a blood-pressure cuff from one maker can report to a hub from another.

13.2.2 From silos to horizontal platforms

Machine-to-machine (M2M) systems have existed for decades: remote meter reading, fleet tracking, alarm panels. They were typically built as **vertical silos**: one application, one type of device, one proprietary communication stack, tightly coupled end to end. A smart-metering system and a street-lighting system in the same city shared nothing, even when they used the same cellular network.

This fragmentation slowed the market. Every new application required building a new stack from the devices up. The response, in the phrase used by ETSI and by Rayes and Salam [1], was to **invert the pipes**: replace vertical silos with **horizontal** layers that many applications share. Devices connect through a common network (IP plus mobile infrastructure), and applications use a common **application services layer** that provides the functions every IoT application needs.

13.2.3 The application services layer

The key gap, in this view, is an abstraction layer offering a common set of services, so that an application can work with potentially any device without knowing its internals in advance. The value of IoT is in the data, not in how it is transported or which hardware produced it.

An application services layer typically provides:

- **network-agnostic message routing**, synchronous and asynchronous;
- a **subscribe/notify** model, with buffering so that devices are not overloaded with requests;
- a **uniform data storage model** and resource naming;
- a **uniform API** independent of programming language;
- **registration and discovery** of devices and applications;
- **group management**, to address many devices at once;
- **security**: authentication, authorization and access control;
- **device management**, such as configuration and firmware updates;
- **location** information.

There are two broad routes to this layer:

- **Middleware**: software that sits between devices and applications and translates each device-specific language into a common internal model. Middleware works with existing devices but must be updated for every new device type.
- **Standards**: devices and applications agree on common interfaces and data models in advance. Standards scale better but require industry agreement and take time.

Real systems usually combine both.

13.2.4 Standards for the application services layer

13.2.4.1 ETSI M2M and oneM2M

In 2012 the European Telecommunications Standards Institute (ETSI) published a service-layer standard for M2M. Its central idea was a **Service Capabilities Layer** (SCL) present in devices, gateways and network servers, exposing common functions (registration, data transfer, identification, group management, security, device management and location) through a **RESTful** API in which all data, device information and access rights are represented as resources.

Later in 2012, seven standards organizations from the US, Europe, Japan, China and Korea, including ETSI, launched **oneM2M** to develop a single global standard for the IoT service layer, and transferred their own overlapping work to it [44]. oneM2M defines a common service layer that can run over HTTP, CoAP, MQTT or WebSockets, and is used mainly in smart-city and telecom-operator platforms.

13.2.4.2 LwM2M

Lightweight M2M (LwM2M), from OMA SpecWorks, focuses on **device management** for constrained devices. It runs over CoAP (and also over MQTT and HTTP in recent versions) and defines a registry of standard **objects**: Device, Firmware Update, Connectivity Monitoring, Location, Temperature and many more, each with numbered resources. A server can update firmware or read battery level on any compliant device, regardless of maker.

13.2.4.3 The W3C Web of Things

Rather than replace existing protocols, the W3C **Web of Things** (WoT) describes them [45]. Its central artifact is the **Thing Description** (TD): a JSON-LD document that lists a device's **properties** (values that can be read or observed), **actions** (operations that can be invoked) and **events** (notifications), and for each one the protocol binding needed to use it (an HTTP URL, a CoAP resource, an MQTT topic):

```
{
  "@context": "https://www.w3.org/2022/wot/td/v1.1",
  "id": "urn:uuid:5f1b0c3e-iot-kit-07",
  "title": "IoT@UCI kit 07",
  "securityDefinitions": { "basic_sc": { "scheme": "basic" } },
  "security": "basic_sc",
  "properties": {
    "temperature": {
      "type": "number",
      "unit": "degree Celsius",
      "readOnly": true,
      "observable": true,
      "forms": [{ "href": "mqtts://broker.example.org/uci/iot/kit-07/temperature",
                  "op": ["readproperty", "observeproperty"] }]
    }
  },
  "actions": {
    "beep": {
       "input": { "type": "integer", "minimum": 50, "maximum": 2000, "description":
"duration in ms" },
      "forms": [{ "href": "mqtts://broker.example.org/uci/iot/kit-07/cmd" }]
    }
  }
}
```

A generic client that understands Thing Descriptions can use any described device without device-specific code.

13.2.4.4 Semantic models and ontologies

Units, names and relationships can be pinned down further with shared **ontologies**: formal vocabularies that define concepts and how they relate. Examples include the W3C **SOSA/SSN** ontology for sensors and observations, **SAREF** (ETSI's Smart Applications REFerence ontology) for appliances and energy, and **Brick** and **Project Haystack** for buildings. Linking a TD's `temperature` property to `saref:Temperature` with a unit from a standard units vocabulary removes the ambiguity in `{"temp": 21}`.

13.2.5 Matter: interoperability in the smart home

For years, smart-home buyers had to check whether a product worked with Amazon Alexa, Apple Home, Google Home or Samsung SmartThings. **Matter**, developed by the Connectivity Standards Alliance with those companies and hundreds of others, addresses all four levels of interoperability at once [46]:

- **Technical**: Matter runs over IPv6 on **Wi-Fi**, **Ethernet** or **Thread** (Chapter 11.), with BLE used for setup.
- **Syntactic and semantic**: a single **data model**. A device has numbered **endpoints**; each endpoint implements standard **clusters** (On/Off, Level Control, Color Control, Door Lock, Thermostat, and so on); each cluster defines **attributes**, **commands** and **events** with precise meanings and units. A "dimmable light" is defined the same way by every maker.
- **Security and onboarding**: every device carries a certificate chain proving it is a genuine, certified product; it is **commissioned** by scanning a QR code or entering a setup code; and all communication is encrypted.
- **Organizational**: certification by the Alliance, and **multi-admin** support, which lets the same device be controlled by several ecosystems at once.

Matter has grown steadily since its first release in 2022. Version 1.5 (late 2025) added cameras, and version 1.6 (mid-2026) added NFC-based commissioning and better support for sharing devices across ecosystems, along with a product-security specification. Thermostats, locks, lights, plugs, sensors, appliances and energy-management devices are all covered.

Matter is a good example of the "standards" route: devices interoperate because they implement the same data model, not because a middleware translates for them. Its cost is that every device must implement the full stack and pass certification, which is why the ESP32 family is popular for Matter products: Espressif provides a certified SDK.

Try it on your kit: describe your device

1. Write a Thing Description for your kit that exposes the potentiometer, the BMP280's temperature and pressure, and the LEDs, with the MQTT topics you used in Chapter 12.. Validate it with an online TD playground.
2. Identify every piece of information in your TD that addresses semantic, rather than syntactic, interoperability.
3. Exchange TDs with another team. Without looking at their code, write a short program on your laptop that reads their kit's temperature and turns on their LED. What was missing or ambiguous in their TD?
4. Which Matter clusters would your kit implement if it were a Matter device? Which kit features have no standard cluster?

13.3 In depth (optional)

Note

Optional, in-depth explanation. This section goes beyond the rest of the chapter into the math and physics behind it. Nothing later in the book depends on it; skip it on a first reading.

13.3.1 Why common models scale, and what encodings cost

The translation problem. If N systems each speak their own data model and every pair must interoperate, you need a translator for each ordered pair: $N(N-1)$ translators, or $N(N-1)/2$ if each is bidirectional. Ten systems need 45 bidirectional translators. If they all map to one shared model, you need only N mappings (to and from the common model): 10 instead of 45. The gap grows quadratically, which is the economic argument for standards such as Matter's data model, OPC UA information models, or W3C Thing Descriptions.

The argument has a cost: each mapping to a common model can lose information that the model does not represent. Two translations (A → common → B) can lose more than one direct translation. Good common models therefore include extension points.

Encoding size. The same reading, `{"t":23.5,"h":41}`, is 17 bytes as compact JSON. In **CBOR** (RFC 8949), a binary encoding with the same data model as JSON, it is 10 bytes:

Bytes	Meaning
`A2`	map with 2 pairs
`61 74`	text string of length 1: "t"
`F9 4D E0`	half-precision float 23.5
`61 68`	text string "h"
`18 29`	unsigned integer 41

The value 23.5 fits a 16-bit half-precision float exactly: $23.5 = 1.46875 \times 2^4$, so the sign is 0, the biased exponent is $4 + 15 = 19 = 10011_2$, and the 10-bit fraction is $0.46875 \times 1024 = 480 = 0111100000_2$, giving `0 10011 0111100000` = `0x4DE0`. With integer keys instead of text keys (as in many IoT profiles), the message shrinks to 8 bytes.

Smaller is not free: binary encodings need a decoder to be read by a person, and every party must agree on the key numbering. That agreement is exactly what a shared data model provides.

Exercise. Encode `{"t":-4,"h":100}` in CBOR by hand. (Negative integers use major type 1 and encode $-1-n$; integers 24–255 need one extra byte.) How many bytes is it, and how many in JSON?

Sources for this section: [45], [47].

13.4 Design trade-offs

Decision	Considerations
Middleware vs. standards	Middleware integrates existing devices; standards scale but need agreement and certification
Rich semantics vs. simplicity	Ontologies remove ambiguity but add complexity that small teams may not need
Ecosystem standard (Matter) vs. open platform	A standard gives consumer reach; a custom platform gives freedom and differentiation
Describing vs. replacing protocols	WoT describes existing devices; Matter and oneM2M ask devices to adopt a new stack

13.5 Common pitfalls

- **Assuming technical interoperability is enough.** Agreeing on MQTT does not mean agreeing on what a message means.
- **Missing units and reference frames.** Always state units, coordinate frames and whether a value is a measurement or a set point.
- **Ad hoc field names.** `temp`, `t`, `temperature_c` and `Temp` across one fleet make every consumer fragile.
- **Undocumented payloads.** If the only specification is the firmware source, nobody else can integrate.
- **Ignoring the organizational level.** Data sharing between organizations needs agreements on ownership, privacy and responsibility, not just formats.

ℹ Advanced track

Mapping between models. Map your kit's Thing Description onto LwM2M objects (for example IPSO Temperature, object 3303) and onto Matter clusters. Which concepts map cleanly, which need extensions, and what is lost in each mapping?

Standards as strategy. Interoperability standards are shaped by companies with commercial interests. Read about the history of Zigbee, Z-Wave, Thread and Matter, and write a short analysis of why earlier smart-home standards did not unify the market and what Matter did differently, organizationally as much as technically.

13.6 Check your understanding

1. Define interoperability, and explain why "exchange" alone is not enough.
2. Name the four levels of interoperability and give an IoT example of a failure at each.
3. What is meant by vertical silos in M2M, and by "inverting the pipes"?
4. List five functions an application services layer typically provides.
5. Compare middleware and standards as ways to achieve interoperability.
6. What three kinds of interaction does a W3C Thing Description describe?
7. In Matter, what are endpoints and clusters?
8. Why might a company choose not to adopt an interoperability standard?

13.7 Further reading

- A. Rayes and S. Salam [1], on interoperability and standardization of the application services layer.
- W3C, *Web of Things (WoT) Thing Description 1.1* [45].
- oneM2M, *Technical Specifications* [44].
- Connectivity Standards Alliance, *Matter specification* [46].
- F. Delmastro, "Pervasive communications in healthcare" [48], on standards for health-device interoperability.

IV Part IV — Trustworthy IoT

14 Security and Privacy 127
14.1 Why it matters 127
14.2 Concepts 128
14.3 In depth (optional) 133
14.4 Design trade-offs 134
14.5 Common pitfalls 134
14.6 Check your understanding 134
14.7 Further reading 135

15 Device Identity and Secure Elements 137
15.1 Why it matters 137
15.2 Concepts 137
15.3 In depth (optional) 140
15.4 Design trade-offs 141
15.5 Common pitfalls 141
15.6 Check your understanding 142
15.7 Further reading 142

16 Over-the-Air Updates and the Device Lifecycle 143
16.1 Why it matters 143
16.2 Concepts 143
16.3 In depth (optional) 147
16.4 Design trade-offs 148
16.5 Common pitfalls 148
16.6 Check your understanding 149
16.7 Further reading 149

17 Compliance and Regulation 151
17.1 Why it matters 151
17.2 Concepts 151
17.3 In depth (optional) 154
17.4 Design trade-offs 155
17.5 Common pitfalls 155
17.6 Check your understanding 156
17.7 Further reading 156

14. Security and Privacy

Learning objectives

After this chapter you should be able to:

1. Explain why security and privacy failures in IoT have physical and personal consequences.
2. Describe the challenges that make IoT harder to secure than traditional IT.
3. State the security requirements of an IoT system, from confidentiality to forward and backward secrecy.
4. Threat-model a device with STRIDE.
5. Describe attacks and countermeasures in the cloud, fog and sensing domains.
6. Use the basic cryptographic building blocks (hashes, MACs, symmetric encryption, signatures, TLS) correctly, and protect messages against tampering and replay.

14.1 Why it matters

When a web service is breached, data leaks. When an IoT system is breached, the damage can be physical. Researchers have shown that cars can be controlled remotely through their entertainment systems, and that implanted medical devices can be reprogrammed wirelessly. Doctors disabled the wireless features of a pacemaker implanted in a former US vice president because of exactly that concern. Botnets built from insecure cameras and routers have taken down major internet services (Chapter 9.).

IoT also raises privacy concerns that traditional computing rarely did. Smart-home sensors, wearables and cameras record intimate details of daily life: when people are home, when they sleep, how they move, even their heartbeats. Facial, speech and activity recognition amplify what raw sensor data can reveal, and **combining metadata** from several devices can expose far more than any one of them. UCI's TIPPERS project, an IoT testbed for privacy-preserving smart buildings, studied exactly these risks.

Security and privacy are not features to add at the end. They shape every layer of the design.

14.2 Concepts

14.2.1 Why IoT is hard to secure

Table 14.1: Why IoT security is difficult.

Challenge	Why it matters
Multiple technologies	An IoT system chains radios, gateways, cloud services, mobile apps and virtualization, each with its own vulnerabilities. Security is set by the weakest link.
Multiple verticals	eHealth, industry, smart homes and smart cities have very different requirements; one solution does not fit all.
Scale	Billions of devices make centralized, manual defenses impossible.
Big data	Each device produces streams of data that must be protected in transit and at rest.
Availability	Many systems must run continuously, sometimes to "five nines" (99.999%, about five minutes of downtime per year). Defenses must not take the system down.
Resource limits	Constrained devices are easy targets for denial of service, and expensive cryptography may not fit.
Remote locations	Unattended devices can be physically tampered with.
Mobility	Devices move between networks and owners' infrastructure.
Delay sensitivity	Security mechanisms must not break real-time deadlines.
Long lifetimes	Devices may stay in service for 10–20 years, far longer than many vendors support them.

14.2.2 Security requirements

Following Dabbagh and Rayes [2], a secure IoT system must provide:

- **Confidentiality**: only the intended parties can understand exchanged messages.
- **Integrity**: messages cannot be altered without detection.
- **Availability**: the service keeps working. Denial-of-service attacks target this.
- **Authentication**: every party is who it claims to be: devices, users, servers.
- **Authorization**: every party may perform only the operations it is permitted to.
- **Freshness**: data is current. **Replay attacks** resend an old, valid message, such as "unlock", to drive a device back into an old state.
- **Non-repudiation**: a party cannot deny an action it performed.
- **Forward secrecy** (in the group-membership sense): a device that leaves the network cannot understand communications exchanged after it leaves.
- **Backward secrecy**: a device that joins cannot understand communications exchanged before it joined.

14.2.3 Threat modeling with STRIDE

Before choosing defenses, identify what can go wrong. **Threat modeling** systematically asks how each part of a system could be attacked. Microsoft's **STRIDE** mnemonic names six threat categories, each violating one security property:

Table 14.2: STRIDE threats in IoT.

Threat	Violates	IoT example
Spoofing	Authentication	A fake sensor reports false readings; a fake server sends commands
Tampering	Integrity	A message changing "set 21 °C" to "set 35 °C" in transit; modified firmware
Repudiation	Non-repudiation	A user denies sending the "unlock" command; no logs exist
Information disclosure	Confidentiality	Unencrypted MQTT reveals when a house is empty
Denial of service	Availability	Jamming the radio; flooding a device until it crashes or drains its battery
Elevation of privilege	Authorization	A guest's phone app gains administrator control of a lock

A simple process: draw the system's data flows (devices, gateways, brokers, cloud services, apps), mark the **trust boundaries** where data crosses from one party's control to another's, and for each element and flow ask which STRIDE threats apply. Then rank threats by likelihood and impact, and choose mitigations for the important ones.

14.2.4 A three-domain view of IoT security

It helps to divide an IoT system into three domains, each with its own attacks. The taxonomy below follows Dabbagh and Rayes [2]:

- the **cloud domain**, which performs heavy computation and storage;
- the **fog domain** (the edge), which aggregates, preprocesses and stores data close to the devices (Chapter 18.);
- the **sensing domain**, the smart objects themselves and their networks.

14.2.4.1 Cloud domain

Cloud IoT applications run in virtual machines (VMs) or containers that share physical servers, managed by a **hypervisor**. Sharing creates attack opportunities:

Table 14.3: Cloud-domain attacks.

Attack	How it works	Countermeasures
Side-channel (hidden-channel)	A malicious VM placed on the same server infers a victim's secrets from shared hardware, for example by timing cache accesses	Hard isolation (dedicated caches or servers), co-residency allow-lists, cache flushing on switch
VM migration attacks	Attackers flood migration requests, advertise false capacity to attract VMs, or sniff and intercept VM memory in transit	Mutual authentication of servers, signed and encrypted control messages, sequence numbers, authenticated ARP
Theft of service	A VM games the scheduler to obtain more than its share of resources	Fair, precise scheduling and accounting
VM escape	A software bug lets a VM break out of the hypervisor and control the host	Patching, minimal hypervisors, nested isolation layers
Insider attacks	A data-center administrator accesses sensitive data	Encryption with keys the provider never sees; homomorphic encryption or secret-sharing storage for sensitive workloads

14.2.4.2 Fog domain

Fog devices virtualize computing resources close to the objects, so they inherit all cloud-domain attacks, and add some of their own:

- **Authentication and trust.** Fog devices may be owned by many, less well-known parties, and a mobile device may hop between them. Devices must authenticate the fog node and decide whether to trust its owner; **reputation systems** can help.
- **Migration over the internet.** VMs that follow mobile users move between fog nodes over the public internet, not a private data-center network, so migrations must be encrypted and authenticated.
- **Denial of service.** Lower computing capacity makes fog nodes easier to overwhelm.
- **Privacy.** A fog node can infer the location and habits of every connected device. Even wireless signals alone can reveal people's presence, location and breathing; signal **obfuscation** can make such inference harder.
- **Containers.** Lightweight containers share the operating system kernel as well as the hardware, which creates more opportunities for leakage and hijacking than full virtualization.

14.2.4.3 Sensing domain

The sensing domain faces attacks on its radios, its batteries and its routing:

Table 14.4: Sensing-domain attacks.

Attack	How it works	Countermeasures
Jamming	A transmitter interferes with legitimate signals at the receiver, or makes senders think the channel is busy. Jammers may be constant, deceptive (mimicking real frames), reactive (only when others transmit) or random.	Frequency hopping, spread spectrum, directional antennas, jamming detection (from signal strength and corrupted-packet ratios)
Vampire attacks	Drain batteries: *denial of sleep* (keeping devices awake, even with replayed encrypted control messages), flooding, and routing tricks such as *carousel* (looping source routes) and *stretch* (needlessly long routes)	Timestamps and freshness checks on control messages, rate limiting, path checks, disabling source routing
Selective forwarding	A malicious relay drops some packets (or all of them: a *black hole*)	Direct links where possible, path redundancy, trusted checkpoint nodes that acknowledge
Sinkhole	A node advertises a falsely short route to attract traffic, then reads or drops it	Intrusion detection using neighbor reports; authenticated routing

Physical access adds further risks: reading secrets from flash memory, attaching a debugger, or replacing firmware. Chapter 15. and Chapter 16. cover the hardware defenses.

14.2.5 Cryptographic building blocks

You do not need to design cryptography, and you never should. You do need to pick the right building block for each job:

Table 14.5: Cryptographic building blocks.

Goal	Building block	Common algorithms
Detect accidental corruption	Checksum / CRC	CRC-32 (not a security mechanism)
Fingerprint data	Cryptographic hash	SHA-256
Integrity + authenticity with a shared key	Message authentication code (MAC)	HMAC-SHA-256, AES-CMAC
Confidentiality with a shared key	Symmetric encryption	AES
Confidentiality + integrity together	Authenticated encryption (AEAD)	AES-GCM, AES-CCM, ChaCha20-Poly1305
Agree on a key over an open channel	Key exchange	ECDH (X25519, P-256)
Integrity + authenticity + non-repudiation	Digital signature	ECDSA (P-256), Ed25519
All of the above for a connection	Secure transport	TLS 1.3, DTLS 1.3

Three rules prevent most mistakes:

1. **Use standard, well-reviewed libraries** (mbedTLS on the ESP32) and standard protocols (TLS) rather than inventing your own.
2. **Encryption is not integrity.** An encrypted message can still be modified in ways that change its meaning unless it is also authenticated. Use AEAD or add a MAC.
3. **Integrity is not freshness.** A correctly signed "unlock" message can be replayed. Include a **counter**, a **timestamp** or a **nonce**, and reject messages that are old or repeated.

14.2.5.1 Signing messages on the kit

The ESP32 includes mbedTLS with hardware-accelerated SHA and AES. The following sketch attaches an HMAC and a monotonic counter to each command, so the receiver can detect both tampering and replay:

```
#include <mbedtls/md.h>

// Shared secret: provisioned per device, never hard-coded in a real product
static const uint8_t KEY[32] = { /* 32 random bytes */ };

String hmacHex(const String& msg) {
  uint8_t mac[32];
  const mbedtls_md_info_t* sha256 = mbedtls_md_info_from_type(MBEDTLS_MD_SHA256);
  mbedtls_md_hmac(sha256, KEY, sizeof KEY,
                  (const uint8_t*)msg.c_str(), msg.length(), mac);
  char hex[65];
  for (int i = 0; i < 32; i++) sprintf(hex + 2 * i, "%02x", mac[i]);
  return String(hex);
}

// Sender: "counter|command|mac"
String signCommand(uint32_t counter, const String& cmd) {
  String body = String(counter) + "|" + cmd;
  return body + "|" + hmacHex(body);
}

// Receiver: accept only authentic messages with a counter it has not seen
bool verifyCommand(const String& msg, String& cmdOut) {
  static uint32_t lastCounter = 0;              // persist this in flash in practice
  int a = msg.indexOf('|'), b = msg.lastIndexOf('|');
  if (a < 0 || b <= a) return false;
  String body = msg.substring(0, b);
  String mac  = msg.substring(b + 1);
  if (hmacHex(body) != mac) return false;       // tampered or wrong key
  uint32_t counter = msg.substring(0, a).toInt();
  if (counter <= lastCounter) return false;     // replayed or out of date
  lastCounter = counter;
  cmdOut = msg.substring(a + 1, b);
  return true;
}
```

In production code, compare MACs in **constant time** (so that timing does not reveal how many characters matched), store the counter in non-volatile memory so a reboot does not reset it, and keep the key in protected storage (Chapter 15.). With a shared key, any device holding the key can also forge messages; digital signatures, where only the sender holds the private key, remove that limitation and also provide non-repudiation.

14.2.6 Privacy by design

Security protects data from attackers; **privacy** protects people, including from the system's own operators. Practical principles include:

- **Data minimization**: collect only what the application needs, at the coarsest resolution that works.

- **Local processing**: compute on the device or at the edge, and send results instead of raw data (Chapter 18.).
- **Purpose limitation and retention limits**: use data only for the stated purpose, and delete it when it is no longer needed.
- **Transparency and control**: tell people what is collected, and let them see, export and delete it.
- **Separation of identity**: decouple device identifiers and data from the owner's identity where possible, while still allowing the owner to prove ownership.

Many of these principles are also legal requirements (Chapter 17.).

Try it on your kit: signed messages and STRIDE

This exercise corresponds to the security lecture activity.

1. Draw a data-flow diagram of the MQTT command center from Chapter 12.: kits, broker, laptop client. Mark the trust boundaries.
2. Apply STRIDE to each element and flow. List at least one threat per category and rank the top three.
3. Add the HMAC scheme above to the command topic. Show that a command with a modified payload is rejected, and that replaying a captured valid command is also rejected.
4. Reboot the receiving kit and replay an old command. Does it succeed? Fix the problem.
5. Which STRIDE threats does your HMAC scheme *not* address?

A guided, step-by-step version for a lecture session is Activity 8.

14.3 In depth (optional)

Note

Optional, in-depth explanation. This section goes beyond the rest of the chapter into the math and physics behind it. Nothing later in the book depends on it; skip it on a first reading.

14.3.1 How strong is strong enough?

Brute force and key length. A secret chosen uniformly from 2^k possibilities takes, on average, 2^{k-1} guesses to find. An attacker testing 10^{12} keys per second needs about $2^{55}/10^{12} \approx 10$ hours for a 56-bit key and $2^{127}/10^{12} \approx 5 \times 10^{18}$ years for a 128-bit key. The difference is not 2.3 times but 2^{72} times. Hence 128 bits as the modern floor for symmetric keys.

Human-chosen secrets are much weaker. A 6-digit PIN has $10^6 \approx 2^{20}$ values; an 8-character password of lowercase letters $26^8 \approx 2^{37.6}$. Default passwords shared by a whole product line have, effectively, zero bits. Rate limiting and lockout help online, but not against an attacker who has captured a hash and guesses offline.

Truncated authentication tags. A message authentication code (such as HMAC-SHA-256) is often truncated to save bytes on constrained links. An attacker who forges a message without the key succeeds with probability 2^{-t} per attempt for a t-bit tag. A 32-bit tag gives one success in about 4 billion attempts: acceptable on a LoRa link limited to a few hundred messages per hour, unacceptable on a fast network that allows millions of attempts.

Nonces and the birthday bound. Many encryption modes need a **nonce** that never repeats under the same key. If nonces are chosen randomly from 2^b values, the probability that n messages contain at least one repeat is approximately

$$P_{\text{collision}} \approx 1 - e^{-n^2/2^{b+1}} \approx \frac{n^2}{2^{b+1}}. \tag{14.1}$$

With 96-bit random nonces (the AES-GCM standard size), $n = 2^{32}$ messages give $P \approx 2^{-33}$: safe. With a 32-bit random nonce, collisions become likely after about $2^{16} = 65\,536$ messages, a few days of a sensor reporting every few seconds. A counter avoids collisions entirely, provided it survives reboots (see Activity 8's extension).

Risk as likelihood times impact. Threat modeling often scores each threat as

$$\text{risk} = \text{likelihood} \times \text{impact}, \tag{14.2}$$

on simple 1–5 scales. The product is crude, but it forces two separate judgments and makes the ranking discussable. A cheap attack with small impact (reading one room's temperature) can rank above an expensive attack with large impact (extracting a key from silicon), and that ranking is a design decision, not a fact.

Exercise. A command channel uses HMAC tags truncated to 24 bits and accepts up to 10 commands per second. How long would an attacker need, on average, to get one forged command accepted? What would you change?

Sources for this section: [49].

14.4 Design trade-offs

Decision	Considerations
Shared keys vs. public-key cryptography	Shared keys are fast and small but any key holder can impersonate others; public keys cost more computation but scale and give non-repudiation
Hop-by-hop vs. end-to-end security	Hop-by-hop (TLS to a broker) is simple but exposes data at each intermediary; end-to-end protects data through brokers and proxies
Security vs. availability	Locking out a device after failed checks can itself become a denial-of-service vector
Data collection vs. privacy	More data enables more features and more harm

14.5 Common pitfalls

- **Default or shared passwords** across all units.
- **Disabling certificate verification** to "make TLS work".
- **Rolling your own cryptography** or using outdated algorithms (MD5, SHA-1, DES, RC4).
- **No replay protection** on commands.
- **Secrets in source code or firmware images**, readable by anyone with the binary.
- **Debug interfaces left enabled** in production (serial consoles, JTAG, open telnet).
- **Logging sensitive data** in plain text.
- **Treating privacy as a policy document** rather than as design decisions about what data exists.

> **i** Advanced track

Red-team review. Exchange your project's architecture and repository with another team. Perform a structured security code review: threat-model their system with STRIDE, review their code for the pitfalls above, and write a short report with findings ranked by severity and concrete fixes. Chapter 22. describes the method in detail.

Lightweight cryptography. In 2023 NIST selected the Ascon family as its standard for lightweight cryptography, aimed at devices too constrained for AES-GCM. Read the NIST announcement and compare Ascon's code size and performance with AES-GCM on a microcontroller.

Post-quantum readiness. Devices deployed today may still be in service when large quantum computers can break ECC and RSA. NIST published its first post-quantum standards (ML-KEM and ML-DSA) in 2024. What would it take to support them on an ESP32-class device, and why does this matter most for firmware-signing keys?

14.6 Check your understanding

1. Give two examples of IoT security failures with physical consequences.
2. Why does combining metadata from several devices threaten privacy more than any single device?

3. Explain the difference between integrity and freshness, with an example attack that violates freshness but not integrity.
4. What are forward and backward secrecy in a group of devices?
5. Apply STRIDE to a smart door lock with a phone app. Give one threat per category.
6. How does a denial-of-sleep attack work, even when control messages are encrypted? How do you defend against it?
7. What is a sinkhole attack, and why is it specific to multi-hop networks?
8. Why is encryption alone not enough to protect a command message?

14.7 Further reading

- M. Dabbagh and A. Rayes, "Internet of Things Security and Privacy" [2], the source of the three-domain attack taxonomy used in this chapter.
- OWASP, *IoT Top 10* [50], a list of the most common IoT security weaknesses.
- V. Hassija et al., "A survey on IoT security: application areas, security threats, and solution architectures" [51].
- A. Shostack, *Threat Modeling: Designing for Security* [49].

15. Device Identity and Secure Elements

Learning objectives

After this chapter you should be able to:

1. Distinguish identifiers from credentials, and explain why every device needs a unique credential.
2. Explain public-key infrastructure, X.509 certificates and mutual TLS in the context of IoT devices.
3. Describe how a device proves its identity without revealing its secret.
4. Explain what a secure element is, and use the kit's ATECC608A to identify the device and generate random numbers.
5. Describe how devices are provisioned with identities during manufacturing and onboarding.
6. Compare hardware roots of trust: secure elements, eFuses, TPMs and trusted execution environments.

15.1 Why it matters

Almost every security property in Chapter 14. rests on one question: **is this really the device it claims to be?** A server that cannot tell a genuine thermostat from an impostor cannot trust its readings or decide who may send it commands. A device that cannot tell the genuine update server from an attacker will install whatever it is given.

Identity on the web is mostly about people, who can type passwords and answer second-factor prompts. Devices cannot. Their identity must be built in, unique to each unit, and protected even from someone who holds the device in their hands. This chapter explains how that is done, using the secure element in your kit as an example.

15.2 Concepts

15.2.1 Identifiers and credentials

Two things are easily confused:

- An **identifier** names a device: a serial number, a MAC address, a UUID, a client ID. Identifiers are not secret. Anyone can read a MAC address from the air.
- A **credential** proves the device is entitled to that name: a password, a shared key, or a private key with a certificate. Credentials must be secret, or at least their private part must be.

A device that authenticates with only its identifier ("I am kit-07") is not authenticated at all.

Credentials must also be **unique per device**. If every unit shares the same key or password, compromising one device, for example by reading its flash memory, compromises the whole fleet. The Mirai botnet (Chapter 9.) and many later attacks exploited exactly this.

15.2.2 Shared secrets and public keys

There are two broad ways for a device to prove its identity:

- **Shared secrets.** The device and the server both know a key, and the device proves it knows the key, for example by computing an HMAC over a random challenge from the server. This is cheap, but the server must store every device's secret, and anything that holds the secret can impersonate the device.
- **Public-key cryptography.** The device holds a **private key** that never leaves it, and the server knows the matching **public key**. The device proves its identity by **signing** a challenge; anyone with the public key can verify the signature, but only the private key could have produced it. The server stores nothing secret about the device.

Elliptic-curve cryptography (ECC) makes public-key operations practical on small devices: a 256-bit ECC key (such as on the NIST P-256 curve) offers security comparable to a 3 072-bit RSA key, with much smaller keys and signatures and much less computation.

15.2.3 Certificates and PKI

How does a server know which public key belongs to which device, without a list of every device in advance? With a **certificate**: a signed statement, in the standard **X.509** format, that binds a public key to an identity ("this public key belongs to device kit-07, made by IoT@UCI"). The statement is signed by a **certificate authority** (CA).

A **public-key infrastructure** (PKI) is the system of CAs, certificates and policies that makes this work. It forms a **chain of trust**:

1. A **root CA** certificate, whose key is kept offline and highly protected, is trusted in advance by the server.
2. The root signs an **intermediate CA** certificate, used day to day, for example at the factory.
3. The intermediate signs each **device certificate**.

To authenticate, the device presents its certificate and the intermediate certificate, and signs a challenge. The server checks each signature in the chain up to the root it trusts, then checks the device's signature with the public key in the device certificate.

The same structure protects the other direction. When your kit connects to a broker with TLS, it verifies the **server's** certificate chain against a root CA certificate stored in the firmware (`setCACert()` in Chapter 12.). When **both** sides present certificates, the connection uses **mutual TLS** (mTLS), the standard way for devices to authenticate to cloud IoT platforms such as AWS IoT Core and Azure IoT Hub.

Certificates also carry **validity periods**, and CAs can **revoke** compromised certificates. Devices that run for years need a plan for both, including a way to check the current time securely.

15.2.4 Protecting the private key

A private key stored in ordinary flash memory can be read by anyone who can attach a programmer to the chip or extract the firmware image. Protecting it is the job of a **hardware root of trust**: hardware that stores keys and performs cryptographic operations so that the key itself is never exposed, even to the device's own software.

15.2.4.1 Secure elements

A **secure element** is a small, dedicated chip designed to store keys and use them without revealing them, and to resist physical attacks: probing, glitching the power supply, and measuring power consumption or timing to infer secrets.

The kit's **Microchip ATECC608A** is a typical example [19]. Connected to the I²C bus at address 0x60 (often listed as 0xC0 in its 8-bit form), it provides:

- **16 key slots** for private keys, public keys, certificates and other data, each with configurable access rules;
- **ECC P-256** key generation inside the chip, **ECDSA** signing and verification, and **ECDH** key agreement;
- **SHA-256** and **HMAC**;
- a **hardware random number generator**;
- **monotonic counters**, useful for replay protection and limiting key use;
- a unique **72-bit serial number**.

A private key generated inside the ATECC608A **never leaves the chip**. Software asks the chip to sign a digest, and the chip returns only the signature. Even if an attacker dumps the ESP32's entire flash, the device's identity remains safe.

15.2.4.2 Configuration and locking

A new ATECC608A has two zones that must be **locked** before it can be used for real cryptography:

- the **configuration zone**, which defines what each slot may hold and how it may be used;
- the **data zone**, which holds keys and certificates.

Locking is permanent. Once the configuration zone is locked, it can never be changed. Until it is locked, some functions are restricted: for example, the random number generator returns a fixed test pattern

rather than random data. In the course, kits are provisioned and locked by the teaching staff; do not run locking examples on your own kit.

15.2.4.3 Using the secure element

Libraries such as SparkFun's ATECCX08A library or Microchip's CryptoAuthLib wrap the chip's command set. Reading the chip's identity and generating random numbers on a provisioned kit looks like this:

```
#include <Wire.h>
#include <SparkFun_ATECCX08a_Arduino_Library.h>

ATECCX08A atecc;

void setup() {
  Serial.begin(115200);
  Wire.begin(21, 22);
  if (!atecc.begin(0x60)) {                    // 7-bit address
    Serial.println("ATECC608A not found");
    while (true) delay(1000);
  }
  atecc.readConfigZone(false);                 // reads serial number and lock bits
  Serial.print("Serial: ");
  for (int i = 0; i < 9; i++) Serial.printf("%02X", atecc.serialNumber[i]);
  Serial.printf("\nConfig locked: %s  Data locked: %s\n",
                atecc.configLockStatus ? "yes" : "no",
                atecc.dataOTPLockStatus ? "yes" : "no");

  if (atecc.configLockStatus) {
   Serial.printf("Random: %lu\n", atecc.getRandomLong());  // true hardware randomness
  }
}

void loop() {}
```

With a key in slot 0, signing a 32-byte message digest is a single call (`atecc.createSignature(digest, 0)` in the SparkFun library), and the matching public key can be read out and placed in a certificate.

15.2.4.4 Other roots of trust

Table 15.1: Hardware roots of trust.

Mechanism	What it is	Examples
Secure element	Separate tamper-resistant chip with its own key storage and crypto	ATECC608A, NXP SE050, Infineon OPTIGA Trust M
eFuses	One-time-programmable bits inside the MCU, readable only by hardware	ESP32 keys for secure boot and flash encryption (Chapter 16.)
Digital signature peripheral	MCU block that signs with a key the CPU cannot read	ESP32-S2, S3, C3, C6 (not the original ESP32)
Trusted Platform Module	Standardized secure chip, common in PCs and gateways	TPM 2.0
Trusted execution environment	Isolated region of the main processor	Arm TrustZone-M, ESP-TEE
eSIM / iSIM	SIM functions, including identity, built into the module or chip	Cellular IoT devices

Certification schemes such as **PSA Certified** and **SESIP** evaluate how well a chip or device protects its roots of trust.

15.2.5 Provisioning: giving a device its identity

Provisioning is the process of giving each device its identity and credentials. It usually happens in two phases.

At manufacturing, each device:

1. generates a key pair inside its secure element (or has one injected in a secure facility);
2. has its public key signed by the manufacturer's intermediate CA, producing a unique device certificate;
3. stores the certificate in the secure element or flash;
4. is registered (or its certificate chain is registered) with the cloud service.

Secure-element vendors offer **pre-provisioned** parts, with keys and certificates loaded in the factory, so that even small companies can ship unique device identities without building their own secure facility.

At onboarding (or **commissioning**), the owner claims the device and connects it to their network and account. Examples include scanning a QR code with a phone app, as in Matter, or **just-in-time registration**, where a cloud platform automatically accepts any device whose certificate chains to a registered CA. Matter combines both phases: every certified device carries a **device attestation certificate** proving it is a genuine, certified product, and commissioning adds operational credentials for the owner's network.

15.2.6 Identity beyond authentication

A strong device identity enables much more than logging in:

- **authorization** policies that refer to specific devices or groups;
- **audit logs** that tie every action to a device, supporting non-repudiation;
- **secure updates**, where devices verify the identity of the update server and the signer of the firmware (Chapter 16.);
- **revocation**: disabling a stolen or compromised device without affecting the rest of the fleet;
- **ownership transfer**: re-provisioning a device when it is sold, without letting the previous owner retain access.

Try it on your kit: who are you?

1. Read and print your kit's ATECC608A serial number and lock status. Compare serial numbers with another team: are they really unique?
2. Compare the ESP32's MAC address (`WiFi.macAddress()`) with the secure element's serial number. Which one is a credential? Which could an attacker copy?
3. If your kit is provisioned, generate 10 random numbers with the secure element and 10 with `esp_random()`. Why might a product prefer one over the other?
4. Design a challenge–response protocol in which the course server verifies that a message really came from your kit's secure element. Which parts must be secret, and where does each live?

15.3 In depth (optional)

Note

Optional, in-depth explanation. This section goes beyond the rest of the chapter into the math and physics behind it. Nothing later in the book depends on it; skip it on a first reading.

15.3.1 Inside an ECDSA signature

The ATECC608A signs with **ECDSA over the NIST P-256 curve**. The arithmetic is worth seeing once, because it explains both why the scheme is secure and how it fails.

The setting. Points (x, y) satisfying $y^2 = x^3 - 3x + b \pmod{p}$, for a fixed 256-bit prime p and constant b, form a group under a geometric "addition" rule. A fixed **generator** point G has a large prime

order n: adding G to itself n times returns to the identity. Computing $Q = dG$ (adding G to itself d times, done efficiently by doubling and adding) is fast; recovering d from Q is the **elliptic-curve discrete logarithm problem**, believed to take about 2^{128} operations for P-256.

- **Private key:** a random integer d with $1 \le d < n$, generated inside the chip.
- **Public key:** the point $Q = dG$ (64 bytes: x and y).

Signing a message m, with z = the leftmost 256 bits of SHA-256(m):

1. Choose a fresh secret random k in $[1, n-1]$.
2. Compute $(x_1, y_1) = kG$ and $r = x_1 \bmod n$.
3. Compute $s = k^{-1}(z + r\,d) \bmod n$.
4. The signature is (r, s): two 32-byte integers, the 64 bytes in Activity 8.

Verifying with the public key Q:

1. Compute $w = s^{-1} \bmod n$, $u_1 = zw \bmod n$, $u_2 = rw \bmod n$.
2. Compute $(x_2, y_2) = u_1 G + u_2 Q$, and accept if $x_2 \equiv r \pmod n$.

It works because $u_1 G + u_2 Q = w(z + rd)\,G = w\,s\,k\,G = kG$.

Why the nonce must never repeat. If two signatures (r, s_1) and (r, s_2) on hashes $z_1 \neq z_2$ use the same k (visible because r is equal), subtracting the two signing equations gives

$$k = \frac{z_1 - z_2}{s_1 - s_2} \bmod n, \qquad d = \frac{s_1 k - z_1}{r} \bmod n. \tag{15.1}$$

Two signatures reveal the private key. Even a slightly biased k leaks it, given enough signatures. Generating k inside a secure element with a hardware random-number generator, or deterministically from the key and message (RFC 6979), removes this failure mode.

Why a signature proves so little about time. Nothing in (r, s) depends on *when* it was made. A valid signature proves who signed which bytes; freshness must be inside the signed bytes (a counter, timestamp or challenge), as Activity 8 shows.

Exercise. Verification needs two scalar multiplications, signing one. On a microcontroller where a P-256 scalar multiplication takes 50 ms in software, how many signatures per second can a gateway verify? What changes if the gateway uses a secure element that verifies in 60 ms?

Sources for this section: [19], [52], [53].

15.4 Design trade-offs

Decision	Considerations
Shared secrets vs. certificates	Shared secrets are simple and cheap; certificates scale, keep servers free of secrets and support revocation
Software keys vs. secure element	A secure element costs about a dollar per unit and a little complexity, and protects keys against physical extraction
Own PKI vs. pre-provisioned parts	Running a CA requires secure processes; pre-provisioned parts outsource them
Long-lived vs. short-lived certificates	Long-lived certificates avoid renewal in the field; short-lived ones limit damage from compromise

15.5 Common pitfalls

- **Using a MAC address or serial number as a credential.** Identifiers are public.
- **The same key in every device.** One extraction compromises the fleet.
- **Private keys in firmware images** or source repositories.
- **Locking a secure element with the wrong configuration.** It is irreversible.
- **Never planning for revocation or renewal.** Certificates expire, and some devices will be compromised.

- **Accepting any certificate.** Disabling verification makes TLS useless against impersonation.

> **i** Advanced track
>
> **Build a mini-PKI.** With OpenSSL on your laptop, create a root CA, an intermediate CA and a device certificate for your kit's public key. Configure a local Mosquitto broker to require client certificates, and connect the kit with mutual TLS. Then revoke the device certificate and show that the broker rejects it.
>
> **Side channels.** Software ECC implementations can leak private keys through timing or power consumption. Read about the "Minerva" and "TPM-Fail" timing attacks, and explain which properties of a secure element are designed to resist them.
>
> **Ownership transfer.** Design the process for reselling a used smart lock: what the seller does, what the buyer does, what the cloud service does, and how you guarantee that the seller can no longer open the door.

15.6 Check your understanding

1. What is the difference between an identifier and a credential? Give an example of each from your kit.
2. Why must credentials be unique per device?
3. What does a certificate bind together, and who vouches for the binding?
4. Describe the chain of trust from a root CA to a device certificate.
5. What is mutual TLS, and why is it common in IoT cloud platforms?
6. How can the ATECC608A sign a message without ever revealing its private key to the ESP32?
7. Why is locking the ATECC608A's configuration zone a decision to make carefully?
8. What happens at manufacturing and at onboarding when a device is provisioned?

15.7 Further reading

- Microchip, *ATECC608A CryptoAuthentication Device Summary Datasheet* [19].
- NIST IR 8259, *Foundational Cybersecurity Activities for IoT Device Manufacturers* [54], including device identity as a core capability.

16. Over-the-Air Updates and the Device Lifecycle

Learning objectives

After this chapter you should be able to:

1. Describe the stages of an IoT device's lifecycle and the security work each requires.
2. Explain why devices must be updatable, and what a robust over-the-air (OTA) update system needs on both server and device.
3. Explain A/B partitions, image verification and rollback on the ESP32.
4. Describe secure boot and the chain of trust from ROM to application.
5. Explain flash encryption and the role of eFuses in protecting a deployed device.
6. Plan staged rollouts, monitoring and end-of-life for a fleet of devices.

16.1 Why it matters

Every non-trivial program has bugs, and some of them are security vulnerabilities. For a laptop or phone, the fix arrives as an update the next week. For an IoT device, someone has to build an update mechanism into the product before it ships, or the bug is permanent. Recalling devices or sending technicians to thousands of sites is rarely possible.

A broken update mechanism is worse than none. An update that fails halfway through, or that installs firmware that crashes, can turn every device in the field into a "brick". An update mechanism that accepts unsigned firmware hands attackers a way to install their own code on the whole fleet. Getting updates right, and securing the boot process that runs them, is one of the most important engineering tasks in IoT.

16.2 Concepts

16.2.1 The device lifecycle

An IoT device passes through several stages, and security must be considered in each:

Table 16.1: The IoT device lifecycle.

Stage	Activities	Security concerns
Design	Architecture, component choice, threat modeling	Choosing hardware with a root of trust; planning for updates
Manufacture	Building and testing; flashing firmware	Protecting signing keys; injecting unique identities (Chapter 15.)
Provisioning and onboarding	Registering the device and connecting it to an owner	Proving genuineness; claiming ownership securely
Operation	Normal use, monitoring, remote configuration	Detecting faults and attacks; protecting data
Update	Fixing bugs and vulnerabilities, adding features	Authentic, reliable updates; recovering from failures
Transfer	Resale or change of owner	Removing the previous owner's access and data
Decommissioning	Retirement and disposal	Wiping data and credentials; revoking identities

Embedded programmers must assume a deployed device is physically unreachable (Chapter 4.). The update mechanism is therefore the only way to change a device's behavior for the rest of its life.

16.2.2 Over-the-air updates

An **over-the-air** (OTA) update delivers new software to a device through its network connection: directly over Wi-Fi or cellular, or through a gateway that relays it over a local link such as Bluetooth. OTA updates are routine in phones and cars, where they update everything from infotainment screens to engine and brake control software.

To reduce download time and cost, updates are often sent as a **delta**: only the differences between the old and new image, which the device applies to its current firmware. The delta, together with metadata and security credentials (the signature, version numbers, target hardware), forms an **update package**.

A complete OTA system has two halves:

Table 16.2: Requirements of an OTA update system.

Server side	Device side
Management of users, devices and groups	Agnostic to the network used to deliver the image
Scalable, highly available distribution (often a CDN)	Small enough to fit in flash alongside the application
Authentication and authorization for publishing updates	Verifies authenticity and integrity before installing
An API for devices to check for and fetch updates	Survives power loss at any moment during the update
Delta generation and compression	Supports rollback and recovery from bad images
Staged rollout, monitoring and analytics	Integrates with secure boot and the device's identity

16.2.3 A/B partitions on the ESP32

The ESP32 stores its firmware in external flash, divided into **partitions** described by a partition table. An OTA-capable layout includes:

- **`otadata`**: a small partition recording which application slot to boot;
- **`ota_0`** and **`ota_1`**: two application slots;
- optionally a **`factory`** slot with a known-good image;
- **`nvs`** for settings and other data partitions.

The device always runs from one slot and downloads updates into the other (Figure 16.1). Only when the new image has been completely written and verified does it update `otadata` to point at the new slot and reboot. If power fails during the download, the old image is untouched and still marked as the one to boot. This **A/B** (dual-bank) scheme is the core of reliable updates.

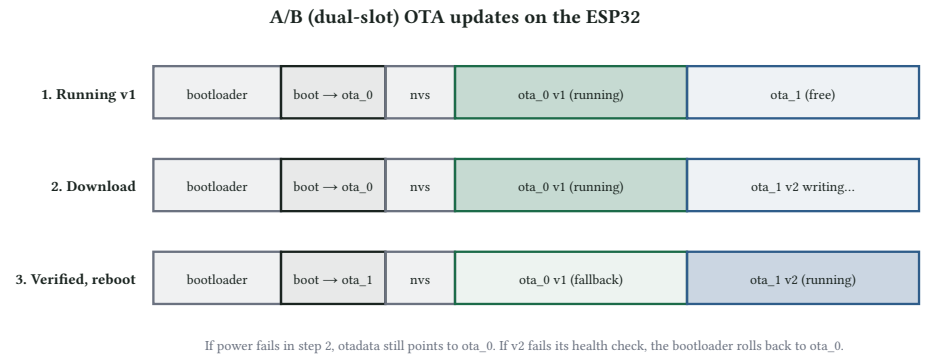


Figure 16.1: A/B updates: the new image is written to the inactive slot, verified, and only then selected for the next boot.

16.2.3.1 Rollback

A new image can be complete and correctly signed, yet still fail: it crashes on start-up, cannot connect to the network, or cannot reach the update server, which would make it impossible to fix with another update. With **rollback** enabled, the bootloader marks a newly installed image as *pending verification*. The new firmware must run its self-tests and then confirm that it is healthy; if it reboots before confirming, the bootloader returns to the previous image.

```
#include <esp_ota_ops.h>

void confirmFirmwareIfHealthy() {
  const esp_partition_t* running = esp_ota_get_running_partition();
  esp_ota_img_states_t state;
  if (esp_ota_get_state_partition(running, &state) == ESP_OK &&
      state == ESP_OTA_IMG_PENDING_VERIFY) {
    bool ok = connectWiFi() && reachUpdateServer() && sensorsRespond();  // your checks
    if (ok) esp_ota_mark_app_valid_cancel_rollback();
    else    esp_ota_mark_app_invalid_rollback_and_reboot();
  }
}
```

(Rollback must be enabled when the bootloader is built, which the default Arduino-ESP32 configuration does not do; it is available with ESP-IDF, or with PlatformIO using the ESP-IDF framework or custom configuration.)

16.2.4 Fetching updates

The Arduino-ESP32 core offers two update paths:

- **ArduinoOTA** lets a developer push new firmware from PlatformIO over the local network instead of over USB. It is convenient during development but does not scale to a fleet.

- **HTTP(S) update** lets a device pull an image from a server:

```
#include <WiFiClientSecure.h>
#include <HTTPUpdate.h>

void checkForUpdate() {
  WiFiClientSecure tls;
  tls.setCACert(ROOT_CA_PEM);                    // authenticate the update server
  httpUpdate.rebootOnUpdate(true);
  t_httpUpdate_return r = httpUpdate.update(
      tls, "https://updates.example.org/kit/firmware.bin", FIRMWARE_VERSION);
  switch (r) {
    case HTTP_UPDATE_NO_UPDATES: Serial.println("Up to date"); break;
    case HTTP_UPDATE_FAILED:
      Serial.printf("Update failed: %s\n", httpUpdate.getLastErrorString().c_str());
      break;
    case HTTP_UPDATE_OK: break;                  // not reached: the device reboots
  }
}
```

TLS authenticates the *server* and protects the image in transit. That is necessary but not sufficient: if the update server itself is compromised, it can serve malicious firmware over a perfectly valid TLS connection. The image itself must be **signed** by the developer, and the device must verify the signature before installing or booting it. That is the job of secure boot.

16.2.5 Secure boot and the chain of trust

Secure boot ensures that a device runs only software signed by its legitimate developer. It builds a **chain of trust** in which each stage verifies the next before handing over control:

1. **Key generation.** The developer creates a signing key pair and keeps the private key in a secure location, ideally a hardware security module.
2. **Signing.** Each firmware image is hashed, and the digest is signed with the private key. The signature is attached to the image.
3. **Verification at boot.**
   - The **ROM bootloader** runs first. It is in mask ROM inside the chip, so it cannot be modified and is trusted by definition.
   - The ROM reads the public key attached to the second-stage bootloader, hashes it, and compares the hash with a **digest stored in eFuses** (below). If they match, it verifies the bootloader's signature with that key.
   - The verified **bootloader** repeats the process for the application image, before running it and before accepting an OTA image.

Why store only a **hash** of the public key in the eFuses rather than the key itself? eFuse space is scarce (a 3 072-bit RSA key would not fit in a 256-bit block), and a hash is enough to confirm that the key presented with the image is the right one.

On the ESP32, **Secure Boot V2** (available on chip revision 3 and later) uses RSA-3072 signatures with the scheme above; newer Espressif chips also support ECDSA. The original Secure Boot V1 used a symmetric scheme and is now deprecated.

16.2.6 eFuses

The ESP32 contains 1 024 bits of **eFuse** memory: one-time-programmable bits, organized in four 256-bit blocks, that are burned by a high current and can never be reset. They hold:

- the **flash-encryption key** and the **secure-boot key digest**, in blocks that can be made **read-protected**, so that only the hardware can use them and no software (including the device's own firmware) can read them;
- **configuration bits** that permanently enable secure boot and flash encryption, and permanently **disable** features an attacker could abuse, such as JTAG debugging and the UART download mode used to flash new firmware over USB.

Because eFuses are permanent, enabling secure boot or flash encryption on a production device is a one-way decision. Mistakes can brick the chip, so these features are practiced on spare boards, never on a kit you need.

16.2.7 Flash encryption

Secure boot stops a device from running unauthorized code, but it does not stop someone from reading the flash chip to extract firmware, Wi-Fi passwords or keys stored in software. **Flash encryption** protects the contents of the external flash with AES:

- Firmware is flashed as plain text, and encrypted in place on the first boot.
- From then on, all memory-mapped reads from flash are decrypted transparently by hardware as code runs, and writes are encrypted, using the key in the eFuses.
- Someone who removes and reads the flash chip sees only ciphertext.

Secure boot and flash encryption are designed to be used together, with JTAG and UART download disabled, for production devices.

16.2.8 Operating a fleet

Shipping an update mechanism is only the start. Running it safely across thousands of devices requires:

- **Version management**: every device reports its firmware version, and the server knows which versions exist where.
- **Staged rollouts**: release to a small percentage of devices first, watch crash rates and health metrics, then expand. Halt automatically if failures rise.
- **Anti-rollback**: prevent attackers from installing an *older*, validly signed but vulnerable version, using a security version number checked against an eFuse or secure-element counter.
- **Monitoring and logging**: health reports, crash dumps and connectivity statistics, so problems are visible before customers report them.
- **Remote configuration**: changing parameters without a full firmware update.
- **A software bill of materials** (SBOM): a list of all software components and versions in each image, so that when a vulnerability is announced in a library, you know which devices are affected.
- **A support period**: a published commitment to how long security updates will be provided, increasingly a legal requirement (Chapter 17.).
- **End of life**: when support ends, devices should fail safe, and owners should be able to wipe them.

Try it on your kit: a safe update

Use a spare board or the Wokwi simulator for steps that change eFuses; never burn eFuses on a kit you need.

1. Print the running partition's label and the firmware version at boot. Use PlatformIO's partition table option to select a layout with two OTA slots.
2. Host two firmware versions on a laptop web server and update the kit over HTTP, then HTTPS. Confirm that the running slot changes.
3. Pull the USB cable in the middle of an update. What does the kit run when power returns, and why?
4. Write the health checks your project firmware should run before confirming a new image.
5. Write a short plan (one page) for updating 500 deployed kits: rollout stages, metrics, halt criteria and rollback.

16.3 In depth (optional)

Note

Optional, in-depth explanation. This section goes beyond the rest of the chapter into the math and physics behind it. Nothing later in the book depends on it; skip it on a first reading.

16.3.1 The numbers behind a safe rollout

How long does an update take to download? Time is size divided by *effective* throughput:

$$t = \frac{8\,S}{R_{\text{eff}}}. \tag{16.1}$$

A 1.2 MB firmware image over Wi-Fi at an effective 2 Mb/s takes about 5 s. Over LoRa at SF7 the raw rate is about 5.5 kb/s, but a 1% duty-cycle limit leaves 55 b/s on average: $8 \times 1.2 \times 10^6 / 55 \approx 1.7 \times 10^5$ s, almost two days, before counting lost packets. Hence delta updates (send only the changed bytes), compression, and firmware-update-over-the-air schemes for LPWANs that multicast fragments with forward error correction.

Canary rollouts catch rare failures. Suppose a new version bricks a fraction p of devices in some unusual configuration. Updating a random canary group of n devices first reveals the problem with probability

$$P_{\text{detect}} = 1 - (1 - p)^n. \tag{16.2}$$

For $p = 1\%$ and $n = 300$, $P \approx 1 - e^{-3} \approx 95\%$. To catch a 0.1% failure with the same confidence you need about 3000 canaries. The rule of thumb $n \approx 3/p$ for 95% confidence comes from $e^{-np} \approx (1-p)^n = 0.05$.

Why A/B partitions matter. If a power cut during an in-place update bricks the device with probability q per update, and the fleet does u updates per year, the expected number of bricked devices per year is $N u q$. For 100 000 devices, four updates a year and $q = 10^{-4}$, that is 40 truck rolls per year. A/B partitions (Figure 16.1) make q effectively zero for power loss: the old image is untouched until the new one has booted and confirmed itself.

Flash wear. NOR flash sectors survive on the order of 10^5 erase cycles. A full image rewrite once a week uses 52 cycles a year, harmless. Logging a sensor reading to the same sector every second would exhaust it in about a day, which is why logs use wear-leveling file systems or rotate across many sectors.

Exercise. Your fleet has 20 000 devices. You want 95% confidence of catching a defect affecting 0.5% of devices before more than 5% of the fleet is updated. Is that possible? How many stages would you use?

Sources for this section: [55], [56].

16.4 Design trade-offs

Decision	Considerations
Full images vs. deltas	Deltas save bandwidth but need the exact old image and more complex tooling
Push vs. pull	Pull (device checks periodically) works through NAT; push (server notifies) gives faster response
Automatic vs. user-approved updates	Automatic updates patch faster; user approval respects control but leaves devices unpatched
Secure boot and flash encryption	Strong protection, at the cost of irreversible configuration and harder debugging
Factory image	A known-good fallback costs flash space

16.5 Common pitfalls

- **One firmware slot.** A failed update bricks the device.
- **TLS without image signing.** A compromised server can push malicious firmware.
- **No health check before confirming.** A new image that cannot reach the update server can never be fixed.
- **Signing keys on a developer laptop** or in a CI system without protection.
- **Updating the whole fleet at once.**
- **No anti-rollback**, allowing downgrade attacks.
- **Burning eFuses on development boards** by accident.
- **Forgetting the end of life.** Unsupported devices stay on networks for years.

> **i** Advanced track
>
> **Update frameworks.** Compare the ESP-IDF OTA APIs with general-purpose update frameworks such as Mender, SWUpdate, RAUC, and MCUboot (used with Zephyr). The IETF SUIT working group defines a standard manifest format for IoT firmware updates; read its architecture document (RFC 9019) and map its concepts onto the ESP32.
>
> **The Uptane framework.** Vehicle updates use Uptane, which separates the roles of image repository and director, and uses multiple keys with different privileges, so that compromising one server is not enough to install malicious firmware. Explain how this design defends against a compromised update server.

16.6 Check your understanding

1. List the stages of an IoT device's lifecycle, with one security task for each.
2. Why is an A/B partition scheme safer than overwriting the running firmware?
3. What is rollback, and what must the new firmware do to prevent it?
4. Why is TLS on the update download not enough to guarantee authentic firmware?
5. Describe the chain of trust in secure boot. Why is the ROM bootloader trusted?
6. Why store a hash of the public key in eFuses rather than the key itself?
7. What does flash encryption protect against that secure boot does not?
8. What is a downgrade attack, and how is it prevented?

16.7 Further reading

- Espressif, *ESP-IDF Security Guides: Secure Boot V2 and Flash Encryption* [57].
- Espressif, *ESP-IDF Over The Air Updates (OTA)* [56].
- RFC 9019, *A Firmware Update Architecture for Internet of Things* [55].

17. Compliance and Regulation

Learning objectives

After this chapter you should be able to:

1. Explain why connected products are regulated, and name the main areas of regulation: radio, safety, environment, cybersecurity and privacy.
2. Describe how radio certification works in the US and the EU, and why pre-certified modules matter.
3. Summarize the main IoT cybersecurity laws and labels in the EU, UK and US, and the baseline requirements they share.
4. Identify the privacy obligations that apply to IoT data.
5. Recognize sector-specific rules, for example for medical devices.
6. Turn regulatory requirements into concrete engineering tasks for a product.

Not legal advice

This chapter explains the landscape as of late 2026 so that engineers know which questions to ask. Regulations change, and their details depend on the product and the market. Real products need review by compliance specialists, and you should always check the current text of any regulation you rely on.

17.1 Why it matters

A prototype on a lab bench is not a product. Before a connected device can be sold, it must usually show that its radio stays within its assigned frequencies and power, that it will not electrocute or burn anyone, that it does not contain banned substances, and, increasingly, that it meets minimum cybersecurity standards. Selling in another country often means another set of rules.

For a long time, cybersecurity was voluntary. That has changed. The European Union's **Cyber Resilience Act** makes security requirements mandatory for almost all products with digital elements sold in the EU, and its first obligations took effect in September 2026. The UK already bans default passwords. Asking "does it ship in the EU?" now changes design decisions from the first week of a project: whether the device has unique credentials, whether it can be updated, and how long the company will support it.

17.2 Concepts

17.2.1 Why connected products are regulated

Regulation protects people and shared resources from harms that individual buyers cannot judge for themselves:

- **Radio spectrum** is shared; one badly designed transmitter can disrupt many other users, including emergency services.
- **Electrical safety and fire** risks are invisible until something fails.
- **Hazardous substances and electronic waste** harm the environment.
- **Insecure devices** harm not only their owners but everyone else, as botnets such as Mirai showed (Chapter 9.).
- **Personal data** collected by devices can be misused.

Regulation typically works through **conformity assessment**: the manufacturer (or an independent test lab) shows that the product meets the relevant requirements, documents the evidence in a **technical file**, and marks the product accordingly.

17.2.2 Radio and electromagnetic compatibility

Every device that transmits radio waves, or that unintentionally emits radio-frequency noise, is regulated:

- **United States.** The **FCC** regulates radio devices. Most IoT radios operate in unlicensed bands under **Part 15** of the FCC rules, which limits transmit power, bandwidth, and, in some bands, how long a device may occupy a channel. Products need **FCC equipment authorization** and display an **FCC ID**.
- **European Union.** The **Radio Equipment Directive** (RED) covers radio products. A manufacturer who shows compliance affixes the **CE mark**. In the EU's 868 MHz band, devices must respect strict **duty-cycle** limits, often 1% of the time.

Testing a radio product costs time and money. **Pre-certified modules**, such as the ESP32-WROOM-32 or the RFM95W, have already passed radio testing. A product that uses one, without changing its antenna or RF design, can often rely on the module's certification for the radio part (in the US, under the **modular approval** rules), and only needs testing for unintentional emissions and the complete product. This is a major reason products use modules rather than bare radio chips.

Hardware is also regional: a LoRa radio built for the US 902–928 MHz band must not be operated in Europe, and vice versa (Chapter 10.).

17.2.3 Safety and environmental rules

- **Electrical safety** standards such as IEC/UL 62368-1 cover audio, video and IT equipment, including most IoT devices; battery-powered products also face battery-safety and transport rules for lithium cells.
- **RoHS** restricts hazardous substances such as lead, mercury and cadmium in electronics sold in the EU, and similar rules apply in other regions.
- **WEEE** requires producers in the EU to finance the collection and recycling of electronic waste, which is why products carry a crossed-out wheelie-bin symbol.

17.2.4 Cybersecurity regulation

17.2.4.1 A common baseline

Most IoT cybersecurity rules draw on the same baseline. The European standard **ETSI EN 303 645** [58] and the US **NIST IR 8259** series [54] are the most influential sources. Their key provisions are:

1. **No universal default passwords.** Each device must have a unique credential or require the user to set one.
2. **A vulnerability disclosure policy.** A public way for researchers to report vulnerabilities, and a process to act on reports.
3. **Software updates.** Devices must be securely updatable, and the manufacturer must state how long updates will be provided.
4. **Secure storage of credentials and sensitive data.**
5. **Secure communication**, using current cryptography.
6. **Minimized attack surface**: unused ports, services and debug interfaces disabled.
7. **Software integrity**: verifying software before it runs.
8. **Protection of personal data.**
9. **Resilience** to outages of networks and power.
10. **Easy deletion** of user data, and ways for users to configure the device securely.

Each of these maps directly onto techniques in Part IV.

17.2.4.2 European Union

- **RED cybersecurity requirements.** Since 1 August 2025, internet-connected radio equipment sold in the EU must meet cybersecurity, privacy and anti-fraud requirements under a delegated act of the Radio Equipment Directive. The harmonized standards **EN 18031-1, –2 and –3** describe how to meet them.
- **The Cyber Resilience Act** (CRA) [59] covers almost all hardware and software "products with digital elements". It entered into force in December 2024 with a staged timeline:
  - **From 11 September 2026**, manufacturers must report actively exploited vulnerabilities and severe security incidents to authorities (through ENISA's single reporting platform) within strict deadlines.
  - **From 11 December 2027**, the full requirements apply: security by design and by default, a vulnerability-handling process, security updates for a defined **support period** (generally at least five years,

or the expected product lifetime if shorter), an SBOM, conformity assessment and CE marking. Products deemed "important" or "critical" face stricter assessment.
- **The EU Data Act**, applying from September 2025, gives users of connected products rights to access and share the data their devices generate.

17.2.4.3 United Kingdom

The **Product Security and Telecommunications Infrastructure (PSTI)** regime, in force since 29 April 2024, requires consumer connectable products sold in the UK to have no universal default passwords, to publish a way to report vulnerabilities, and to state the minimum period for which security updates will be provided.

17.2.4.4 United States

The US has no single federal IoT security law for consumer products. Instead:

- **California's SB-327** (in force since 2020), and a similar Oregon law, require connected devices to have "reasonable security features", including unique passwords or a forced password change on first use.
- The **IoT Cybersecurity Improvement Act** of 2020 sets security requirements for IoT devices bought by federal agencies, based on NIST guidance.
- The **US Cyber Trust Mark** is a voluntary FCC labeling program for consumer IoT products, based on NIST criteria. After its first lead administrator withdrew, the FCC selected the ioXt Alliance in April 2026; as of late October 2026 the FCC was still finalizing testing procedures, and no labeled products had yet reached store shelves [60].
- Sector regulators add their own rules (below).

17.2.5 Privacy and data protection

IoT devices often collect **personal data**, which data-protection laws regulate regardless of the product's security:

- The EU's **General Data Protection Regulation** (GDPR) requires a lawful basis for processing, data minimization, purpose limitation, security, and rights for individuals to access, correct and delete their data. Its principle of **data protection by design and by default** is effectively an engineering requirement.
- In the US, state laws such as California's **CCPA** (as amended by the CPRA) give consumers rights over personal information, and the **FTC** acts against companies whose security or privacy practices are deceptive or unfair.
- Data that crosses borders may face transfer restrictions.

The design principles in Chapter 14. (minimization, local processing, retention limits, transparency) are the practical way to meet these obligations.

17.2.6 Sector-specific rules

Some domains add their own requirements on top of the general ones:

- **Medical devices.** In the US, the FDA regulates connected medical devices; since 2023, manufacturers of "cyber devices" must submit cybersecurity plans, an SBOM and evidence of secure design with their applications. The EU Medical Device Regulation includes similar requirements. Health data may also fall under HIPAA.
- **Automotive.** UN Regulations No. 155 (cybersecurity management) and No. 156 (software updates) apply to new vehicles in many markets.
- **Industrial systems.** The **IEC 62443** series defines security levels and processes for industrial automation and control systems.
- **Critical infrastructure and government procurement** often add their own lists of approved or prohibited components and suppliers.

17.2.7 From regulation to engineering tasks

Regulatory requirements become concrete work items:

Table 17.1: Turning regulation into engineering work.

Requirement	Engineering task
No default passwords	Per-device credentials from provisioning (Chapter 15.); first-use setup flow
Secure updates for a support period	Signed OTA with rollback (Chapter 16.); budget for maintenance
Vulnerability handling and reporting	A `security.txt` or disclosure page; an internal triage process; ability to ship fixes quickly
SBOM	Generate a component list at build time
Secure storage and communication	Secure element or flash encryption; TLS with verification
Minimal attack surface	Disable debug interfaces and unused services in production builds
Data protection	Data inventory; minimization; deletion and export features
Radio compliance	Pre-certified modules; correct regional frequency plans; respect duty cycles

Try it on your kit: does it ship in the EU?

This exercise corresponds to the compliance discussion in lecture.

1. Assume your course project will be sold as a consumer product in the EU and the UK. Go through the ETSI EN 303 645 baseline above and mark each provision as *met*, *partly met* or *not met* by your current design.
2. List the three changes that would close the biggest gaps, and estimate the effort for each.
3. Your kit's LoRa radio is a US 915 MHz part. What would have to change to sell the product in Europe?
4. Write a one-paragraph "security support statement" for your product, stating how long it will receive security updates and how people can report vulnerabilities.

17.3 In depth (optional)

Note

Optional, in-depth explanation. This section goes beyond the rest of the chapter into the math and physics behind it. Nothing later in the book depends on it; skip it on a first reading.

17.3.1 Decibels, power limits and duty cycles

Radio rules are written in units that are easy to misread, and a factor of 10 is easy to miss.

Decibels relative to a milliwatt. Power in dBm is

$$P_{\mathrm{dBm}} = 10 \log_{10} \frac{P}{1\,\mathrm{mW}}, \tag{17.1}$$

so 1 mW = 0 dBm, 25 mW ≈ 14 dBm, 100 mW = 20 dBm and 1 W = 30 dBm. Each 3 dB doubles the power; each 10 dB multiplies it by ten. Gains and losses then simply add.

EIRP and ERP. Limits usually apply to the power that leaves the antenna in its strongest direction, not to the transmitter's output. **EIRP** (equivalent isotropically radiated power) is transmitter power plus antenna gain in dBi, minus cable loss. **ERP** uses a half-wave dipole as the reference instead of an ideal isotropic antenna, and a dipole has 2.15 dBi of gain:

$$\mathrm{EIRP} = \mathrm{ERP} + 2.15\ \mathrm{dB}\ . \tag{17.2}$$

A rule of "25 mW ERP" is therefore 14 dBm ERP, or about 16.15 dBm EIRP. Swapping the kit's spring antenna for a 6 dBi directional antenna adds 6 dB of EIRP without touching the radio, and can turn a compliant device into a non-compliant one.

Duty cycle. Where a band limits transmit time to a fraction δ, a device may transmit $3600\,\delta$ seconds per hour; at 1%, 36 s. Chapter 10's time-on-air formula converts that into packets per hour. Compliance is per device, so a firmware bug that retries forever is a regulatory problem as well as a battery problem.

Certification is about the whole product. A pre-certified radio module simplifies compliance only if it is used within the conditions of its grant: the listed antennas, power settings and host constraints. Changing any of them can require new testing.

Exercise. A module transmits at 20 dBm into a cable with 1.5 dB loss and a 3 dBi antenna. What is its EIRP in dBm and in milliwatts? What is its ERP?

Sources for this section: [58].

17.4 Design trade-offs

Decision	Considerations
Pre-certified module vs. custom radio design	Modules cost more per unit but save testing time, money and risk
Global vs. regional products	One design for all markets simplifies manufacturing but must meet the strictest rules; regional variants add SKUs
Long support periods	Required by law in some markets; costly for low-margin products
Collecting data	Each data type collected adds legal obligations as well as value

17.5 Common pitfalls

- **Treating compliance as a final step.** Many requirements, such as unique credentials and updatability, cannot be added at the end.
- **Modifying a pre-certified module's antenna** and assuming the certification still applies.
- **Using the wrong regional radio frequencies.**
- **Shipping without a vulnerability disclosure contact.**
- **Promising updates without budgeting for them.**
- **Collecting personal data "just in case".**

> ℹ Advanced track
>
> **Read the source.** Read the requirements annex of the Cyber Resilience Act (Annex I) and compare it, provision by provision, with ETSI EN 303 645. Where does the CRA go further?
>
> **Compliance as competitive advantage.** Some companies treat strict compliance as a cost; others market it. Choose a consumer IoT company and examine its published security policy, support periods and certifications. Write a short assessment of whether its practices would meet the EU's 2027 requirements.

17.6 Check your understanding

1. Name five areas in which connected products are regulated.
2. Why can a product that uses a pre-certified radio module often skip some radio testing?
3. List five provisions shared by most IoT cybersecurity baselines.
4. What changed for manufacturers selling in the EU on 11 September 2026, and what changes on 11 December 2027?
5. What three things does the UK PSTI regime require?
6. How does GDPR's "data protection by design and by default" translate into engineering decisions?
7. Give an example of a sector with additional IoT requirements, and one such requirement.

17.7 Further reading

- ETSI EN 303 645, *Cyber Security for Consumer Internet of Things: Baseline Requirements* [58].
- Regulation (EU) 2024/2847, the *Cyber Resilience Act* [59].
- NIST IR 8259 series, *Foundational Cybersecurity Activities for IoT Device Manufacturers* [54].

V

Part V — Intelligence at the Edge

18 AI on IoT and Edge ML 159
18.1 Why it matters 159
18.2 Concepts 159
18.3 In depth (optional) 163
18.4 Design trade-offs 164
18.5 Common pitfalls 164
18.6 Check your understanding 164
18.7 Further reading 164

19 Where IoT Is Heading 165
19.1 Why it matters 165
19.2 Concepts 165
19.3 In depth (optional) 167
19.4 Design trade-offs 168
19.5 Common pitfalls 168
19.6 Check your understanding 169
19.7 Further reading 169

18. AI on IoT and Edge ML

Learning objectives

After this chapter you should be able to:

1. Explain the limits of cloud-only processing for IoT, and place computation along the device–edge–cloud continuum.
2. Describe Edge AI and TinyML, and give examples of where they are used.
3. Review the supervised-learning concepts most used in IoT: classification, regression, features, training and testing.
4. Explain why model size matters on microcontrollers, and how quantization and pruning shrink models.
5. Follow the TinyML workflow from data collection to on-device inference.
6. Evaluate an edge model on accuracy, latency, memory and energy.

18.1 Why it matters

IoT devices produce enormous volumes of raw data: accelerations sampled hundreds of times a second, audio, images, vibration spectra. The first generation of IoT systems sent all of it to the cloud for analysis. That works for a few devices with good connections, but it breaks down at scale, for battery-powered devices, and whenever a decision is needed immediately or the data is too sensitive to leave the device.

Advances in algorithms, model compression and microcontroller hardware now make it possible to run machine-learning models directly on devices like the ESP32. A smart doorbell can tell a person from a package without contacting a server; a wearable can detect a fall in milliseconds; a vibration sensor can flag a failing bearing and send one alert instead of streaming data all day. This chapter explains how.

18.2 Concepts

18.2.1 The limits of cloud-only processing

Sending all raw data to the cloud has four costs:

- **Latency.** A round trip to the cloud takes tens to hundreds of milliseconds, and much longer on poor networks, too slow for emergency stops, fall detection or control loops.
- **Bandwidth.** Streaming audio or video from many devices consumes network capacity and, on cellular links, money.
- **Energy.** Transmitting data usually costs far more energy than processing it locally (Chapter 10.).
- **Privacy.** Raw data, such as audio from a living room, must leave the device, increasing the risk of leaks and misuse (Chapter 14.).

There is also **availability**: a device that depends on the cloud to make every decision stops working when the network does.

18.2.2 The device–edge–cloud continuum

Computation can happen at several places between the sensor and the data center:

Table 18.1: The device–edge–cloud continuum.

Tier	Examples	Strengths	Limits
Device (extreme edge)	Microcontroller in a sensor or wearable	Lowest latency, works offline, raw data never leaves	Kilobytes of RAM, tight energy budget
Edge / fog	Gateway, router, phone, on-premises server	More compute, sees many devices, still local	Must be deployed and managed; shared by many devices
Cloud	Data-center servers	Effectively unlimited compute and storage; fleet-wide view	Latency, bandwidth, connectivity and privacy costs

Fog computing describes the middle tier: computing, storage and networking resources between devices and the cloud, often virtualized and shared, close to where the data is produced. Real systems split work across tiers. A camera might detect motion on the device, identify objects at the edge, and train improved models in the cloud.

18.2.3 Edge AI and TinyML

Edge AI means running machine-learning models on devices or edge nodes instead of only in the cloud. Only the result ("person detected", "fall", "abnormal vibration") needs to be sent. Its benefits are real-time response, offline operation, lower bandwidth and energy, and privacy.

TinyML is the part of Edge AI that runs on the smallest devices: microcontrollers with kilobytes of memory and milliwatts of power. It sits at the overlap of machine learning and embedded systems, and applies when rule-based logic is not enough but the device is too small for conventional ML.

Typical applications include:

- **keyword spotting** ("Hey…") in smart speakers, running continuously at very low power;
- **activity recognition** and **fall detection** in wearables;
- **anomaly detection** in industrial machines (predictive maintenance);
- **visual wake words** (is there a person in the image?) in cameras and doorbells;
- **gesture recognition** from IMUs.

18.2.4 A machine-learning refresher

Machine learning teaches a computer rules from data instead of hard-coding them. Instead of writing "if acceleration > X then running", you collect labeled acceleration data and **train** a model that learns the boundary.

- **Supervised learning** uses labeled examples. **Classification** predicts a category (walking or running); **regression** predicts a number (the temperature in an hour, a motor's speed from its sound).
- **Unsupervised learning** finds structure without labels, for example clustering, or **anomaly detection**: learning what "normal" looks like and flagging what is not.
- **Training** adjusts the model's parameters to reduce errors on labeled examples. **Testing** evaluates it on data it has never seen, to check that it **generalizes** rather than memorizing.

Common supervised models for edge IoT include:

Model	Idea	Edge suitability
k-nearest neighbors	Predict from the labels of the most similar stored examples	Simple, but must store training data
Decision trees and small forests	A sequence of threshold tests	Tiny, fast, interpretable; easy to export as code
Linear and logistic regression	Weighted sum of features	Tiny and fast

Model	Idea	Edge suitability
Neural networks (MLPs, 1D/2D CNNs, small RNNs)	Layers of learned weights; CNNs capture spatial or temporal patterns	Most powerful; need compression to fit

18.2.5 Features from sensor data

Raw sensor streams are usually turned into **features** before (or inside) a model. For time series, a common pipeline is:

1. **Windowing**: split the stream into fixed-length windows, for example 2 s of accelerometer data at 50 Hz (100 samples), often overlapping.
2. **Feature extraction**: compute summary statistics for each window and axis, such as mean, standard deviation, minimum, maximum, energy, zero crossings or the magnitudes of frequency components.
3. **Classification** of each window.

Neural networks can learn their own features from raw windows, at the cost of more computation and data.

18.2.6 Why model size matters

A model that works in a notebook may be far too large for a microcontroller:

	Cloud server	ESP32
Memory	Hundreds of GB of RAM	520 KB of SRAM (part used by Wi-Fi and the RTOS)
Storage	Terabytes	4 MB of flash, shared with the firmware
Compute	GPUs, TFLOPS	Two 240 MHz cores, single-precision FPU
Power	Hundreds of watts	Tens to hundreds of milliwatts

Edge models must therefore be **tiny and efficient**, trading some accuracy for size and speed. The main compression techniques are:

- **Quantization**: storing weights and activations as 8-bit integers instead of 32-bit floats. This makes models about **four times smaller** and much faster on hardware without a strong floating-point unit, usually with only a small loss of accuracy. **Post-training quantization** converts a trained model; **quantization-aware training** simulates quantization during training for better accuracy.
- **Pruning**: removing weights or whole neurons that contribute little.
- **Knowledge distillation**: training a small "student" model to imitate a large "teacher".
- **Architecture choice**: using small, efficient model families designed for embedded devices.

18.2.7 The TinyML toolchain

Tool	Role
LiteRT for Microcontrollers (formerly TensorFlow Lite for Microcontrollers)	The most widely used inference runtime for MCUs; runs quantized models converted from TensorFlow, Keras, PyTorch or JAX
Edge Impulse	End-to-end platform: collect data from a device, label, train, optimize and generate firmware
ExecuTorch	PyTorch's lightweight runtime for edge and embedded devices
Vendor libraries	Espressif ESP-DL, esp-tflite-micro and ESP-DSP; Arm CMSIS-NN; ST X-CUBE-AI; NXP eIQ. These

Tool	Role
	use each chip's accelerators and special instructions.

The typical workflow is:

1. **Collect and label data** on the real device, in realistic conditions.
2. **Train** a model on a PC, often in a Jupyter notebook.
3. **Optimize**: quantize, prune, and check accuracy after each step.
4. **Convert** to the runtime's format (for LiteRT, a `.tflite` flat buffer, embedded in firmware as a C array).
5. **Deploy** to the device and run inference on live data.
6. **Evaluate on the device**: accuracy, latency, memory and energy, then iterate.

18.2.8 A tiny example: classifying motion with a decision tree

Not every edge model needs a neural network. A decision tree trained in Python (for example with scikit-learn) can be exported as plain C code. This one classifies 2-second accelerometer windows from the MPU-6050 (Chapter 6.) as still, walking or running, using two features:

```
// Features computed over a 2 s window of acceleration magnitude (in g)
struct Features { float meanMag; float stdMag; };

Features extract(const float* mag, int n) {
  float sum = 0, sq = 0;
  for (int i = 0; i < n; i++) { sum += mag[i]; sq += mag[i] * mag[i]; }
  float mean = sum / n;
  return { mean, sqrtf(fmaxf(sq / n - mean * mean, 0.0f)) };
}

// Exported from a depth-2 decision tree (thresholds learned from labeled data)
const char* classify(const Features& f) {
  if (f.stdMag < 0.05f) return "still";
  if (f.stdMag < 0.45f) return "walking";
  return "running";
}
```

The whole model is a few comparisons and takes microseconds. The thresholds above are illustrative; yours must come from your own labeled data, collected on your kit and with the sensor mounted the way it will be used.

18.2.9 Evaluating an edge model

Accuracy alone does not decide whether a model is suitable for a device. Measure, on the device:

- **accuracy** on held-out data collected separately from the training data, preferably from different people or machines; also precision and recall when classes are imbalanced;
- **latency** per inference;
- **memory**: flash for the model and code, RAM for activations ("tensor arena" in LiteRT);
- **energy** per inference and per day, given how often inference runs.

Two practical risks recur:

- **Distribution shift.** Models trained in the lab fail in the field when conditions differ: a different user, a different mounting position, a noisier machine.
- **Silent failure.** A model always returns *some* answer. Monitoring confidence and occasionally sending samples for review helps detect when it has stopped working.

18.2.10 Larger models at the edge

More capable edge hardware runs much larger models. Camera boards such as the ESP-EYE run face and object detection with Espressif's ESP-WHO framework. Gateways and single-board computers with

neural accelerators run real-time vision, and even small language models, on premises. The same principles apply: move computation toward the data when latency, bandwidth, energy or privacy demand it.

Try it on your kit: from rules to a learned classifier

1. Record 2-second windows of MPU-6050 data while standing still, walking and running, at least 30 windows of each. Stream them over the serial port to a CSV file.
2. In a notebook, compute the mean and standard deviation of the acceleration magnitude per window, and train a small decision tree. Report its accuracy on windows recorded by a different team member.
3. Export the tree as C code and run it on the kit, showing the predicted activity on the OLED.
4. Measure the classification time with `micros()`, and the RAM used by your window buffer.
5. Optional: train a small neural network on the same data, quantize it to int8, and run it with LiteRT for Microcontrollers. Compare accuracy, size and latency with the tree.

18.3 In depth (optional)

Note

Optional, in-depth explanation. This section goes beyond the rest of the chapter into the math and physics behind it. Nothing later in the book depends on it; skip it on a first reading.

18.3.1 Fitting a model into a microcontroller

Sampling first. To capture motion up to frequency $f_{\max}$, the Nyquist criterion requires sampling at $f_s > 2f_{\max}$. Human walking has most of its energy below about 3 Hz, and arm gestures below about 10–15 Hz, so 50 Hz accelerometer sampling is ample; sampling slower lets faster movements alias into false low-frequency patterns.

Features from raw axes. The acceleration magnitude

$$|a| = \sqrt{a_x^2 + a_y^2 + a_z^2} \tag{18.1}$$

does not depend on how the device is oriented, which makes it a robust input for step counting. Over a window of W samples, simple features such as the mean, standard deviation and number of threshold crossings summarize the motion into a few numbers a small model can use.

Counting the work. A fully connected layer with n_{in} inputs and n_{out} outputs performs $n_{\text{in}} \times n_{\text{out}}$ multiply-accumulate operations (MACs) and stores the same number of weights. A 2-D convolution producing an $H \times W \times C_{\text{out}}$ output with $K \times K$ kernels over C_{in} input channels performs

$$\text{MACs} = H\,W\,C_{\text{out}}\,K^2\,C_{\text{in}}. \tag{18.2}$$

A gesture model with layers 300⟶32⟶16⟶4 needs $9600 + 512 + 64 \approx 10\,200$ MACs and as many weights. At a few cycles per MAC on a 240 MHz core, inference takes well under a millisecond.

Quantization. Storing weights as 8-bit integers instead of 32-bit floats cuts memory four times and allows faster integer arithmetic. The common affine scheme maps a real value x to an integer q with a scale s and zero point z:

$$q = \text{round}\left(\frac{x}{s}\right) + z, \qquad x \approx s\,(q - z), \tag{18.3}$$

with $s = (x_{\max} - x_{\min})/255$ for int8. The rounding error is at most $s/2$ per value; accuracy usually drops by around a percentage point or less if the model was trained or calibrated with quantization in mind, and can drop much more if it was not.

Energy per inference. Energy is power times time: a core at 30 mA and 3.3 V running a 1 ms inference uses about 0.1 mJ. Sending the raw 300-sample window over Wi-Fi instead (hundreds of bytes plus connection overhead) costs far more. That ratio, compute is cheap and radio is expensive, is the main argument for edge intelligence.

Exercise. Your model has 25 000 float32 weights. How much flash does it need before and after int8 quantization? Would it fit alongside a 1.2 MB application in one 1.9 MB OTA partition?

Sources for this section: [61], [62].

18.4 Design trade-offs

Decision	Considerations
Device vs. edge vs. cloud inference	Latency, energy, privacy and offline operation versus model capability and ease of updating
Hand-crafted features vs. learned features	Features plus small models are tiny and interpretable; neural networks need less feature design but more data and compute
Quantization	Smaller and faster, with some accuracy loss
Updating models	Models need OTA updates too (Chapter 16.), and versioning alongside firmware

18.5 Common pitfalls

- **Testing on the training data**, or on windows from the same recording, which overstates accuracy.
- **Collecting data in only one condition.** Models fail on other users, placements or environments.
- **Ignoring RAM for activations.** A model that fits in flash can still run out of RAM at inference time.
- **Measuring latency on a PC** instead of on the device.
- **Using ML where a threshold would do.** Start with the simplest model that meets the requirement.
- **Shipping without monitoring.** Model accuracy degrades silently.

> ℹ Advanced track

Federated learning. Instead of sending raw data to the cloud, devices can train locally and send only model updates, which a server aggregates. Read about federated averaging and explain what it protects and what it does not (model updates can still leak information).

Energy-aware inference. Design a cascade: a tiny, always-on model (such as a motion threshold) wakes a larger model only when needed. Estimate the energy saved compared with running the larger model continuously.

Benchmarks. MLPerf Tiny defines standard tasks (keyword spotting, visual wake words, image classification, anomaly detection) and reports accuracy, latency and energy on microcontrollers. Compare published results for two microcontroller families.

18.6 Check your understanding

1. Give four reasons not to send all raw IoT data to the cloud.
2. What is the difference between Edge AI and TinyML?
3. Distinguish classification from regression with an IoT example of each.
4. Why must test data be collected separately from training data?
5. How does int8 quantization reduce model size, and what does it cost?
6. List the six steps of the TinyML workflow.
7. A model fits in flash but crashes at inference time. What is the likely cause?
8. When would you choose a decision tree over a neural network on a microcontroller?

18.7 Further reading

- P. Warden and D. Situnayake, *TinyML* [62].
- Google AI Edge, *LiteRT for Microcontrollers* documentation [63].
- C. R. Banbury et al., “MLPerf Tiny Benchmark” [64].

19. Where IoT Is Heading

Learning objectives

After this chapter you should be able to:

1. Summarize the main ideas of the book as one connected picture.
2. Explain how continuous integration and continuous deployment apply to IoT firmware.
3. Describe several current directions in IoT: new connectivity, batteryless devices, digital twins, AI agents and sustainability.
4. Evaluate claims about emerging technologies, such as blockchain for IoT, against the requirements of real IoT systems.

19.1 Why it matters

The technologies in this book will change. The ESP32 will be replaced by newer chips; today's protocol versions will be superseded; regulations will tighten. What lasts are the questions and trade-offs: how to sense and act reliably, how to move data at the right cost, how to keep devices trustworthy for years, and where computation should happen.

This chapter looks back at the whole stack, then looks forward at directions that are shaping the next generation of IoT systems, and offers a way to evaluate new technologies as they appear.

19.2 Concepts

19.2.1 Looking back: one connected picture

Every chapter in this book addressed part of a single system:

- **The things** (Parts I and II): microcontrollers, embedded software, buses, sensors and actuators, tied together by the Observe–Decide–Act loop.
- **The connections** (Part III): a layered stack, from radios that trade range for power, through IPv6 adapted to small networks, to application protocols and the shared semantics that make devices interoperate.
- **Trust** (Part IV): security requirements, threat modeling, device identity, secure updates and the regulations that now require them.
- **Intelligence and systems** (Parts V and VI): where computation should run, and how to build, test and review a complete system.

The recurring theme is **trade-offs under constraints**: energy, memory, bandwidth, latency, cost, security and time to market. A good IoT engineer makes those trade-offs explicitly and can defend them.

19.2.2 Scalable deployment: CI/CD for firmware

Web services are often updated many times a day, thanks to **continuous integration** (CI) and **continuous deployment** (CD). The same practices increasingly apply to device firmware:

- **Continuous integration**: every change to the repository triggers an automated build for every supported board, unit tests on the host, static analysis, and, ideally, tests on real hardware in a test rack ("hardware in the loop").
- **Continuous deployment**: successful builds are signed and stored as release candidates. They flow first to internal test devices, then to a **staging** group, and only then, in stages, to customer devices, using the OTA mechanisms and rollout safeguards from Chapter 16..

CI/CD shortens the time to deliver features and fixes, reduces risk by making each change small and quick to reverse, improves code quality through automation, and produces the build records (including SBOMs) that regulators increasingly expect.

19.2.3 Directions to watch

19.2.3.1 Connectivity everywhere

- **Cellular IoT keeps evolving.** LTE-M and NB-IoT continue to replace 2G and 3G, and **5G RedCap** ("reduced capability") brings 5G to wearables and industrial sensors at lower cost and power.
- **Satellite IoT.** Standards for non-terrestrial networks let small devices send messages directly to satellites, extending coverage to oceans, deserts and farms with no terrestrial network.
- **Wi-Fi and Bluetooth** add features aimed at IoT: target wake time, longer range, and precise distance measurement.

19.2.3.2 Batteryless and ambient IoT

Batteries limit device lifetime and create waste. **Energy harvesting** from light, vibration, heat differences or radio waves can power devices that use very little energy. **Backscatter** communication lets a device send data by reflecting an existing radio signal instead of generating its own, as passive RFID tags do. Cellular standards bodies are defining "ambient IoT" devices that work this way, aiming at billions of very low-cost tags for logistics and retail.

19.2.3.3 Digital twins

A **digital twin** is a continuously updated software model of a physical asset (a machine, a building, a city) fed by IoT data. Engineers use it to monitor the asset, test changes in simulation before applying them, and predict failures. Digital twins depend on good sensing, reliable data pipelines and shared semantics, so they build directly on Parts II and III.

19.2.3.4 AI agents and natural-language control

Language models are beginning to act as interfaces to devices: "make the house comfortable before I get home" is translated into commands for thermostats, blinds and lights. This raises new questions in the spirit of Part IV: what an AI agent is authorized to do, how its actions are logged and reversed, how it handles ambiguous or malicious instructions, and how fail-safe defaults (Chapter 7.) still apply when the "decide" step is a model rather than code you wrote.

19.2.3.5 Interoperable homes and buildings

Matter and Thread (Chapter 11., Chapter 13.) are pushing the smart home toward devices that work across ecosystems. Similar efforts in buildings and industry, using shared semantic models, aim to make data from different vendors usable together.

19.2.3.6 Regulation and long-term support

The EU Cyber Resilience Act and similar laws (Chapter 17.) make security, vulnerability handling and multi-year support mandatory. This changes product economics: maintenance becomes a planned cost from day one, and devices that cannot be updated become hard to sell.

19.2.3.7 Sustainability

Billions of devices consume energy and materials, and become electronic waste. Sustainable IoT design means low-power operation, repairability, long support periods, recyclable materials and graceful end-of-life, and asking honestly whether a product needs to be connected at all.

19.2.4 Evaluating emerging technologies: the case of blockchain

New technologies are often proposed as the solution to IoT's problems. A useful way to evaluate them is to compare their properties with the requirements and constraints in this book.

Blockchain has been widely proposed for IoT. A blockchain is a ledger, replicated across many parties, to which records can be appended but not changed, with agreement among parties reached without a central authority. Proposed uses include:

- tamper-evident logs of sensor data, for example in supply chains (where was this shipment, and was it kept cold?);
- device identity and ownership records that no single company controls;
- automated payments between devices ("machine-to-machine micropayments") using smart contracts.

Measured against IoT requirements, the picture is mixed:

Requirement	Blockchain's fit
Constrained devices	Poor: devices cannot store a ledger or run consensus; they must rely on gateways
Throughput and latency	Public blockchains confirm transactions in seconds to minutes, with limited throughput
Energy	Proof-of-work chains are very energy-intensive; proof-of-stake chains much less so
Privacy	Public ledgers are permanent and visible to all, which conflicts with data minimization and deletion rights
Data integrity	Good for records once written, but cannot verify that the sensor reading was true in the first place
Trust among parties	Useful when several organizations must share records and none is trusted to run a central database

The honest conclusion is that blockchain helps in a narrow set of cases: multi-party settings without a trusted operator, where tamper-evidence matters more than speed or privacy. For most IoT systems, a conventional database with signed records (Chapter 14.) and good access control meets the same needs more cheaply. The same method of evaluation applies to the next technology to be proposed.

Try it on your kit: a CI pipeline for your project

1. Add a GitHub Actions (or similar) workflow to your project repository that builds your firmware with PlatformIO on every push.
2. Make the build fail if the code does not compile for the kit's board, or if a simple unit test of your non-hardware logic (for example, your debounce or classification function) fails.
3. Have the workflow publish the compiled `firmware.bin` as a build artifact, named with the commit hash.
4. Describe the additional steps needed to turn this into continuous deployment to a fleet of kits, safely.

19.3 In depth (optional)

Note

Optional, in-depth explanation. This section goes beyond the rest of the chapter into the math and physics behind it. Nothing later in the book depends on it; skip it on a first reading.

19.3.1 Can a device run on harvested energy?

A recurring vision for the future of IoT is the battery-free device. Whether it works is an energy-balance question.

The balance. Over a long period, the energy harvested must at least equal the energy used:

$$\bar{P}_{\text{harvest}} \geq \bar{P}_{\text{device}} = D\,P_{\text{active}} + (1 - D)\,P_{\text{sleep}}. \tag{19.1}$$

A storage element (a supercapacitor or small rechargeable cell) covers the gap between moments of plenty and moments of need, but it cannot fix a negative average.

What sources offer. Orders of magnitude, which vary widely with conditions:

Source	Typical harvested power density
Outdoor sunlight on a small solar cell	~10 mW/cm²
Indoor office lighting on a solar cell	~10–100 µW/cm²
Body heat (thermoelectric, small ΔT)	~10–50 µW/cm²
Vibration (machinery)	~10–100 µW/cm³
Ambient radio signals	~0.1–1 µW/cm² near transmitters, far less elsewhere

An example. An indoor sensor with a 10 cm² solar cell harvesting 20 µW/cm² gets 200 µW on average while the lights are on, say 10 hours a day: a daily average of about 83 µW. A BLE beacon that sleeps at 3 µW (≈10 µW with its regulator) and spends 30 µJ per advertising event can afford

$$\frac{(83 - 10)\ \mu\ \mathrm{W}}{30\ \mu\ \mathrm{J}} \approx 2.4\ \text{events per second}, \tag{19.2}$$

comfortably above one advertisement per second. A Wi-Fi device that spends tens of millijoules per connection could afford roughly one report every few minutes, and only if its sleep current is in the microamperes. The radio, again, decides what is possible.

Why the trend helps. For decades, the energy needed per computation fell by roughly half every year and a half to two and a half years (an observation often called Koomey's law). Radios and sensors improve more slowly, but each halving moves another class of application into the range of harvested energy.

Exercise. Using the table, what solar-cell area would a device that averages 1 mW need indoors? Outdoors? What does that suggest about where battery-free Wi-Fi devices make sense?

Sources for this section: [65].

19.4 Design trade-offs

Decision	Considerations
Adopting new technology early	Possible advantage versus immature tools, changing standards and unknown failure modes
Automated deployment	Speed versus the risk of shipping bad firmware quickly; requires strong testing and staged rollouts
Connectivity choice for the long term	Networks are retired (2G, 3G); choose technologies with a long expected life

19.5 Common pitfalls

- **Choosing technology for novelty**, not because it meets a requirement better than simpler options.
- **Ignoring total lifetime cost**: support, updates, connectivity fees and disposal.
- **Automating deployment without automating testing.**
- **Assuming today's network will exist for the device's whole life.**

> ℹ Advanced track
>
> **Technology assessment memo.** Choose one emerging IoT technology (for example satellite IoT, ambient backscatter devices, digital twins or LLM-based home agents). Write a two-page memo for a product team: what problem it solves, its maturity, how it measures against the requirements in Chapters 9, 14 and 17, the risks of adopting it now, and your recommendation.

19.6 Check your understanding

1. Summarize, in one paragraph, how the parts of this book fit together into one IoT system.
2. What do continuous integration and continuous deployment add to an IoT development process, and what safeguards does CD need for firmware?
3. Explain how a batteryless device can communicate.
4. What is a digital twin, and what does it require from the underlying IoT system?
5. Give two security questions raised by letting an AI agent control physical devices.
6. In which situations does blockchain help an IoT system, and why is it a poor fit for most?

19.7 Further reading

- IoT Analytics, *State of IoT* reports [3], for current market and technology trends.
- M. Ali et al., "Applications of blockchains in the Internet of Things: A comprehensive survey" [66].

VI Part VI — Building an IoT System

20 From Idea to Vertical Slice 173
20.1 Why it matters 173
20.2 Concepts 173
20.3 In depth (optional) 176
20.4 Design trade-offs 177
20.5 Common pitfalls 177
20.6 Check your understanding 177
20.7 Further reading 177

21 Testing in the Field 179
21.1 Why it matters 179
21.2 Concepts 179
21.3 In depth (optional) 182
21.4 Design trade-offs 183
21.5 Common pitfalls 183
21.6 Check your understanding 183
21.7 Further reading 183

22 Reviewing Someone Else's System . 185
22.1 Why it matters 185
22.2 Concepts 185
22.3 In depth (optional) 188
22.4 Design trade-offs 188
22.5 Common pitfalls 188
22.6 Check your understanding 189
22.7 Further reading 189

20. From Idea to Vertical Slice

Learning objectives

After this chapter you should be able to:

1. Turn an IoT product idea into a problem statement, users and measurable requirements.
2. Write a project proposal with a system architecture, risks and a plan.
3. Explain what a vertical slice is and why it beats building one layer at a time.
4. Plan milestones that retire the biggest risks first.
5. Organize a small team with clear roles, version control and working agreements.

20.1 Why it matters

Most failed student IoT projects do not fail because a sensor was too hard. They fail because the team spent weeks perfecting one part (a beautiful dashboard, an elaborate enclosure, a complex sensor driver) and discovered in the last week that the parts did not fit together: the radio could not reach the gateway, the battery lasted four hours, or the broker rejected the device's messages.

The remedy is a disciplined path from idea to a thin, working, end-to-end system, followed by deliberate improvement. This chapter describes that path. It applies equally to a ten-week course project and to the first months of a real product.

20.2 Concepts

20.2.1 From idea to problem statement

Start with a problem, not a technology. A good **problem statement** says who has the problem, what it is, and why current solutions fall short:

> *Weak*: "A LoRa-based smart garden."
>
> *Better*: "Community-garden volunteers water every plot daily because they cannot tell which plots are dry, wasting water and time. A low-cost soil-moisture monitor visible from a phone would let them water only plots that need it."

From the problem statement, identify:

- **Users and stakeholders**: who uses the system, who installs and maintains it, and who else is affected (neighbors, building managers, people whose data is collected).
- **Use scenarios**: short stories of the system in use, including what goes wrong (the network is down; a sensor is stolen; the battery is low).

20.2.2 Requirements

Turn scenarios into **requirements**: statements that can be checked.

- **Functional requirements** say what the system does: "The dashboard shows each plot's moisture, updated at least every 30 minutes."
- **Non-functional requirements** say how well: battery life, range, latency, reliability, security, cost, size, privacy. They drive most IoT design decisions.

Make requirements measurable. "Long battery life" cannot be tested; "runs at least 30 days on two AA cells" can. Mark each requirement **must**, **should** or **could**, so that you know what to cut if time runs short.

20.2.3 Architecture

Sketch the **system architecture** on the four levels of Chapter 1.: devices, network, services platform and applications. For each element, choose technologies using the trade-offs from earlier chapters:

Question	See
Which sensors and actuators, and how are they connected?	Chapter 5., Chapter 6., Chapter 7.
Which link technology meets the range, power and data-rate needs?	Chapter 10.
Which application protocol and message format?	Chapter 12., Chapter 13.
Where does processing happen: device, edge or cloud?	Chapter 18.
What are the threats, and how are devices identified and updated?	Chapter 14., Chapter 15., Chapter 16.

Include a **data-flow diagram** showing every message, its topic or endpoint, its format and how often it is sent. It will be the basis of your threat model and of your testing.

20.2.4 Risks

List what could stop the project from working, and rank each risk by likelihood and impact. Typical IoT risks include:

- a sensor that is less accurate or noisier than its datasheet suggests;
- radio range or reliability that is worse than expected indoors;
- power consumption that is far higher than estimated;
- parts that arrive late or not at all;
- a cloud service or library that does not do what you assumed;
- integration problems between team members' parts.

For each major risk, plan a **spike**: a quick, throwaway experiment that answers the question early. Measure the range with two kits before designing the network; measure the current before promising battery life.

20.2.5 The proposal

A project proposal brings these together. A useful structure is:

1. **Problem statement and users**
2. **Proposed solution** and how it will be used
3. **Requirements**, with priorities and measurable targets
4. **System architecture** and data-flow diagram
5. **Hardware and software**: parts (from the kit and any extras), libraries, services
6. **Risks** and how you will reduce them
7. **Plan**: milestones and who does what
8. **Evaluation**: how you will show that the requirements are met

A proposal is a starting point, not a contract. Feedback on a proposal is usually about making the project achievable and testable, not replacing it.

20.2.6 The vertical slice

A **vertical slice** is a thin version of the whole system that works **end to end**, through every layer of the architecture:

> one real sensor reading → over the real network → through the real broker or server → stored → shown on the real dashboard, and one real command flowing back to one real actuator.

It does not need to be pretty, complete or efficient. It needs to be **real**: the actual hardware, protocols and services you intend to use, not stand-ins.

Building the vertical slice first has three benefits:

1. **It retires integration risk early.** Most surprises in IoT appear at the boundaries between layers, so you meet them while there is time to change course.
2. **It gives everyone something to build on.** Team members can improve their layer while the rest of the system keeps working.
3. **It is always demonstrable.** From then on, the project works at every moment, and progress means adding features to a working system.

The alternative, building each layer completely before connecting them ("horizontal" development), postpones integration to the end, where it is most expensive.

20.2.7 Milestones

Plan milestones that build on the vertical slice:

Milestone	Typical content
Proposal	Problem, requirements, architecture, risks, plan
Spikes	Answers to the riskiest questions (range, power, sensor quality)
Vertical slice	One end-to-end path working on real hardware and services
Progress report	Status against the plan; what changed and why; updated risks
Feature complete	All *must* requirements implemented
Security review	Threat model, review by another team, fixes (Chapter 22.)
Reliability and field testing	Long-duration and real-environment tests (Chapter 21.)
Final demonstration and report	Evidence that requirements are met; lessons learned

20.2.8 Working as a team

A team of three can easily lose a week to coordination problems. A few practices prevent most of them:

- **Clear ownership.** Assign an owner to each part of the architecture (for example device firmware, networking and back end, application and data), plus shared responsibility for integration and testing.
- **Version control.** Keep all code, configuration and documentation in one repository. Use branches and pull requests, and review each other's code. Never commit secrets: keep them in an ignored `secrets.h` and provide a `secrets.example.h`.
- **Interfaces first.** Agree early on message formats and topics, written down in the repository (a Thing Description, Chapter 13., works well). Then each member can work against the agreement.
- **Short, regular check-ins.** Ten minutes, twice a week: what was done, what is next, what is blocking.
- **A shared hardware log.** Note wiring changes, pin assignments and anything that was damaged.
- **Working agreements.** How quickly to answer messages, how decisions are made, and what happens when someone falls behind.

Try it on your kit: your first vertical slice

1. Write your project's problem statement and five measurable requirements.
2. Draw the architecture and data-flow diagram.
3. Build a vertical slice: one sensor reading from your kit reaches your dashboard or app through your chosen protocol and services, and one command from the dashboard changes an actuator on the kit.
4. Record a short video of the slice working, and list the three things that surprised you while building it.

20.3 In depth (optional)

Note

Optional, in-depth explanation. This section goes beyond the rest of the chapter into the math and physics behind it. Nothing later in the book depends on it; skip it on a first reading.

20.3.1 Sizing and reliability of a whole system

A vertical slice crosses several components: device, network, broker, database, dashboard. Two simple models let you reason about the whole chain before it exists.

Little's law. In any stable system, the average number of items inside, L, equals the arrival rate λ times the average time each spends inside, W:

$$L = \lambda W. \tag{20.1}$$

If 500 devices each publish once every 10 s, the broker receives $\lambda = 50$ messages per second. If the database writer takes $W = 40$ ms per message on average, about $L = 2$ messages are in the pipeline at any time. If a slow query raises W to 400 ms, 20 messages queue up; if W exceeds $1/\lambda$ per writer for long, the queue grows without bound. The law holds regardless of the arrival pattern, which makes it a quick sanity check for any queue in your design.

Peaks, not averages. If every device reports on the minute, the arrival rate in that second is N, not $N/60$. Adding a random offset to each device's schedule spreads the load; this is called **jitter**, and it is one of the cheapest scalability fixes.

Reliability multiplies along a chain. If a reading must pass through k components in series, each available with probability A_i, the end-to-end availability is

$$A_{\text{chain}} = \prod_{i=1}^{k} A_i. \tag{20.2}$$

Five components at 99% each give $0.99^5 \approx 95\%$: more than a day of outage per month. Each component's availability can be estimated as $A = \text{MTBF}/(\text{MTBF} + \text{MTTR})$, mean time between failures over that plus mean time to repair.

Redundancy adds in parallel. Two independent components in parallel, either of which suffices, fail only if both do: $A_{\text{par}} = 1 - (1 - A_1)(1 - A_2)$. Two 99% gateways give 99.99%. Buffering readings on the device until the network returns has a similar effect: it turns a network outage from data loss into delay.

Exercise. Your slice is device (99.5%), Wi-Fi (98%), broker (99.9%), database (99.9%), dashboard (99%). What is the end-to-end availability? Which single change would help most?

Sources for this section: [67], [68].

20.4 Design trade-offs

Decision	Considerations
Ambition vs. feasibility	An ambitious project is more interesting; a feasible one gets finished. Use must/should/could priorities to have both.
Buy vs. build	Cloud services, libraries and dashboards save time but add dependencies and costs
Extra parts vs. kit only	Extra parts expand what you can build but add delivery, integration and budget risk
Polishing vs. integrating	Time spent polishing one layer is time not spent finding integration problems

20.5 Common pitfalls

- **Starting with the technology** instead of the problem.
- **Unmeasurable requirements** such as “fast”, “secure” or “low power”.
- **Building horizontally** and integrating in the last week.
- **Ignoring power and range** until the demo.
- **One person holding all the knowledge** of a critical part.
- **Undocumented interfaces** between team members’ code.

> **i** Advanced track
>
> **Requirements traceability.** Build a table linking each requirement to the design elements that implement it and the tests that verify it. Use it to show, at the end of the project, which requirements are met and how you know.
>
> **Cost model.** Estimate the bill of materials for 1 000 units of your device, plus monthly connectivity and cloud costs per device. At what price would the product break even over three years, including the support period required by regulation (Chapter 17.)?

20.6 Check your understanding

1. Rewrite “a smart parking system” as a problem statement.
2. Give one functional and two non-functional requirements for a fall-detection wearable, all measurable.
3. What is a spike, and when should you use one?
4. Define a vertical slice. Why must it use the real hardware and services?
5. Why is integration risk the main reason to build vertically?
6. What should a team agree on first to let members work in parallel?

20.7 Further reading

- A. Cockburn, *Writing Effective Use Cases* [69].
- F. P. Brooks, *The Mythical Man-Month* [70], on why adding people and postponing integration rarely helps.

21. Testing in the Field

Learning objectives

After this chapter you should be able to:

1. Explain why IoT systems need testing beyond unit tests and bench demos.
2. Plan tests at several levels: unit, hardware-in-the-loop, integration, long-duration and field.
3. Design and run a multi-day reliability test with clear metrics and logging.
4. Conduct a field test in a realistic environment and report its results.
5. Instrument a device so that failures can be diagnosed remotely.
6. Prepare a robust final demonstration.

21.1 Why it matters

A device that works for five minutes on a desk has proven very little. In the field it will meet weak signals, crowded radio bands, power glitches, network outages, memory leaks that only appear after two days, sensors that drift, and users who press buttons in the wrong order. Many IoT failures appear only after hours or days, or only in a particular place.

Testing in IoT therefore has two dimensions that ordinary software testing often lacks: **time** and **environment**. This chapter explains how to test along both.

21.2 Concepts

21.2.1 Levels of testing

Table 21.1: Levels of testing for an IoT system.

Level	What it checks	How
Unit tests	Logic that does not need hardware: parsing, state machines, filters, classifiers	Run on a laptop or in CI (PlatformIO's native environment and Unity test framework)
Hardware-in-the-loop	Drivers and firmware on the real board	Automated tests on a board connected to a test machine
Integration tests	End-to-end paths: device → network → broker → back end → app	Scripted scenarios against the vertical slice
Fault-injection tests	Behavior when things go wrong	Disconnect Wi-Fi, stop the broker, unplug a sensor, cut power mid-operation
Long-duration (reliability) tests	Behavior over days: leaks, drift, reconnection, uptime	Run unattended for 24–72 hours or more, with logging
Field tests	Behavior in the real environment with real users	Deploy where the system will actually be used

Each level catches problems the others miss. Unit tests are fast and cheap but know nothing about radios; field tests are realistic but slow and hard to repeat.

21.2.2 Designing for testability

Testing is much easier if the firmware is designed for it:

- **Separate logic from hardware.** Put decision logic (state machines, thresholds, message formatting) in functions that take values and return results, so they can be tested on a laptop.
- **Log meaningfully.** Every important event (boot, reset reason, connect, disconnect, error, command received) should be logged with a timestamp and enough context to diagnose it later (Chapter 4.).
- **Report health.** Publish a periodic **heartbeat** with uptime, free heap, firmware version, signal strength (RSSI), reconnect counts and last error. A missing heartbeat is itself a signal.
- **Record why the device restarted.** `esp_reset_reason()` distinguishes power-on, software reset, watchdog timeouts, brownouts and crashes.

```
#include <esp_system.h>

const char* resetReasonText() {
  switch (esp_reset_reason()) {
    case ESP_RST_POWERON:  return "power-on";
    case ESP_RST_SW:       return "software";
    case ESP_RST_PANIC:    return "crash";
    case ESP_RST_INT_WDT:
    case ESP_RST_TASK_WDT:
    case ESP_RST_WDT:      return "watchdog";
    case ESP_RST_BROWNOUT: return "brownout";
    case ESP_RST_DEEPSLEEP:return "deep-sleep wake";
    default:               return "other";
  }
}

void publishHeartbeat() {
  char msg[160];
  snprintf(msg, sizeof msg,
              "{\"up_s\":%lu,\"heap\":%u,\"rssi\":%d,\"fw\":\"%s\",\"reset\":\"%s\",
\"reconnects\":%u}",
           millis() / 1000, ESP.getFreeHeap(), WiFi.RSSI(), FIRMWARE_VERSION,
           resetReasonText(), reconnectCount);
  mqtt.publish(HEARTBEAT_TOPIC, msg);
}
```

21.2.3 Fault injection

Deliberately break things and watch what happens:

Fault	Expected behavior to verify
Wi-Fi access point switched off for 10 minutes	Device keeps sensing and acting locally; buffers or drops data as designed; reconnects automatically
Broker or server unreachable	Back-off between retries; no crash; no battery drain from tight retry loops
Sensor unplugged or returning nonsense	Plausibility checks reject bad data; actuator goes to its fail-safe state (Chapter 7.)
Power cut during a write or an update	Settings and firmware remain valid (Chapter 16.)
Burst of commands or malformed messages	Device rejects invalid input without crashing
Clock not yet synchronized	Timestamps and certificate checks behave sensibly

21.2.4 Long-duration reliability testing

A **reliability test** runs the complete system, unattended, for an extended period, typically 24 to 72 hours, and measures how well it keeps working.

Plan it like an experiment:

1. **Define the configuration**: firmware version, hardware, location, network, power source.
2. **Define the metrics** and how each is measured:
   - **uptime** and the number and causes of **resets**;
   - **message delivery ratio**: messages received ÷ messages sent, using sequence numbers to count gaps;
   - **latency** distribution (median and 95th percentile), using timestamps;
   - **reconnections** and how long each outage lasted;
   - **free heap over time** (a steady decline indicates a memory leak);
   - **power** or battery voltage over time, if battery-powered;
   - application-specific correctness, such as sensor values within plausible ranges.
3. **Log everything to a place that survives the device**: the broker and a back-end database, not just the serial port.
4. **Run it without touching it.** Intervening invalidates the result; if you must, record what you did and when.
5. **Analyze and report**: plots of each metric over time, a list of every incident with its cause, and what you changed as a result.

A simple sequence-number scheme makes message loss measurable:

```
RTC_DATA_ATTR uint32_t seq = 0;      // survives deep sleep; resets on power loss

void publishReading(float value) {
  char msg[96];
  snprintf(msg, sizeof msg, "{\"seq\":%lu,\"t_ms\":%lu,\"v\":%.2f}", ++seq, millis(),
value);
  mqtt.publish(DATA_TOPIC, msg);
}
// On the server: delivery ratio = received / (max_seq - min_seq + 1), per boot
```

Expect the first long test to find problems. That is its purpose. Typical findings include memory leaks from repeated `String` concatenation, connections that never recover after a router restart, watchdog resets from blocking network calls, and TLS sessions that fail after the device's clock drifts.

21.2.5 Field testing

A **field test** places the system where it will actually be used, with the people who will actually use it. It reveals problems no lab can: a wall that blocks the signal, a sensor that reads differently in sunlight, a user who misunderstands the display.

A useful field-test plan includes:

- **objectives**: which requirements and risks the test addresses;
- **site and setup**: where devices are placed, with photos and a map;
- **procedure**: what testers and users do, and for how long;
- **data collected**: logs, measurements, observations, user feedback;
- **safety and permissions**: permission to install devices, electrical and physical safety, and the privacy of anyone whose data is collected (Chapter 14., Chapter 17.).

Report the results honestly. A field test that shows the range is half what you expected is valuable, provided you explain what you learned and what you would change.

21.2.6 Preparing the final demonstration

A live demo is a small field test under pressure. Reduce the risk:

- Have a **rehearsed script** with a clear story: the problem, the system, the evidence that it works.
- **Bring your own network** if possible (a travel router), or confirm the venue's network in advance.
- Keep a **backup**: a recorded video of the system working and screenshots of dashboards and long-test results.

- **Charge everything**, and bring spare cables.
- Show **evidence**, not just features: the reliability plots and field-test results are often more convincing than the live demo.

Try it on your kit: a 72-hour reliability test

1. Add a heartbeat and sequence-numbered data messages to your project firmware.
2. Write a small script on a laptop or server that subscribes to your topics and stores every message with its arrival time.
3. Run the complete system unattended for 72 hours.
4. Produce plots of message delivery ratio per hour, latency, free heap and RSSI, and a table of every reset and disconnection with its cause.
5. Write one page on what failed, why, and what you changed.

21.3 In depth (optional)

Note

Optional, in-depth explanation. This section goes beyond the rest of the chapter into the math and physics behind it. Nothing later in the book depends on it; skip it on a first reading.

21.3.1 How much testing is enough?

Field tests produce numbers such as “no failures in 72 hours” or “997 of 1000 packets delivered”. Statistics tells you how much those numbers actually prove.

Zero failures is not zero failure rate. If a test of n independent trials sees no failures, the 95% upper confidence bound on the failure probability is approximately

$$p_{\text{upper}} \approx \frac{3}{n} \qquad \text{(the "rule of three")}, \tag{21.1}$$

because $(1-p)^n = 0.05$ gives $p \approx -\ln(0.05)/n \approx 3/n$. 1000 successful packets show, with 95% confidence, a loss rate below 0.3%, not a loss rate of zero.

Mean time between failures from a short test. For failures that occur at a constant rate, a test of total duration T with no failures supports, at confidence C,

$$\text{MTBF} \geq \frac{T}{-\ln(1-C)}. \tag{21.2}$$

A 72-hour run with no failures shows MTBF ≥ 72/3.0 ≈ 24 hours at 95% confidence. To claim a month, you would need about three months of failure-free device-hours, which is why teams test many devices in parallel: 30 kits for 72 hours give 2160 device-hours.

Estimating a rate with an interval. For x successes in n trials, the observed rate is $\hat{p} = x/n$, and a simple 95% interval is

$$\hat{p} \pm 1.96\sqrt{\frac{\hat{p}(1-\hat{p})}{n}}. \tag{21.3}$$

997 of 1000 gives 99.7% ± 0.34%. The approximation is poor near 0% and 100%, where the Wilson interval is better, but it shows the key point: to halve the uncertainty, you need four times the trials.

Comparing before and after. If a fix raises delivery from 96% to 98%, is that real? With 200 packets each, the intervals (about ±2.7% and ±1.9%) overlap; with 2000 each they do not. Decide the sample size before the test, not after seeing the result.

Exercise. Your team’s device must deliver at least 99% of readings. How many consecutive successful readings, with no failures, would let you claim that with 95% confidence?

Sources for this section: [71].

21.4 Design trade-offs

Decision	Considerations
Test duration	Longer tests find rarer problems but delay feedback
Logging detail	More detail aids diagnosis but costs bandwidth, storage, energy and possibly privacy
Automated vs. manual testing	Automation is repeatable and runs in CI; some field behavior can only be observed by people
Lab vs. field	Labs are controlled and repeatable; the field is realistic

21.5 Common pitfalls

- **Testing only the happy path.**
- **Logging only to the serial monitor**, so the record disappears when the laptop sleeps.
- **No sequence numbers or timestamps**, so loss and latency cannot be measured.
- **Changing firmware during a long test** without restarting the clock.
- **Ignoring resets** because "it came back".
- **A demo that depends on unfamiliar Wi-Fi.**

> **i** Advanced track
>
> **Statistics of rare failures.** If your device runs 72 hours with no failures, what can you say about its mean time between failures? Using the "rule of three", estimate a 95% upper bound on the failure rate, and discuss how many device-hours you would need to claim a mean time between failures of one year.
>
> **Chaos testing.** Write a script that randomly disconnects the broker, delays messages and restarts the access point during a long test, and compare the system's behavior with an undisturbed run.

21.6 Check your understanding

1. Name four levels of testing for an IoT system and one problem each is good at finding.
2. Why should decision logic be separated from hardware access?
3. What should a heartbeat message contain, and why is a *missing* heartbeat useful?
4. How do sequence numbers let you measure message loss?
5. What does a steadily falling free-heap value during a long test indicate?
6. List five elements of a field-test plan.

21.7 Further reading

- PlatformIO, *Unit Testing* documentation [11].
- M. T. Nygard, *Release It!* [68], on designing systems that survive production.

22. Reviewing Someone Else's System

Learning objectives

After this chapter you should be able to:

1. Explain why independent review finds problems that the builders miss.
2. Conduct a hands-on device review: use, observe and probe another team's device.
3. Conduct a structured security code review of another team's repository.
4. Report findings with clear severity, evidence and recommended fixes.
5. Respond to a review constructively and track fixes to completion.

22.1 Why it matters

Builders test their systems the way they expect them to be used. They know which button not to press, which order to power things up in, and which warning to ignore. Attackers and real users know none of this. An independent reviewer, with fresh eyes and a different set of assumptions, finds failures and vulnerabilities that the builders cannot see.

Professional teams build review into their process: code review on every change, design reviews before major decisions, and penetration tests or security audits before release. Many regulations now expect evidence of such reviews (Chapter 17.). This chapter describes two forms of review you can apply to a peer team's project: a **device review**, where you receive and probe their device, and a **security code review** *(Advanced track)*, where you examine their source code and design.

22.2 Concepts

22.2.1 Ground rules

Reviews work only when both sides trust the process:

- **Scope and permission.** Review only the systems you have been given, within the agreed scope. Do not attack shared infrastructure (the course broker, campus Wi-Fi) or other teams' systems. This is the same rule that separates professional security testing from attacks.
- **Do no harm.** Do not damage hardware, delete data or leave changes behind. Report what you could have done instead of doing it.
- **Critique the system, not the people.** Findings describe behavior and code, with evidence, not judgments about the builders.
- **Confidentiality.** Findings go to the team (and the teaching staff), not to everyone, until they are fixed.

22.2.2 Preparing for a review

The team being reviewed provides a short **review package**:

- a one-page description of the system and its intended use;
- the architecture and data-flow diagram, with trust boundaries (Chapter 14.);
- instructions to set up and use the device;
- the repository (for a code review), with build instructions;
- the team's own threat model and any known issues.

The reviewers read the package first, then plan their review around the most valuable targets: where trust boundaries are crossed, where commands reach actuators, and where secrets are stored.

22.2.3 The device review

In a device review, reviewers receive the other team's working device, use it as intended, then try to make it fail. A useful structure follows the STRIDE categories (Table 14.2) and the reliability concerns from Chapter 21.:

Area	Questions to investigate
First use	Can you set it up from the instructions alone? Are there default credentials?
Normal use	Does it meet its stated requirements? Is feedback clear?
Physical access	Are secrets readable over the serial console? Is a debug menu exposed? What happens if you press buttons during boot?
Network observation	Using a desktop MQTT client or packet capture on a network you control: are messages encrypted? Do they reveal personal or sensitive information?
Spoofing and tampering	Can you send commands that the device accepts without authorization? Can you replay a captured command?
Denial of service	What happens with a flood of messages, malformed payloads, a disconnected network or a removed sensor?
Recovery	After power loss or network loss, does it recover on its own, and into a safe state?

Record every test, including the ones where the device behaved well. A finding of "replay rejected" is useful evidence.

22.2.4 The security code review *(Advanced track)*

In a security code review, reviewers read another team's repository with an attacker's eye. Work through it systematically rather than skimming:

1. **Build and run it.** Confirm that the instructions work. Many problems surface here.
2. **Map the attack surface.** Find every place where external input enters: network callbacks, serial input, configuration pages, BLE characteristics, sensor readings.
3. **Trace each input** to where it is used. Is it validated? Can it overflow a buffer, change a command, or reach an actuator without authorization?
4. **Check secrets.** Search the repository and its *history* for passwords, keys and tokens (`git log -p`, or a secret-scanning tool). Check that `secrets.h` is ignored.
5. **Check cryptography and TLS.** Is certificate verification disabled (`setInsecure()`)? Are standard libraries used correctly? Is there replay protection on commands (Chapter 14.)?
6. **Check the update path.** Can firmware be replaced over the network? Is it authenticated (Chapter 16.)?
7. **Check failure handling.** Do network calls have timeouts? Are fail-safe states defined?
8. **Check privacy.** What personal data is collected, logged or retained, and is all of it needed?

A checklist of common findings in student IoT code:

- Wi-Fi passwords, broker credentials or API keys committed to the repository, sometimes later "removed" but still in the history;
- `setInsecure()` or disabled certificate checks;
- one MQTT topic for everything, with any client able to send commands;
- commands accepted without authentication, or authenticated without replay protection;
- `strcpy`, `sprintf` and fixed-size buffers fed with network input;
- `String` concatenation in loops, causing heap fragmentation over days;

- blocking calls without timeouts, triggering watchdog resets;
- verbose debug output in production builds, including secrets.

22.2.5 Writing findings

Each finding should let the receiving team understand, reproduce and fix the problem:

Field	Content
Title	A short description: "Unlock command can be replayed"
Severity	Critical, high, medium or low, based on impact and likelihood
Category	STRIDE category, or reliability, privacy, usability
Evidence	Steps to reproduce, captured messages, file and line references, screenshots
Impact	What an attacker or a failure could cause in the real use case
Recommendation	A concrete fix, with references to standard practices

A simple severity guide:

- **Critical**: remote control of actuators or full device compromise without credentials.
- **High**: exposure of credentials or sensitive personal data; reliable denial of service.
- **Medium**: weaknesses that need special access or conditions to exploit.
- **Low**: hardening and best-practice improvements.

End the report with a short summary: the most important findings, what the team did well, and the overall risk.

22.2.6 Responding to a review

Receiving a review is part of the exercise. The team should:

1. **Acknowledge** each finding, and ask clarifying questions if needed.
2. **Triage**: agree or disagree with each severity, with reasons.
3. **Fix** the important findings, and **verify** each fix, ideally with the reviewers.
4. **Document** what was fixed, what was accepted as a known risk and why, and what was deferred.

The response is evidence of engineering maturity, and is often worth as much as the fixes themselves.

Try it on your kit: review a peer team

1. Prepare your team's review package and exchange it with your assigned peer team.
2. Perform a device review of their kit (or, on the Advanced track, a security code review of their repository) using the structure above, within the agreed scope.
3. Write a findings report with at least five findings, each with severity, evidence and a recommendation.
4. When you receive your own report, write a response that triages every finding and documents your fixes.

22.3 In depth (optional)

> Note
>
> **Optional, in-depth explanation.** This section goes beyond the rest of the chapter into the math and physics behind it. Nothing later in the book depends on it; skip it on a first reading.

22.3.1 How many defects are left?

A review finds some defects. The question a team really wants answered is how many it did *not* find.

Independent reviewers find more. If each reviewer independently finds a given defect with probability p, then k reviewers find it with probability

$$P_{\text{found}} = 1 - (1 - p)^k. \tag{22.1}$$

With $p = 0.4$, one reviewer finds 40% of defects, two find 64%, three 78%. Each extra reviewer adds less; and in practice reviewers are not independent (they share blind spots), so real gains are smaller. Checklists and different review roles (one reviewer per STRIDE category, for instance) make reviewers less alike.

Estimating what remains: capture–recapture. Ecologists estimate a population by catching animals, marking them, and seeing how many marked ones are caught again. The same idea estimates the number of defects. If reviewer A finds n_A defects, reviewer B finds n_B, and m of them are the same, then an estimate of the total number of defects is

$$\hat{N} = \frac{n_A \, n_B}{m}. \tag{22.2}$$

If A finds 12 issues in a peer team's device, B finds 10, and 6 are common, $\hat{N} = 120/6 = 20$. Together they found $12 + 10 - 6 = 16$, so about 4 are probably still there. A small overlap is a warning: it means the reviewers are sampling a large population of defects.

The estimate assumes both reviewers look at the same scope and that every defect is equally easy to find; easy defects are found by both, so it tends to underestimate. It is still far better than assuming the count of found defects is the count of defects.

Severity, not just count. Scoring systems such as CVSS combine how a weakness is exploited (network or physical access, complexity, privileges needed) with its impact on confidentiality, integrity and availability. Whatever the scale, rank findings so that the team fixes the few that matter before the many that do not.

Exercise. Two reviewers of the same repository report 8 and 5 findings with 1 in common. Estimate the total. What would you tell the team about the review's coverage?

Sources for this section: [72], [73].

22.4 Design trade-offs

Decision	Considerations
Device review vs. code review	A device review tests what is actually built and deployed; a code review finds issues that are hard to trigger from outside
Breadth vs. depth	A broad checklist finds common issues; depth on the riskiest path finds serious ones
Fixing everything vs. accepting risk	Not every finding is worth fixing before release; document accepted risks explicitly

22.5 Common pitfalls

- **Reviewing outside the agreed scope**, or disrupting shared systems.
- **Vague findings** ("security is weak") without evidence or a fix.

- **Severity inflation**, which hides the findings that matter.
- **Checking only the current code**, not the repository history, for secrets.
- **Defensive responses** instead of triage and fixes.

> ℹ Advanced track
>
> **Coordinated disclosure.** In industry, researchers who find vulnerabilities in products they do not own follow coordinated disclosure: report privately to the vendor, give a reasonable time to fix, then publish. Read ISO/IEC 29147 (vulnerability disclosure) or a vendor's published policy, and write a disclosure policy for your project as if it were a product.
>
> **Automated analysis.** Run a static analyzer (such as cppcheck or clang-tidy) and a secret scanner (such as gitleaks) on the repository you reviewed. Which of your manual findings did the tools catch, which did they miss, and what did they find that you missed?

22.6 Check your understanding

1. Why do independent reviewers find problems that builders miss?
2. What is the most important ground rule when reviewing someone else's system?
3. Give four areas to investigate in a device review.
4. Why must a code review check the repository's history, not just its current files?
5. What six fields make a finding actionable?
6. How should a team respond to a finding it disagrees with?

22.7 Further reading

- OWASP, *IoT Security Testing Guide* [73].
- A. Shostack, *Threat Modeling: Designing for Security* [49].

22.7 Recognizing good and bad joints 194
22.7 What to show 194
22.7 Questions 194

22.7 Lab 1: Build the Kit in Stages 197
22.7 Goals 197
22.7 The layout 197
22.7 The stages 198
22.7 Procedure 199
22.7 What to show and submit 199
22.7 Troubleshooting 200
22.7 Questions 200

22.7 2: Traffic Controller 201
22.7 Goals 201
22.7 The problem 201
22.7 Design 201
22.7 A starting structure 202
22.7 Testing 203
22.7 What to show and submit 203
22.7 Analysis questions 204

22.7 Lab 3: Serial Communication 205
22.7 Goals 205
22.7 Part 1: I²C, reading the BMP280 from its datasheet 205
22.7 Part 2: SPI, showing data on the OLED 206
22.7 Part 3: UART, talking to another device 206
22.7 Stretch: hardware vs. software SPI 206
22.7 What to show and submit 206
22.7 Analysis questions 206

22.7 Lab 4: BLE Step Counter 207
22.7 Goals 207
22.7 Part 1: detecting steps 207
22.7 Part 2: publishing over BLE 208
22.7 Part 3: evaluating accuracy 208
22.7 What to show and submit 208
22.7 Analysis questions 208

22.7 Lab 5: Cloud Communication 209
22.7 Goals 209
22.7 What you need 209
22.7 Part A: the cloud side 209
22.7 Part B: credentials in flash, not in code 210
22.7 Part C: TLS, Last Will, back-off and telemetry . 210
22.7 Part D: dashboard and alert 211
22.7 Part E: project analysis 211
22.7 What to show and submit 211
22.7 Troubleshooting 211
22.7 Stretch goal 212
22.7 Analysis questions 212

22.7 Lab 6: Harden Your Device 213
22.7 Goals 213
22.7 Part 1: threat model 213
22.7 Part 2: verify the broker 213
22.7 Part 3: get secrets out of the source code 213
22.7 Part 4: authenticate commands 214
22.7 Part 5: reduce the attack surface 214
22.7 Part 6: eFuses, read-only 214
22.7 What to show and submit 214
22.7 Analysis questions 214

VII Part VII — Lab Companion

Lab 0: Bench Safety and Soldering

> **i** Lab Companion
>
> The Lab Companion describes the labs as run with the current version of the IoT@UCI kit. Pin numbers, libraries and procedures are updated each term. Where this text and your term's handout differ, **the handout wins**.

Goals

- Work safely at an electronics bench.
- Make reliable through-hole solder joints on header pins.
- Recognize and fix common soldering defects.

Safety first

Hazard	Precaution
Burns (iron tips reach 300–450 °C)	Always return the iron to its stand. Never touch the tip or the metal shaft. Assume every iron is hot.
Fumes from flux	Use the fume extractor or work in a well-ventilated area; keep your face out of the plume.
Lead (in leaded solder)	Prefer lead-free solder. Do not eat or drink at the bench. Wash your hands afterwards.
Eye injury from flying solder or clipped leads	Wear safety glasses. Hold leads when clipping and point them down.
Electrical	Check the iron's cord before use. Keep liquids away. Never solder a powered circuit.
Fire	Work on a heat-resistant mat. Turn the iron off when you leave the bench, even briefly.

Always **unplug the USB cable before changing any wiring** on the kit.

Background

Soldering joins metal parts with a low-melting alloy (solder), making a joint that is both an electrical connection and a mechanical one. Electronics uses **soft soldering**, at roughly 180–450 °C. You will do **through-hole** soldering: component leads or header pins pass through holes in a circuit board and are soldered to the copper pads around them. (Surface-mount soldering, where parts sit on pads on the surface, needs finer technique and is not required.)

Tools and materials

- **Soldering iron or station**, ideally temperature-controlled (25–40 W is typical). Around 330–370 °C suits most lead-free work on small pads.
- **Tips**: a **conical** tip for precise work, a **chisel** tip for general use (headers and wires), **bevel** and **knife** tips for drag soldering.

- **Solder**: lead-free (such as SAC305) or leaded (60/40, 63/37); thin (0.5–0.8 mm) wire is easiest for headers. Most solder wire has a **rosin flux core**; a **flux pen** helps when solder will not flow.
- **Tip cleaner**: brass wool or a damp (not wet) sponge.
- **Workholding**: helping hands or a board holder; a spare breadboard holds header pins straight.
- **Corrections**: desoldering braid (wick) and a solder sucker.

Procedure: soldering header pins

The breakout boards in the kit (the OLED, BMP280, MPU-6050, ATECC608A and LoRa radio) come with loose header pins.

1. **Set up.** Heat the iron, clean the tip, and **tin** it: melt a small amount of solder onto the tip so it is shiny.
2. **Hold the pins straight.** Push the long ends of the header into a breadboard, then place the breakout board on top, with the pins through its holes and the board level.
3. **Tack one pin.** Touch the iron to *both* the pad and the pin for 1–2 seconds, then feed solder into the joint (not onto the iron). Remove the solder, then the iron.
4. **Check alignment.** If the board is tilted, reheat the single joint and straighten it. This is much easier with one pin soldered than with all of them.
5. **Solder the rest.** Heat the pad and pin together for 2–3 seconds, feed solder until it flows around the pin and covers the pad, then remove solder and iron. Move steadily; do not hold the iron on the board for long.
6. **Inspect.** Each joint should be shiny (or slightly satin for lead-free), smooth and **cone- or volcano-shaped**, fully covering the pad.
7. **Clean up.** Return the iron to its stand, clean and re-tin the tip, and switch off the iron when done.

Recognizing good and bad joints

Joint	Appearance	Cause	Fix
Good	Shiny, smooth, concave cone covering the pad	—	—
Cold	Dull, grainy or lumpy; may not wet the pad	Not enough heat, or movement while cooling	Reheat with a touch of flux until it flows
Bridge	Solder joining two adjacent pins	Too much solder	Drag the iron between pins, or use desoldering braid
Insufficient	Pad only partly covered; gap around pin	Too little solder or heat	Add a little solder while heating
Excess	Large ball hiding the pin	Too much solder	Remove some with braid
Lifted pad	Copper pad detached from board	Too much heat or force	Ask a TA; may need a wire repair

After soldering, run the kit check from Lab 1 or an I²C scan (Chapter 5.). A device that does not appear often has a cold joint or a bridge on its SDA or SCL pin.

What to show

- A breakout board with all header pins soldered, straight, and passing visual inspection by a TA.
- The board working in a quick test (for example, appearing in an I²C scan).

Questions

1. Why should solder be fed into the joint rather than onto the iron tip?
2. Why tack a single pin first?

3. A sensor appears in an I²C scan only when you press on it. What is the most likely cause, and how do you fix it?

Lab 1: Build the Kit in Stages

> **i** Lab Companion
>
> Pin numbers and parts reflect the current kit. If your term's handout or `iot_pins.h` differs, **they win**.

Goals

- Assemble the IoT@UCI team kit: two breadboards, the ESP32 and Stage 1 components.
- Learn to wire carefully and verify each stage before moving on.
- Understand why the kit is built in stages, just before each part is needed.

Kit assembly is part of the course: you will cut and route wires, solder headers (Lab 0), place parts and test the board yourselves.

The layout

The kit uses **two 830-point breadboards** side by side along their long edges, with the **ESP32 DevKit V1** at the bottom end, bridging the seam so that each row of pins lands on a different board.

- **Left board**: LEDs, buttons, potentiometer and buzzer; at the top, the servo and its 470 µF capacitor on a short, separate **5 V rail**.
- **Right board**: from top to bottom, the ATECC608A, the OLED, the MPU-6050 and the BMP280.
- A printed holder at the top (download link in Appendix A.) carries the servo and, in some kits, the LoRa radio and its antenna; otherwise the radio sits on the right board.

Figure A.1 in Appendix A shows the finished layout, and Figure A.3 to Figure A.6 show the wiring for each stage.

Run the ESP32's **3V3** and **GND** pins to the power rails of both boards. Keep wires short, flat and color-coded: red for 3.3 V, black for ground, a different color for each signal type.

> **⚠** Warning
>
> The servo's **5 V rail** (fed from the ESP32's VIN pin) must **never** be connected to the 3.3 V rails. The ESP32's pins are not 5 V tolerant.

The stages

Stage 1: basic I/O and the display (this lab)

Table 22.1: Stage 1 wiring.

Part	Connection	Notes
Red LED (LED 1)	GPIO 2 ⟶ 220 Ω ⟶ LED ⟶ GND	Longer leg (anode) toward the resistor
Yellow LED (LED 2)	GPIO 4 ⟶ 220 Ω ⟶ LED ⟶ GND	
Green LED (LED 3)	GPIO 15 ⟶ 220 Ω ⟶ LED ⟶ GND	
Button 1	GPIO 32 ⟶ button ⟶ GND	`INPUT_PULLUP`; reads LOW when pressed
Button 2	GPIO 33 ⟶ button ⟶ GND	`INPUT_PULLUP`
Potentiometer	Outer legs to 3V3 and GND; wiper ⟶ GPIO 34	GPIO 34 is input-only (ADC1)
Passive buzzer	GPIO 27 ⟶ buzzer + ; buzzer − ⟶ GND	
OLED (SPI)	GND⟶GND, VDD⟶3V3, SCK⟶GPIO 18, SDA⟶GPIO 23, RES⟶GPIO 17, DC⟶GPIO 16, CS⟶GPIO 5	"SDA" on this module is **SPI data** (COPI), not I²C

Solder the headers on all breakout boards in this lab, even those you will wire later, so later stages only need wiring.

Stage 2: the I²C bus (before the serial-protocols activity)

All three devices share **SDA** = **GPIO 21** and **SCL** = **GPIO 22**, plus 3V3 and GND.

Table 22.2: Stage 2 wiring.

Device	Address	Extra connections
BMP280	0x76	CSB ⟶ 3V3 (selects I²C), SDO ⟶ GND (selects 0x76)
MPU-6050	0x68	AD0 ⟶ GND (selects 0x68); INT ⟶ GPIO 35 (optional)
ATECC608A	0x60	—

Stage 3: the servo rail (before the actuators activity)

Table 22.3: Stage 3 wiring.

Part	Connection
Servo power	Red wire → 5 V rail (from VIN); brown/black → GND
Servo signal	Orange/yellow → GPIO 13
Capacitor	470 µF across the 5 V rail and GND; **observe polarity**: the stripe marks the negative lead

Stage 4: the LoRa radio (before the LoRa activity)

Table 22.4: Stage 4 wiring.

RFM95W pin	ESP32	Notes
VIN (3.3V on bare modules)	3V3	
GND	GND	
SCK	GPIO 18	Shared with the OLED
MOSI	GPIO 23	Shared with the OLED
MISO	GPIO 19	
CS (NSS)	GPIO 14	
RST (RESET)	GPIO 26	
G0 (DIO0)	GPIO 25	Interrupt on packet received/sent

> ⚠ Warning
>
> Solder the **antenna** to the radio before powering it. Transmitting without an antenna can damage the radio.

Later labs need no new wiring.

Procedure

1. **Inventory** your parts against the kit list in Appendix A.. Report anything missing or damaged.
2. **Solder** headers on the breakout boards (Lab 0).
3. **Place** the breadboards and ESP32, and wire the power rails. With the USB cable connected and nothing else wired, measure 3.3 V between the rails (a TA can help with the multimeter). Unplug again.
4. **Wire Stage 1** one part at a time, following Table 22.1. After each part, check the wiring against the table with a teammate.
5. **Run the kit check**: open the provided kit-check project in PlatformIO, build, upload and open the Serial Monitor at 115200 baud. It blinks each LED, reads each button and the potentiometer, beeps the buzzer, and writes to the OLED. Fix anything that fails before moving on.
6. **Photograph** your finished Stage 1 board from above, and label your kit with your team number.

What to show and submit

- A TA checkoff of the Stage 1 kit passing the kit check.
- A top-down photo of the board.

- The answers to the questions below.

Troubleshooting

Symptom	Check
LED never lights	LED orientation; resistor in the same row as the LED; correct GPIO
Button always reads pressed or never	Button orientation (try rotating it 90°); `INPUT_PULLUP` in code
Potentiometer reads 0 or 4095 only	Wiper on GPIO 34; outer legs on 3V3 and GND
OLED blank	All seven wires; SDA on GPIO 23 (not 21); RES and DC not swapped
Upload fails	Data-capable USB cable; driver installed; hold BOOT while upload starts
Board resets when the buzzer or servo runs	Loose ground; servo on 3.3 V instead of its 5 V rail; missing capacitor

See also Appendix C..

Questions

1. Why are the buttons wired to ground with `INPUT_PULLUP`, rather than to 3.3 V?
2. Why is the potentiometer on GPIO 34 rather than an ADC2 pin?
3. The OLED's pin is labeled "SDA", yet it is not connected to the I²C bus. Explain.
4. Why is the kit built in stages instead of all at once?

Lab 2: Traffic Controller

> **Lab Companion**
>
> Pin numbers reflect the current kit. If your term's handout differs, **it wins**.

> **Note**
>
> This lab is adapted from an earlier traffic-light lab handout by Biswadip Maity and Sergio Gago-Masagué.

Goals

- Design and implement a **finite-state machine** on a microcontroller.
- Use non-blocking timing with `millis()` so several behaviors run at once (Chapter 3.).
- Think about safe default states and accessibility (Chapter 7.).

Uses: Stage 1 of the kit (Lab 1): LED 1 red (GPIO 2), LED 2 yellow (GPIO 4), LED 3 green (GPIO 15), button 1 (GPIO 32), buzzer (GPIO 27).

The problem

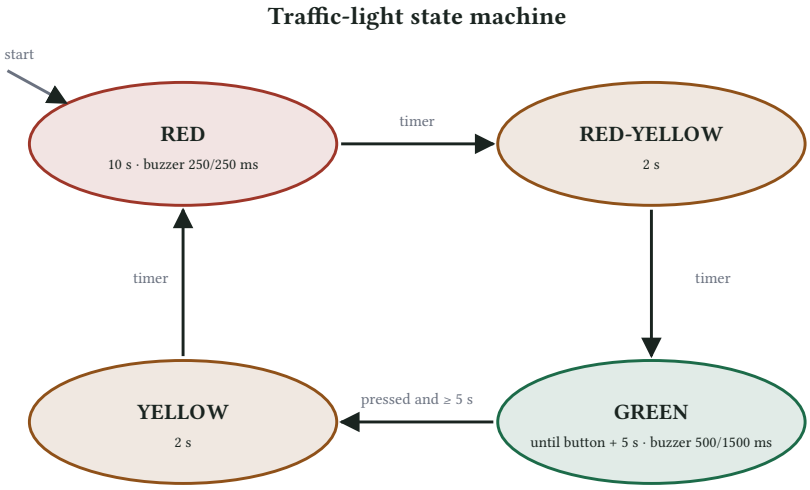
Build a simplified pedestrian traffic light for a road crossing:

1. The light cycles **green → yellow → red → red-yellow → green**.
2. **Red** lasts **10 seconds**. **Yellow** and **red-yellow** each last **2 seconds**.
3. The light **stays green** until a pedestrian presses the button. After the button is pressed, it stays green for **at least 5 more seconds** before changing to yellow.
4. A **buzzer** helps people with visual impairments: while green, it sounds **500 ms on, 1 500 ms off**; while red, it sounds **250 ms on, 250 ms off**. It is silent in the other states.

(A fingertip on top of the buzzer lowers the volume, which your lab neighbors will appreciate.)

Design

Model the light as a **state machine** with four states: `RED`, `RED_YELLOW`, `GREEN` and `YELLOW`, as shown below.



Each state has:

- **outputs**: which LEDs are on, and the buzzer pattern;
- **transitions**: the conditions (timer expired, button pressed) that move to the next state.

Keep the **buzzer** as a second, independent state machine (on/off, with its own timer) that reads the current traffic-light state to choose its pattern. Neither machine may use `delay()`: the button must be noticed immediately, whatever the buzzer is doing.

A starting structure

```
#include <Arduino.h>
#include "iot_pins.h"

enum class Light { RED, RED_YELLOW, GREEN, YELLOW };

constexpr uint32_t RED_MS = 10000, AMBER_MS = 2000, MIN_GREEN_AFTER_PRESS_MS = 5000;

Light state = Light::RED;            // what is the safest initial state?
uint32_t stateStart = 0;
bool requested = false;              // pedestrian pressed the button
uint32_t requestTime = 0;

void setLights(bool r, bool y, bool g) {
  digitalWrite(IoTPins::LED_1, r);
  digitalWrite(IoTPins::LED_2, y);
  digitalWrite(IoTPins::LED_3, g);
}

void enter(Light s) { state = s; stateStart = millis(); }

void setup() {
  pinMode(IoTPins::LED_1, OUTPUT);
  pinMode(IoTPins::LED_2, OUTPUT);
  pinMode(IoTPins::LED_3, OUTPUT);
  pinMode(IoTPins::BUTTON_1, INPUT_PULLUP);
  enter(Light::RED);
}

void updateLight() {
  uint32_t inState = millis() - stateStart;
  switch (state) {
```

```
    case Light::RED:
      setLights(1, 0, 0);
      if (inState >= RED_MS) enter(Light::RED_YELLOW);
      break;
    case Light::RED_YELLOW:
      // TODO: outputs and transition to GREEN
      break;
    case Light::GREEN:
      // TODO: stay green until requested, then at least 5 s after the request
      break;
    case Light::YELLOW:
      // TODO: outputs and transition to RED; clear the request
      break;
  }
}

void updateButton() {
  // TODO: detect a debounced press; set requested and requestTime
}

void updateBuzzer() {
  // TODO: second state machine; pattern depends on `state`
}

void loop() {
  updateButton();
  updateLight();
  updateBuzzer();
}
```

Use `tone(IoTPins::BUZZER, frequency)` and `noTone(IoTPins::BUZZER)` to switch the buzzer on and off.

Testing

Write a short test plan before demonstrating:

Test	Expected result
Power on	The light starts in the state you chose as safest
No button press	Light cycles to green and stays green indefinitely
Press during green, more than 5 s after entering green	Green lasts 5 more seconds, then yellow (2 s), red (10 s), red-yellow (2 s), green
Press during red	Does the press carry over? Decide, document and justify
Hold the button down	No repeated or stuck requests
Buzzer	500/1500 ms pattern in green, 250/250 ms in red, silent otherwise, never blocking the button

Time each state with a stopwatch or by printing timestamps over the serial port.

What to show and submit

- A live demonstration of every requirement to a TA.
- Your code, with the state machine clearly structured.
- A text file answering the analysis questions.

Analysis questions

1. Why does the light stay green if the button is never pressed? Is that a sensible design for a road crossing?
2. What is the safest initial state when the controller powers up or resets? Why?
3. What would you add to make the crossing safer for people with auditory or visual impairments?
4. What should the controller do if it detects that a lamp has failed?

Lab 3: Serial Communication

> **Lab Companion**
>
> Pin numbers and parts reflect the current kit. If your term's handout differs, **it wins**.

> **Note**
>
> This lab is adapted from an earlier serial-communication lab handout by Biswadip Maity and Sergio Gago-Masagué.

Goals

- Read a sensor over I²C at the register level, using its datasheet.
- Drive a display over SPI.
- Exchange messages over UART.
- Compare the three buses in practice (Chapter 5.).

Uses: Stages 1 and 2 of the kit (Lab 1).

Part 1: I²C, reading the BMP280 from its datasheet

Libraries hide the details of a sensor. In this part you will not use the BMP280 library; you will use only the `Wire` library and the datasheet [17].

1. **Scan** the bus (Chapter 5.) and confirm the BMP280 at 0x76.
2. **Identify the chip**: read register `0xD0` (`id`). It should return `0x58`.
3. **Configure** it: write `ctrl_meas` (`0xF4`) to select temperature and pressure oversampling and **normal mode**, and `config` (`0xF5`) for standby time and filtering. Choose and justify your settings from the datasheet.
4. **Read the calibration coefficients** `dig_T1`…`dig_T3` and `dig_P1`…`dig_P9` from `0x88`–`0x9F`. They are little-endian; note which are signed.
5. **Read the raw data**: a burst read of 6 bytes from `0xF7` gives 20-bit raw pressure and temperature.
6. **Compensate**: implement the integer compensation formulas from the datasheet to compute temperature in °C and pressure in Pa. Compare your results with the library's.

```
uint8_t read8(uint8_t reg) {
  Wire.beginTransmission(0x76);
  Wire.write(reg);
  Wire.endTransmission(false);           // repeated START
  Wire.requestFrom((uint8_t)0x76, (uint8_t)1);
  return Wire.read();
}

uint16_t read16LE(uint8_t reg) {          // calibration words are little-endian
  Wire.beginTransmission(0x76);
  Wire.write(reg);
  Wire.endTransmission(false);
  Wire.requestFrom((uint8_t)0x76, (uint8_t)2);
  uint16_t lo = Wire.read(), hi = Wire.read();
  return (hi << 8) | lo;
}
```

Part 2: SPI, showing data on the OLED

1. Initialize the OLED over SPI (Chapter 7.) using the kit pins.
2. Show the temperature and pressure from Part 1, updated once per second, plus a counter of SPI frame updates.
3. Measure how long one full-screen update takes with `micros()`. Estimate the number of bytes sent per update (128 × 64 pixels at 1 bit each) and the effective data rate.

Part 3: UART, talking to another device

1. On your laptop, open the Serial Monitor at 115200 baud and implement a small **command interface**: typing `temp`, `pres` or `led 1 on` should produce a reply or an action.
2. Make the parser robust: ignore case, reject unknown commands with an error message, and never block while waiting for input.
3. **Optional**: connect two kits with a second UART (`Serial2`) on free pins, crossing TX and RX and sharing ground, and send readings from one kit to the other's display.

Stretch: hardware vs. software SPI

Drive the OLED with a **software** (bit-banged) SPI on different free pins, using the display library's software-SPI constructor. Measure the update time again and compare it with hardware SPI. When is software SPI worth using anyway?

What to show and submit

- Demonstrate Parts 1–3 to a TA, including the BMP280 values computed from your own compensation code.
- Submit your code and a text file with the analysis answers and your measurements.

Analysis questions

1. Which oversampling and filter settings did you choose for the BMP280, and why?
2. Why must the temperature be computed before the pressure in the BMP280's compensation?
3. How many bytes does one OLED update send, and what effective rate did you measure? How does this compare with what I²C at 400 kHz could achieve?
4. When is it reasonable to use a communication method that is not native to your hardware, such as software SPI?

Lab 4: BLE Step Counter

> **i** Lab Companion
>
> Pin numbers, libraries and settings reflect the current kit. If your term's handout differs, **it wins**.

Goals

- Turn raw accelerometer data into a meaningful event: a **step**.
- Expose the result over **Bluetooth Low Energy** using GATT services and characteristics (Chapter 10.).
- Evaluate a detector's accuracy with real walking data.

Uses: the MPU-6050 (Stage 2) and the OLED (Stage 1). You also need a phone with a BLE explorer app, such as nRF Connect.

Part 1: detecting steps

When you walk with the kit in your hand or pocket, each step produces a peak in the **magnitude** of the acceleration:

$$|a| = \sqrt{a_x^2 + a_y^2 + a_z^2} \tag{22.1}$$

Using the magnitude makes the detector independent of how the kit is oriented. At rest, $|a| \approx 1$ g.

1. Configure the MPU-6050 for a **±4 g** range (register `0x1C`, `ACCEL_CONFIG`), where the sensitivity is **8 192 counts per g**. Sample at a steady rate, such as 50 Hz, using `millis()`.
2. **Smooth** the magnitude with a short moving average or an exponential moving average (Chapter 6.).
3. **Detect peaks**: count a step when the smoothed magnitude rises above an upper threshold and then falls below a lower threshold (hysteresis), and ignore new peaks for a short **refractory period** (around 250–300 ms) after each step, since people do not take more than three or four steps per second.
4. Show the step count on the OLED.

```
// Sketch of a peak detector with hysteresis and a refractory period
constexpr float HIGH_G = 1.20f, LOW_G = 1.05f;      // starting values: tune them
constexpr uint32_t REFRACTORY_MS = 300;

uint32_t steps = 0;

void onSample(float magSmoothed) {
  static bool above = false;
  static uint32_t lastStep = 0;
  uint32_t now = millis();
  if (!above && magSmoothed > HIGH_G && now - lastStep > REFRACTORY_MS) {
    above = true;
    steps++;
    lastStep = now;
  } else if (above && magSmoothed < LOW_G) {
    above = false;
  }
}
```

The thresholds above are starting points. **Tune them with data**: print the smoothed magnitude over the serial port while walking, plot it, and choose thresholds that separate steps from noise.

Part 2: publishing over BLE

1. Create a BLE peripheral named after your team (for example `IoT-Team-07`) with a custom **Step Counter service**.
2. Add a **step-count characteristic** (read and notify) that sends the count as a 32-bit unsigned integer, and notify connected clients whenever the count changes.
3. Add a **reset characteristic** (write): writing `1` resets the count to zero.
4. Connect with the phone app, enable notifications, and watch the count update as you walk.

The BLE stack uses a lot of flash. If the build reports that the program is too large, select a larger application partition in `platformio.ini`:

```
board_build.partitions = huge_app.csv
```

Part 3: evaluating accuracy

1. Walk exactly **100 steps** (have a teammate count) at a normal pace, three times. Record the counts.
2. Repeat walking slowly, walking quickly and climbing stairs.
3. Shake the kit by hand for 10 seconds without walking. How many false steps are counted?
4. Report the **error** for each condition as $(\text{counted} - \text{actual})/\text{actual}$.

What to show and submit

- A live demonstration: walk 20 steps while the count updates on both the OLED and the phone, then reset it from the phone.
- Your code, your tuned thresholds and how you chose them, and your accuracy table.

Analysis questions

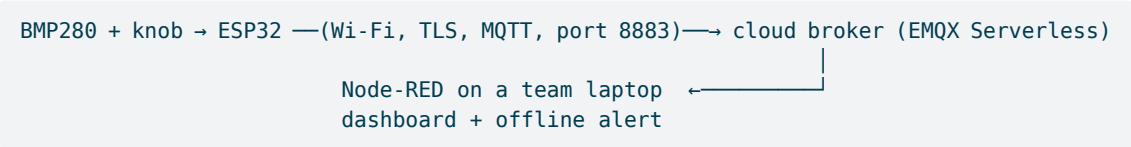
1. Why does the detector use the magnitude of acceleration rather than one axis?
2. What do the hysteresis and the refractory period each prevent?
3. Where was your detector least accurate, and why?
4. How would you reduce the energy this device uses, if it had to run on a coin cell for a month? Consider sampling rate, BLE advertising and connection intervals, and the sensor's own features.
5. How could you replace the hand-tuned thresholds with a learned model (Chapter 18.)?

Lab 5: Cloud Communication

> **i** Lab Companion
>
> Services, libraries and settings reflect the current term. Free cloud plans change often; if your term's handout names a different broker or dashboard, **it wins**.

Goals

- Publish sensor data to a cloud MQTT broker over TLS (Chapter 12.).
- Keep secrets out of the source code: store credentials in the board's flash and enter them over Serial.
- Choose a publish rate on purpose and defend it.
- Use MQTT's **Last Will** so that the broker announces when the device disappears.
- Build a dashboard and an offline alert in Node-RED.



This pipeline is the backbone of the team project: the reliability test (Chapter 21.) reuses the same broker, status topic and dashboard, and the lab is a natural point to build your project's **vertical slice** (Chapter 20.).

What you need

- The kit as wired for Lab 3; nothing new. The lab uses the BMP280 (I²C, 0x76), the potentiometer (GPIO 34) and the OLED.
- A laptop with VS Code and PlatformIO, and Node.js 18 or newer (or Docker) for Node-RED.
- A 2.4 GHz WPA2-Personal network. Enterprise networks such as eduroam need a username and certificate that a plain `WiFi.begin(ssid, password)` cannot provide, and campus networks may block port 8883. A phone hotspot (on an iPhone, turn on *Maximize Compatibility*) or the lab's travel router works (Appendix B.).
- The term's starter project, which compiles before you start.

Topic (replace `T07` with your team ID)	Payload	Published by	Retained
`iot/T07/telemetry`	JSON, every `PERIOD_MS`	the device	no
`iot/T07/status`	`online`	the device, right after it connects	yes
`iot/T07/status`	`offline`	**the broker**, as the device's Last Will	yes

Part A: the cloud side

A1. Create a broker

In Fall 2026 the lab uses **EMQX Serverless**, which has a free plan with no credit card. (The free serverless plan used earlier, HiveMQ Cloud, was closed to new clusters in November 2026; existing clusters could

be kept until the end of the year. Any broker that offers MQTT over TLS with username and password authentication works the same way.)

1. Sign up at the EMQX Cloud site (one account per team) and create a **Serverless** deployment on the free plan. Check the current free-tier limits on the provider's site; they are far above what this lab needs.
2. From the deployment's overview page, copy its **address** (the MQTT host, of the form `xxxxxxxx.ala.us-east-1.emqxsl.com`). The port for MQTT over TLS is **8883**.
3. Under **Access Control › Authentication**, add a credential: a username and a long password. Better: create **two**, one for the device and one for Node-RED, so either can be revoked without breaking the other. Keep them in a password manager, never in a file in your repository.
4. Optional check: connect a desktop client such as MQTTX to the address on port 8883 with TLS on, subscribe to `iot/#`, and leave it open.

A2. Node-RED and the dashboard

Install Node-RED with npm (`npm install -g --unsafe-perm node-red`, then `node-red`) or Docker (`docker run -it -p 1880:1880 -v node_red_data:/data nodered/node-red`), open `http://localhost:1880`, and install **`@flowfuse/node-red-dashboard`** (Dashboard 2.0) from *Manage palette*.

Import the starter flow, replace the team ID, and edit the MQTT broker node: your EMQX address, port 8883, **Use TLS** ticked with broker verification left on, and the Node-RED credential on the *Security* tab. Deploy; each MQTT-in node should show **connected**.

Node-RED stores the password separately in `flows_cred.json` in its own folder, so an exported flow does not contain it. Never commit that file.

Part B: credentials in flash, not in code

A `secrets.h` file ends up in Git, in zip files and on course sites. In this lab the board instead asks for its credentials once over Serial and keeps them in **NVS** (non-volatile storage, a key-value area of the flash) with the `Preferences` library. The source code contains no secrets at all.

1. `loadCredentials()`: open the namespace read-only, read the Wi-Fi SSID and password, MQTT host, username and password with `getString()`, and close it. Return `true` only if the SSID, host and username are present.
2. `saveCredentials()`: open it read-write and `putString()` the same keys.
3. On first boot the board prompts for each value and echoes passwords only as `********`. After a reset it prints *Credentials loaded from NVS* and asks nothing; the values survive power cycles and new uploads.
4. Type `PROVISION` within 3 seconds of a reset to change the values, or `FORGET` to erase them. **Erase them before the kit is returned**: kits are reused.

Part C: TLS, Last Will, back-off and telemetry

1. **TLS client.** Configure `WiFiClientSecure` and PubSubClient: server and port 8883, a keepalive of about 15 s, a short socket timeout and a buffer of about 512 bytes. In this lab the client may call `setInsecure()`. Put a comment above it, in your own words, saying that **the connection is encrypted but the broker is not verified**, so a fake hotspot could impersonate the broker and collect the MQTT password. Lab 6 fixes this; do not fix it here.
2. **Connect with a Last Will.** Use the seven-argument `connect()`: client ID, username, password, will topic (`iot/T07/status`), will QoS 1, will retained `true`, will message `offline`. On success, publish `online` to the same topic, **retained**, so that a dashboard started later still sees the current state.
3. **Reconnect with back-off.** `loop()` calls `maintainWiFi()`, `maintainMqtt()` and `mqtt.loop()` on every pass, and never uses `delay()`. In `maintainMqtt()`, retry only when the back-off interval has passed; on failure, print the client's state, double the interval up to 30 s and add a little random **jitter**, so that 40 boards do not all retry at the same instant after a router reboot.
4. **Telemetry at a rate you can defend.** Set `PERIOD_MS` and justify it in a comment: how fast does room temperature change, how many messages per day is `86400000 / PERIOD_MS`, and what does that add up to over a 72-hour test? Then publish, not retained:

```
{"seq":42,"temp_c":23.41,"press_hpa":1012.60,"pot_pct":37,"rssi":-61,"uptime_s":420}
```

`seq` increases by one per message, so a gap reveals a lost message.

Why does offline take about 20 seconds?

The broker cannot see a pulled cable. It waits for a message or a keepalive ping and gives up after **1.5 × keepalive** of silence (about 22 s with a 15 s keepalive), then publishes the Will. A shorter keepalive detects failures faster but costs more pings, and therefore more traffic and energy. The telemetry rate does not change detection time; the keepalive does. A *clean* `disconnect()` tells the broker the device left on purpose, so no Will is sent; publish `offline` yourself before a planned disconnect.

Part D: dashboard and alert

1. **Dashboard.** Feed the parsed telemetry to a **gauge** and a **chart** of the temperature, plus at least one more value (pressure or the knob).
2. **Alert.** Complete the *offline detector* function so that it outputs an alert message (with the time) only when the status **changes** to `offline`, remembering the previous state in the node's context. It drives a dashboard notification.
3. **Test.** Unplug the board: within about 25 s the status changes to offline and the notification appears. Plug it back in and the status returns to online.

Part E: project analysis

Where is the weakest point in this pipeline, and what would it cost to fix? Follow the data from the sensor to the alert on the screen: sensor, ESP32, Wi-Fi, the local network, TLS, the free cloud broker, the laptop running Node-RED, the browser. Pick the link most likely to fail first or easiest to attack, give your evidence, and estimate the cost of fixing it in money, time, power and complexity. Candidates include `setInsecure()`, a laptop that must stay awake, one shared password, a free tier with no service-level agreement, flaky Wi-Fi and QoS 0 telemetry.

What to show and submit

- A live demonstration: data reaching the broker, the dashboard updating, the Last Will marking the device offline when unplugged, and the alert. Any team member should be able to explain the NVS, Last Will, back-off and `mqtt.loop()` code.
- Your source code with the `PERIOD_MS` justification, and the written analysis. **No passwords anywhere**: search your repository (`git grep -i` part of the password) before pushing. If a password ever reaches Git, change it in the broker console; deleting the commit is not enough.

Troubleshooting

PubSubClient state	Meaning	Usual cause
−4	Connection timeout	Weak Wi-Fi, or the network drops port 8883; try a hotspot
−3	Connection lost	`loop()` blocked (a `delay()`), or Wi-Fi dropped
−2	Connect failed	Wrong host (the console URL instead of the deployment address), wrong port, or port 8883 blocked
3	Server unavailable	Deployment stopped; check the console
4	Bad credentials	Wrong username or password; `PROVISION` again (case-sensitive)

PubSubClient state	Meaning	Usual cause
5	Not authorized	Credential lacks permission for `iot/#`; check the broker's authorization rules

- **Status flips online/offline every few seconds:** two boards share a client ID, or `loop()` is blocked longer than the keepalive.
- **Dashboard shows offline right after deploying:** that is the retained Will from the last unplug; power the board.
- **Alert fires on every message:** the detector does not compare with the previous state.
- **Board resets after connecting:** brown-out on a weak USB port; try another cable or port.

Stretch goal

Add a **stale-data watchdog** in Node-RED: if no telemetry arrives for 3 × `PERIOD_MS`, show a warning even while the status says online. A frozen sensor loop still looks online to the broker.

Analysis questions

1. Why does the device connect out to the broker instead of the dashboard connecting to the device?
2. Which QoS and retain settings did you use for each topic, and why?
3. What does the Last Will give you that a heartbeat alone does not?
4. What is the risk of `setInsecure()`? Describe an attack it allows.
5. How would your topic design change for 1000 devices owned by different customers?

Lab 6: Harden Your Device

> **i** Lab Companion
>
> Procedures reflect the current term. If your term's handout differs, **it wins**.

> **⚠** Nothing irreversible on kit boards
>
> Kits are reused every term. **Do not burn eFuses, enable flash encryption or secure boot, or change the ATECC608A's configuration** on your kit. Those steps are demonstrated by the teaching staff on a dedicated board. Every step in this lab is reversible.

Goals

- Apply the security practices from Part IV to your own device and system.
- Remove credentials from source code and authenticate the broker properly.
- Protect commands against tampering and replay.
- Reduce the device's attack surface.
- Understand, without burning anything, how the ESP32's eFuses protect production devices.

Uses: your Lab 5 system.

Part 1: threat model

Draw the data-flow diagram of your Lab 5 system with its trust boundaries, and apply **STRIDE** (Chapter 14.). List your top five threats with likelihood and impact. Keep this list: each part below should address one or more of them.

Part 2: verify the broker

Replace `setInsecure()` with real certificate verification:

1. Find the root CA certificate that issued your broker's certificate (your broker's documentation names it; for example, many services use Let's Encrypt's ISRG Root X1).
2. Store it in the firmware as a PEM string and pass it to `setCACert()`.
3. Make sure the device has the correct time (with `configTime()` and NTP) before connecting, since certificate validity depends on it.
4. Show that connecting to a different host with the same code now **fails**.

Part 3: get secrets out of the source code

1. Remove Wi-Fi and broker credentials from `secrets.h` in the firmware.
2. Implement **serial provisioning**: at first boot (or when both buttons are held at start-up), the device asks for its credentials over the Serial Monitor and stores them in **NVS** (non-volatile storage) using the `Preferences` library. Normal boots read them from NVS.
3. Check your repository's **history** for secrets that were ever committed (`git log -p | grep -i pass`), and rotate (change) any you find.

NVS on this ESP32 is not encrypted unless flash encryption is enabled. Note in your report what an attacker with physical access could still read, and what a production device would do instead (Chapter 16.).

Part 4: authenticate commands

Protect your command topic against spoofing, tampering and replay, using either:

- an **HMAC** with a per-device shared key and a monotonically increasing counter (Chapter 14.), with the counter stored in NVS so that a reboot does not reset it; or
- **ECDSA signatures** verified on the device, if your kit's ATECC608A has been provisioned with a key by the staff (read-only use; Chapter 15.).

Show that a modified command and a replayed command are both rejected, and that a valid one is accepted.

Part 5: reduce the attack surface

1. Add a `PRODUCTION` build flag that removes debug output containing secrets or internal state.
2. Validate every field of every incoming message: types, ranges and lengths. Reject anything else.
3. Rate-limit commands, so a flood cannot keep the buzzer on or the CPU busy.
4. Use per-device broker credentials with permissions limited to your own topics, if your broker supports access control.

Part 6: eFuses, read-only

Run the read-only summary command from the ESP-IDF tools, with the kit connected:

```
espefuse.py --port /dev/ttyUSB0 summary
```

(On Windows the port is `COM3` or similar; on macOS, `/dev/cu.usbserial-…` or `/dev/cu.SLAB_USBtoUART`.)

Identify the eFuses for flash encryption, secure boot, JTAG and UART download mode. Are any of them set on your board? Then watch the staff demonstration of flash encryption on a dedicated board.

What to show and submit

- A demonstration: provisioning over serial, a verified TLS connection, and rejection of tampered and replayed commands.
- Your threat model, showing which threats each change addresses and which remain.
- Your code (with no secrets in it or in its history).
- Answers to the analysis questions.

Analysis questions

1. Which of your top five threats did this lab address, and which remain? What would you do next?
2. Why does certificate verification depend on the device knowing the correct time?
3. What could an attacker with physical access to your kit still extract, and how would secure boot and flash encryption change that?
4. Why must the replay counter survive a reboot?
5. Explain, from your eFuse summary, why enabling these protections on a production device is irreversible, and why that is a feature rather than a flaw.

22.7 Starter code . . . 221
22.7 Hints . . . 222
22.7 Expected result . . . 222
22.7 Extension (optional) . . . 222
22.7 Troubleshooting . . . 222
22.7 Questions . . . 222

22.7 Activity 3: Scan the I²C Bus, Report SPI . . . 223
22.7 Goal . . . 223
22.7 What the starter code . . . 223
22.7 Expected result . . . 223
22.7 Extension (optional) . . . 224
22.7 Troubleshooting . . . 224
22.7 Questions . . . 224

22.7 Activity 4: The Observe–Decide–Act Loop . . . 225
22.7 Goal . . . 225
22.7 Before you start . . . 225
22.7 TODOs . . . 225
22.7 Expected result . . . 226
22.7 Extension (optional) . . . 226
22.7 Troubleshooting . . . 226
22.7 Questions . . . 226

22.7 Activity 5: A Classroom Poll over HTTP . . . 227
22.7 Goal . . . 227
22.7 Before you start . . . 227
22.7 Steps and TODOs . . . 227
22.7 Expected result . . . 228
22.7 Status codes you may see . . . 228
22.7 Extension (optional) . . . 228
22.7 Questions . . . 228

22.7 Activity 6: A LoRa Emergency Beacon . . . 229
22.7 Goal . . . 229
22.7 TODOs . . . 229
22.7 Expected result . . . 229
22.7 Class signal test . . . 230
22.7 Extension (optional) . . . 230
22.7 Troubleshooting . . . 230
22.7 Questions . . . 230

22.7 Activity 7: MQTT Command Center and Picking the QoS . . . 231
22.7 Goal . . . 231
22.7 Part A: the command center . . . 231
22.7 Part B: pick the QoS . . . 231
22.7 Extension (optional) . . . 232
22.7 Troubleshooting . . . 232
22.7 Questions . . . 232

22.7 Activity 8: Signed Messages and STRIDE . . . 233
22.7 Goal . . . 233
22.7 TODOs . . . 233
22.7 Running it . . . 233
22.7 Extension (optional) . . . 234
22.7 Questions . . . 234

VIII Part VIII — Project Activities

About the Activities

The eight activities in this part are short, hands-on exercises designed for a lecture session. Each one takes 15 to 45 minutes and turns one idea from a chapter into something that runs on a board.

Table 22.1: The lecture activities.

#	Activity	Chapter	Hardware
1	Blink and basic circuits	Chapter 2.	Simulator (Wokwi)
2	Buttons, debouncing and interrupts	Chapter 3.	Simulator (Wokwi)
3	Scan the I²C bus, report on SPI	Chapter 5.	Kit, Stages 1–2
4	The Observe–Decide–Act loop	Chapter 7.	Kit, Stage 3
5	A classroom poll over HTTP	Chapter 8.	Kit, Stage 1 + Wi-Fi
6	A LoRa emergency beacon	Chapter 10.	Kit, Stage 4
7	MQTT command center and picking the QoS	Chapter 12.	Kit + Wi-Fi
8	Signed messages and STRIDE	Chapter 14.	Kit, Stage 2

How each activity works

- **Starter code with TODOs.** Each kit activity starts from a small PlatformIO project in which the interesting lines are left as numbered TODOs. The handout gives a hint for each one. The activities in this book describe the TODOs; the starter projects are distributed by the course and will be published alongside the book.
- **Required part and extension.** The TODOs are the required part. The *Extension* at the end is optional, for teams that finish early.
- **Roles.** Kit activities work best in teams of two or three with rotating roles: a *Driver* at the keyboard and board, a *Navigator* who reads the handout and hints aloud, and a *Recorder* who keeps notes for the questions.
- **Questions.** Each activity ends with four short questions. They are meant to be answered in a sentence or two, and they check understanding rather than code.
- **Grading.** In IoT@UCI, activities are graded on participation, not correctness. Solutions are not published.

Helpers used by the starter projects

The starter projects share a small support library, so that activity code stays short:

Table 22.2: Support helpers used in the activities.

Helper	What it does
`IoTPins::…`	Pin names from `iot_pins.h` (Appendix A.)
`iotInitI2C()`	Starts the I²C bus on SDA 21 / SCL 22
`iotInitSPI()`	Starts the SPI bus on SCK 18 / MOSI 23 / MISO 19
`iotInitDisplay()`	Initializes the OLED; returns `true` on success
`iotDisplayLines(a, b, c, d)`	Writes up to four lines to the OLED
`iotConnectWiFi()`	Joins the network in `secrets.h`; returns `true` on success

i For instructors

The network activities (5 and 7) need a 2.4 GHz WPA2-Personal network that every board can join, such as a travel router (Appendix E.), and a server or broker on the instructor's laptop. Activity 6 needs one receiving radio at the front of the room. Activity 8 needs ATECC608A chips that were provisioned **before** the term: students must never configure or lock the chips themselves.

Activity 1: Blink and Basic Circuits

Time: about 15 minutes · **Format:** individual · **Hardware:** none, the free Wokwi ESP32 simulator in a web browser · **Chapter:** Chapter 2.

Goal

Build the simplest useful embedded circuit, an LED with a current-limiting resistor, and make it blink from software. The code runs unchanged on a real ESP32 later.

After this activity you should be able to:

- build an LED circuit with a series resistor;
- configure a GPIO pin as an output and drive it HIGH and LOW;
- control timing with `delay()`;
- run and debug ESP32 code in a simulator.

Set up the simulator

1. Open wokwi.com (an optional free account lets you save and share projects).
2. Start a new **ESP32** project from the **Arduino** template. The editor is on the left, the simulated board on the right.
3. Add parts with the **+** button; make a wire by clicking one pin and then another; click a part to change its settings.
4. Press ▸ **Start** to run. Serial output appears below the board.

Build the circuit

Add one **LED** and one **resistor** (set it to **220 Ω**), and wire:

- ESP32 pin **D26** (GPIO 26) ⟶ resistor;
- resistor ⟶ LED **anode** (A, the longer leg);
- LED **cathode** (C) ⟶ **GND**.

Starter code

```
#include <Arduino.h>

const int LED_PIN = 26;   // GPIO wired to the resistor and LED

void setup() {
  Serial.begin(115200);
  Serial.println("Blink LED");
  // TODO 1: configure LED_PIN as an output
}

void loop() {
  // TODO 2: turn the LED on
  // TODO 3: wait 500 ms
  // TODO 4: turn the LED off
  // TODO 5: wait 500 ms
}
```

Hints

TODO	Hint
1	A pin must be made an output once, in `setup()`, with `pinMode()`.
2, 4	`digitalWrite()` drives the pin to 3.3 V (`HIGH`) or 0 V (`LOW`).
3, 5	`delay()` takes milliseconds.

Expected result

The LED is on for about half a second, off for about half a second, and repeats; the Serial Monitor shows the title line once.

Extension (optional)

Add a second LED with its own 220 Ω resistor on GPIO 27 and make the two LEDs alternate, or blink SOS (three short, three long, three short). The simulator may light an LED wired without a resistor: what would happen on a real board? (Chapter 7. has the arithmetic.)

Troubleshooting

- **LED never lights:** the anode must face the GPIO pin, the cathode GND.
- **Nothing happens:** the circuit must be on the same pin as `LED_PIN`.
- **Compile error:** read the first error line; a missing semicolon is the usual cause.
- **LED stays on or off:** check that both delays are present and that TODO 1 is inside `setup()`.

Questions

1. Why does the circuit need a resistor in series with the LED?
2. Why must a GPIO pin be configured with `pinMode()` before it is used?
3. What would you observe if both `delay()` calls were removed? Why?
4. What would you change to make the LED blink twice as fast?

Activity 2: Buttons, Debouncing and Interrupts

Time: about 15 minutes · **Format:** individual · **Hardware:** none, the Wokwi ESP32 simulator · **Chapter:** Chapter 3.

Goal

Count button presses with an **interrupt service routine (ISR)** instead of polling, and add a software **debounce** so that one press counts once. Wokwi's pushbutton imitates the contact bounce of a real button, so you will see the problem before you fix it.

After this activity you should be able to:

- wire a pushbutton with `INPUT_PULLUP`;
- register an ISR with `attachInterrupt()` and explain what an ISR is for;
- implement a time-based debounce;
- explain why variables shared with an ISR are `volatile`.

Build the circuit

Add one **pushbutton**: one side to **D27** (GPIO 27), the other to **GND**. No resistor is needed: the internal pull-up holds the pin HIGH until the button pulls it LOW. In the simulator, click and hold the button to press it.

Starter code

```
#include <Arduino.h>

const int BUTTON_PIN = 27;

volatile unsigned long pressCount = 0;      // changed inside the ISR
volatile unsigned long lastInterruptMs = 0;
unsigned long lastPrinted = 0;

void IRAM_ATTR onButtonPress() {
  // TODO 1: ignore presses within 150 ms of the last accepted one
  // TODO 2: increment pressCount
}

void setup() {
  Serial.begin(115200);
  Serial.println("Button interrupts");
  // TODO 3: configure BUTTON_PIN as an input with the internal pull-up
  // TODO 4: attach onButtonPress to BUTTON_PIN on the FALLING edge
}

void loop() {
  if (pressCount != lastPrinted) {
    lastPrinted = pressCount;
    // TODO 5: print the current press count
  }
}
```

Hints

TODO	Hint
1	Read `millis()`; accept the press only if more than 150 ms have passed since `lastInterruptMs`, and then update `lastInterruptMs`.
2	Increment inside the accepted branch only.
3	`pinMode()` with `INPUT_PULLUP`.
4	`attachInterrupt(digitalPinToInterrupt(pin), handler, FALLING)`.
5	`Serial.print()` then `Serial.println()`.

Keep the ISR short: never call `Serial` or `delay()` inside it (Chapter 4.).

Expected result

Each press prints one new line, such as `Button count: 3`.

Extension (optional)

Set the debounce window to 0 ms and press several times: how often does one press count more than once? Then add an LED (with a 220 Ω resistor) that toggles on every valid press.

Troubleshooting

- **Count jumps by several per press:** the debounce check is missing or the window is too small.
- **Count never changes:** check the wiring and `INPUT_PULLUP`.
- **Counts on release:** use `FALLING`, not `RISING`.
- **Simulation crashes when pressing:** something slow is inside the ISR.

Questions

1. What is an interrupt, and why is it useful compared with checking the button in `loop()`?
2. What problem does debouncing solve? What did you see with a 0 ms window?
3. Why does `INPUT_PULLUP` let you wire the button without an external resistor?
4. Why is the falling edge used here?

Activity 3: Scan the I²C Bus, Report on SPI

Time: about 45 minutes · **Format:** team of 2–3 with the kit · **Hardware:** kit Stage 1 (ESP32 and OLED) and at least one Stage 2 I²C device · **Chapter:** Chapter 5.

Goal

Find every device on the kit's shared I²C bus by its address, name it, and show the result on the OLED, which is on the SPI bus. One program, two buses.

After this activity you should be able to:

- explain how several devices share two I²C wires using addresses;
- initialize the I²C and SPI buses on the ESP32;
- scan a bus and interpret the addresses that answer;
- compare I²C and SPI in wires, speed and addressing.

If only some of your Stage 2 devices are wired, do the activity anyway: a scan that finds one device, or none, still shows the idea.

What the starter does

The starter project contains a scan loop that, for each 7-bit address from 1 to 126, starts a transmission and checks whether any device acknowledges. You complete six TODOs:

TODO	Task	Hint
1	Initialize the I²C bus	`iotInitI2C()`
2	Print each address that answers, in hexadecimal, with a leading zero below 0x10	`Serial.print(addr, HEX)`; use `print`, not `println`, because the starter finishes the line
3	In `identifyI2CDevice(addr)`, return a name for each known address	0x60 ATECC608A, 0x68 MPU-6050, 0x76 or 0x77 BMP280
4	Run the scan and keep the count	Store the return value of `scanI2CBus()`
5	Initialize SPI, then the display	`iotInitSPI()` before `iotInitDisplay()`
6	Show the count and up to three names on the OLED	`iotDisplayLines(...)`

Expected result

```
Found I2C device at 0x60 -> ATECC608A secure element
Found I2C device at 0x68 -> MPU-6050 motion sensor
Found I2C device at 0x76 -> BMP280 pressure sensor
Total I2C devices found: 3
```

The OLED shows the same count and names, which proves the SPI side works too.

Extension (optional)

In `loop()`, show the live potentiometer value on the OLED and count how many full-screen updates you complete per second. Use the result to estimate the SPI throughput, and compare it with what the same 1 KB frame would take on I²C at 400 kHz (Section 5.3 works through the arithmetic).

Troubleshooting

- **0 devices found:** TODO 1 must run before the scan; check the four bus wires (3V3, GND, SDA 21, SCL 22).
- **0x60 missing although wired:** the ATECC608A sleeps until woken and can miss a scan. Press EN to scan again.
- **Addresses print as `0x8`:** add the leading-zero check.
- **OLED blank:** SPI must start before the display; the module's "SDA" pin is SPI data and goes to GPIO 23, not 21.

Questions

1. How can three devices share the same two I²C wires without their messages getting mixed up?
2. What would happen if two devices on one bus had the same address? How do some sensors avoid this?
3. Why does the OLED use SPI instead of I²C? What does SPI cost in exchange?
4. Count the wires each bus uses in the kit. Which bus would you choose for a board with very few free pins, and why?

Activity 4: The Observe–Decide–Act Loop

Time: about 45 minutes · **Format:** team of 2–3 with the kit · **Hardware:** kit Stage 3 (servo on its 5 V rail) · **Chapters:** Chapter 6. and Chapter 7.

Goal

Make the servo follow the potentiometer. Every 100 ms the program will:

- **Observe:** read the potentiometer with the ADC;
- **Decide:** map the reading to a servo pulse width and convert it to a PWM duty value;
- **Act:** output the PWM signal that moves the servo.

Before you start

Check the Stage 3 wiring (Lab 1): servo signal on GPIO 13, red wire on the **5 V rail** from VIN, brown or black on GND, and the 470 µF capacitor across the rail with its stripe to GND. **Do not power the servo from 3V3**: its current spikes reset the ESP32.

TODOs

TODO	Task	Hint
1	Make the potentiometer pin an input	`pinMode(IoTPins::POT_ADC, INPUT)`
2	Set up a 50 Hz, 16-bit PWM channel on the servo pin	`ledcSetup(SERVO_CH, 50, 16)` and `ledcAttachPin(...)`; the starter uses channel 2 because `tone()` uses channel 0
3	Observe: read the ADC	`analogRead()` returns 0–4095
4	Decide: map to 600–2400 µs, then to a duty value	`map()`, then the provided `usToDuty()`
5	Act: write the duty value	`ledcWrite(SERVO_CH, duty)`

i The duty arithmetic

At 50 Hz one period is 20 ms. With 16-bit resolution the period is divided into $2^{16} = 65\,536$ counts, so a pulse of t microseconds needs a duty of

$$\text{duty} = \frac{t}{20\,000\ \mu\text{ s}} \times 65\,536. \tag{22.1}$$

A centred servo (1500 µs) is therefore about 4915 counts.

Expected result

```
Pot  328 ->  744 us -> duty 2437
Pot 2048 -> 1500 us -> duty 4915
Pot 4095 -> 2400 us -> duty 7864
```

The OLED shows the reading, the pulse width and the approximate angle, and the servo follows the knob.

Extension (optional)

Add a **dead band**: only write a new duty when the pulse width differs from the last one sent by more than 20 µs. Does the servo stop buzzing when you let go of the knob? Why would a battery-powered product want this?

Troubleshooting

- **Servo does not move:** signal on GPIO 13; servo ground shared with the ESP32; `ledcAttachPin()` in `setup()`.
- **Board resets (“Brownout detector was triggered”):** the servo is on 3V3 or the capacitor is missing.
- **Servo buzzes at the ends:** it is pushing against its end stops; narrow the range to about 700–2300 µs.
- **`ledcSetup` does not compile:** the project needs Arduino core 2.x (`espressif32@6.9.0`); core 3.x renamed these functions.

Questions

1. Which lines of your code are Observe, which Decide, and which Act?
2. What duty value gives a 1.5 ms pulse? Why does the servo never use most of the 0–65 535 range?
3. What would change if the loop ran once per second instead of every 100 ms?
4. Why is the servo powered from the 5 V rail with a capacitor instead of the 3V3 pin?

Activity 5: A Classroom Poll over HTTP

Time: about 35 minutes · **Format:** team of 2–3 with the kit · **Hardware:** kit Stage 1 and a classroom Wi-Fi network · **Chapter:** Chapter 8.

Goal

Turn the ESP32 into an HTTP **client**. It joins the room's Wi-Fi network, reads the potentiometer as a percentage, and, when Button 1 is pressed, sends the team's answer to a poll **server** on the instructor's laptop. A dashboard at the front of the room shows every team's answer and the class average.

Every layer of the TCP/IP model is at work:

Layer	In this activity
Application	HTTP: `GET` for the question, `POST` for the answer, and a status code back
Transport	TCP connection to port 8080, reliable byte delivery
Internet	IP routing to the server's private address on the room router
Link	Wi-Fi (802.11) frames between the board and the router

Before you start

No new wiring. The network name, password and server address are shown in the room. Use the class router, not the campus enterprise network, which blocks unregistered devices (Appendix B.).

Steps and TODOs

Set `WIFI_SSID`, `WIFI_PASSWORD`, `SERVER_BASE_URL` and your `TEAM_ID` in `include/secrets.h`, then complete:

TODO	Task	Hint
1	Configure the potentiometer and Button 1	`INPUT` and `INPUT_PULLUP`
2	Join Wi-Fi	Store the result of `iotConnectWiFi()`
3	Read the knob as 0–100 %	`map()` then `constrain()`
4	Open the request with a JSON content type	`http.begin(url)`; `http.addHeader("Content-Type", "application/json")`
5	Send the answer	`http.POST(body)`, with a body such as `{"team":"T07","value":42}`
6	Interpret the status code	200 means accepted; anything else is a failure to report

Expected result

```
Connected. IP: 192.168.8.23
GET /api/question -> 200
POST http://192.168.8.10:8080/api/answer  {"team":"T07","value":64}
Server replied: {"ok": true, "team": "T07", "value": 64, "replaced": false}
Submission successful.
```

Pressing again replaces your team's answer: the server keeps one answer per team.

Status codes you may see

Code	Meaning here
200 OK	Answer accepted
400 Bad Request	Body not valid JSON, value outside 0–100, or the default team ID
404 Not Found	Wrong path (for example a trailing slash in the base URL)
415 Unsupported Media Type	Missing `Content-Type` header
Negative (e.g. −1)	**Not an HTTP response**: the board never reached the server

Extension (optional)

Measure the round-trip time: record `millis()` just before and after `http.POST()`. Submit several times. How much does it vary, and why? Which part of the time is TCP's handshake (Chapter 12.)?

Questions

1. For each TCP/IP layer, name the part of your program or hardware that handles it.
2. What status code did you get when it worked? What would a 400 tell you, and what does a negative code tell you?
3. Why does the client, not the server, start the conversation in this design?
4. What would have to change for the dashboard to push a new question to every board instead of each board asking?

Activity 6: A LoRa Emergency Beacon

Time: about 30 minutes · **Format:** team of 2–3 with the kit · **Hardware:** kit Stage 4 (RFM95W radio with its antenna) · **Chapter:** Chapter 10.

Goal

Send a short packet directly from radio to radio, with no Wi-Fi and no Internet. The beacon carries the team, a sequence number and a sensor reading; a receiver at the front of the room shows each packet with its **RSSI** and **SNR**.

After this activity you should be able to:

- initialize an SX1276-based radio on the SPI bus;
- build a compact text payload for a low-data-rate link;
- interpret RSSI and SNR;
- explain where LoRa fits compared with Wi-Fi and BLE.

> ⚠ Warning
>
> Keep the antenna attached whenever the radio is powered. Do not change the frequency: every transmitter and the receiver must use the same one, and it must be legal in your region (915 MHz in North America).

TODOs

Set your two-digit `TEAM_ID` first.

TODO	Task	Hint
1	Start the radio	`LoRa.begin(LORA_FREQUENCY_HZ)`; keep the Boolean result. The starter already sets the CS, RST and G0 pins.
2	Read the knob as 0–100 %	`map()` then `constrain()`
3	Build the packet	Exactly `TEAM=<team>;SEQ=<seq>;SENSOR=<percent>`, e.g. `TEAM=07;SEQ=3;SENSOR=62`
4	Transmit	`LoRa.beginPacket()`, `LoRa.print(packet)`, `LoRa.endPacket()`; success when it returns 1

Expected result

```
Packet: TEAM=07;SEQ=1;SENSOR=62
Payload size: 23 bytes
Transmission: SUCCESS
```

Class signal test

Transmit once with the kit in its normal orientation (round A), then rotate it about 90° without moving and transmit again (round B). Record RSSI and SNR for both. RSSI closer to 0 dBm is stronger: −65 dBm is stronger than −90 dBm.

Extension (optional)

Use the time-on-air formula in the chapter's in-depth section (Section 10.3) to estimate how long your 23-byte packet occupies the channel at the starter's spreading factor. How many such packets per hour fit within a 1% duty cycle?

Troubleshooting

- **Radio fails to start:** check the antenna and Stage 4 wiring, power-cycle and upload again.
- **Packet does not appear at the receiver:** wait for other teams, transmit once more, and check that the team ID is not `00`.
- **Different RSSI each time:** expected. Orientation, people, furniture and multipath all change it.

Questions

1. Which orientation gave the stronger signal, and what is one plausible reason?
2. Why is the payload a short text string rather than JSON with long field names?
3. Your packet reached the receiver with no router and no Internet. What did you give up compared with Wi-Fi?
4. Give one IoT application for which LoRa is a better choice than BLE, and one for which it is worse.

Activity 7: MQTT Command Center and Picking the QoS

Time: about 30 minutes · **Format:** team of 2–3 with the kit · **Hardware:** kit Stage 1 and a classroom Wi-Fi network with an MQTT broker · **Chapter:** Chapter 12.

Goal

Connect the board to a classroom MQTT **broker**. It publishes potentiometer telemetry to a team topic and subscribes to a class-wide command topic, so that one message from the instructor changes every kit in the room. Then decide which QoS, retain and Last Will settings fit four real situations.

Term	Meaning
QoS 0	At most once: no acknowledgment; may be lost
QoS 1	At least once: resent until acknowledged; may be duplicated
QoS 2	Exactly once: a four-step handshake, at extra cost
Retained	The broker keeps the last message on a topic and gives it to new subscribers
Last Will (LWT)	A message the broker publishes for a client that disconnects unexpectedly

Part A: the command center

Set the Wi-Fi values, `MQTT_HOST`, `MQTT_PORT` (1883) and `MQTT_TOPIC_BASE` in `include/secrets.h`, and your `TEAM_ID`.

TODO	Task	Hint
1	Point the client at the broker and register the callback	`mqttClient.setServer(...)`, `mqttClient.setCallba`
2	Subscribe to the command topic after connecting	Keep the Boolean result of `subscribe()`
3	Publish telemetry on Button 1	Topic `<base>/team/<TEAM_ID>/telemetry`; payload `TEAM=<team>;SEQ=<n>`
4	Act on commands	`ALERT` → `activateAlert()`; `NORMAL` → `restoreNormalMode()` (case-sensitive)

When the instructor publishes `ALERT`, every connected kit shows an alert and changes its LEDs; `NORMAL` restores them.

Part B: pick the QoS

The starter's library publishes at QoS 0, so this part is a design exercise. For each situation choose a QoS level, decide whether the message is retained, and justify it in one sentence. Then choose **one** situation where a Last Will helps and write its topic and payload.

1. A greenhouse sensor publishes temperature every 10 seconds to a live dashboard.
2. A phone app sends an “unlock” command to a smart door lock.
3. A smart electricity meter reports a daily billing reading that must be counted exactly once.
4. A dashboard must show whether each device is online, including devices that just lost power.

Extension (optional)

On Button 2, publish `HELLO from team <TEAM_ID>` to `<base>/team/<TEAM_ID>/chat`. Would other teams receive it? What would they need to subscribe to? Try a wildcard (`+` or `#`).

Troubleshooting

- **Broker connection fails:** check the host, port and that the board is on the class network; note the state code printed.
- **Your messages do not appear:** the topic base must match exactly.
- **`ALERT` does nothing:** check the subscription result and the string comparison.

Questions

1. Situations 1 and 2: which QoS and retain setting did you choose for each, and why?
2. Situation 3: which QoS did you choose? Could QoS 1 work, and what would the receiver have to do?
3. Situation 4: write the Last Will topic and payload, and say whether it should be retained.
4. Why could the instructor alert every kit with one publish, without knowing how many were connected?

Activity 8: Signed Messages and STRIDE

Time: about 20 minutes · **Format:** team of 2–3 with the kit · **Hardware:** kit Stage 2 with a **pre-provisioned** ATECC608A · **Chapters:** Chapter 14. and Chapter 15.

Goal

Protect a sensor message with a digital signature made by the kit's **ATECC608A** secure element, whose private key never leaves the chip. Then show two things: changing a signed message makes verification fail (**tampering**), and resending an old, valid message is only caught by a **freshness** check (**replay**). Finally, map what you saw to **STRIDE**.

> ! Important
>
> The code only *uses* the chip. It never configures or locks it and never reads private keys. If the chip reports that it is not provisioned, stop and ask the teaching staff.

TODOs

TODO	Task	Hint
1	Convert the ADC reading to 0–100 %	`map()`
2	Build the message	Exactly `DEVICE=<chip serial>;SEQ=<seq>;SENSOR=<p` the serial is `cryptoContext.serialNumber`
3	Sign it in hardware	`IoTCrypto::signMessage(packet.message, packet.signature)`, a 64-byte ECDSA P-256 signature from key slot 0
4	Verify the authentic message	`IoTCrypto::verifyMessage(message, signature, cryptoContext.publicKey)`
5	Tamper test	Verify the **modified** message with the **original** signature; do not re-sign
6	Replay test	(A) the old message still verifies; (B) accept it only if `sequence > lastAcceptedSequence`

Running it

Turn the knob, press **Button 1** to create, sign and verify a message. Press **Button 2** once for the tamper test, and again for the replay test.

Test	Expected OLED
Authentic	`AUTHENTIC MESSAGE` · `Signature: VALID`

Test	Expected OLED
Tamper	`TAMPER DETECTED · Signature: INVALID`
Replay	`REPLAY DETECTED · Signature: VALID · Freshness: REJECT`

The replay row is the key lesson: a valid signature proves the message is authentic and unmodified, but only the sequence check shows it is new.

Extension (optional)

Reset the board and press Button 1. The sequence number starts over. Would a receiver that remembered the last accepted sequence now reject every new message? Propose one way a real device keeps freshness working across reboots (Chapter 15.).

Questions

1. Which STRIDE category does the tamper test demonstrate, and which part of your code defends against it?
2. A signature does not hide the sensor value. Which STRIDE category is still open, and what would close it?
3. Why does a valid signature not stop a replay? Besides a sequence number, name another way to prove freshness.
4. What does keeping the private key inside the ATECC608A protect against, compared with storing it in the ESP32's flash?

Appendices

A. Kit Reference

> **Note**
>
> This appendix describes the IoT@UCI kit as built in Fall 2026, in enough detail for another course to build compatible kits. If your term's kit or starter code differs, **the starter's `iot_pins.h` is authoritative**. Photos of a finished kit will be added in a later release; the illustrations below show the layout.

A.1 What the finished kit looks like

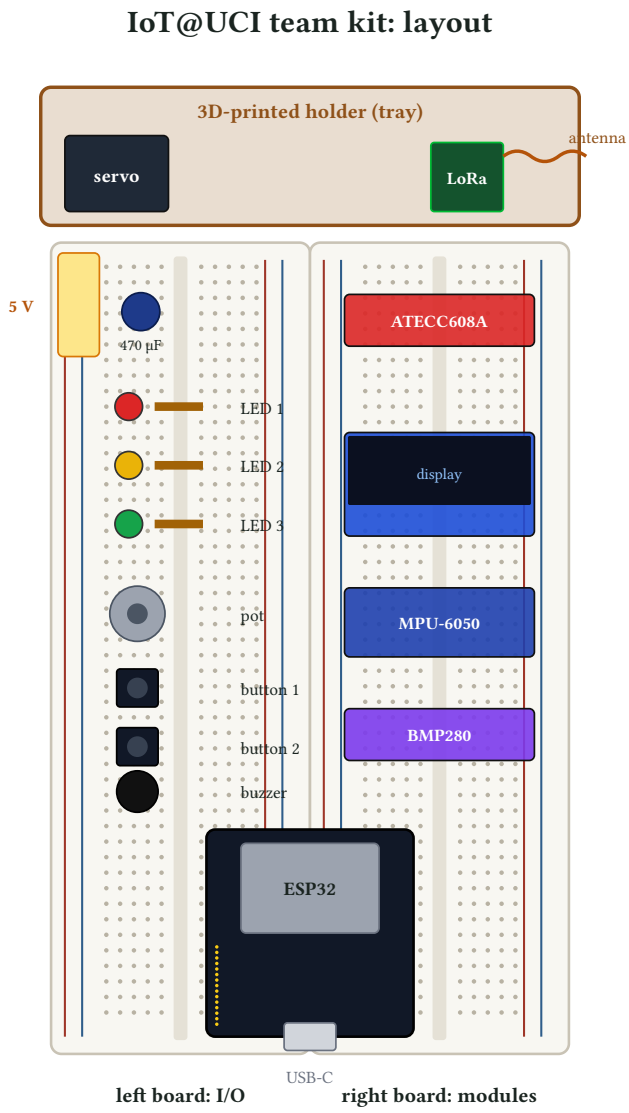


Figure A.1: The team kit, top view. The ESP32 bridges the two breadboards at the bottom; I/O parts are on the left board, bus-connected modules on the right, and a printed holder at the top carries the servo and the radio.

The layout follows a few deliberate rules:

- **Two 830-point breadboards**, joined along their long edges, give every module its own rows and leave space for project additions.
- **The ESP32 straddles the seam** between the boards, at the end nearest the edge of the desk, so the USB cable does not cross the circuit.
- **Left board, top to bottom**: the 5 V servo rail and its capacitor, three LEDs, the potentiometer, two buttons and the buzzer. These are the parts students touch.
- **Right board, top to bottom**: ATECC608A, OLED, MPU-6050, BMP280. All four sit on shared buses, so their header pins line up with the bus wiring.
- **The holder** at the top keeps the servo and radio from pulling on their wires.

A.2 Bill of materials

Each kit is shared by a team of two or three students. Part numbers are the ones used in Fall 2026; equivalent parts work if they match the noted specifications.

Table A.1: Kit bill of materials.

Qty	Part	Specification that matters	Fall 2026 source / part no.
1	ESP32 DevKit V1 (ESP32-WROOM-32), 30-pin, USB-C	30-pin pinout shown in Figure A.3	AITRIP 4-pack, Amazon B0GBVDJLSF
1	USB-C to USB-C cable, 1 m	**Data** cable, not charge-only	SMAYS 10-pack, Amazon B0CC5J8GB7
2	830-point solderless breadboard	Split power rails are common; check	ELEGOO 3-pack, Amazon B01EV6LJ7G
1 pack	Jumper wires (male–male) and 22 AWG solid-core wire	Solid core for flat wiring	ELEGOO 120-pack B01EV70C78; Adafruit #1311
3	LEDs: red, yellow, green, 5 mm		Assorted 600-pack, Amazon B0FL1T4ZHT
3	220 Ω resistors, ¼ W		Resistor assortment, Amazon B0F4P352BB
2	6 × 6 mm tactile push buttons	4-pin, fits across the center channel	DAOKI 100-pack, Amazon B01CGMP9GY
1	10 kΩ linear potentiometer, breadboard-friendly	0.1 in (2.54 mm) pin pitch	Adafruit #4133 (or #562)
1	Passive piezo buzzer	**Passive** (needs a tone), ≈12 mm	Adafruit #160 (PS1240)
1	0.96 in 128 × 64 OLED, SSD1306, **7-pin SPI**	Pins GND, VDD, SCK, SDA, RES, DC, CS	UMLIFE 6-pack, Amazon B08SJW4J4P
1	BMP280 breakout, 3.3 V	6-pin with CSB and SDO broken out	Amazon B0B38CBXG1
1	MPU-6050 (GY-521) breakout	AD0 and INT broken out	HiLetgo 3-pack, Amazon B00LP25V1A
1	ATECC608A breakout	I²C, address 0x60	SparkFun DEV-18077 (DigiKey)
1	SG90-class micro servo, 180°	4.8–6 V, three-wire	Miuzei 10-pack, Amazon B072V529YD
1	470 µF, 16 V electrolytic capacitor	≥ 10 V rating	Nichicon UHE1C471MPD6 (DigiKey)
1	LoRa radio, 915 MHz (North America)	SX1276-based; small 16 × 16 mm module fits the holder	Adafruit RFM95W #3072, or bare RFM95W-915S2
1	915 MHz spring antenna (or 78 mm of 22 AWG wire)	Quarter-wave at 915 MHz	Adafruit #4269
—	Header pins (male), 40-pin strips	Cut to length	Amazon B0774VBJ3J
1	Printed holder (tray)	See below	Breadboard Snap-On Component Tray (830-point), designed by Alvin Ma; free download, printed in PLA or PETG

In Fall 2026 the parts for one new kit cost roughly **$55–60** at pack prices, excluding the printed holder, tax and shipping; the radio and the secure element account for about $23 of that.

Why a quarter-wave antenna is 78 mm

A simple wire antenna works best at a quarter of the wavelength. At 915 MHz the wavelength is $\lambda = c/f = 3 \times 10^8 / 9.15 \times 10^8 \approx 0.328$ m, so $\lambda/4 \approx 82$ mm. A real wire is made slightly shorter (about 95%) because the signal travels a little slower along a conductor than in free space, which gives the familiar ≈78 mm.

A.3 The printed holder

The holder is a single printed part that clips over the outer edges of the two breadboards and carries the servo and the radio, so that they do not pull on their wires.

Get the holder file from its designer

The holder is the **Breadboard Snap-On Component Tray (830-point)**, designed by **Alvin Ma** and shared as a free download on Cults3D: https://cults3d.com/en/3d-model/gadget/breadboard-snap-on-component-tray-830-point. Download the STL there rather than from a copy: the designer keeps it up to date based on feedback from users, and instructors who need a variation for their own kits can find his other designs on the same page. This book does not redistribute the file.

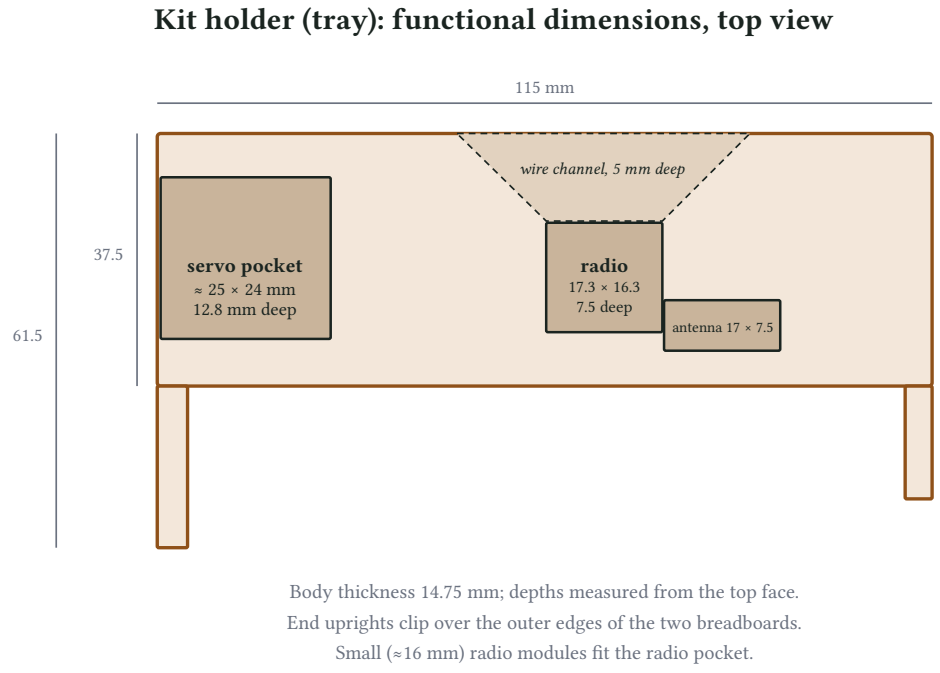


Figure A.2: Functional dimensions of the holder used in Fall 2026, measured from the printed part. Download the part itself from its designer (link above); the dimensions are given to check a print and to plan compatible parts.

Key dimensions (useful to check a print against the kit):

Feature	Value	Why
Body	115 × 37.5 mm, 14.75 mm thick	Spans two 55 mm-wide breadboards plus clip walls

Feature	Value	Why
End uprights	≈ 4 mm wide, extending to 61.5 mm	Clip over the breadboards' outer edges
Servo pocket	≈ 25 × 24 mm, 12.8 mm deep	SG90 body (≈ 23 × 12 × 23 mm) lying on its side, with clearance
Radio pocket	≈ 17.3 × 16.3 mm, 7.5 mm deep	Bare 16 × 16 mm RFM95W module
Antenna arm	≈ 17 × 7.5 mm	Keeps the spring antenna away from the board
Wire channel	Tapered, 5 mm deep	Routes the radio's wires down to the right breadboard

Larger breakout-style radios (such as the 29 × 25 mm RadioFruit board) do **not** fit the radio pocket; they sit on the right breadboard instead.

Printing tips: PLA or PETG, 0.2 mm layers, 15–20% infill, no supports if printed pocket-side up. Print one, test-fit a servo, a radio and both breadboards, then print the rest.

A.4 Wiring by stage

The kit is wired in four stages, each just before the chapter that first needs it (Lab 1).

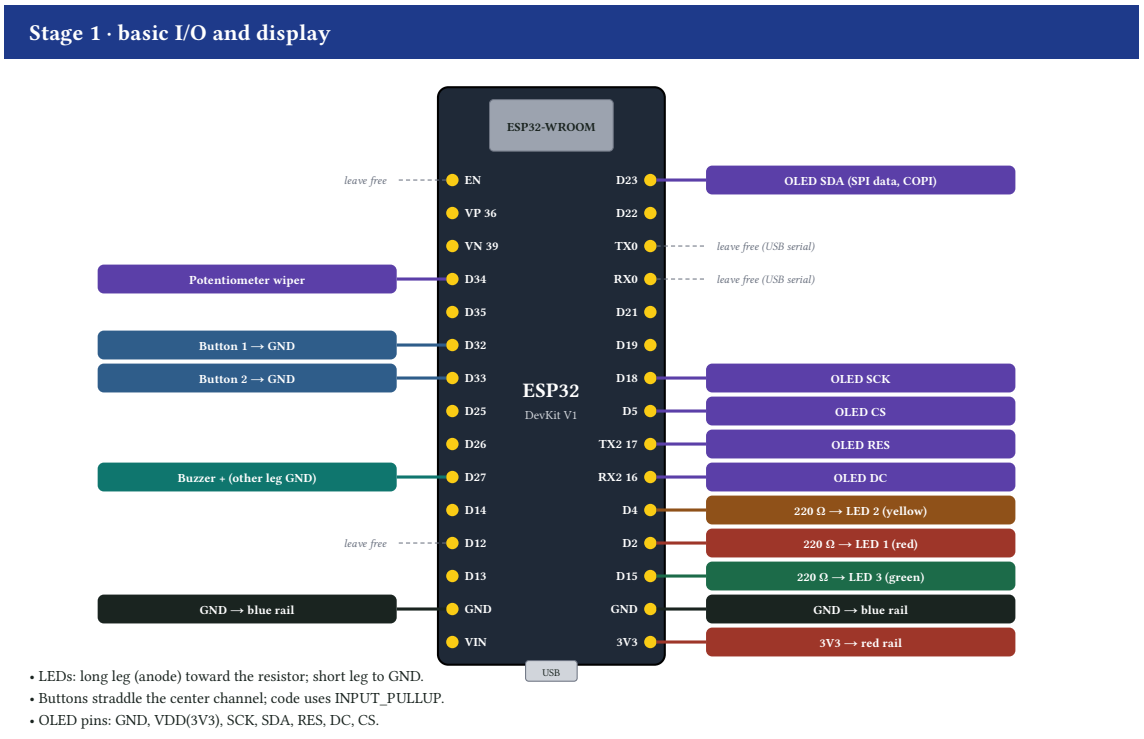


Figure A.3: Stage 1: LEDs, buttons, potentiometer, buzzer and the SPI display.

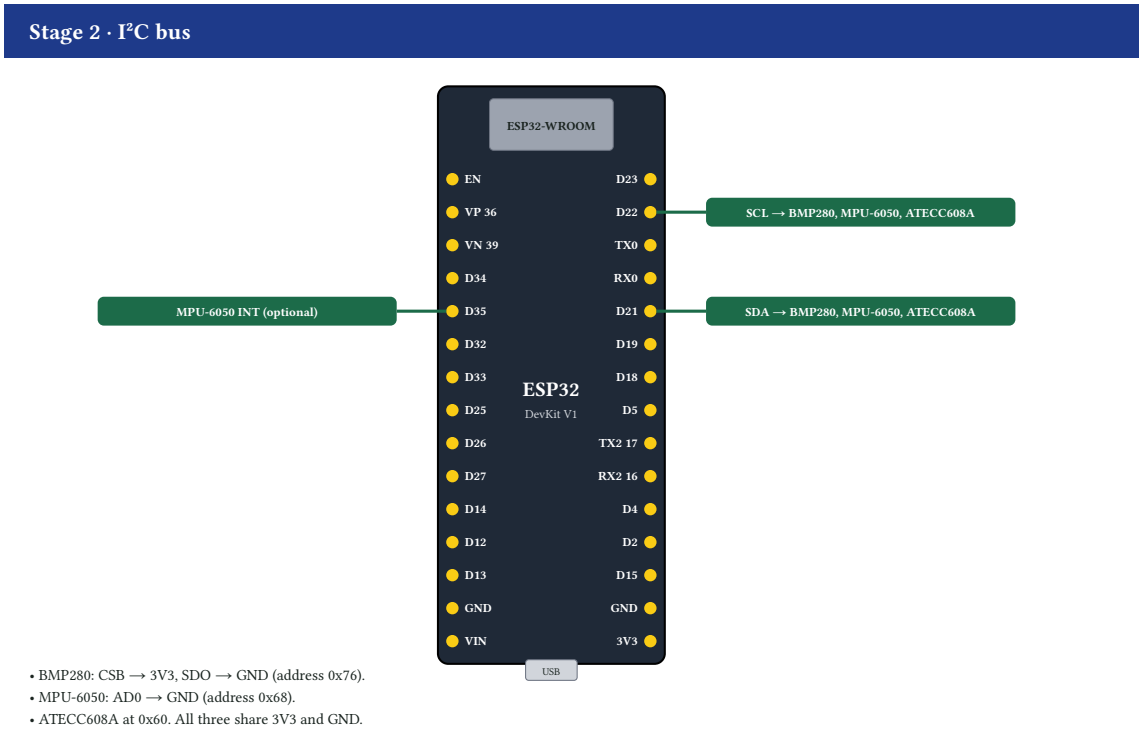


Figure A.4: Stage 2: the shared I²C bus for the BMP280, MPU-6050 and ATECC608A.



Figure A.5: Stage 3: the servo and its separate 5 V rail.

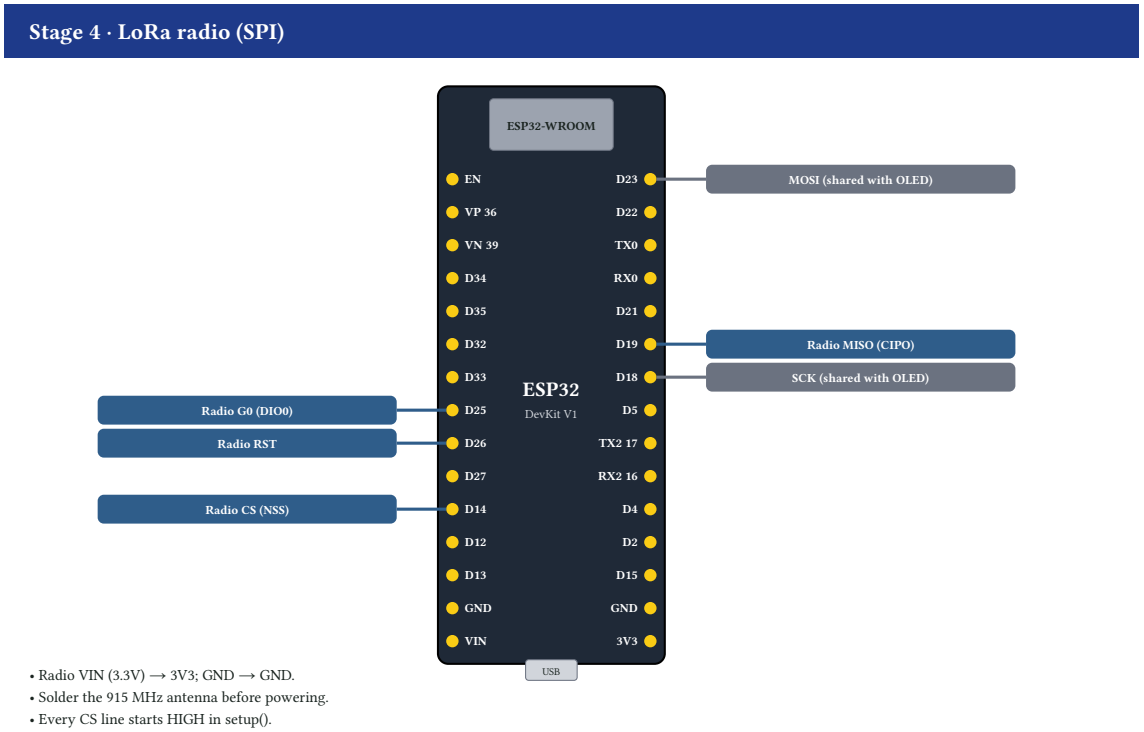


Figure A.6: Stage 4: the LoRa radio on the SPI bus, sharing SCK and MOSI with the display.

A.5 Pin map

Table A.2: Kit pin map.

Function	GPIO	Notes
LED 1 (red)	2	Strapping pin; also the board's blue LED on many DevKits
LED 2 (yellow)	4	
LED 3 (green)	15	Strapping pin
Button 1	32	`INPUT_PULLUP`, active low
Button 2	33	`INPUT_PULLUP`, active low
Potentiometer wiper	34	Input-only, ADC1
Buzzer	27	LEDC channel shared with `tone()`
Servo signal	13	On its own LEDC channel; powered from the 5 V rail
I²C SDA	21	BMP280 0x76, MPU-6050 0x68, ATECC608A 0x60
I²C SCL	22	
MPU-6050 INT (optional)	35	Input-only
SPI SCK	18	Shared: OLED, LoRa
SPI COPI (MOSI)	23	Shared: OLED ("SDA" on the module), LoRa
SPI CIPO (MISO)	19	LoRa only
OLED CS	5	Strapping pin
OLED DC	16	
OLED RES	17	
LoRa CS (NSS)	14	
LoRa RST	26	
LoRa G0 (DIO0)	25	

Free pins for project additions include GPIO 12 (a strapping pin: must not be pulled high at boot), 36 and 39 (input-only). Check Table 2.2 before using any pin.

A.6 `iot_pins.h`

The starter projects define the pins in one header, so code refers to names rather than numbers:

```
#pragma once
#include <stdint.h>

namespace IoTPins {
  // Stage 1
  constexpr uint8_t LED_1    = 2;    // red
  constexpr uint8_t LED_2    = 4;    // yellow
```

```
  constexpr uint8_t LED_3    = 15;   // green
  constexpr uint8_t BUTTON_1 = 32;
  constexpr uint8_t BUTTON_2 = 33;
  constexpr uint8_t POT_ADC  = 34;
  constexpr uint8_t BUZZER   = 27;
  constexpr uint8_t SPI_SCK  = 18;
  constexpr uint8_t SPI_MOSI = 23;
  constexpr uint8_t SPI_MISO = 19;
  constexpr uint8_t OLED_CS  = 5;
  constexpr uint8_t OLED_DC  = 16;
  constexpr uint8_t OLED_RES = 17;
  // Stage 2
  constexpr uint8_t I2C_SDA  = 21;
  constexpr uint8_t I2C_SCL  = 22;
  constexpr uint8_t MPU6050_INT = 35;
  // Stage 3
  constexpr uint8_t SERVO    = 13;
  // Stage 4
  constexpr uint8_t LORA_CS  = 14;
  constexpr uint8_t LORA_RST = 26;
  constexpr uint8_t LORA_G0  = 25;
}

namespace IoTAddr {
  constexpr uint8_t BMP280    = 0x76;
  constexpr uint8_t MPU6050   = 0x68;
  constexpr uint8_t ATECC608A = 0x60;
}
```

A.7 Power

- The board is powered from USB (5 V), regulated to **3.3 V** for the ESP32 and the 3V3 rails.
- **VIN** carries the USB 5 V (minus a diode drop on some boards) and feeds only the **servo rail**.
- Budget: the 3.3 V regulator on a typical DevKit supplies several hundred milliamperes, and Wi-Fi transmission alone can draw up to about 250 mA in bursts. Do not power motors or additional radios from 3V3.

A.8 Kit care

- Unplug USB before changing wiring.
- Keep the antenna attached to the LoRa radio.
- Store the kit with wires pressed flat and parts seated; loose parts are the most common cause of “it worked yesterday”.
- Report damaged parts to the teaching staff rather than swapping in parts from elsewhere.

B. Toolchain Setup

B.1 What you need

- A laptop running macOS, Windows or Linux, with a free USB port (USB-A or USB-C, matching your cable).
- **Visual Studio Code** and the **PlatformIO IDE** extension.
- **Git**, and a GitHub (or similar) account for your team repository.
- Optionally: **Wokwi** (browser-based ESP32 simulator), an **MQTT client** such as MQTT Explorer, **Node-RED**, and **Python 3** with Jupyter for data analysis.

B.2 Installing PlatformIO

1. Install VS Code from code.visualstudio.com.
2. In VS Code, open the Extensions view and install **PlatformIO IDE**. The first installation downloads its own Python environment and tools; this can take several minutes.
3. Restart VS Code. The PlatformIO (alien-head) icon appears in the left sidebar.

B.3 USB drivers

The DevKit's USB-to-UART bridge is usually a **Silicon Labs CP210x** or a **WCH CH340** chip. macOS and Linux usually recognize both; Windows may need the vendor's driver.

To check that the board is detected:

- **macOS**: `ls /dev/cu.*` should show `cu.usbserial-…`, `cu.SLAB_USBtoUART` or `cu.wchusbserial…`.
- **Linux**: `ls /dev/ttyUSB*`; add yourself to the `dialout` (or `uucp`) group if uploads fail with a permission error.
- **Windows**: Device Manager → Ports (COM & LPT) should list a new COM port.

If nothing appears, try a different cable: many USB cables carry power only.

B.4 Opening a project

Starter projects are complete PlatformIO projects.

1. Unzip the starter.
2. In VS Code: **PlatformIO → Open Project**, and select the folder that contains `platformio.ini`. Do not open a single `.cpp` file.
3. Wait for PlatformIO to install the platform and libraries listed in `platformio.ini`.

A typical project looks like this:

```
my-project/
├── platformio.ini       build configuration: board, platform version, libraries
├── include/
│   ├── iot_pins.h       kit pin map
│   ├── secrets.h        your credentials (git-ignored)
│   └── secrets.example.h
├── lib/                 project-specific libraries
├── src/
│   └── main.cpp         your code
└── test/                unit tests
```

B.5 Build, upload and monitor

Use the buttons in the blue status bar at the bottom of VS Code:

Button	Action
✓	Build
→	Upload
🔌	Serial Monitor
🗑️	Clean

Or from the PlatformIO terminal:

```
pio run                 # build
pio run -t upload       # upload
pio device monitor      # Serial Monitor (uses monitor_speed from platformio.ini)
pio test -e native      # run unit tests on the laptop (if configured)
```

If an upload hangs at "Connecting…", hold the **BOOT** button on the board until the upload starts.

B.6 Secrets

Never commit passwords or keys.

1. Keep credentials in `include/secrets.h`, and list that file in `.gitignore`.
2. Commit a `secrets.example.h` with placeholder values so teammates know what to fill in.

```
// secrets.example.h — copy to secrets.h and fill in
#pragma once
#define WIFI_SSID      "your-network"
#define WIFI_PASSWORD  "your-password"
#define MQTT_HOST      "xxxxxxxx.ala.us-east-1.emqxsl.com"
#define MQTT_USER      "device-user"
#define MQTT_PASS      "device-password"
```

B.7 Wokwi

Wokwi simulates an ESP32 with LEDs, buttons, displays and many sensors in the browser, and can also run inside VS Code. It is useful for the early activities, before the kit is built, and for testing logic without hardware. Simulated timing and radios are not identical to real hardware, so always confirm on the kit.

B.8 Networks

Campus enterprise Wi-Fi (WPA2-Enterprise) and networks with captive portals generally do not work with small devices. Use the course IoT network, a phone hotspot (2.4 GHz), or a travel router.

C. Troubleshooting

Work from the bottom of the stack up: power and wiring first, then the bus, then the code, then the network. Change one thing at a time, and write down what you changed.

C.1 Power and upload

Symptom	Likely causes	What to try
Board not detected by the laptop	Charge-only cable; missing driver; bad USB port	Another cable; install CP210x/CH340 driver; another port (Appendix B.)
Upload stuck at "Connecting..."	Auto-reset not working; strapping pin held at the wrong level	Hold BOOT during upload; disconnect parts on GPIO 0, 2, 5, 12, 15 and retry
"A fatal error occurred: ... No serial data received"	Serial Monitor holding the port	Close the monitor, then upload
Board resets repeatedly	Brownout (servo, short circuit); crash loop	Check the reset reason (Chapter 21.); remove the servo; look for shorts between rails
Board gets hot	Short circuit between 3V3/5V and GND	Unplug immediately; inspect the rails

C.2 Serial Monitor

Symptom	Likely causes	What to try
Garbled characters	Baud-rate mismatch	Set the monitor to the speed in `Serial.begin()` (115200)
Nothing printed	Wrong port; code stuck before printing; `Serial.begin()` missing	Print as the first line of `setup()`; check the port
Output only after a reset	Monitor opened late	Press EN after opening the monitor

C.3 GPIO, buttons and analog input

Symptom	Likely causes	What to try
LED does not light	Reversed LED; resistor in the wrong row; wrong pin	Swap LED orientation; trace the row; check `pinMode`
Button reads random values	Floating input	`INPUT_PULLUP`; check the button is wired to GND
Button counts several presses	Bounce	Debounce (Chapter 3.)

Symptom	Likely causes	What to try
Analog reads fail after Wi-Fi starts	ADC2 pin	Use GPIO 32–39
Analog reading stuck at 0 or 4095	Wiper or rail disconnected	Check all three potentiometer legs

C.4 I²C

Symptom	Likely causes	What to try
Scanner finds nothing	SDA/SCL swapped or loose; no power to breakout; cold solder joints	Check 21/22, 3V3 and GND; reflow joints (Lab 0)
Device at an unexpected address	Address pin left floating or tied differently	BMP280 SDO→GND (0x76); MPU-6050 AD0→GND (0x68)
Device appears intermittently	Bad joint; long wires; missing pull-ups	Press-test joints; shorten wires
MPU-6050 returns zeros	Still in sleep mode	Write 0 to register 0x6B (Chapter 6.)
Readings are nonsense	Byte order or sign errors	Check endianness and signedness in the datasheet

C.5 SPI and the OLED

Symptom	Likely causes	What to try
OLED blank	Module "SDA" on 21 instead of 23; DC/RES swapped; `display()` not called	Re-check Table 22.1; call `display()` after drawing
OLED works until LoRa is added	Both chip selects low; CS not initialized high	Set every CS pin HIGH in `setup()` before using SPI
Garbled display	Wrong SPI mode or too high a clock	Use the library defaults

C.6 LoRa

Symptom	Likely causes	What to try
`LoRa.begin()` fails	Wiring (CS 14, RST 26, G0 25); MISO missing; no 3.3 V	Re-check Table 22.4
Nothing received	Different frequency, spreading factor, bandwidth or sync word	Match all settings on both kits
Very short range	Missing or poorly soldered antenna	Attach and check the antenna

C.7 Wi-Fi and MQTT

Symptom	Likely causes	What to try
Never connects to Wi-Fi	5 GHz-only network; WPA2-Enterprise; captive portal; typo	Use a 2.4 GHz WPA2-Personal network (Appendix B.)
TLS connection fails	Wrong CA certificate; clock not set; wrong port	Set time with NTP; check the CA; use 8883
MQTT connects then drops repeatedly	Two clients with the same client ID; `mqtt.loop()` not called	Unique IDs; call `loop()` every pass
Messages not received	Topic typo; subscribed before connecting; payload larger than the buffer	Watch with a desktop client subscribed to `#`; `setBufferSize()`
Works on the bench, fails overnight	Memory leak; no reconnection logic; watchdog	Log free heap and reset reasons (Chapter 21.)

C.8 When you are stuck

1. Write down exactly what you expected and what happened.
2. Reduce the problem to the smallest program that shows it.
3. Swap one component with a known-good one (another team's sensor, another cable).
4. Ask for help with the evidence from steps 1–3: the error message, the minimal program, a photo of the wiring.

D. Glossary

6LoWPAN
IPv6 over Low-Power Wireless Personal Area Networks; an adaptation layer that compresses and fragments IPv6 packets for small radio frames (Chapter 11.).

ACK / NACK
Acknowledgment / negative acknowledgment, confirming whether data was received.

Actuator
A device that converts a control signal into a physical effect such as motion, light or sound (Chapter 7.).

ADC
Analog-to-digital converter; measures a voltage and converts it to a number (Chapter 3.).

AEAD
Authenticated encryption with associated data; encryption that also guarantees integrity, such as AES-GCM (Chapter 14.).

BLE
Bluetooth Low Energy (Chapter 10.).

Border router
A router connecting a low-power network such as 6LoWPAN or Thread to the wider IP network (Chapter 11.).

Broker
In publish/subscribe systems such as MQTT, the server that receives messages and forwards them to subscribers (Chapter 12.).

Certificate (X.509)
A signed statement binding a public key to an identity (Chapter 15.).

CoAP
Constrained Application Protocol; a REST protocol over UDP for constrained devices (Chapter 12.).

Constrained device
A device with limited memory, processing and energy (Chapter 9.).

Debouncing
Filtering the rapid on/off transitions of a mechanical switch (Chapter 3.).

Deep sleep
An ESP32 low-power mode in which only the RTC domain stays powered (Chapter 10.).

DODAG
Destination-oriented directed acyclic graph; the routing structure built by RPL (Chapter 11.).

Duty cycle
The fraction of time a signal is on (PWM), or that a device or radio is active.

eFuse
One-time-programmable memory bits inside a chip, used for keys and security configuration (Chapter 16.).

Edge / fog computing
Processing close to where data is produced, between devices and the cloud (Chapter 18.).

Encapsulation
Wrapping data with each protocol layer's header as it moves down the stack (Chapter 8.).

Fail-safe state
The state that causes the least harm when something fails (Chapter 7.).

Flash encryption
Transparent encryption of a device's external flash memory (Chapter 16.).

GATT
Generic Attribute Profile; the BLE data model of services and characteristics (Chapter 10.).

GPIO
General-purpose input/output pin.

HMAC
Hash-based message authentication code; proves integrity and authenticity with a shared key (Chapter 14.).

Hysteresis
Using separate on and off thresholds to prevent rapid switching near a set point (Chapter 7.).

I²C
Inter-Integrated Circuit; a two-wire, addressed serial bus (Chapter 5.).

IMU
Inertial measurement unit; an accelerometer and gyroscope, sometimes with a magnetometer (Chapter 6.).

Interrupt / ISR
A hardware signal that pauses normal execution to run an interrupt service routine (Chapter 3.).

Interoperability
The ability of systems to exchange information and use it (Chapter 13.).

LLN
Low-power and lossy network (Chapter 11.).

LoRa / LoRaWAN
A long-range, low-power radio modulation, and the network protocol built on it (Chapter 10.).

LPWAN
Low-power wide-area network.

Matter
A smart-home interoperability standard running over IP on Wi-Fi, Ethernet and Thread (Chapter 13.).

MEMS
Micro-electro-mechanical systems; microscopic mechanical structures in silicon, used in most motion and pressure sensors.

MQTT
A lightweight publish/subscribe messaging protocol (Chapter 12.).

mTLS
Mutual TLS, in which both client and server present certificates (Chapter 15.).

NAT
Network address translation, which maps private addresses to a public one (Chapter 8.).

ODA loop
Observe–Decide–Act, the cycle connecting sensing to actuation (Chapter 7.).

OTA update
Over-the-air update: delivering new firmware through the network (Chapter 16.).

PKI
Public-key infrastructure; the certificate authorities, certificates and policies that establish trust (Chapter 15.).

Provisioning
Giving a device its identity, credentials and configuration (Chapter 15.).

Pull-up / pull-down resistor
A resistor that holds an input at a defined level when nothing else drives it (Chapter 3.).

PWM
Pulse-width modulation; switching a signal rapidly to control average power or encode a value (Chapter 3.).

QoS
Quality of service; in MQTT, the delivery guarantee level (Chapter 12.).

Real-time system
A system that must respond within a specified time (Chapter 4.).

Replay attack
Resending a captured valid message to repeat its effect (Chapter 14.).

RPL
IPv6 Routing Protocol for Low-Power and Lossy Networks (Chapter 11.).

RSSI / SNR
Received signal strength indicator / signal-to-noise ratio.

RTOS
Real-time operating system (Chapter 4.).

SBOM
Software bill of materials; a list of all software components in a product (Chapter 16.).

Secure boot
Verifying the signature of each piece of software before running it (Chapter 16.).

Secure element
A tamper-resistant chip that stores keys and performs cryptography without exposing keys (Chapter 15.).

SoC
System on a chip; a microcontroller or processor integrated with larger subsystems such as radios.

SPI
Serial Peripheral Interface; a fast synchronous bus with a chip-select line per device (Chapter 5.).

STRIDE
Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege; a threat-modeling mnemonic (Chapter 14.).

Thing Description
A W3C JSON-LD document describing a device's properties, actions and events (Chapter 13.).

Thread
An IPv6 mesh networking protocol on 802.15.4 radios (Chapter 11.).

TinyML
Machine learning on microcontrollers (Chapter 18.).

TLS / DTLS
Transport Layer Security, and its datagram (UDP) variant.

TSN
Time-Sensitive Networking; IEEE standards for deterministic Ethernet (Chapter 10.).

UART
: Universal asynchronous receiver/transmitter; point-to-point serial communication without a clock wire (Chapter 5.).

Vertical slice
: A thin, end-to-end working version of a whole system (Chapter 20.).

Watchdog timer
: A timer that resets the processor unless software services it regularly (Chapter 4.).

E. Setting Up a Teaching Lab

This appendix is for instructors and lab managers who want to run the course, or a course like it. It describes what a room needs so that every team can build, solder, program and test its kit safely. Quantities assume a section of about 30 students working in teams of two or three, with up to 10 people soldering at once.

E.1 The room

Table E.1: Room requirements.

Need	Recommendation	Why
Layout	A dedicated soldering bench along one wall; separate tables for pair work with laptops; an instrument shelf; lockable storage; a visible safety station	Keeps hot irons away from laptops, paper and foot traffic
Power	Hard-wired or properly rated outlets at each bench; no daisy-chained power strips; cords routed off the floor	Every station draws power at once, and crossing cords are a trip and burn hazard
Ventilation	A fume extractor at each soldering station, and confirmation from your environmental health and safety office that room ventilation is adequate	Rosin flux fumes are a respiratory irritant
Surfaces	Heat-resistant, wipeable bench tops or silicone mats at the soldering bench	Solder spatter and dropped irons
Seating	Fixed, wipeable stools at the soldering bench	Fabric chairs on casters collect spatter, build static, and roll over cords
Lighting	Task lighting at the soldering bench	Small pads and fine pitch need good light
Network	A 2.4 GHz WPA2-Personal network able to handle about 30 devices, or a dedicated travel router	Enterprise and captive-portal networks block small devices (Appendix B.)
Safety station	First-aid kit including burn dressings, fire extinguisher, eyewash if required, posted soldering procedure and emergency contacts	Required by most institutions; makes the rules visible

E.2 Shared equipment (per 10 soldering stations)

Table E.2: Shared equipment.

Qty	Item	Notes (Fall 2026 choice)
10	Temperature-controlled soldering stations with stands	60 W, with spare tips, helping hands and a solder sucker (YIHUA 926 III)
10	Fume extractors	One per station
10	Silicone work mats	Heat-resistant, ≈ 34 × 23 cm
30+	Safety glasses	One per person soldering, plus spares
2	Spools of lead-free solder, 0.8 mm, rosin core	SAC305
5	Flux pens	No-clean
10	Desoldering braid	One spool per station
6	Sets of 22 AWG solid-core wire, six colors	For the cut-and-strip activity and flat breadboard wiring
12	Automatic wire strippers	Covering 22 AWG
15	Flush cutters	One per station plus spares
5	Digital multimeters	With continuity beeper
12	USB-C inline power meters	For measuring board current; most cannot resolve deep-sleep currents
1	Travel router	A private 2.4 GHz network for demos
2	Burn first-aid kits	Supplementing the room's general kit
1	Small-parts organizer	Spare LEDs, resistors, buttons and headers

In Fall 2026 this shared equipment cost about **$1,900** for 10 stations, and kit parts for 36 new kits about **$2,000**.

E.3 Before the first lab

- **Kits**: count, label and test every kit; flash a kit-check sketch; pre-solder or pre-provision anything students should not do themselves (for example, locking secure elements).
- **Network**: test that 30 boards can join the room network and reach your broker or server at the same time.
- **Safety briefing**: present the soldering procedure, show the safety station and require safety glasses.
- **Staffing**: plan at least one teaching assistant per 10 students during soldering.
- **Kit policy**: decide whether kits are returned at the end of the term or can be bought, and how damaged parts are replaced.

E.4 Running costs

Budget each term for consumables (solder, flux, wire, resistors and LEDs), replacement parts (an ESP32 or a sensor per few kits is typical), and spare tips for the soldering stations.

References

Bibliography

[1] A. Rayes and S. Salam, *Internet of Things from Hype to Reality: The Road to Digitization*, 3rd ed. Cham: Springer, 2022. doi: 10.1007/978-3-030-90158-5.

[2] M. Dabbagh and A. Rayes, "Internet of Things Security and Privacy," in *Internet of Things from Hype to Reality: The Road to Digitization*, A. Rayes and S. Salam, Eds., Cham: Springer, 2017, pp. 195–223. doi: 10.1007/978-3-319-44860-2_8.

[3] IoT Analytics, "State of IoT 2025: Number of connected IoT devices growing 14% to 21.1 billion globally." [Online]. Available: https://iot-analytics.com/number-connected-iot-devices/

[4] D. Evans, "The Internet of Things: How the Next Evolution of the Internet Is Changing Everything," White paper, 2011.

[5] Espressif Systems, "ESP-IDF Programming Guide: Sleep Modes." [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/sleep_modes.html

[6] "Recommendation Y.2060: Overview of the Internet of Things," technical report, 2012.

[7] "ESP32 Series Datasheet." [Online]. Available: https://www.espressif.com/en/support/documents/technical-documents

[8] M. Margolis, B. Jepson, and N. R. Weldin, *Arduino Cookbook*, 3rd ed. O'Reilly Media, 2020.

[9] "ESP32 Technical Reference Manual." [Online]. Available: https://www.espressif.com/en/support/documents/technical-documents

[10] Espressif Systems, "Arduino-ESP32 Documentation." [Online]. Available: https://docs.espressif.com/projects/arduino-esp32/

[11] PlatformIO Labs, "PlatformIO Documentation." [Online]. Available: https://docs.platformio.org/

[12] C. L. Liu and J. W. Layland, "Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.

[13] R. Barry, *Mastering the FreeRTOS Real Time Kernel: A Hands-On Tutorial Guide*. Real Time Engineers Ltd., 2016. [Online]. Available: https://www.freertos.org/Documentation/02-Kernel/07-Books-and-manual/01-RTOS_book

[14] Espressif Systems, "ESP-IDF Programming Guide: FreeRTOS." [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/freertos.html

[15] "UM10204: I2C-bus Specification and User Manual." 2021. [Online]. Available: https://www.nxp.com/docs/en/user-guide/UM10204.pdf

[16] "SPI Block Guide." 2003.

[17] "BMP280 Digital Pressure Sensor Datasheet." [Online]. Available: https://www.bosch-sensortec.com/products/environmental-sensors/pressure-sensors/bmp280/

[18] "MPU-6000 and MPU-6050 Product Specification."

[19] "ATECC608A CryptoAuthentication Device Summary Datasheet." [Online]. Available: https://www.microchip.com/

[20] SparkFun Electronics, "Serial Communication." [Online]. Available: https://learn.sparkfun.com/tutorials/serial-communication/all

[21] "SX1276/77/78/79 Datasheet: 137 MHz to 1020 MHz Low Power Long Range Transceiver." [Online]. Available: https://www.semtech.com/products/wireless-rf/lora-connect/sx1276

[22] J. Fraden, *Handbook of Modern Sensors: Physics, Designs, and Applications*, 5th ed. Springer, 2016.

[23] K. J. Åström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*, 2nd ed. Princeton University Press, 2021. [Online]. Available: https://fbswiki.org/

[24] Adafruit Industries, "Adafruit GFX Graphics Library." [Online]. Available: https://learn.adafruit.com/adafruit-gfx-graphics-library

[25] A. L. Russell, "OSI: The Internet That Wasn't," *IEEE Spectrum*, vol. 50, no. 8, pp. 39–43, 2013.

[26] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed. Pearson, 2021.

[27] C. Bormann, M. Ersue, and A. Keranen, "Terminology for Constrained-Node Networks," RFC 7228, 2014.

[28] M. Antonakakis and others, "Understanding the Mirai Botnet," in *Proceedings of the 26th USENIX Security Symposium*, 2017, pp. 1093–1110.

[29] C. E. Shannon, "A Mathematical Theory of Communication," *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948, doi: 10.1002/j.1538-7305.1948.tb01338.x.

[30] "LoRaWAN Specification." [Online]. Available: https://lora-alliance.org/resource_hub/
[31] H. T. Friis, "A Note on a Simple Transmission Formula," *Proceedings of the IRE*, vol. 34, no. 5, pp. 254–256, 1946, doi: 10.1109/JRPROC.1946.234568.
[32] "LoRa Modem Designer's Guide (AN1200.13)." 2013.
[33] K. Townsend, C. Cuf\i, Akiba, and R. Davidson, *Getting Started with Bluetooth Low Energy*. O'Reilly Media, 2014.
[34] G. Montenegro, N. Kushalnagar, J. Hui, and D. Culler, "Transmission of IPv6 Packets over IEEE 802.15.4 Networks," RFC 4944, 2007.
[35] J. Hui and P. Thubert, "Compression Format for IPv6 Datagrams over IEEE 802.15.4-Based Networks," RFC 6282, 2011.
[36] T. Winter and others, "RPL: IPv6 Routing Protocol for Low-Power and Lossy Networks," RFC 6550, 2012.
[37] D. S. J. De Couto, D. Aguayo, J. Bicket, and R. Morris, "A High-Throughput Path Metric for Multi-Hop Wireless Routing," in *Proceedings of the 9th Annual International Conference on Mobile Computing and Networking (MobiCom)*, 2003, pp. 134–146. doi: 10.1145/938985.939000.
[38] Z. Shelby and C. Bormann, *6LoWPAN: The Wireless Embedded Internet*. Wiley, 2009.
[39] Z. Shelby, K. Hartke, and C. Bormann, "The Constrained Application Protocol (CoAP)," RFC 7252, 2014.
[40] K. Hartke, "Observing Resources in the Constrained Application Protocol (CoAP)," RFC 7641, 2015.
[41] "MQTT Version 5.0." 2019. [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html
[42] HiveMQ, "MQTT Essentials." [Online]. Available: https://www.hivemq.com/mqtt/
[43] A. Tolk and J. A. Muguira, "The Levels of Conceptual Interoperability Model," in *Proceedings of the 2003 Fall Simulation Interoperability Workshop*, 2003.
[44] oneM2M, "oneM2M Technical Specifications." [Online]. Available: https://www.onem2m.org/technical/published-specifications
[45] "Web of Things (WoT) Thing Description 1.1." 2023. [Online]. Available: https://www.w3.org/TR/wot-thing-description11/
[46] "Matter Specification." [Online]. Available: https://csa-iot.org/developer-resource/specifications-download-request/
[47] C. Bormann and P. Hoffman, "Concise Binary Object Representation (CBOR)," RFC 8949, 2020. doi: 10.17487/RFC8949.
[48] F. Delmastro, "Pervasive Communications in Healthcare," *Computer Communications*, vol. 35, no. 11, pp. 1284–1295, 2012.
[49] A. Shostack, *Threat Modeling: Designing for Security*. Wiley, 2014.
[50] OWASP Foundation, "OWASP Internet of Things Top 10." [Online]. Available: https://owasp.org/www-project-internet-of-things/
[51] V. Hassija, V. Chamola, V. Saxena, D. Jain, P. Goyal, and B. Sikdar, "A Survey on IoT Security: Application Areas, Security Threats, and Solution Architectures," *IEEE Access*, vol. 7, pp. 82721–82743, 2019.
[52] National Institute of Standards and Technology, "Digital Signature Standard (DSS)," FIPS 186–5, 2023. doi: 10.6028/NIST.FIPS.186-5.
[53] T. Pornin, "Deterministic Usage of the Digital Signature Algorithm (DSA) and Elliptic Curve Digital Signature Algorithm (ECDSA)," RFC 6979, 2013. doi: 10.17487/RFC6979.
[54] M. Fagan, K. N. Megas, K. Scarfone, and M. Smith, "Foundational Cybersecurity Activities for IoT Device Manufacturers," NIST IR 8259, 2020.
[55] B. Moran, H. Tschofenig, D. Brown, and M. Meriac, "A Firmware Update Architecture for Internet of Things," RFC 9019, 2021.
[56] Espressif Systems, "ESP-IDF Programming Guide: Over The Air Updates (OTA)." [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/ota.html
[57] Espressif Systems, "ESP-IDF Programming Guide: Security (Secure Boot V2, Flash Encryption)." [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/security/index.html
[58] "ETSI EN 303 645: Cyber Security for Consumer Internet of Things: Baseline Requirements." 2020.
[59] "Regulation (EU) 2024/2847 of the European Parliament and of the Council on horizontal cybersecurity requirements for products with digital elements (Cyber Resilience Act)." [Online]. Available: https://eur-lex.europa.eu/eli/reg/2024/2847/oj
[60] Nextgov/FCW, "FCC commissioner hopes Cyber Trust Mark products reach shoppers within a year." [Online]. Available: https://www.nextgov.com/cybersecurity/2026/10/fcc-commissioner-hopes-cyber-trust-mark-products-reach-shoppers-within-year/416878/

[61] B. Jacob *et al.*, "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.
[62] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media, 2019.
[63] Google AI Edge, "LiteRT for Microcontrollers." [Online]. Available: https://ai.google.dev/edge/litert/microcontrollers/overview
[64] C. Banbury and others, "MLPerf Tiny Benchmark," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.
[65] J. G. Koomey, S. Berard, M. Sanchez, and H. Wong, "Implications of Historical Trends in the Electrical Efficiency of Computing," *IEEE Annals of the History of Computing*, vol. 33, no. 3, pp. 46–54, 2011, doi: 10.1109/MAHC.2010.28.
[66] M. S. Ali, M. Vecchio, M. Pincheira, K. Dolui, F. Antonelli, and M. H. Rehmani, "Applications of Blockchains in the Internet of Things: A Comprehensive Survey," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 2, pp. 1676–1717, 2019.
[67] J. D. C. Little, "A Proof for the Queuing Formula: L = \lambda W", *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961, doi: 10.1287/opre.9.3.383.
[68] M. T. Nygard, *Release It! Design and Deploy Production-Ready Software*, 2nd ed. Pragmatic Bookshelf, 2018.
[69] A. Cockburn, *Writing Effective Use Cases*. Addison-Wesley, 2000.
[70] J. Brooks Frederick P., *The Mythical Man-Month: Essays on Software Engineering*, Anniversary. Addison-Wesley, 1995.
[71] J. A. Hanley and A. Lippman-Hand, "If Nothing Goes Wrong, Is Everything All Right? Interpreting Zero Numerators," *JAMA*, vol. 249, no. 13, pp. 1743–1745, 1983, doi: 10.1001/jama.1983.03330370053031.
[72] S. G. Eick, C. R. Loader, M. D. Long, L. G. Votta, and S. Vander Wiel, "Estimating Software Fault Content Before Coding," in *Proceedings of the 14th International Conference on Software Engineering (ICSE)*, 1992, pp. 59–65. doi: 10.1145/143062.143090.
[73] OWASP Foundation, "OWASP IoT Security Testing Guide." [Online]. Available: https://owasp.org/owasp-istg/